CẨM NANG NGÀNH DEVOPS

Bản thảo tích hợp Đợt 2 · 143/178 bài có nội dung

Chưa phải bản phát hành hoàn chỉnh

MỤC LỤC

PHẦN 01 — Máy tính và tiến trình 7

Chương P01.C01 — Công việc, thời gian và đường đi dữ liệu . . . 8

Bài P01.C01.L01 — Máy thực thi . . . 8

Bài P01.C01.L02 — Đường thời gian . . . 9

Bài P01.C01.L03 — Dữ liệu đi qua I/O . . . 9

Chương P01.C02 — Chương trình, tiến trình và ranh giới nhân . . . 11

Bài P01.C02.L01 — Ba dạng ứng dụng . . . 11

Bài P01.C02.L02 — Tạo và thay process . . . 11

Bài P01.C02.L03 — Descriptor và identity . . . 13

PHẦN 02 — Linux và vận hành máy chủ 14

Chương P02.C01 — Syscall, đường dẫn và hệ thống tệp . . . 15

Bài P02.C01.L01 — Vào nhân . . . 15

Bài P02.C01.L02 — Phân giải tên . . . 15

Bài P02.C01.L03 — Ghi và bền vững . . . 16

Chương P02.C02 — Danh tính và quyền Linux . . . 17

Bài P02.C02.L01 — Danh tính tiến trình . . . 17

Bài P02.C02.L02 — Bước đi qua thư mục . . . 17

Bài P02.C02.L03 — Điều tra và sửa . . . 18

Chương P02.C03 — Tài nguyên và chẩn đoán hiệu năng Linux . . . 20

Bài P02.C03.L01 — CPU và scheduler . . . 20

Bài P02.C03.L02 — Memory . . . 20

Bài P02.C03.L03 — I/O và độ bền . . . 21

Bài P02.C03.L04 — Điều tra đa tín hiệu . . . 22

Chương P02.C04 — systemd và vòng đời dịch vụ . . . 24

Bài P02.C04.L01 — Unit graph . . . 24

Bài P02.C04.L02 — Process lifecycle . . . 25

Bài P02.C04.L03 — Quan sát dịch vụ . . . 25

Chương P02.C05 — Vận hành host và phục hồi có kiểm chứng . . . 27

Bài P02.C05.L01 — Thay đổi host . . . 27

Bài P02.C05.L02 — Dữ liệu host . . . 28

Bài P02.C05.L03 — Phục hồi . . . 28

PHẦN 03 — Mạng và giao thức ứng dụng 30

Chương P03.C01 — Địa chỉ, subnet và đường đi gói IP . . . 31

Bài P03.C01.L01 — Định danh logic . . . 31

Bài P03.C01.L02 — Chọn chặng . . . 31

Bài P03.C01.L03 — Đóng frame . . . 32

Chương P03.C02 — Transport, TCP, UDP và NAT . . . 33

Bài P03.C02.L01 — Bắt tay và byte stream . . . 33

Bài P03.C02.L02 — Socket và lỗi . . . 34

Bài P03.C02.L03 — NAT và đối xứng đường . . . 35
Chương P03.C03 — Tên, TLS và ngữ nghĩa HTTP . . . 36
Bài P03.C03.L01 — DNS và cache . . . 36
Bài P03.C03.L02 — TLS . . . 36
Bài P03.C03.L03 — HTTP . . . 37
Chương P03.C04 — Reverse proxy và điều tra request hai chặng . . . 38
Bài P03.C04.L01 — Chọn upstream . . . 38
Bài P03.C04.L02 — Hai chặng . . . 38
Bài P03.C04.L03 — Đường lỗi . . . 39
PHẦN 04 — Git và lịch sử thay đổi 41
Chương P04.C01 — Git object, index và ref . . . 42
Bài P04.C01.L01 — Object graph . . . 42
Bài P04.C01.L02 — Trạng thái cục bộ . . . 42
Chương P04.C02 — Tích hợp và sửa lịch sử . . . 44
Bài P04.C02.L01 — Merge . . . 44
Bài P04.C02.L02 — Rebase . . . 44
Bài P04.C02.L03 — Phục hồi . . . 45
PHẦN 05 — Tự động hóa hệ thống 46
Chương P05.C01 — Bash và ngữ nghĩa lỗi của shell . . . 47
Bài P05.C01.L01 — Parsing . . . 47
Bài P05.C01.L02 — I/O và status . . . 48
Bài P05.C01.L03 — Control flow . . . 48
Chương P05.C02 — Python và Go điều phối tiến trình . . . 50
Bài P05.C02.L01 — Chọn ngôn ngữ . . . 50
Bài P05.C02.L02 — Subprocess . . . 51
Bài P05.C02.L03 — Timeout . . . 56
Chương P05.C03 — Workflow an toàn và chạy lại . . . 58
Bài P05.C03.L01 — Hợp đồng bước . . . 58
Bài P05.C03.L02 — Retry . . . 58
Bài P05.C03.L03 — Phục hồi . . . 59
PHẦN 06 — Build, kiểm thử và phát hành 61
Chương P06.C01 — Build và danh tính artifact . . . 62
Bài P06.C01.L01 — Đầu vào build . . . 62
Bài P06.C01.L02 — Artifact . . . 62
Chương P06.C02 — Test gates và CI/CD . . . 63
Bài P06.C02.L01 — CI vs CD . . . 63
Bài P06.C02.L02 — Test model . . . 63
Bài P06.C02.L03 — Gate . . . 64
Chương P06.C03 — Rollout và quyết định dừng . . . 65
Bài P06.C03.L01 — Chiến lược . . . 65
Bài P06.C03.L02 — Readiness . . . 65
Bài P06.C03.L03 — Điều phối . . . 65

Chương P06.C04 — Rollback và tương thích dữ liệu . . . 67
Bài P06.C04.L01 — Dữ liệu qua bản phát hành . . . 67
Bài P06.C04.L02 — Partial deployment . . . 67
Bài P06.C04.L03 — Phục hồi . . . 68
PHẦN 07 — Container và OCI 69
Chương P07.C01 — Namespace, cgroup v2 và runtime boundary . . . 70
Bài P07.C01.L01 — Tạo/tham gia . . . 70
Bài P07.C01.L02 — View . . . 70
Bài P07.C01.L03 — Limit . . . 71
Chương P07.C02 — OCI image, build và registry . . . 74
Bài P07.C02.L01 — Build . . . 74
Bài P07.C02.L02 — OCI . . . 74
Bài P07.C02.L03 — Distribution . . . 75
Chương P07.C03 — Container lifecycle và dữ liệu . . . 76
Bài P07.C03.L01 — Lần chạy . . . 76
Bài P07.C03.L03 — Mount . . . 76
Chương P07.C04 — Compose và ứng dụng nhiều service . . . 78
Bài P07.C04.L01 — Mô hình project . . . 78
Bài P07.C04.L02 — Khởi động . . . 78
Bài P07.C04.L03 — Dữ liệu . . . 78
PHẦN 08 — Cloud và AWS 80
Chương P08.C01 — AWS identity và IAM evaluation . . . 81
Bài P08.C01.L01 — Danh tính . . . 81
Bài P08.C01.L02 — Quyết định . . . 81
Bài P08.C01.L03 — Điều tra . . . 82
Chương P08.C02 — VPC, subnet và đường đi AWS . . . 83
Bài P08.C02.L01 — Topology . . . 83
Bài P08.C02.L02 — Chính sách gói . . . 83
Bài P08.C02.L03 — Private access . . . 84
Chương P08.C03 — EC2, load balancer và miền lỗi . . . 85
Bài P08.C03.L02 — Traffic . . . 85
Chương P08.C04 — Block, file và object storage . . . 86
Bài P08.C04.L01 — Block . . . 86
Bài P08.C04.L02 — File . . . 86
Bài P08.C04.L03 — Object . . . 86
PHẦN 09 — Terraform và trạng thái hạ tầng 87
Chương P09.C01 — HCL, provider và dependency DAG . . . 88
Bài P09.C01.L01 — Ngôn ngữ . . . 88
Bài P09.C01.L03 — DAG . . . 88
Chương P09.C02 — Plan, state và backend . . . 90
Bài P09.C02.L01 — Ba trạng thái . . . 90
Bài P09.C02.L02 — Plan/apply . . . 90

Bài P09.C02.L03 — Backend . . . 91
Chương P09.C03 — Drift, import và refactor . . . 92
Bài P09.C03.L01 — Drift . . . 92
Chương P09.C04 — Thay đổi hạ tầng có dữ liệu . . . 93
Bài P09.C04.L01 — Replacement . . . 93
Bài P09.C04.L02 — Điều phối . . . 93
PHẦN 10 — Ansible và hội tụ cấu hình 95
Chương P10.C01 — Inventory và đường điều khiển Ansible . . . 96
Bài P10.C01.L01 — Topology . . . 96
Bài P10.C01.L02 — Đơn vị chạy . . . 96
Bài P10.C01.L03 — Quyền/biến . . . 97
Chương P10.C02 — Hội tụ, handler và thất bại từng host . . . 98
Bài P10.C02.L01 — Idempotency . . . 98
Bài P10.C02.L02 — Handler . . . 99
Bài P10.C02.L03 — Failure . . . 101
Chương P10.C03 — Vai trò, secrets và triển khai theo đợt . . . 102
Bài P10.C03.L01 — Tái sử dụng . . . 102
Bài P10.C03.L02 — Phân đợt . . . 102
PHẦN 11 — Kubernetes và cloud native 104
Chương P11.C01 — API, etcd và reconciliation . . . 105
Bài P11.C01.L01 — Control plane . . . 105
Bài P11.C01.L02 — Reconciliation . . . 105
Bài P11.C01.L03 — Node boundary . . . 106
Chương P11.C02 — Pod, Deployment và readiness . . . 107
Bài P11.C02.L01 — Workload objects . . . 107
Bài P11.C02.L02 — Probe . . . 107
Bài P11.C02.L03 — Rollout . . . 109
Chương P11.C03 — Scheduler và ngân sách tài nguyên . . . 111
Bài P11.C03.L01 — Placement . . . 111
Bài P11.C03.L02 — Runtime budget . . . 112
Bài P11.C03.L03 — Resource metric tối thiểu . . . 112
Chương P11.C04 — Service, EndpointSlice và NetworkPolicy . . . 113
Bài P11.C04.L01 — Discovery . . . 113
Bài P11.C04.L02 — Dataplane . . . 113
Bài P11.C04.L03 — Policy . . . 114
Chương P11.C05 — PVC, CSI và dữ liệu workload . . . 115
Bài P11.C05.L01 — Tài nguyên . . . 115
Bài P11.C05.L02 — Thực thi . . . 115
Bài P11.C05.L03 — Vòng đời . . . 116
Chương P11.C06 — HPA và điều khiển mở rộng . . . 117
Bài P11.C06.L01 — Phép đo . . . 117
Bài P11.C06.L02 — Vòng tính . . . 117

Bài P11.C06.L03 — Hội tụ . . . 118
Chương P11.C07 — Gateway API và ranh giới request ngoài cluster . . . 119
Bài P11.C07.L01 — API objects . . . 119
Bài P11.C07.L02 — Attachment . . . 120
Bài P11.C07.L03 — Request path . . . 121
Chương P11.C08 — Helm chart và vòng đời release . . . 122
Bài P11.C08.L01 — Chart . . . 122
Bài P11.C08.L02 — Release . . . 124
Bài P11.C08.L03 — Rollback . . . 124
PHẦN 12 — Observability và telemetry 126
Chương P12.C01 — Phép đo, nhãn và chuỗi thời gian . . . 127
Bài P12.C01.L01 — Đo . . . 127
Bài P12.C01.L02 — Tập hợp . . . 127
Bài P12.C01.L03 — Thu thập . . . 128
Chương P12.C02 — Prometheus, PromQL và Grafana . . . 129
Bài P12.C02.L01 — PromQL . . . 129
Bài P12.C02.L02 — Histogram . . . 130
Bài P12.C02.L03 — Dashboard . . . 131
Chương P12.C03 — Logs, traces và OpenTelemetry . . . 132
Bài P12.C03.L01 — Log . . . 132
Bài P12.C03.L02 — Trace . . . 133
Bài P12.C03.L03 — Pipeline . . . 135
Chương P12.C04 — Alerting và sức khỏe pipeline quan sát . . . 137
Bài P12.C04.L01 — Rule . . . 137
Bài P12.C04.L03 — Pipeline . . . 137
PHẦN 13 — SRE và phục hồi 138
Chương P13.C01 — SLI, SLO và error budget . . . 139
Bài P13.C01.L01 — SLI . . . 139
Bài P13.C01.L02 — SLO . . . 139
Bài P13.C01.L03 — Burn . . . 140
Chương P13.C02 — Incident response và postmortem . . . 143
Bài P13.C02.L01 — Phản ứng . . . 143
Bài P13.C02.L02 — Giảm tác động . . . 144
Bài P13.C02.L03 — Học lại . . . 144
Chương P13.C03 — Backup, DR và phục hồi dữ liệu . . . 147
Bài P13.C03.L01 — Mục tiêu . . . 147
Bài P13.C03.L02 — Bản sao . . . 148
Bài P13.C03.L03 — Diễn tập . . . 148
Chương P13.C04 — Hiệu năng, capacity và thử giới hạn . . . 150
Bài P13.C04.L01 — Workload . . . 150
Bài P13.C04.L02 — Capacity . . . 150
Bài P13.C04.L03 — Experiment . . . 151

PHẦN 01 — Máy tính và tiến trình

Một dòng mã trở thành công việc có thể quan sát bằng cách nào?

Chương trình, tiến trình, luồng và tài nguyên là các thực thể khác nhau; thời gian là tổng của chạy, chờ và phối hợp.

Chương P01.C01 — Công việc, thời gian và đường đi dữ liệu

Phân tích một request thành tính toán, chờ và tác dụng phụ.

Bài P01.C01.L01 — Máy thực thi

Một chương trình chỉ trở thành công việc khi hệ điều hành tạo tiến trình, cấp không gian địa chỉ và cho một hoặc nhiều luồng của nó cơ hội chạy. Tệp chương trình trên ổ lưu trữ là dữ liệu; lõi xử lý thực thi lệnh của luồng đã được bộ lập lịch chọn. Vì vậy, "máy có tám CPU" thường nói về tám đơn vị xử lý logic mà hệ điều hành có thể lên lịch, không hứa tám tác vụ sẽ tiến nhanh gấp tám lần. Hai luồng cùng sửa một dữ liệu còn phải phối hợp; nhiều lõi không xóa được thời gian chờ khóa hay thiết bị.

Ta có thể lần theo một request giả định: nhận byte từ mạng, giải mã trong bộ nhớ, đọc một tệp, tính kết quả rồi ghi câu trả lời. CPU thực hiện lệnh và di chuyển dữ liệu qua các tầng nhớ; RAM giữ mã, dữ liệu hoạt động và các trang cache; bộ điều khiển thiết bị cùng kernel đưa dữ liệu giữa bộ nhớ và mạng hoặc ổ lưu trữ. Khi tác vụ phải đợi dữ liệu chưa có, luồng có thể ngừng chạy để luồng khác dùng lõi. Một lần đọc sau có thể nhanh hơn vì dữ liệu đã nằm trong page cache; kết quả ấy không chứng minh ổ lưu trữ vừa nhanh hơn. Cũng không thể suy dung lượng bộ nhớ "bị ứng dụng dùng hết" chỉ từ lượng RAM đã được kernel dùng làm cache.

Đồng hồ trả lời một câu hỏi khác với bộ đếm CPU. Thời gian từ lúc bắt đầu tới lúc kết thúc request bao gồm cả chạy lẫn chờ; CPU time chỉ cộng thời gian được tính cho tiến trình hoặc luồng trong khi dùng CPU. Trên Linux, đồng hồ đơn điệu phù hợp để lấy hiệu giữa hai thời điểm của cùng máy vì không nhảy lùi khi chỉnh giờ hệ thống; đồng hồ thời gian thực phục vụ mốc lịch nhưng có thể được chỉnh. Đồng hồ đơn điệu thông thường không đếm thời gian máy suspend. Khi so log từ hai máy, phải xét đồng bộ giờ và vị trí đo; không lấy hiệu hai mốc từ những đồng hồ không có quan hệ đã kiểm chứng.

Ví dụ tính toán, không phải phép đo: request bắt đầu ở 0 ms, chờ lịch chạy 8 ms, chạy CPU 12 ms, chờ I/O 60 ms, chạy thêm 20 ms rồi hoàn tất. Wall time là 100 ms, CPU time theo mô hình này là 32 ms. Tăng gấp đôi tốc độ phần tính toán nhưng giữ nguyên hai khoảng chờ chỉ hạ wall time xuống 84 ms. Ngay cả con số đó là giả định: cache, tranh chấp và thời điểm hoàn tất I/O có thể đổi khi chương trình thật chạy. Bài kế tiếp dùng phân biệt này để đọc latency mà không nhầm nó với mức bận CPU.

Bài tập P01.C01.L01.Q01

Trong timeline giả định 8 ms chờ lịch chạy, 12 ms CPU, 60 ms chờ I/O và 20 ms CPU, hãy tính wall time và CPU time. Nếu chỉ phần CPU nhanh gấp đôi, kết quả tối thiểu theo mô hình là bao nhiêu?

Bài P01.C01.L02 — Đường thời gian

Hãy xét một ví dụ tính toán, không phải kết quả đo: yêu cầu A hoàn tất sau 100 ms theo đồng hồ, trong đó CPU xử lý 40 ms và các khoảng chờ cộng lại 60 ms. Độ trễ của A là 100 ms, còn CPU time gắn với phần xử lý của A là 40 ms. Nếu chỉ giảm thời gian CPU một nửa mà giữ nguyên mọi khoảng chờ, độ trễ mới tốt nhất trong mô hình đơn giản này là 80 ms. Kết quả đó không tiên đoán hệ thống thực; nó chỉ cho thấy không thể lấy tỷ lệ CPU bận để suy trực tiếp độ trễ của từng yêu cầu.

Thông lượng là số công việc hoàn thành trong một khoảng thời gian; độ trễ là thời gian một công việc mất từ đầu đến cuối. Hai đại lượng liên hệ nhưng không thay thế nhau. Một hệ thống có thể giữ thông lượng bằng cách xếp thêm yêu cầu vào hàng đợi, trong khi người dùng chờ lâu hơn. Vì thế, khi nói "cần thêm máy", phải biết mục tiêu là tăng thông lượng, giảm độ trễ ở một mức tải hay giữ hệ thống ổn định khi có đột biến.

Bài tập P01.C01.L02.Q01

Trong một ví dụ giả định, độ trễ yêu cầu là 100 ms gồm 40 ms CPU và 60 ms chờ. Nếu chỉ giảm phần CPU xuống 20 ms, độ trễ thấp nhất theo mô hình cộng đơn giản là bao nhiêu? Nêu một lý do khiến kết quả thực tế khác phép tính.

Bài P01.C01.L03 — Dữ liệu đi qua I/O

Một lời gọi ghi tệp đi qua nhiều ranh giới: bộ nhớ của tiến trình, lời gọi hệ thống, cache của kernel, filesystem, bộ điều khiển và thiết bị lưu trữ. "Ghi thành công" trước hết nghĩa là lời gọi đã chấp nhận số byte theo hợp đồng của nó; không đồng nghĩa mọi byte đã bền vững sau mất điện. Tầng đệm cho phép ứng dụng và thiết bị chạy với nhịp khác nhau. Page cache có thể giữ trang mới sửa ở trạng thái dirty, rồi kernel đưa chúng xuống thiết bị trong một lần writeback sau đó. Nếu chậm ở bước này, latency có thể xuất hiện muộn hơn lời gọi ghi ban đầu.

Hãy xét một thao tác thay cấu hình bằng tệp mới, với đường cũ vẫn còn cho tới khi thay tên. Ứng dụng ghi nội dung vào tệp tạm, yêu cầu đồng bộ tệp, đổi tên trong cùng filesystem, rồi yêu cầu đồng bộ thư mục chứa tên mới. Đây là một trình tự thiết kế độ bền trên hệ thống hỗ trợ các thao tác ấy; chỉ đồng bộ dữ liệu tệp chưa đủ để bảo đảm mục nhập thư mục mới đã bền. Ngược lại, thao tác đổi tên giúp người đọc tránh thấy một tệp đang ghi dở trong nhiều trường hợp, nhưng tính nguyên tử của tên không tự bảo đảm byte đã ở thiết bị. Cần xử lý từng lỗi trả về, ghi nhận filesystem và loại thiết bị, rồi thử khôi phục trong điều kiện mục tiêu; không biến một chuỗi lời gọi thành lời hứa tuyệt đối về mọi phần cứng.

Đọc cũng có hai đường. Nếu trang cần đọc đã ở page cache, kernel có thể phục vụ mà không chờ thiết bị. Nếu chưa có, tác vụ có thể phải đợi I/O vật lý; readahead và truy cập của tác vụ khác làm phép so "lần một so với lần hai" không còn là thí nghiệm sạch. Độ trễ thiết bị, độ trễ hàng đợi và thời gian hệ thống gọi lại ứng dụng là các thành phần khác nhau. Một counter tổng byte đọc từ ổ không chứng minh request cụ thể đã đọc byte ấy; cần tương quan thời điểm và tác vụ trước khi quy nguyên nhân.

Ví dụ tính toán: dịch vụ báo đã nhận 64 KiB và trả HTTP thành công sau 5 ms; sau đó máy mất điện. Nếu không có hợp đồng đồng bộ hoặc hệ lưu trữ giao dịch, câu trả lời HTTP chỉ là bằng chứng ứng dụng đã phát phản hồi, không là bằng chứng dữ liệu có thể đọc lại sau reboot. Với yêu cầu quan trọng, phải định nghĩa rõ điểm xác nhận bền vững, kiểm lỗi ghi và thử đọc lại từ bản phục hồi. Bài về Linux và backup sau này sẽ nối ranh giới này với snapshot nhất quán và phép thử restore.

Bài tập P01.C01.L03.Q01

Một dịch vụ trả thành công ngay sau khi ghi tệp tạm và đổi tên. Sau mất điện, tên tệp mới biến mất. Nêu hai ranh giới độ bền cần kiểm và một điều chưa thể kết luận từ HTTP thành công.

Chương P01.C02 — Chương trình, tiến trình và ranh giới nhân

Lần theo một bản build đến hai tiến trình và kết quả từng nhánh.

Bài P01.C02.L01 — Ba dạng ứng dụng

Một dịch vụ có thể có mã nguồn đúng mà vẫn không phục vụ được yêu cầu. Tệp thực thi có thể tồn tại, nhưng chưa có tiến trình nào đang chạy; tiến trình có thể chạy, nhưng không mở được cổng; cổng có thể mở, nhưng tiến trình không đọc được tệp cấu hình. Để chẩn đoán những tình huống này, trước hết cần biết hệ điều hành đã biến chương trình thành một thực thể đang hoạt động như thế nào. Chương này xây dựng mô hình ấy trước khi nói đến container, điều phối hay giám sát.

Mã nguồn là văn bản mô tả hành vi. Tệp thực thi hoặc chương trình được thông dịch là vật liệu để khởi chạy. Tiến trình là một lần thực thi cụ thể: nó có định danh, không gian địa chỉ, trạng thái xử lý, thông tin định danh người dùng và tập tài nguyên đang mở. Một chương trình có thể được chạy nhiều lần, tạo ra nhiều tiến trình độc lập. Ngược lại, một tiến trình có thể thay chương trình đang thực thi mà vẫn giữ định danh tiến trình, khi dùng cơ chế `execve`.

Sự phân biệt này giải thích vì sao "build thành công" không đồng nghĩa "dịch vụ sống". Build xác nhận một số tính chất của artifact; nó không chứng minh hệ điều hành cấp đủ tài nguyên, địa chỉ mạng sẵn có hay cấu hình runtime hợp lệ. Đối với người vận hành, câu hỏi cần tách làm hai: artifact nào được đưa tới máy, và tiến trình nào đang thực thi artifact đó? Nếu hai câu trả lời khác nhau, một lần triển khai có thể đã thay tệp trên đĩa mà chưa thay phiên bản đang phục vụ.

CPU thực thi lệnh của tiến trình theo các khoảng thời gian; bộ nhớ chứa mã và dữ liệu cần cho lần chạy; nhân hệ điều hành quản lý tài nguyên và cung cấp các lời gọi hệ thống. Đó là mô hình khởi đầu, chưa phải toàn bộ chi tiết của Linux. Trong một tiến trình còn có thể có nhiều luồng thực thi cùng chia sẻ phần lớn không gian địa chỉ. Vì thế, số tiến trình không phải số công việc đồng thời, và mức dùng CPU không thể suy ra chỉ từ số PID.

Bài tập P01.C02.L01.Q01

Một bản triển khai thay tệp thực thi trên đĩa nhưng không khởi động lại dịch vụ. Giải thích vì sao không được kết luận tiến trình đang phục vụ đã chạy phiên bản mới. Nêu phép kiểm tra cần làm.

Bài P01.C02.L02 — Tạo và thay process

Trên Linux, `fork` tạo tiến trình con. Tiến trình cha nhận PID của con, còn con nhận số 0; nếu không tạo được con, chỉ cha nhận `-1` và mã lỗi. Nhờ vậy hai tiến trình đi theo hai nhánh xử lý khác nhau. Không nên hình dung rằng toàn bộ bộ nhớ vật lý bị sao chép ngay lập tức: triển khai Linux dùng các trang copy-on-write, tạo bản riêng khi cần ghi. Đây là chi tiết triển khai Linux, không phải định nghĩa phổ quát của mọi hệ điều hành.

Sau bước tạo con, một cách khởi chạy chương trình khác là gọi `execve`. Lời gọi này thay process image hiện tại bằng chương trình mới; nó không tạo thêm một tiến trình mới. Nếu thành công, luồng lệnh cũ không tiếp tục ngay sau lời gọi. Nếu lỗi, lời gọi trả về `-1`, đặt `errno` và chương trình cũ còn

cơ hội xử lý. Cặp cơ chế tạo con rồi thay chương trình giúp hiểu vì sao một shell có thể tiếp tục tồn tại sau khi khởi chạy lệnh khác, đồng thời vì sao tiến trình của lệnh có PID riêng.

Ví dụ chạy trên Linux bằng `python3 fork_exec.py`; thêm `fail` để cố ý chọn đường dẫn không tồn tại. PID do hệ điều hành cấp. Cha in trạng thái sau khi `waitpid` trả về.

```
"""Linux-only trace of fork and execve; run with optional argument 'fail'."""

import os
import sys


child = os.fork()
if child == 0:
    path = "/definitely/missing" if sys.argv[1:] == ["fail"] else "/bin/echo"
    os.write(2, f"child: fork returned {child}, PID {os.getpid()}\n".encode())
    try:
        os.execve(path, ["echo", "child: execve succeeded"], os.environ.copy())
    except OSError as exc:
        os.write(2, f"child: execve failed: {exc.strerror}\n".encode())
        os._exit(127)

print(f"parent: fork returned child PID {child}", flush=True)
_, status = os.waitpid(child, 0)
if os.WIFEXITED(status):
    print(f"parent: child exited with {os.WEXITSTATUS(status)}")
else:
    print("parent: child did not exit normally")
```

Ở cả hai lần chạy, `fork` thành công tạo hai PID. Trong cha, `child` là PID dương của con; trong con, `child` bằng 0. Nhánh thành công thay chương trình của con bằng `/bin/echo`, giữ nguyên PID của con và không vào khối `except`. Nhánh lỗi khiến lời gọi Python ném `OSError`; con in lỗi rồi thoát với mã 127 theo quy ước riêng của ví dụ, còn cha nhận mã ấy qua `waitpid`. Ở giao diện lời gọi hệ thống C, lỗi tương ứng là giá trị `-1` và `errno`; Python chuyển nó thành ngoại lệ. Nếu `fork` thất bại, Python cũng ném `OSError` và không tạo con để chờ. Mã 127 ở đây là lựa chọn của chương trình, không phải mã lỗi cố định do `execve` áp đặt.

Ví dụ chỉ dành cho tiến trình Python một luồng. Từ Python 3.12, `os.fork()` có thể phát `DeprecationWarning` nếu phát hiện tiến trình có nhiều luồng: các khóa do luồng khác giữ có thể khiến tiến trình con kẹt trước khi thay chương trình. Không dùng đoạn mã này làm mẫu khởi chạy trong ứng dụng Python đa luồng.

Mô hình này cũng giải thích một lỗi vận hành phổ biến: chương trình con kế thừa tài nguyên mà người gọi quên đóng. Theo mặc định, một file descriptor có thể còn mở qua `execve`; cờ close-on-exec thay đổi hành vi đó. Nếu vô tình giữ đầu ghi của một pipe ở tiến trình khác, đầu đọc có thể chưa nhìn thấy kết thúc luồng dù bên tạo dữ liệu đã xong. Kết luận này là hệ quả của vòng đời tài nguyên, không phải một quy tắc riêng của Bash.

Bài tập P01.C02.L02.Q01

`fork` thành công, `execve` thất bại. Trong mô hình của chương này, có bao nhiêu tiến trình sau `fork`, và tiến trình nào có thể nhận biết lỗi `execve`? Vì sao không được coi `execve` là thao tác tạo tiến trình?

Bài P01.C02.L03 — Descriptor và identity

Khi tiến trình gọi `open`, nó nhận một số nguyên nhỏ không âm: file descriptor. Số này là chỉ số trong bảng descriptor của chính tiến trình. Descriptor không phải tên tệp, cũng không phải bản thân nội dung tệp. Nó tham chiếu tới một open file description do nhân quản lý, nơi giữ vị trí đọc/ghi hiện tại và các cờ trạng thái. Hai descriptor có thể cùng tham chiếu một open file description; khi đó chúng chia sẻ vị trí đọc/ghi.

Sự khác biệt có ý nghĩa khi đọc log. Nếu một tiến trình đã mở tệp log và người vận hành đổi tên hoặc xóa đường dẫn, descriptor đang mở vẫn có thể trỏ tới đối tượng cũ. Tạo một tệp mới ở cùng tên đường dẫn không buộc tiến trình chuyển đầu ghi sang tệp mới. Phần thực hành về log rotation ở các chương sau sẽ dựa trên quan hệ này để giải thích vì sao có trường hợp dung lượng đĩa chưa được giải phóng và vì sao cần yêu cầu dịch vụ mở lại log.

Ba descriptor quen thuộc khi khởi chạy chương trình là đầu vào chuẩn, đầu ra chuẩn và đầu lỗi chuẩn. Chuyển hướng trong shell thay đổi các luồng mà tiến trình nhận được; nó không sửa ý nghĩa của chương trình. Khi một dịch vụ ghi thông báo lỗi vào đầu ra chuẩn, người thu thập log vẫn có thể lấy được dữ liệu, nhưng việc phân loại mức độ lỗi trở nên khó hơn. Do vậy, quy ước đầu ra là một phần của hợp đồng vận hành.

Chương này không yêu cầu thuộc lòng mọi lời gọi hệ thống. Điều cần giữ là ba câu hỏi: thứ gì tồn tại trên đĩa, tiến trình nào đang chạy, và tiến trình ấy đang nắm những tài nguyên nào? Phần sau sẽ dùng cùng ba câu hỏi khi đi từ một tiến trình trên máy tới container, rồi tới workload trong cụm.

PHẦN 02 — Linux và vận hành máy chủ

Nhân cho phép, giới hạn và duy trì một dịch vụ như thế nào?

Lời gọi hệ thống đi qua định danh, namespace tên, quyền, tài nguyên rồi tới supervisor; mỗi ranh giới có bằng chứng riêng.

Chương P02.C01 — Syscall, đường dẫn và hệ thống tệp

Giải thích tại sao cùng tên file có thể trỏ đối tượng khác.

Bài P02.C01.L01 — Vào nhân

Một dịch vụ có thể chạy dưới đúng người dùng mà vẫn không đọc được cấu hình. Một tệp có bit đọc nhưng đường dẫn tới nó lại không thể đi qua. Một lệnh `systemctl start` có thể trả thành công trong khi ứng dụng chưa sẵn sàng nhận yêu cầu. Những hiện tượng này thuộc các lớp khác nhau: nhân kiểm tra lời gọi hệ thống và quyền truy cập, hệ thống tệp ánh xạ tên sang đối tượng, còn systemd điều hành vòng đời tiến trình. Chương này nối các lớp đó qua một trường hợp vận hành có thể tự kiểm chứng.

Tiến trình thực thi phần lớn mã của nó ở user space. Khi cần mở tệp, tạo socket hay đổi quyền, nó gọi một giao diện mà cuối cùng đi vào nhân. Ví dụ `openat` nhận đường dẫn, cờ mở và ngữ cảnh của tiến trình; nhân phân giải đường dẫn, kiểm tra quyền, rồi trả về file descriptor hoặc lỗi. Một hàm thư viện hay một lệnh shell có thể gọi thêm nhiều bước trước và sau lời gọi này. Vì vậy, thông báo "Permission denied" của ứng dụng cần được lần tới thao tác thực sự thất bại; nó không tự cho biết bit quyền nào sai.

Nhân là nơi thực thi các quy tắc về tiến trình, bộ nhớ, đối tượng tệp và thiết bị. systemd là một tiến trình ở user space dùng các giao diện ấy để khởi động và giám sát dịch vụ. `systemctl` lại là công cụ yêu cầu systemd thực hiện thao tác. Ba tên này không thay thế nhau: nhân không đọc mục `Restart=` trong unit, còn systemd không bỏ qua được phép kiểm tra của nhân chỉ vì unit đã nạp thành công.

Sau một lời gọi thất bại, chương trình có thể nhận `EACCES` khi thiếu quyền tìm kiếm hoặc truy cập, `ENOENT` khi thành phần đường dẫn không tồn tại, hay `ENOTDIR` khi một thành phần trung gian không phải thư mục. Đó là mã lỗi ở ranh giới lời gọi; ứng dụng có thể gộp hoặc đổi thông báo khi hiển thị. Trước khi sửa quyền, phải xác định đường dẫn thực, người dùng hiệu lực và thao tác đọc, ghi hay thực thi mà tiến trình đã yêu cầu.

Bài P02.C01.L02 — Phân giải tên

Đường dẫn là chuỗi các thành phần được tra cứu từ một điểm bắt đầu. Mỗi thư mục tham gia phép tra cứu tên; tệp cuối cùng có metadata như loại, quyền và số inode. Tên không đồng nhất với inode: đổi tên thay quan hệ trong thư mục, còn một descriptor đang mở vẫn tham chiếu đối tượng đã mở như Chương 1 đã giải thích. Một hard link khác có thể trỏ tới cùng inode trên cùng hệ thống tệp, còn symbolic link chứa một đường dẫn cần được phân giải tiếp. Vì thế, xem riêng `ls -l` của tệp đích có thể bỏ sót vấn đề ở thư mục cha hoặc một liên kết trung gian.

Bài tập P02.C01.L02.Q01

Một chương trình vẫn ghi vào tệp log đã bị đổi tên; tệp mới cùng tên vẫn rỗng. Hãy dùng quan hệ giữa đường dẫn, descriptor và open file description để giải thích. Đề xuất một cách xử lý an toàn.

Bài P02.C01.L03 — Ghi và bền vững

Đọc một tệp thường đi qua page cache của Linux. Dữ liệu vừa đọc có thể được giữ trong RAM để lần truy cập sau không phải lấy lại từ thiết bị. Cache vì thế không đơn giản là "RAM bị chiếm vô ích"; nó là một cách đổi bộ nhớ lấy độ trễ I/O thấp hơn. Khi thiếu bộ nhớ, nhân có thể thu hồi những trang phù hợp. Tuy nhiên, khả năng thu hồi và chi phí thu hồi khác nhau giữa trang sạch của tệp, trang đã sửa nhưng chưa ghi xuống thiết bị, và vùng nhớ ẩn danh.

Ghi vào tệp cũng có thể hoàn tất ở tầng lời gọi ứng dụng trước khi mọi thay đổi đã nằm trên thiết bị bền vững. Trang đã sửa trong cache được đánh dấu dirty và sau đó cần được đồng bộ. Vì vậy, "lời gọi ghi trả về" và "dữ liệu sống sót qua mất điện" là hai hợp đồng khác nhau. Trên Linux, `fsync` yêu cầu đồng bộ dữ liệu cùng metadata của tệp tới thiết bị theo hợp đồng của lời gọi; khi tạo hoặc đổi tên tệp, việc đồng bộ riêng descriptor của tệp chưa nhất thiết bảo đảm mục thư mục chứa tên đó đã bền vững. Ứng dụng có dữ liệu quan trọng phải xác định chính xác mình cần bảo đảm điều gì, thay vì gọi chung là "đã lưu".

Một thao tác sao lưu an toàn vì thế không chỉ là sao chép bytes rồi thấy lệnh trả 0. Cần kiểm tra nhất quán dữ liệu đầu vào, kết quả ghi, ranh giới bền vững cần thiết và khả năng phục hồi. Hệ thống tệp, cơ sở dữ liệu và object storage cung cấp các hợp đồng khác nhau; chương về lưu trữ sẽ đi sâu vào từng loại. Ở đây điều cần nhớ là I/O có nhiều tầng và độ trễ ở từng tầng không hiện ra từ một phép đo CPU.

Bài tập P02.C01.L03.Q01

Dịch vụ ghi một tệp cấu hình mới rồi đổi tên tệp để thay bản cũ. Việc lời gọi ghi trả về thành công đã đủ chứng minh tên tệp mới sống sót sau mất điện chưa? Giải thích ranh giới cần kiểm tra.

Chương P02.C02 — Danh tính và quyền Linux

Chẩn đoán một lần mở file bị từ chối qua mọi thư mục cha.

Bài P02.C02.L01 — Danh tính tiến trình

Một tiến trình mang thông tin định danh và chịu kiểm tra quyền khi truy cập tài nguyên. Có quyền đọc một đường dẫn không đồng nghĩa có quyền thực thi tệp, và có quyền thực thi một tệp không đồng nghĩa được đọc mọi tệp mà chương trình sẽ mở sau đó. Nhìn từ bên ngoài, cả hai tình huống có thể hiện thành "ứng dụng không khởi động"; bên trong, điểm lỗi nằm ở những bước khác nhau. Về sau, mô hình này sẽ nối với quyền Linux, IAM và quyền của tài khoản dịch vụ trong Kubernetes.

Tiến trình còn hoạt động trong các giới hạn tài nguyên. Một lần tạo tiến trình con có thể thất bại khi đạt giới hạn số tác vụ; `fork` báo lỗi thay vì bảo đảm thành công. Vì vậy, một bộ điều phối không thể chữa một chương trình tạo tiến trình vô hạn chỉ bằng cách tự khởi động lại nó. Tự phục hồi là chính sách phản ứng với lỗi; ngăn lỗi lặp lại đòi sửa nguyên nhân hoặc đặt giới hạn thích hợp.

Lưu ý rằng quyền truy cập và khả năng nhìn thấy tài nguyên là hai câu hỏi khác nhau. Một namespace có thể cho tiến trình một góc nhìn riêng về một loại tài nguyên của hệ thống. Nhưng góc nhìn riêng không tự chứng minh rằng tiến trình an toàn hay không thể tác động đến thứ bên ngoài ranh giới đó. Ở phần container, ta sẽ kết hợp namespace với giới hạn tài nguyên, quyền và chính sách bảo mật thay vì xem "cách ly" là một thuộc tính nhị phân.

Bài P02.C02.L02 — Bước đi qua thư mục

Với chế độ quyền Linux thông thường, mỗi nhóm bit đọc, ghi, thực thi áp cho chủ sở hữu, nhóm và người khác. Trên tệp thường, bit `x` cho phép thực thi theo các điều kiện của định dạng tệp; trên thư mục, bit `x` là quyền tìm kiếm/đi qua thành phần tên, không phải "chạy thư mục". Bit `r` của thư mục cho phép liệt kê tên; biết sẵn một tên và tra cứu nó cần quyền tìm kiếm. Khi tạo hoặc xóa tên trong thư mục, quyền ở thư mục có vai trò riêng, không thể suy chỉ từ bit ghi trên tệp. ACL, mount option, namespace, chính sách bảo mật bổ sung và quyền đặc biệt có thể làm kết quả khác mô hình ba bộ bit cơ bản; mô hình đó là điểm bắt đầu để khoanh vùng, không phải toàn bộ chính sách.

Xét cây `/srv/atlas/config/app.toml`: `app.toml` là `0640`, thuộc `root:atlas`; tiến trình chạy với nhóm `atlas`. Tệp có vẻ đọc được qua bit nhóm, nhưng nếu `/srv/atlas` là `0700 root:root`, tiến trình không thể đi qua thư mục này và thao tác mở có thể trả `EACCES`. Cách sửa đúng phụ thuộc hợp đồng truy cập: có thể cấp quyền tìm kiếm tối thiểu trên thư mục cho nhóm phù hợp, hoặc đặt cấu hình ở một cây do dịch vụ được phép đi qua. Đổi `app.toml` thành `0666` không sửa được đường đi, lại mở rộng người có thể đọc và sửa bí mật trong tệp. Không dùng `chmod -R 777` như cách chẩn đoán.

Để kiểm chứng trên máy Linux thử nghiệm do mình kiểm soát, ghi nhận danh tính tiến trình bằng `id`, xem từng thành phần bằng `namei -l /srv/atlas/config/app.toml`, rồi thử đọc dưới đúng tài khoản dịch vụ, chẳng hạn `sudo -u atlas cat /srv/atlas/config/app.toml`. Trước khi thay quyền, xác nhận các đường dẫn và tài khoản thực sự tồn tại; các tên trong ví dụ là giả

định. `namei` giúp nhìn chuỗi quyền nhưng kết quả đọc cuối cùng mới kiểm tra đúng tổ hợp tài khoản, nhóm, ACL và mount đang có hiệu lực. Nếu unit cấp thêm sandbox hoặc thay root directory, phải kiểm tra từ ngữ cảnh của dịch vụ thay vì chỉ dùng một shell bên ngoài.

Bài tập P02.C02.L02.Q01

Tệp cấu hình có mode `0640 root:atlas`, tiến trình thuộc nhóm `atlas`, nhưng thư mục cha `/srv/atlas` có mode `0700 root:root`. Vì sao thao tác đọc có thể bị từ chối? Nêu một thay đổi tối thiểu cần kiểm tra và một thay đổi không nên dùng.

Bài P02.C02.L03 — Điều tra và sửa

Ở bài trước, tệp cấu hình đọc được theo bit nhóm nhưng tiến trình không đi qua được thư mục cha. Phép sửa cần xuất phát từ lời gọi bị từ chối, chứ không xuất phát từ một ảnh chụp `ls -l` của tệp đích. Trước hết ghi đường dẫn mà ứng dụng thật sự mở, loại thao tác và mã lỗi; một đường dẫn tương đối còn phụ thuộc thư mục làm việc hoặc descriptor mà `openat` dùng. Sau đó xác định UID dùng để kiểm quyền tệp, GID chính và các nhóm phụ của tiến trình ở thời điểm lỗi. Tài khoản người vận hành trong terminal không tự đại diện cho tiến trình do systemd tạo.

Kernel lần lượt tra từng thành phần đường dẫn. Thiếu quyền tìm kiếm trên một thư mục trung gian có thể trả `EACCES` trước khi kernel xét bit đọc của tệp cuối. Một symbolic link có thể chuyển phép tra sang cây khác; mount namespace hoặc root directory của dịch vụ cũng có thể làm cùng chuỗi ký tự trỏ tới đối tượng khác. Vì vậy, phép chẩn đoán phải đọc cả cây, owner/group/ACL và ngữ cảnh chạy. Chỉ khi xác định được điểm chặn mới sửa một điều kiện nhỏ nhất, rồi thử lại chính thao tác đó dưới cùng danh tính. Sửa theo giả thuyết mà không thử hậu điều kiện có thể vừa để lỗi còn nguyên vừa mở rộng quyền ngoài ý muốn.

Tình huống giả định: `/srv/atlas` là `0700 root:root`, `/srv/atlas/config` là `0750 root:atlas`, còn `app.toml` là `0640 root:atlas`. Tiến trình thuộc nhóm `atlas` vẫn dừng ở `/srv/atlas`. Nếu chính sách cho phép nhóm này đi qua nhưng không liệt kê cây gốc, có thể cân nhắc cấp quyền tìm kiếm hẹp trên đúng thư mục đó hoặc chuyển tệp đến cây được quản lý cho dịch vụ; lựa chọn cuối cùng còn phụ thuộc ACL và chính sách bảo mật của máy. Chuyển cả cây thành `0777` hoặc chạy dịch vụ với quyền root phá ranh giới least privilege mà không chứng minh nguyên nhân. Nếu phép thử đọc thành công nhưng ứng dụng vẫn báo lỗi, quay lại kiểm đường dẫn, tài khoản và thời điểm thực tế thay vì tiếp tục tăng quyền.

Một bằng chứng đủ tốt ghi trạng thái trước, thay đổi được duyệt và trạng thái sau: lỗi `EACCES` ở phép mở nào, danh tính nào, quyền tra cứu ở thành phần nào, rồi cùng phép mở đã thành công hay chưa. Nó không khẳng định mọi yêu cầu ứng dụng đã khỏe; kiểm tra readiness và request ở bài vận hành host vẫn cần riêng.

Bài tập P02.C02.L03.Q01

Dịch vụ thuộc nhóm atlas vẫn nhận EACCES với cây 0700 root:root /srv/atlas, 0750 root:atlas /srv/atlas/config, 0640 root:atlas app.toml. Chỉ ra điểm chặn, cách kiểm lại và vì sao chmod 777 cho tệp không giúp.

Chương P02.C03 — Tài nguyên và chẩn đoán hiệu năng Linux

Phân biệt CPU runnable/quota, memory reclaim và I/O wait bằng counter.

Bài P02.C03.L01 — CPU và scheduler

Một yêu cầu HTTP mất hai giây không có nghĩa CPU đã làm việc trong hai giây. Trong khoảng ấy, tiến trình có thể đợi lịch chạy, đợi ổ lưu trữ, đợi mạng hoặc chờ khóa do chính chương trình giữ. Nếu không phân biệt những khoảng thời gian này, ta rất dễ tăng số CPU cho một dịch vụ đang nghẽn ở dữ liệu, hoặc tăng bộ nhớ cho một tiến trình đang bị giới hạn thời gian CPU. Chương này xây dựng một cách phân rã độ trễ từ máy tính trước khi dùng công cụ giám sát.

CPU chỉ thực hiện lệnh của một luồng tại một thời điểm trên mỗi đơn vị xử lý logic. Khi nhiều luồng cùng sẵn sàng, bộ lập lịch chọn luồng nào được chạy tiếp. Một luồng có thể đang chạy, sẵn sàng nhưng chưa được chọn, hoặc không sẵn sàng vì đang chờ một sự kiện. Trạng thái "chờ" không đồng nhất: chờ dữ liệu mạng khác với chờ khóa, và cả hai khác với việc sẵn sàng nhưng bị giới hạn quota CPU. Những lý do này đòi cách sửa khác nhau.

Lập lịch công bằng là một mục tiêu chính sách, không phải lời hứa mỗi tiến trình luôn nhận đúng một phần CPU cố định. Chính sách, độ ưu tiên, số tác vụ sẵn sàng, ràng buộc nhóm và phiên bản nhân đều ảnh hưởng kết quả. Tài liệu Linux mô tả quá trình chuyển từ CFS sang EEVDF ở các nhân mới; cuốn sách này giữ mô hình chung "tác vụ sẵn sàng được chọn theo chính sách" ở phần nền tảng, và chỉ bàn thuật toán cụ thể khi có phiên bản nhân làm mốc.

Để phân biệt, hãy đặt câu hỏi theo chuỗi. Công việc có sẵn sàng chạy nhưng không nhận CPU không? Bộ nhớ nào đang tăng, và đó là cache có thể thu hồi hay dữ liệu hoạt động? Có thao tác I/O nào bị chặn? Giới hạn nào áp cho tiến trình, container hoặc cả máy? Nếu quan sát cho thấy CPU bị giới hạn theo quota, tăng số replica có thể chia tải, nhưng cũng có thể làm áp lực lên phụ thuộc chung. Nếu quan sát cho thấy ứng dụng chờ cơ sở dữ liệu, thêm CPU cho nó không loại bỏ thời gian chờ ấy.

Những câu hỏi trên là mô hình phân tích, không phải một runbook tự động. Chúng chuẩn bị cho các chương sau về đo lường, HPA và SLO. Trước khi dùng một metric để tự thay đổi số bản sao, phải biết metric đại diện cho tài nguyên nào, cửa sổ thời gian nào và điều gì xảy ra khi phép đo bị thiếu. Một bộ điều khiển ra quyết định từ tín hiệu sai vẫn có thể hoạt động đúng theo thuật toán của nó nhưng tạo kết quả vận hành xấu.

Bài P02.C03.L02 — Memory

Mỗi tiến trình nhìn thấy một không gian địa chỉ ảo. Chương trình dùng các địa chỉ trong không gian ấy; phần cứng và nhân phối hợp ánh xạ chúng tới trang bộ nhớ vật lý khi cần. Hai tiến trình có thể dùng cùng một giá trị địa chỉ ảo mà không mặc nhiên đọc cùng dữ liệu. Điều này tạo ranh giới bảo vệ và cho phép quản lý bộ nhớ linh hoạt hơn một sơ đồ "biến nằm ở ô RAM cố định".

Một vùng địa chỉ được dành cho tiến trình chưa nhất thiết đã có một trang RAM riêng. Chẳng hạn, vùng ánh xạ ẩn danh có thể được tạo trước, còn trang vật lý được cấp khi truy cập hoặc ghi phù hợp.

Khi một địa chỉ chưa có ánh xạ vật lý cần thiết, phần cứng báo page fault để nhân xử lý. Page fault không tự đồng nghĩa lỗi chương trình: nhiều fault là một phần của cấp phát theo nhu cầu. Lỗi thật xuất hiện khi truy cập không hợp lệ hoặc không thể đáp ứng điều kiện ánh xạ.

Vì vậy, một con số "bộ nhớ ảo" lớn không đủ để kết luận sắp hết RAM. Ngược lại, nhìn một con số bộ nhớ của riêng tiến trình cũng chưa cho biết áp lực bộ nhớ toàn máy: trang có thể được chia sẻ, nằm trong cache, hoặc chịu quy tắc tính của nhóm tài nguyên. Khi chẩn đoán, cần xác định phép đo đang đếm địa chỉ, trang cư trú, trang dùng chung hay bộ nhớ tính cho một cgroup. Nhầm tên đại lượng dẫn tới tăng giới hạn sai chỗ.

Bài tập P02.C03.L02.Q01

Một tiến trình có không gian địa chỉ ảo lớn nhưng bộ nhớ cư trú thấp. Vì sao chưa thể kết luận nó đang chiếm gần hết RAM? Cần phân biệt ít nhất hai đại lượng nào?

Bài P02.C03.L03 — I/O và độ bền

Một request ghi dữ liệu có thể trả nhanh khi kernel còn giữ trang dirty, trong khi writeback xuống thiết bị diễn ra sau đó. Cũng có thể request đọc bị chậm vì trang chưa ở cache, hoặc một luồng bị chặn bởi hàng đợi thiết bị. Cần phân biệt ba đại lượng: byte ứng dụng muốn đọc/ghi, byte kernel thực sự chuyển tới thiết bị và thời gian một tác vụ phải chờ. Chúng không nhất thiết tăng cùng lúc. `fsync` đặt thêm ranh giới: nó yêu cầu đồng bộ dữ liệu và metadata cần thiết của tệp, rồi trả lỗi nếu không thực hiện được; khi tạo tên mới còn phải xét thư mục chứa tên ấy.

Xét dữ liệu tính toán giả định trong cửa sổ mười giây, không phải output của máy đã chạy. Số byte ghi của ứng dụng tăng 120 MiB, số byte ghi của thiết bị tăng 40 MiB, lượng trang dirty tăng 80 MiB, và p95 latency của thao tác ghi tăng từ 6 lên 90 ms. Chênh lệch 80 MiB phù hợp giả thuyết writeback chưa theo kịp, nhưng không chứng minh tất cả 80 MiB thuộc ứng dụng này: máy còn tác vụ khác, phép đo có thể ở các thời điểm khác và các counter không cùng ranh giới kế toán. Nếu `fsync` của cùng request tăng thời gian chờ, giả thuyết thiết bị hoặc filesystem cần kiểm tiếp; nếu không có `fsync`, latency HTTP tăng có thể nằm ở khóa, CPU hoặc phụ thuộc mạng. Chỉ nhìn CPU idle không phân biệt được các đường đó.

Một quy trình đo có sức phân biệt lấy delta counter cùng cửa sổ, chỉ rõ process, cgroup và thiết bị, rồi ghép request ID với thời điểm gọi I/O. Trên Linux có thể xem `Dirty` và `Writeback` trong `/proc/meminfo` để thấy trạng thái toàn máy, nhưng đây không phải số byte của riêng request. Counter block I/O và pressure stall cho biết áp lực theo ranh giới đo của chúng; chúng không tự chỉ ra tệp hay câu lệnh SQL thủ phạm. Trong bài thử cục bộ, tạo một tệp riêng trên filesystem tạm có đủ dung lượng, đo trước và sau, đồng bộ tệp rồi đọc lại; không dùng môi trường lab hoặc dữ liệu thật để ép I/O.

Kết luận chẩn đoán phải ghi cả phép phủ định. Nếu dữ liệu đã đọc từ cache, không quy độ trễ đọc cho ổ chỉ vì ổ đang bận. Nếu `fsync` thành công, vẫn phải xét thư mục cho tên mới và đặc tính thiết bị trước khi tuyên bố phục hồi sau mất điện. Bài đầu Phần I đã đặt ranh giới bền vững; ở đây các counter giúp tìm nơi nào trong đường đi ấy cần kiểm chứng thực nghiệm.

Bài tập P02.C03.L03.Q01

Trong 10 giây, ứng dụng báo ghi 120 MiB, thiết bị báo ghi 40 MiB, Dirty tăng 80 MiB và latency tăng. Nêu một giả thuyết phù hợp, một phép đo bổ sung và điều chưa thể quy cho request.

Bài P02.C03.L04 — Điều tra đa tín hiệu

Xét một dịch vụ đọc tệp trong một cgroup v2. Bảng sau là dữ liệu giả định để tập tính và phản biện, không phải đầu ra đo từ máy hay bằng chứng cho một sự cố có thật. Hai cửa sổ đều dài 60 giây, có cùng số yêu cầu và cùng giới hạn tài nguyên. Các giá trị `usage_usec`, `throttled_usec` và `io.pressure some total` là mức tăng của counter tích lũy giữa đầu và cuối mỗi cửa sổ; `memory.current` là giá trị ở cuối cửa sổ. Người đo phải lấy đúng cgroup chứa workload, cùng ranh giới thời gian với số liệu yêu cầu, và kiểm tra các tệp này có sẵn trên nhân đang chạy.

Quan sát trong 60 giây	Bình thường	Độ trễ tăng
p95 yêu cầu	120 ms	870 ms
`cpu.stat usage_usec` tăng	18.000.000 µs	19.200.000 µs
`cpu.stat nr_throttled` tăng	0	0
`memory.current` cuối kỳ	310 MiB	315 MiB
`memory.events high / oom` tăng	0 / 0	0 / 0
`io.pressure some total` tăng	1.200.000 µs	18.000.000 µs

Chia CPU time cho 60 giây cho ta mức dùng trung bình 0,30 rồi 0,32 lõi, không phải 30% rồi 32% độ trễ của từng yêu cầu. `nr_throttled` không tăng nên trong hai cửa sổ này chưa có dấu hiệu cgroup đó bị giới hạn bởi quota CPU; việc luồng phải đợi lịch chạy vì tác vụ khác vẫn có thể xảy ra. Bộ nhớ cuối kỳ tăng 5 MiB và không có sự kiện `high` hay `oom`, nên không có dấu vết vượt hai ngưỡng ấy trong counter đã xem; điều đó không loại trừ thu hồi bộ nhớ toàn máy, page fault hoặc một ngưỡng ở cgroup tổ tiên. Muốn xét áp lực bộ nhớ, đọc thêm `memory.stat`, `memory.pressure` và giới hạn thực tế theo cùng cửa sổ.

Với I/O, `some total` tăng 1,2 giây rồi 18 giây: lần lượt khoảng 2% và 30% của mỗi cửa sổ có ít nhất một tác vụ trong nhóm bị nghẽn I/O. Đây là dấu vết mạnh hơn cho giả thuyết chờ I/O trong cửa sổ chậm, nhưng nó không nói yêu cầu nào chờ, chờ thiết bị nào, hay I/O là nguyên nhân duy nhất của p95 tăng. Bước kế tiếp là ghép thời điểm yêu cầu chậm với thời gian thao tác đọc, bộ đếm bytes/thao tác và tình trạng thiết bị hoặc phụ thuộc tương ứng; cũng phải xét khóa trong ứng dụng và mạng. Nếu đó là đọc từ page cache, số bytes ở thiết bị có thể không phản ánh số lần ứng dụng đọc. Kết luận đúng ở giai đoạn này là ưu tiên kiểm tra đường I/O, chưa phải "ổ đĩa hỏng" hoặc "thêm CPU sẽ hết chậm".

Bài tập P02.C03.L04.Q01

Dùng số liệu giả định ở mục 2.5: tính mức dùng CPU trung bình và tỷ phần thời gian `io.pressure some` của hai cửa sổ; nêu dấu vết cho hoặc chống lại ba giả thuyết quota CPU, giới hạn bộ nhớ và chờ I/O. Chỉ ra một điều bảng chưa thể kết luận và phép đo kế tiếp.

Chương P02.C04 — systemd và vòng đời dịch vụ

Tách start thành công khỏi listener, readiness và dữ liệu đúng.

Bài P02.C04.L01 — Unit graph

Dịch vụ là một hợp đồng vận hành đặt lên tiến trình: khởi động khi nào, cấu hình từ đâu, báo trạng thái ra sao, dừng theo tín hiệu nào, có khởi động lại không và dữ liệu bền vững nằm ở đâu. Không phải mọi tiến trình đều là dịch vụ. Một công cụ chạy xong rồi thoát có vòng đời khác một HTTP server tồn tại lâu và tiếp nhận yêu cầu mới.

Khi một tiến trình thoát, hệ điều hành có thể ghi nhận mã kết thúc. Mã này cho biết chương trình kết thúc theo quy ước nào; tự nó chưa cho biết nguyên nhân gốc. Cùng một mã khác không có bảo đảm diễn giải giống nhau giữa mọi chương trình. Một hệ thống giám sát tốt cần xem thêm thời điểm, stderr, tình trạng tài nguyên và sự kiện từ bộ quản lý dịch vụ. Đọc mỗi trạng thái "đã dừng" rồi suy ra lỗi mạng, lỗi quyền hay thiếu bộ nhớ là bước nhảy thiếu chứng cứ.

Một dịch vụ cũng có thể còn tiến trình nhưng không còn khả năng làm việc có ích. Vòng lặp bị kẹt, hàng đợi đầy hoặc phụ thuộc phía sau không phản hồi đều có thể khiến PID vẫn tồn tại. Vì vậy, giám sát sự sống của tiến trình và kiểm tra khả năng phục vụ là hai phép đo khác nhau. Ta sẽ gặp lại khác biệt này khi phân tích healthcheck của Docker, probe của Kubernetes và SLI của SRE.

```
[Unit]
Description=Atlas example service
After=network.target

[Service]
Type=exec
User=atlas
Group=atlas
WorkingDirectory=/opt/atlas
ExecStart=/opt/atlas/bin/atlas --config /srv/atlas/config/app.toml
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Dòng `After=network.target` trong unit chỉ đặt thứ tự nếu cả hai unit cùng được đưa vào giao dịch khởi động; nó không tự kéo `network.target` vào, cũng không xác nhận máy đã có địa chỉ IP. `Wants=` kéo một unit khác vào với quan hệ phụ thuộc yếu; `Requires=` mạnh hơn và có thể làm khởi động thất bại khi unit được yêu cầu thất bại, nhưng cả hai không tự đặt thứ tự nếu thiếu `After=`. `network.target` là mốc thụ động của ngăn xếp mạng, không phải tín hiệu mạng đã cấu hình đầy đủ. Dịch vụ thật sự cần mạng được cấu hình trước khi bắt đầu có thể dùng `Wants=network-online.target` cùng `After=network-online.target`; ý nghĩa "online" do dịch vụ quản lý mạng xác định và vẫn không bảo đảm máy ở xa hay ứng dụng của nó sẵn sàng. Với dịch vụ chạy lâu dài, còn phải thiết kế cách chịu mất mạng và kết nối lại sau khởi động, thay vì coi thứ tự boot là phép kiểm tra sức khỏe.

Bài P02.C04.L02 — Process lifecycle

Một unit service mô tả chương trình chính, danh tính chạy, thứ tự phụ thuộc và chính sách khi nó kết thúc. Ví dụ minh họa sau dùng đường dẫn giả định; trước khi cài phải tạo tài khoản, triển khai chương trình và cấu hình đúng trên máy đích. `Type=exec` khiến thao tác khởi động chờ tới khi thực thi được binary; nó bắt được trường hợp executable hoặc user không tồn tại tốt hơn `Type=simple`. Nó vẫn không chứng minh mã ứng dụng đã đọc xong cấu hình, mở socket hoặc sẵn sàng phục vụ.

Trong ví dụ, systemd tạo tiến trình với `User=atlas` và `Group=atlas`, đặt thư mục làm việc ở `/opt/atlas`, rồi thực thi chương trình được chỉ ra ở `ExecStart`. Thư mục làm việc tách khỏi cây cấu hình để lỗi quyền ở `/srv/atlas` xuất hiện khi ứng dụng gọi `openat`, sau bước khởi chạy. Khi ấy nhân vẫn kiểm tra đường dẫn và quyền theo ngữ cảnh đã thiết lập. `Restart=on-failure` yêu cầu thử chạy lại khi tiến trình kết thúc theo điều kiện lỗi được systemd định nghĩa; nó không chữa lỗi quyền cố định. `RestartSec=5s` chỉ tạo khoảng chờ giữa các lần thử và giới hạn tần suất khởi động vẫn có thể chặn vòng lặp lỗi.

Đường lỗi có thể lần theo như sau. Nếu `/opt/atlas/bin/atlas` không tồn tại, bước thực thi thất bại và `Type=exec` báo lỗi khởi động. Nếu binary chạy được nhưng không đi qua `/srv/atlas`, chương trình nhận lỗi khi mở cấu hình và tự thoát với mã lỗi; systemd ghi nhận tiến trình chính đã thoát, áp chính sách restart, rồi có thể chạm giới hạn khởi động. Nếu binary còn chạy nhưng kẹt ở phụ thuộc khác, trạng thái `active` chỉ nói tiến trình được quản lý còn ở trạng thái ấy; nó không phải healthcheck nghiệp vụ. Mỗi đường lỗi cần xem `systemctl status atlas.service`, `journalctl -u atlas.service` và phép thử khả năng phục vụ của chính ứng dụng. Không lấy riêng trạng thái `active` để tuyên bố đã triển khai thành công.

Khi yêu cầu dừng, systemd gửi tín hiệu theo cấu hình dừng và chờ tiến trình kết thúc trong thời hạn quy định; nếu tiến trình không dừng, chính sách có thể dẫn tới tín hiệu cưỡng bức. Ứng dụng phải xử lý tín hiệu phù hợp, ngừng nhận công việc mới, hoàn tất hoặc hủy công việc đang làm theo hợp đồng, đóng tài nguyên và trả mã kết thúc rõ ràng. `Restart=` mô tả phản ứng sau khi tiến trình mất; nó không bảo đảm dữ liệu được ghi bền vững hoặc yêu cầu đang xử lý được giữ lại. Muốn có ranh giới an toàn, phải thiết kế riêng giao thức dừng và kiểm tra nó dưới tải.

Bài tập P02.C04.L02.Q01

Với `Type=exec`, `systemctl start` thành công. Điều này xác nhận được bước nào và chưa xác nhận được bước nào của một HTTP service? Nêu hai quan sát tiếp theo.

Bài P02.C04.L03 — Quan sát dịch vụ

Giả sử sau triển khai, trình duyệt báo không kết nối được tới dịch vụ. Cách suy nghĩ theo lớp bắt đầu từ artifact và tiến trình: bản build đã đến máy chưa; tiến trình có được tạo không; nó còn tồn tại hay đã thoát? Nếu đã thoát, cần xem mã kết thúc và thông báo lỗi. Nếu còn tồn tại, xem nó đã tạo socket lắng nghe đúng địa chỉ và cổng chưa. Sau đó mới xem đường đi mạng và phụ thuộc ở xa. Trật tự này không bảo đảm tìm ra mọi lỗi, nhưng ngăn việc sửa DNS khi chương trình thực ra chưa mở cổng.

Khi kiểm tra, phải phân biệt quan sát với kết luận. “Không thấy PID trong danh sách” là quan sát tại thời điểm đo; “build lỗi” chỉ là một giả thuyết. “Có PID” cũng chưa chứng minh dịch vụ khỏe. Một hồ sơ sự cố tốt ghi mốc thời gian, lệnh hoặc nguồn quan sát, artifact đang chạy, cấu hình có hiệu lực và thay đổi vừa xảy ra. Nhờ thế, người khác có thể phản biện kết luận hoặc tái dựng chuỗi nhân quả thay vì tin vào trí nhớ của người trực hệ thống.

Unit có thể được kiểm tra cú pháp và một phần tham chiếu bằng `systemd-analyze verify` trên tệp unit. Sau khi cài unit vào nơi systemd đọc và chạy `daemon-reload`, cần kiểm tra user/group và đường dẫn đã tồn tại, rồi mới khởi động. Trên máy đang quản lý dịch vụ, quan sát `systemctl show` các thuộc tính như `MainPID`, `ExecMainStatus`, `ActiveState` và `SubState` giúp phân biệt “chưa tạo được process”, “process thoát” và “process còn sống”. Nhật ký đặt các bước vào trục thời gian; cùng một mã thoát chỉ là dấu vết, không phải lời giải thích nguyên nhân.

Một kiểm tra nhỏ nhưng có sức phân biệt là chủ động gây lỗi trong môi trường thử nghiệm: tạm trỏ unit thử tới một binary không tồn tại để quan sát lỗi ở bước exec, rồi trả lại; tiếp đó thử một đường dẫn cấu hình mà tài khoản dịch vụ không đi qua được để quan sát lỗi sau khi process đã khởi chạy. Hai phép thử không nên làm trên dịch vụ production. Chúng dạy cách đọc ranh giới: trường hợp đầu thuộc thiết lập tiến trình của systemd, trường hợp sau là lời gọi tệp trong ứng dụng bị nhân từ chối. Sau mỗi thay đổi phải hoàn nguyên cấu hình thử, kiểm tra trạng thái và dữ liệu; không dùng restart lặp vô hạn làm bằng chứng tự phục hồi.

Ở điểm này, một dịch vụ có bốn câu hỏi kiểm chứng khác nhau: artifact có ở đúng đường dẫn không; tiến trình có được tạo dưới đúng danh tính không; các lời gọi tài nguyên của nó có thành công không; và người dùng có nhận được chức năng mong đợi không. Chúng lần lượt nối Chương 1, Chương 2 và chương này. Phần mạng sẽ bổ sung đường đi của yêu cầu qua địa chỉ và giao thức, nơi một PID khỏe vẫn có thể không phục vụ được người dùng ở xa.

Bài tập P02.C04.L03.Q01

Phân biệt hai lỗi: binary trong `ExecStart` không tồn tại và binary chạy được nhưng `openat` cấu hình trả `EACCES`. Thành phần nào ghi nhận lỗi đầu tiên, systemd có thể phản ứng ra sao, và vì sao tăng `RestartSec` không giải quyết nguyên nhân?

Chương P02.C05 — Vận hành host và phục hồi có kiểm chứng

Đặt thay đổi cấu hình, log rotation và backup vào quy trình có hậu điều kiện.

Bài P02.C05.L01 — Thay đổi host

Một thay đổi host không chỉ là sửa tệp rồi khởi động lại dịch vụ. Host đang chứa tiến trình, cấu hình, quyền và dữ liệu có vòng đời khác nhau; mỗi bước có một hậu điều kiện riêng. Tình huống xuyên suốt là dịch vụ Atlas đang phục vụ request, nhưng một bản cấu hình mới đổi đường đọc dữ liệu. Người vận hành thấy lỗi mở tệp, trong khi unit vẫn `active`. Trước khi sửa, phải biết lỗi bắt đầu từ revision nào, request nào bị ảnh hưởng, trạng thái dữ liệu và liệu có đường quay lại hay không. Cửa sổ bảo trì dành cho những thao tác có thể ngắt phục vụ hoặc làm đổi dữ liệu, không phải tấm giấy chứng nhận rằng thay đổi chắc chắn an toàn.

Trình tự thực dụng là lưu cấu hình hiện hành và hash, kiểm bản sao có thể đọc, ghi điều kiện dừng, rồi so diff mong muốn với hiện trạng. Kiểm cú pháp unit và cấu hình bằng công cụ của chính ứng dụng nếu có. Xem danh tính mà systemd sẽ dùng, quyền trên toàn đường dẫn và biến môi trường cần thiết; không đưa bí mật vào log kiểm thử. Chỉ áp thay đổi khi đã có cách quay lại tương ứng với kiểu lỗi: hoàn nguyên config cho lỗi config, phục hồi dữ liệu cho lỗi dữ liệu. Một snapshot hạ tầng không thay được bản sao ứng dụng nhất quán nếu dữ liệu đang được sửa trong lúc chụp.

Trong cửa sổ triển khai, ghi thời điểm bắt đầu, người chịu trách nhiệm và các phép quan sát trước/sau. `systemctl restart` thành công chỉ chứng minh systemd đã xử lý yêu cầu theo hợp đồng của unit; tiến trình còn sống vẫn có thể chưa sẵn sàng hoặc đang đọc cấu hình sai. Cần thử request thực đi qua đường người dùng, đọc đúng dữ liệu và kiểm không có lỗi mới trong nhật ký. Nếu một điều kiện dừng đã định bị vi phạm, dừng triển khai trước khi tiếp tục sửa nhiều biến cùng lúc. Việc sửa quyền trên host phải ở mức tối thiểu đã kiểm chứng; chạy lại bằng root để “thử nhanh” làm phép chẩn đoán mất giá trị.

Không có một thứ tự thao tác bất biến cho mọi dịch vụ. Nếu thay đổi gồm schema dữ liệu không tương thích, hoàn nguyên binary/config có thể làm dữ liệu mới không đọc được. Khi ấy kế hoạch phải chỉ rõ phạm vi phục hồi dữ liệu và mức mất dữ liệu chấp nhận được trước khi bắt đầu, thay vì nghĩ rollback ứng dụng luôn đầy đủ.

Bài tập P02.C05.L01.Q01

Atlas vẫn active sau restart nhưng request đọc dữ liệu lỗi. Nêu ba chứng cứ trước thay đổi và hai hậu điều kiện trước khi tuyên bố xong.

Bài P02.C05.L02 — Dữ liệu host

Backup có ích khi nó phục hồi được một trạng thái có ý nghĩa cho ứng dụng. Một bản sao thư mục được tạo trong lúc dịch vụ đang ghi có thể chứa tệp A ở thời điểm trước và tệp B ở thời điểm sau, chưa từng cùng tồn tại ở trạng thái ấy. Với dữ liệu giao dịch, phải dùng cơ chế backup của ứng dụng, dừng ghi theo hợp đồng, hoặc snapshot nhất quán của toàn bộ tập dữ liệu cần thiết. Ví dụ PostgreSQL 17: sao chép trực tiếp các tệp dữ liệu bằng công cụ tệp khi server đang chạy không tạo bản backup dùng được; một snapshot nhất quán phải bao gồm dữ liệu liên quan và WAL theo điều kiện tài liệu nêu. Đây là ví dụ về tính nhất quán, không là hướng dẫn chạy backup trên cơ sở dữ liệu thật.

Đặt ba câu hỏi trước khi nhận một archive: nó chụp dữ liệu nào và ở mốc nào; có thể đọc lại từ nơi khác không; cần mất bao lâu và mất tối đa bao nhiêu thay đổi để phục hồi. Hash của archive giúp phát hiện byte đổi sau khi tạo, nhưng không chứng minh dữ liệu bên trong nhất quán hoặc đầy đủ. Phép thử restore trên đường dẫn tạm cô lập phải mở được tệp, kiểm cấu trúc hoặc truy vấn bản ghi đại diện và ghi kết quả. Không phục hồi đè lên dữ liệu đang chạy để "xác nhận nhanh". Nếu không có công cụ ứng dụng hay quyền cần thiết, ghi `SKIP` thay vì coi sự tồn tại của file `.tar` là PASS.

Log rotation là một thay đổi dữ liệu khác: tiến trình có thể giữ descriptor mở tới inode cũ sau khi tên tệp được đổi. Cần biết ứng dụng có hỗ trợ mở lại log khi nhận tín hiệu hay không. Chế độ `copytruncate` giữ tệp gốc rồi xóa phần đã copy, nhưng có khoảng giữa copy và truncate khiến bản ghi mới có thể mất. Do đó chỉ dùng khi chấp nhận rủi ro ấy và kiểm theo phiên bản công cụ; một hệ thống yêu cầu audit log đầy đủ cần cơ chế ghi và luân chuyển phù hợp hơn. Sau rotation, kiểm tiến trình vẫn ghi vào đích dự kiến và các bản lưu còn đọc được; không suy từ việc lệnh rotation trả mã 0.

Trong tình huống Atlas, bản backup trước đổi config phải gắn với cùng `change_id`, thời điểm, phạm vi dữ liệu và phép thử restore. Rotation log trong cửa sổ đó phải giữ đủ chứng cứ chẩn đoán trước/sau; xóa log ngay khi sự cố đang diễn ra có thể làm mất đường suy nguyên nhân.

Bài tập P02.C05.L02.Q01

Một file tar của thư mục PostgreSQL được tạo khi server đang ghi, hash tar khớp sau copy. Điều gì đã được kiểm, điều gì chưa, và phép thử restore nên diễn ra ở đâu?

Bài P02.C05.L03 — Phục hồi

Phục hồi host bắt đầu từ việc đóng băng chứng cứ vừa đủ, không bắt đầu bằng restart lặp lại. Ghi revision artifact đang chạy, hash cấu hình, UID/GID hiệu lực, lỗi mở tệp, trạng thái dịch vụ, một request thất bại có ID và mốc thời gian. Không đưa nội dung secret vào bản ghi. Sau đó chọn ranh giới sửa nhỏ nhất: nếu chỉ một mode thư mục ngăn đọc, sửa quyền đã duyệt; nếu config mới trỏ sai đường, quay về bản config trước; nếu dữ liệu đã đổi, phải dùng kế hoạch phục hồi dữ liệu riêng. Các hành động này không hoán đổi được.

Ví dụ giả định, trước thay đổi request `r-01` đọc bản ghi `order-7` thành công. Sau đổi config, `r-02` trả lỗi vì tiến trình không đi qua `/srv/atlas`. Hoàn nguyên config khiến `r-03` trả HTTP thành

công, nhưng hậu điều kiện chưa hoàn tất nếu `order-7` trả nội dung khác hoặc yêu cầu ghi mới đã mất. Phép kiểm phải đọc lại bản ghi cũ, ghi và đọc bản ghi thử mới trong phạm vi an toàn, xác nhận revision đang phục vụ, rồi so log lỗi theo cùng cửa sổ. ID ví dụ là dữ liệu giả định, không phải output thực nghiệm. Nếu cấu trúc dữ liệu đã thay, cần đánh giá tương thích trước khi chạy binary cũ.

Một bản phục hồi có thể được tuyên bố đạt khi bằng chứng sau sửa đáp ứng hợp đồng đã ghi trước: request thành công, dữ liệu đúng, quyền không bị mở rộng và log không còn đường lỗi ấy. `active` của systemd là một quan sát về vòng đời tiến trình; nó không thay phép thử request. Ngược lại, một request thành công đơn lẻ không chứng minh toàn bộ dữ liệu hoặc mọi đường nghiệp vụ đã phục hồi. Nếu còn thiếu dữ liệu, ghi phạm vi chưa kiểm và dừng ở trạng thái chưa đạt, thay vì sửa câu chữ báo cáo thành PASS.

Giữ bản ghi trước/sau cùng một trục thời gian giúp người khác phân biệt nhân quả với trùng hợp. Nếu thay nhiều biến cùng lúc, khó biết thay đổi nào sửa lỗi hoặc gây tác dụng phụ. Vì vậy, sau mỗi bước phải ghi diff, chạy phép kiểm tương ứng và chỉ tiến tiếp khi hậu điều kiện của bước đó được quan sát. Quy trình này chuẩn bị cho runbook sự cố nhiều dịch vụ ở phần sau, nhưng không biến host đơn lẻ thành toàn bộ hệ thống.

Bài tập P02.C05.L03.Q01

Sau khi hoàn nguyên config, r-03 trả HTTP thành công nhưng bản ghi order-7 khác trước. Có được báo phục hồi hoàn tất không? Cần thêm chứng cứ nào?

PHẦN 03 — Mạng và giao thức ứng dụng

Một request đi từ tên tới byte phản hồi qua những ranh giới nào?

Mỗi tầng chọn một đích hoặc hợp đồng: route/neighbor, kết nối, bảo mật phiên, ngữ nghĩa HTTP và proxy.

Chương P03.C01 — Địa chỉ, subnet và đường đi gói IP

Từ đích IP tìm route và địa chỉ chặng kế tiếp.

Bài P03.C01.L01 — Định danh logic

Một client báo "không kết nối được" chưa chỉ ra lỗi nằm ở đâu. Nó có thể chọn sai đích, không tìm thấy đường ra, không chuyển được gói tới gateway, bị lọc trên đường, hoặc đến đúng máy nhưng không có chương trình lắng nghe ở địa chỉ và cổng ấy. Muốn sửa đúng, ta phải tách địa chỉ IP của đích, địa chỉ ở từng chặng truyền, trạng thái của kết nối TCP và trạng thái mà Linux đang giữ. Chương này dùng IPv4 và Ethernet làm mô hình cụ thể; IPv6 có cơ chế tìm láng giềng và biểu diễn địa chỉ khác, không được suy nguyên xi từ ARP của IPv4.

Địa chỉ IP là định danh logic dùng để chuyển gói qua nhiều mạng. Nó không phải địa chỉ vật lý của card mạng, cũng không phải tên dịch vụ. Một giao diện có thể có nhiều địa chỉ; một máy có thể có nhiều giao diện; cùng một dịch vụ có thể được gọi bằng tên nhưng cuối cùng client vẫn phải chọn một địa chỉ và cổng để thử kết nối. Phần DNS và HTTP ở chương sau mới giải thích đầy đủ bước từ tên tới địa chỉ. Ở đây ta bắt đầu từ đích đã biết là `198.51.100.20:443`.

Giả sử client mang `10.20.1.10/24`. Prefix `/24` nghĩa là 24 bit đầu thuộc phần mạng; trong ví dụ, `10.20.1.10` và `10.20.1.1` cùng prefix `10.20.1.0/24`, còn `198.51.100.20` thì không. Phép tính này chỉ so phần bit theo mask; nó không tự chứng minh hai máy thực sự cùng một miền truyền Ethernet, cũng không cho biết đích có hoạt động. Những địa chỉ `198.51.100.20` và `203.0.113.5` trong chương là địa chỉ dành cho tài liệu, không phải endpoint để độc giả thử trên Internet. Địa chỉ `10.20.1.10` thuộc phạm vi riêng trong ví dụ, được dùng để minh họa một client phía trong gateway.

Linux giữ địa chỉ gán cho giao diện, prefix, trạng thái liên kết và bảng route theo network namespace. `ip address` cho biết cấu hình địa chỉ mà máy đang thấy; nó không chứng minh địa chỉ đã được quảng bá đúng ra mạng hoặc thiết bị phía kia có thể trả lời. Trước khi kết luận "mất mạng", cần kiểm tra client đang chạy trong namespace nào: một container có thể nhìn thấy giao diện và route khác host. Tên giao diện cũng là chi tiết của máy, không phải một hằng số của giao thức.

Bài P03.C01.L02 — Chọn chặng

Với đích ngoài prefix trực tiếp, client tra bảng định tuyến để chọn đường ra và next hop. Trong ví dụ, route mặc định dẫn tới gateway `10.20.1.1` trên giao diện nội bộ. Khi có nhiều route khớp, prefix cụ thể hơn thắng prefix ngắn hơn; route mặc định `/0` chỉ dùng nếu không có đường phù hợp hơn theo các quy tắc chọn route có hiệu lực. Trên Linux, policy routing, nguồn được chọn, bảng route và namespace còn có thể thay đổi kết quả. `ip route get 198.51.100.20` hữu ích vì cho thấy quyết định route đối với một đích cụ thể, gồm gateway và địa chỉ nguồn được chọn trong ngữ cảnh gọi lệnh. Đó là phép xem quyết định ở máy, chưa phải bằng chứng gói đã tới gateway.

Bài tập P03.C01.L02.Q01

Một máy có route mặc định và route `198.51.100.0/24` qua hai gateway khác nhau. Với đích `198.51.100.20`, route nào được ưu tiên? Nêu một lý do khiến kết quả thực tế trên Linux vẫn cần kiểm tra bằng `ip route get` trong đúng namespace.

Bài P03.C01.L03 — Đóng frame

Sau khi chọn next hop `10.20.1.1`, client cần địa chỉ lớp liên kết của gateway để gửi frame Ethernet. ARP của IPv4 hỏi "ai giữ địa chỉ IP này?" trên liên kết cục bộ rồi lưu ánh xạ IP–MAC trong bảng láng giềng. Client hỏi MAC của gateway, không hỏi MAC của `198.51.100.20` ở mạng xa. `ip neigh` cho thấy cache láng giềng và trạng thái như `REACHABLE`, `STALE` hoặc `FAILED`; `STALE` không tự nghĩa là mất kết nối vì kernel có thể xác minh lại khi dùng. Một ánh xạ tốt chỉ chứng minh một phần đường cục bộ, không xác nhận route, NAT hay server phía sau.

Router nhận frame ở chặng đầu, bỏ lớp đóng gói liên kết cục bộ, xem địa chỉ IP đích, chọn chặng tiếp theo rồi đóng gói thành frame mới phù hợp với liên kết kế. IP đích thường vẫn là `198.51.100.20` qua các router thông thường, trong khi MAC nguồn và MAC đích đổi theo từng chặng. Nếu router cần chuyển qua nhiều mạng, mỗi chặng tự phân giải láng giềng của nó. Đường trả lời cũng cần route hợp lệ theo chiều ngược; việc client gửi được SYN không có nghĩa SYN-ACK sẽ quay về được.

Bài tập P03.C01.L03.Q01

Client `10.20.1.10/24` gửi tới `198.51.100.20`. Route mặc định qua `10.20.1.1`. Client cần tìm MAC của địa chỉ nào trước khi gửi frame đầu tiên? Vì sao không hỏi trực tiếp MAC của server?

Chương P03.C02 — Transport, TCP, UDP và NAT

Lần theo SYN, seq/ack, retransmission, datagram UDP và dịch tuple hai chiều.

Bài P03.C02.L01 — Bắt tay và byte stream

TCP đặt cổng và trạng thái lên trên IP để hai đầu phân biệt các cuộc hội thoại. Với kết nối thường, client gửi SYN từ một cổng nguồn tạm thời tới cổng server, server trả SYN-ACK, rồi client gửi ACK. Các số thứ tự và xác nhận giúp hai đầu đồng bộ trạng thái và nhận biết byte nào đã được chấp nhận. Server phải có socket lắng nghe trên địa chỉ/cổng phù hợp; một tiến trình còn sống hoặc một unit `active` không đồng nghĩa đã có socket lắng nghe ở địa chỉ client đang gọi. `ss -ltn` trên đúng network namespace cho thấy socket TCP đang `LISTEN` và địa chỉ local mà socket bind; `127.0.0.1:443` chỉ nhận kết nối đi vào loopback của chính namespace đó, không nhận gói đến `198.51.100.20:443` qua giao diện ngoài.

Có thể lần theo một trace tối giản với số khởi đầu giả định. Client gửi `SYN seq=1000`; server đáp `SYN-ACK seq=5000 ack=1001`; client gửi `ACK seq=1001 ack=5001`. Mỗi SYN chiếm một vị trí trong không gian số thứ tự dù không mang dữ liệu. Sau đó client gửi `ACK seq=1001 ack=5001 len=4`, tức các byte được đánh số 1001–1004; server đáp `ACK seq=5001 ack=1005 len=0`, nghĩa là đã nhận liên tục mọi byte trước 1005 và đang chờ byte 1005. Nếu riêng ACK này mất, client có thể truyền lại chính bốn byte với `seq=1001 ack=5001 len=4`; server nhận ra dữ liệu lặp, không giao lần hai cho ứng dụng và lại báo `ack=1005`. ACK là cộng dồn, không phải dấu xác nhận riêng từng packet. Trace bỏ qua TCP option, dữ liệu trong SYN, đảo thứ tự, vòng số thứ tự và cách bộ hẹn giờ quyết định truyền lại; capture ở một điểm cũng không đủ chứng minh vì sao bản truyền lại xuất hiện.

Sau bắt tay, TCP không chỉ "gửi byte rồi hy vọng". Bên gửi giữ dữ liệu chưa được xác nhận để truyền lại khi phát hiện mất; ACK xác nhận tiến độ của luồng byte. Cửa sổ nhận giới hạn lượng dữ liệu bên nhận còn nhận được, còn điều khiển nghẽn giới hạn lượng dữ liệu bên gửi đưa vào mạng theo tín hiệu của đường truyền. Hai giới hạn khác nhau: cửa sổ nhận nhỏ có thể phản ánh ứng dụng đọc chậm, còn điều khiển nghẽn phản ứng với tình trạng mạng theo thuật toán đang dùng. Việc bắt tay thành công không bảo đảm tốc độ truyền tiếp theo. TCP đóng có trạng thái riêng; reset chấm dứt đột ngột, còn đóng thông thường trao đổi FIN và có thể để lại `TIME-WAIT` nhằm xử lý các segment cũ. Không coi mọi socket không còn `ESTABLISHED` là lỗi.

Bài tập P03.C02.L01.Q01

Trong trace mục 4.3, vì sao SYN của client được đáp bằng `ack=1001`, còn bốn byte ở `seq=1001` được đáp bằng `ack=1005`? Nếu ACK đầu cho dữ liệu bị mất, client truyền lại cùng bốn byte thì server nên báo số ACK nào, và vì sao ứng dụng không nhận dữ liệu hai lần?

Bài P03.C02.L02 — Socket và lỗi

Một kết nối được nhận diện bởi giao thức và bộ địa chỉ, cổng ở hai đầu. `SYN-SENT` ở client cho biết nỗ lực bắt tay chưa hoàn tất; `ESTABLISHED` cho biết hai TCP endpoint đã thiết lập trạng thái, chưa chứng minh HTTP, TLS hay ứng dụng trả kết quả đúng. Nếu cổng đích không có người lắng nghe, client có thể nhận reset và `connect` báo `ECONNREFUSED`. Nếu SYN hoặc phản hồi không trở lại, TCP có thể thử lại và cuối cùng báo hết thời gian; `ETIMEDOUT` chỉ là triệu chứng, không tự chỉ ra firewall, mất route hay server quá tải. Một firewall `REJECT` cũng có thể tạo phản hồi rõ, còn `DROP` thường làm client chờ; cần xem gói thực tế và chính sách có hiệu lực trước khi quy nguyên nhân.

Đây là tình huống giả định có kết quả quan sát được định nghĩa trước để tập suy luận, không phải log của một hệ thống thật. Client `10.20.1.10/24` cần kết nối `198.51.100.20:443` qua gateway `10.20.1.1`; gateway dịch nguồn sang `203.0.113.5`. Ứng dụng báo `ECONNREFUSED`. Mục tiêu không phải thử mọi lệnh, mà tìm điểm đầu tiên nơi trạng thái quan sát khác đường đi mong đợi.

Trước hết, xác định lời gọi kết nối dùng đúng IP và cổng, rồi xem route trong namespace của client. Giả sử `ip route get` chọn `10.20.1.1` và địa chỉ nguồn `10.20.1.10`; `ip neigh` cho gateway có ánh xạ hợp lệ. Hai quan sát loại bớt giả thuyết thiếu route hoặc chưa tìm được MAC ở client tại thời điểm đo, nhưng không chứng minh gateway đang chuyển tiếp. Một capture ngắn ở giao diện ra của client thấy SYN `10.20.1.10:51514` tới `198.51.100.20:443`; như vậy lỗi không nằm ở việc client chưa phát SYN. Nếu không có quyền quan sát gateway, phải nói rõ khoảng đường giữa hai điểm đo vẫn chưa xác minh.

Trong tình huống này, người vận hành có quyền xem gateway và server. Capture ở phía ngoài gateway thấy SYN với nguồn đã đổi thành `203.0.113.5:40001`. Capture trên server thấy đúng SYN tới `198.51.100.20:443`, rồi một RST từ phía server quay lại. Điều đó cho thấy route đi và ánh xạ ra đã làm việc cho gói này; reset là manh mối mạnh cho việc kết nối bị từ chối tại hoặc gần server. Nó chưa tự phân biệt kernel vì không có listener với firewall chủ động từ chối. Ta kiểm tra đúng namespace của server: `ss -ltn` chỉ thấy `127.0.0.1:443`, không thấy listener trên `198.51.100.20:443` hay địa chỉ bao quát thích hợp. Đồng thời kiểm tra quy tắc lọc có hiệu lực để loại trừ reset do chính sách. Với bộ quan sát giả định này, nguyên nhân trực tiếp là ứng dụng chỉ bind vào loopback; sửa cấu hình bind theo hợp đồng truy cập rồi xác nhận `ss` và bắt tay SYN, SYN-ACK, ACK trên đúng đường. Sau đó mới thử một yêu cầu ứng dụng: bắt tay TCP thành công chưa kiểm tra được TLS hoặc HTTP.

Nếu trace khác đi, kết luận cũng đổi. Không có SYN tại client dẫn ta về địa chỉ đích, route hoặc lỗi cục bộ; SYN rời client nhưng không tới gateway dẫn ta về link, neighbor hoặc lọc tại chặng đầu. SYN tới server mà không có phản hồi cần kiểm tra lọc, trạng thái máy và việc server nhận gói; SYN-ACK rời server nhưng không về client hướng chú ý sang route trả, NAT và lọc theo chiều về. Những nhánh này là giả thuyết có thứ tự, không phải bảng ánh xạ một triệu chứng với một nguyên nhân duy nhất. Capture là ảnh chụp ở một điểm và một thời gian: mất gói khi capture, offload, namespace sai hoặc lưu lượng qua đường khác đều có thể làm nó thiếu bức tranh. Bằng chứng tốt nhất ghép cùng tuple kết nối, thời gian và hai đầu của đoạn đường đang nghi vấn.

`ss -ltn` trên đúng network namespace cho thấy socket TCP đang `LISTEN` và địa chỉ local mà socket bind; `127.0.0.1:443` chỉ nhận kết nối đi vào loopback của chính namespace đó, không nhận gói đến `198.51.100.20:443` qua giao diện ngoài.

Bài tập P03.C02.L02.Q01

Client thấy SYN rời máy; server thấy SYN và gửi SYN-ACK; client không thấy SYN-ACK. Nêu ít nhất hai điểm cần kiểm tra theo chiều về và giải thích vì sao chưa thể quy lỗi cho ứng dụng HTTP.

Bài tập P03.C02.L02.Q02

`connect` trả `ECONNREFUSED`; server chỉ có `127.0.0.1:443` ở trạng thái `LISTEN`. Hãy nêu giả thuyết mạnh nhất, một nguyên nhân cạnh tranh vẫn phải kiểm tra, và phép xác nhận sau khi sửa. Vì sao `systemctl active` chưa đủ?

Bài P03.C02.L03 — NAT và đối xứng đường

Để client dùng địa chỉ riêng ra mạng khác, gateway trong ví dụ thực hiện NAPT: nó đổi nguồn `10.20.1.10:51514` thành `203.0.113.5:40001` khi gửi tới `198.51.100.20:443`, đồng thời giữ ánh xạ để dịch gói trả lời về đúng client. Số cổng `40001` chỉ là giá trị giả định; triển khai có thể giữ hoặc đổi cổng theo khả năng và xung đột. Server thấy nguồn sau dịch, không thấy địa chỉ riêng của client ở tầng IP. Nếu không có ánh xạ trả về còn hiệu lực, hoặc gói đáp đi qua một thiết bị khác không biết ánh xạ, phản hồi có thể thất lạc dù SYN ban đầu đã rời client.

NAT không phải định tuyến: route quyết định chặng chuyển tiếp, còn NAT thay địa chỉ hoặc cổng trong gói ở điểm dịch và cập nhật checksum cần thiết. Trạng thái dịch theo luồng của gateway cũng khác trạng thái TCP trong hai endpoint. Linux có thể dùng hệ thống theo dõi kết nối của netfilter để gắn các gói với cùng luồng và áp dụng NAT, nhưng việc thấy một mục theo dõi không tự chứng minh server đã chấp nhận kết nối. Ngược lại, một capture sau NAT có tuple khác capture trước NAT; phải ghi rõ mình quan sát ở giao diện nào và phía nào của điểm dịch trước khi so hai trace. NAT không phải cơ chế bảo mật thay cho lọc gói và chính sách truy cập.

Bài tập P03.C02.L03.Q01

Gateway đổi `10.20.1.10:51514` thành `203.0.113.5:40001`. Server trả gói về tuple nào, và gateway cần điều kiện gì để đưa gói về client? Phân biệt trạng thái NAT với trạng thái TCP endpoint.

Chương P03.C03 — Tên, TLS và ngữ nghĩa HTTP

Tách lỗi DNS, TCP, chứng chỉ và HTTP bằng dữ kiện tối thiểu.

Bài P03.C03.L01 — DNS và cache

Một trang không mở được có thể bắt đầu từ tên miền sai, nhưng cũng có thể thất bại sau khi client đã đến đúng IP và hoàn tất TCP. Ta sẽ theo một yêu cầu giả định tới `https://api.example.test/orders/42` qua một reverse proxy rồi tới ứng dụng phía sau. Đường chính của chương là DNS chọn địa chỉ, TCP nối tới địa chỉ ấy, TLS xác thực và bảo vệ kênh client–proxy, HTTP mang yêu cầu, proxy chọn upstream, rồi phản hồi đi ngược về. Đây là mô hình HTTPS trên TCP với HTTP/1.1 và TLS 1.3 để thấy rõ từng ranh giới; Không nên dùng mô hình này để khẳng định mọi lần tải trang đều mở một kết nối mới hoặc đi đủ mọi bước khi cache và kết nối tái sử dụng còn hiệu lực.

Tên `api.example.test` không tự mang địa chỉ IP. Ứng dụng nhờ bộ phân giải ở máy client tìm bản ghi phù hợp, thường là A cho IPv4 hoặc AAAA cho IPv6. Bộ phân giải có thể dùng thông tin cục bộ hoặc hỏi recursive resolver; resolver này dùng cache hay hỏi các máy chủ có thẩm quyền theo hệ phân cấp. Kết quả DNS là dữ liệu để client chọn địa chỉ thử kết nối, không phải phép thử cổng 443, chứng chỉ, hay tình trạng ứng dụng. Cùng một tên có thể trả nhiều địa chỉ; client và mạng có thể chọn các đường khác nhau. Khi đo, cần ghi rõ loại bản ghi, resolver, thời điểm và địa chỉ thật mà tiến trình đã dùng, thay vì coi một kết quả tra cứu trên máy khác là bằng chứng của request lỗi.

Mỗi bản ghi có TTL giới hạn thời gian lưu ở cache DNS. Thay bản ghi ở authoritative server không buộc mọi cache và mọi tiến trình chuyển ngay sang địa chỉ mới; một số ứng dụng còn có cache hoặc giữ kết nối của riêng mình. Kết quả không tồn tại tên (NXDOMAIN) cũng có thể được cache theo quy tắc riêng; nó khác việc resolver chờ quá hạn vì không nhận được câu trả lời đáng tin. Vì thế "DNS sai" cần nói sai ở đâu: dữ liệu authoritative, cache của resolver, cấu hình resolver ở client, hay tên mà ứng dụng thật sự hỏi. Giảm TTL trước một thay đổi có kế hoạch có thể rút ngắn thời gian giữ dữ liệu cũ, đổi lại tăng số lần tra cứu; nó không xóa được cache đã nhận TTL trước đó. DNS thành công chỉ mở cửa cho bước kết nối tiếp theo.

Bài tập P03.C03.L01.Q01

Client A vẫn lấy IP proxy cũ cho `api.example.test`, trong khi một resolver khác trả IP mới. Nêu ba dữ kiện cần ghi trước khi quy lỗi cho authoritative DNS. Vì sao một lần tra cứu từ máy của người vận hành không đủ?

Bài P03.C03.L02 — TLS

Với địa chỉ đã chọn, client dùng route và TCP của Chương 4 để nối tới cổng 443 của proxy. Không có đường trả về hoặc SYN bị lọc có thể gây timeout trước khi có bất kỳ HTTP status nào. TCP `ESTABLISHED` chỉ chứng minh luồng byte đã mở giữa client và đầu nhận TCP; nó chưa nói đầu kia là đúng tên dịch vụ. Một phép thử trực tiếp bằng IP có thể bỏ qua DNS, nhưng nếu không giữ nguyên tên dịch vụ cho TLS và HTTP thì phép thử đó có thể đi vào cấu hình virtual host khác.

Trên TCP, client bắt đầu TLS bằng ClientHello, thường mang tên đích qua SNI để máy phục vụ chọn cấu hình và chứng chỉ phù hợp. Trong TLS 1.3 với xác thực bằng chứng chỉ, hai bên thỏa thuận khóa; server gửi chứng chỉ và chứng minh nắm khóa tương ứng, còn các thông điệp hoàn tất kiểm tra tính toàn vẹn của bắt tay. Client phải kiểm tra chuỗi tin cậy, thời hạn và danh tính trong chứng chỉ có khớp tên dịch vụ yêu cầu. Nếu tên không khớp hoặc chuỗi không đáng tin, client phải dừng theo chính sách xác thực của mình; tắt xác minh chỉ che lỗi và mở đường cho kẻ khác mạo danh. SNI giúp chọn chứng chỉ, không phải chứng cứ rằng chứng chỉ đã hợp lệ. Tên trong HTTP `Host` được dùng ở tầng định tuyến yêu cầu; nó cũng không thay thế phép xác minh TLS.

Khi TLS kết thúc ở reverse proxy, chỉ kênh client–proxy được bảo vệ bởi phiên TLS ấy. Proxy có thể mở một kết nối HTTP thường hoặc một phiên TLS khác tới upstream; cần xác định hợp đồng mạng và mối đe dọa của đoạn sau trước khi chọn. TLS tới proxy không tự mã hóa đoạn proxy–upstream. TLS handshake thành công cũng chưa chứng minh proxy chọn đúng ứng dụng hay ứng dụng xử lý được yêu cầu. Tái sử dụng kết nối và phiên có thể giảm chi phí bắt tay, nhưng làm cho chẩn đoán “một request bằng một kết nối mới” sai; khi kiểm tra cần biết client vừa mở hay đang dùng kết nối cũ.

Bài tập P03.C03.L02.Q01

TCP tới proxy đã `ESTABLISHED` nhưng client báo chứng chỉ không khớp tên. Nêu thông tin cần đối chiếu ở client và proxy. Vì sao đổi HTTP `Host` hoặc tắt xác minh TLS không phải cách sửa gốc?

Bài P03.C03.L03 — HTTP

HTTP/2 có thể ghép nhiều luồng trên một kết nối TCP; HTTP/3 vận chuyển cùng ngữ nghĩa HTTP trên QUIC. Vì vậy khi chẩn đoán cần biết giao thức đã thương lượng, thay vì giả định một request luôn ứng với một kết nối TCP mới.

Sau TLS, client gửi một HTTP request có phương thức, đích yêu cầu, các trường như `Host` và có thể có body. Với ví dụ này, `GET /orders/42` yêu cầu một tài nguyên; phản hồi có status, header và có thể có nội dung. Status mô tả kết quả ở tầng HTTP, nhưng một `200` không chứng minh nội dung đúng, còn một `404` có thể do proxy hoặc ứng dụng trả. Nếu HTTP request chưa hề được gửi vì DNS, TCP hoặc TLS hỏng thì không có status HTTP từ đích để diễn giải. Trong HTTP/1.1, `Host` giúp chọn authority khi nhiều tên cùng dùng một IP; một TCP connection tới đúng IP vẫn có thể bị route sang virtual host khác nếu tên yêu cầu sai.

Cache HTTP lưu phản hồi đủ điều kiện và dùng độ tươi hoặc xác minh lại để giảm độ trễ và tải upstream. Nó khác cache DNS: một bên lưu dữ liệu tên, bên kia lưu phản hồi ứng dụng. Cấu hình cache thiếu phân biệt người dùng có thể làm lộ nội dung riêng; việc thấy nội dung cũ cũng chưa đủ kết luận DNS còn cũ. Cần kiểm tra `Cache-Control`, khóa cache, `Age` và đường đi thực tế theo loại phản hồi. Không dùng cache để che một upstream hỏng nếu dữ liệu sai hoặc quá cũ có hậu quả nghiệp vụ.

Chương P03.C04 — Reverse proxy và điều tra request hai chặng

Xác định ai sinh status và chặng nào timeout.

Bài P03.C04.L01 — Chọn upstream

Reverse proxy nhận request ở phía client, chọn route theo tên và đường dẫn, rồi gửi một request sang upstream đã cấu hình. Đây là hai cuộc trao đổi có thể dùng hai kết nối và hai bộ timeout khác nhau. Proxy có thể thêm thông tin về client ban đầu cho upstream, nhưng ứng dụng chỉ nên tin các trường chuyển tiếp do proxy đáng tin cậy đặt hoặc chuẩn hóa; client ngoài cũng có thể tự gửi header cùng tên. Proxy còn có thể cân bằng tải, giới hạn truy cập, đổi header hoặc cache theo cấu hình. Mỗi tính năng tạo thêm điểm cần quan sát: proxy trả lời từ cache thì upstream có thể không thấy request; upstream trả nhanh nhưng proxy gửi chậm thì độ trễ người dùng vẫn cao. Log nên ghép thời điểm, tên đích, route đã chọn, upstream, thời gian kết nối, thời gian chờ phản hồi và một định danh request không chứa bí mật.

Bài P03.C04.L02 — Hai chặng

Trace dưới đây là dữ liệu giả định để đọc một lần trao đổi thành công, không phải output thu từ mạng. Các dòng phân cách chặng là chú giải, dòng trống kết thúc phần header; byte trên dây HTTP/1.1 dùng CRLF. Client xác thực TLS với `api.example.test` rồi gửi request bên trong kênh mã hóa tới proxy. Proxy giải mã ở điểm kết thúc TLS và, theo cấu hình giả định này, mở HTTP thường tới upstream; đoạn sau cần biện pháp bảo vệ riêng nếu đi qua miền mạng không tin cậy.

```
client -> proxy (HTTP inside TLS)
GET /orders/42 HTTP/1.1
Host: api.example.test
Accept: application/json

proxy -> upstream (HTTP, illustrative)
GET /orders/42 HTTP/1.1
Host: orders.internal.test
X-Forwarded-Host: api.example.test
X-Forwarded-Proto: https

upstream -> proxy
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}

proxy -> client (HTTP inside TLS)
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}
```

`Host: api.example.test` giúp proxy chọn virtual host công khai. Ở chặng sau, proxy chủ động đổi `Host` thành `orders.internal.test` để upstream định tuyến, đồng thời ghi lại tên và giao

thức phía client bằng hai trường chuyển tiếp do chính proxy ghi đè; upstream chỉ nên tin chúng khi request đến từ proxy được tin cậy. `200 OK` trong trace do upstream tạo và proxy chuyển tiếp nguyên trạng, không phải bằng chứng proxy luôn giữ nguyên header hoặc body. `Content-Length: 9` đếm chín byte ASCII của `{"id":42}`. Nếu upstream không trả header kịp ngưỡng chờ, proxy có thể tự sinh `504` thay cho phản hồi này; khi đó cần log của cả hai chặng để biết request có tới upstream và có còn chạy sau lúc proxy ngừng chờ hay không. Trace bỏ qua kết nối tái sử dụng, cache và cách đóng kết nối để tập trung vào ranh giới trách nhiệm.

Bài P03.C04.L03 — Đường lỗi

Chương này dừng ở việc TCP đưa được luồng byte giữa hai endpoint. Người dùng còn có thể báo "không vào được trang" vì tên miền trỏ sai, chứng chỉ bị từ chối, reverse proxy chọn sai upstream hoặc ứng dụng trả lỗi. Các bước ấy thuộc tầng tiếp theo. Giữ ranh giới giúp ta không sửa route khi TCP đã bắt tay, và không sửa HTTP khi SYN chưa tới server.

Nếu proxy, trong vai gateway, nhận phản hồi không hợp lệ từ upstream, nó có thể tạo `502 Bad Gateway`; nếu không nhận phản hồi kịp để hoàn tất yêu cầu, nó có thể tạo `504 Gateway Timeout`. Đây là nghĩa của status, không phải bảng kết luận nguyên nhân: một upstream hoặc tầng trung gian cũng có thể tự trả mã giống vậy, và log proxy phải cho biết mã nào do nó tạo. Lỗi nối upstream, TLS tới upstream, header sai và ứng dụng đóng kết nối là những giả thuyết khác nhau cần phân biệt bằng log và phép đo ở đúng đoạn. Một timeout ngắn giảm thời gian chờ nhưng có thể loại yêu cầu hợp lệ chậm; timeout dài giữ tài nguyên và kéo dài trải nghiệm lỗi. Thử upstream khác có thể cải thiện khả dụng, nhưng gửi lại một yêu cầu thay đổi trạng thái khi chưa biết lần đầu đã thực thi chưa có thể tạo tác dụng phụ lặp. Chính sách retry phải dựa vào tính idempotent, khóa chống lặp hoặc hợp đồng ứng dụng, không chỉ dựa vào status.

Xét một mô hình giả định để luyện chẩn đoán, không phải trace từ hệ thống thật. Theo cấu hình mong đợi, `api.example.test` trỏ tới IP của proxy mới; proxy nhận HTTPS và chuyển `/orders/42` tới upstream đang phục vụ. Người dùng báo "không mở được đơn hàng" sau thay đổi cấu hình. Trước khi sửa, người vận hành ghi lại URL, thời điểm, client, lỗi hiển thị và request ID nếu có. Cùng một câu báo lỗi có thể tương ứng năm điểm dừng sau đây; mỗi phép quan sát chỉ chứng minh một đoạn đường.

Nếu máy bị lỗi phân giải tên ra IP của proxy cũ, còn authoritative server và một resolver khác đã trả IP mới, giả thuyết đầu là cache hoặc cấu hình resolver trên đường của máy ấy. Hỏi đúng resolver mà client dùng, xem TTL còn lại và địa chỉ tiến trình thực sự kết nối; không sửa upstream của proxy mới khi request chưa đến đó. Nếu DNS trả đúng IP mới nhưng SYN rời client mà không có SYN-ACK, chuyển sang route, lọc và đường trả TCP như Chương 4. Một DNS answer đúng không chứng minh cổng mở; timeout TCP chưa phải lỗi TLS hay HTTP. Quan sát ở proxy xem SYN có đến hay không giúp thu hẹp đoạn lỗi, nhưng một capture im lặng tại sai giao diện không đủ kết luận gói chưa tới máy.

Nếu TCP hoàn tất rồi client báo certificate error, xem tên mà client yêu cầu, SNI, chứng chỉ proxy đã chọn, chuỗi tin cậy và đồng hồ của máy. Trong nhánh giả định này, proxy mới đưa chứng chỉ chỉ

chứa `old.example.test`, nên phép xác minh tên `api.example.test` thất bại; sửa binding/chứng chỉ đúng tên là hành động phù hợp. Không chuyển sang HTTP hay bỏ xác minh chứng chỉ để "thử cho qua". Khi TLS hợp lệ và client nhận `502`, đối chiếu log theo request ID: proxy chọn upstream nào, có nối được không, có nhận header HTTP hợp lệ không, hay upstream tự trả `502`. Giả sử log cho thấy upstream chấp nhận TCP rồi đóng trước khi gửi header, nguyên nhân nằm ở giao tiếp proxy–upstream hoặc ứng dụng phía sau, chưa thể suy riêng từ mã `502` là tiến trình đã chết.

Cuối cùng, nếu proxy nhận request nhưng chờ upstream quá hạn rồi tự trả `504`, xem thời gian ở từng bước và log của cùng request ở upstream. Giả sử upstream ghi nhận request và kẹt ở truy vấn dữ liệu lâu hơn ngưỡng chờ của proxy: việc nâng timeout có thể giảm `504` trước mắt nhưng không sửa truy vấn chậm, đồng thời giữ tài nguyên lâu hơn. Cũng phải kiểm tra upstream có thể đã hoàn tất tác dụng phụ sau khi proxy bỏ chờ; retry tự động một lệnh ghi có thể tạo đơn hàng trùng. Mỗi nhánh cần một phép xác nhận sau sửa từ đúng đường người dùng: tên phân giải đúng, TCP bắt tay, TLS xác thực, proxy chọn đúng upstream và response có nội dung đúng. Status, log hay capture riêng lẻ chỉ là một phần chứng cứ.

Bài tập P03.C04.L03.Q01

Cùng URL lần lượt gặp timeout trước TLS, certificate error, `502`, `504`. Với mỗi dấu hiệu, hãy chỉ ra ranh giới xa nhất chắc chắn đã vượt qua và một quan sát tiếp theo có giá trị phân biệt. Có thể kết luận upstream chết chỉ từ `502` không?

Bài tập P03.C04.L03.Q02

Proxy hết thời gian chờ một yêu cầu tạo đơn hàng rồi định retry sang upstream khác. Nêu rủi ro và điều kiện để retry an toàn. Phân biệt tác dụng của tăng timeout với sửa nguyên nhân upstream chậm.

PHẦN 04 — Git và lịch sử thay đổi

Làm sao biết, tích hợp và khôi phục đúng thay đổi đã chia sẻ?

Blob/tree/commit là đồ thị bất biến; ref, index và working tree là trạng thái có thể dịch.

Chương P04.C01 — Git object, index và ref

Dự đoán byte nào sẽ nằm trong commit kế tiếp.

Bài P04.C01.L01 — Object graph

Khi hai người cùng sửa một dịch vụ, câu hỏi không chỉ là "tệp cuối cùng trông ra sao" mà còn là thay đổi nào đã được ghi, thay đổi nào chưa, và hai dòng phát triển có chung tổ tiên nào. Git lưu các snapshot có liên kết cha–con. Hiểu liên kết ấy giúp ta chọn cách tích hợp, đọc conflict và phục hồi sự cố mà không làm mất lịch sử của đồng đội. Các ký tự `A`, `B`, `C` trong chương là nhãn tượng trưng cho commit, không phải SHA thật; tình huống là mô hình giả định, không phải log của repo này.

Một blob chứa nội dung tệp, không chứa tên tệp. Một tree ghép tên, mode và định danh object của blob hoặc tree con thành snapshot thư mục. Commit trỏ tới một tree gốc, ghi tác giả, người tạo commit, thông điệp và không hoặc nhiều commit cha. Vì vậy đổi tên tệp có thể đổi tree mà blob nội dung vẫn giữ nguyên; hai commit khác nhau có thể trỏ tới cùng snapshot tệp nhưng khác cha hoặc metadata. Git lưu lịch sử bằng các quan hệ này, không phải bằng cách chép nguyên một thư mục có nhãn thời gian cho mỗi commit.

Định danh object là giá trị băm của loại, độ dài và nội dung object theo định dạng kho đang dùng. Nhiều kho dùng SHA-1; Git cũng hỗ trợ kho SHA-256. Vì commit chứa định danh tree và cha, thay nội dung một tệp làm đổi blob, tree chứa nó và các commit mới đi qua thay đổi ấy. Định danh cho phép kiểm tra tính nhất quán nội dung trong kho; nó không tự chứng minh ai là tác giả hay rằng mã an toàn. Tên branch và tên file không nằm trong blob. Để tự đối chiếu mô hình trên một kho được phép đọc, `git cat-file -p HEAD` cho thấy trường tree và parent của commit; `git ls-tree HEAD` cho thấy các mục ở tree gốc. Một SHA rút gọn chỉ là cách viết đủ phân biệt trong kho hiện tại, không phải một loại object khác.

Bài P04.C01.L02 — Trạng thái cục bộ

Working tree là những tệp đang thấy và sửa bằng trình biên tập. Index là snapshot dự kiến cho commit kế tiếp; `git add` chép nội dung hiện tại của đường dẫn vào index tại thời điểm chạy. `HEAD` thường là tham chiếu tượng trưng tới branch đang checkout, còn branch ref trỏ tới một commit. Khi tạo commit, Git ghi tree từ index, đặt commit mới có cha là commit tại `HEAD`, rồi dịch ref của branch hiện tại sang commit mới. Sửa tệp sau `git add` không tự cập nhật index; commit có thể chứa bản cũ trong khi working tree vẫn có sửa đổi mới.

Giả sử `timeout_ms` trong commit hiện tại là `1000`. Ta sửa working tree thành `2000`, stage, rồi sửa tiếp thành `3000`. Lúc ấy `HEAD` giữ `1000`, index giữ `2000`, working tree giữ `3000`. `git diff --cached` so `HEAD` với index, còn `git diff` mặc định so index với working tree. Nếu commit ngay, snapshot mới ghi `2000`; muốn ghi `3000` phải stage lại và kiểm tra diff. Đây là lý do `git status` và diff trước commit có giá trị hơn lời khẳng định "tôi đã lưu file". Tệp chưa được theo dõi chưa tự có trong index.

Bài tập P04.C01.L02.Q01

Commit `B` chứa `timeout_ms=1000`. Sau `git add` bản `2000`, người học sửa working tree thành `3000` nhưng chưa stage lại. `HEAD`, index và working tree lần lượt giữ gì? Commit ngay sẽ ghi giá trị nào và phép so sánh nào chứng minh điều đó?

Chương P04.C02 — Tích hợp và sửa lịch sử

Giải conflict mà vẫn giữ ý nghĩa thay đổi và lịch sử chia sẻ.

Bài P04.C02.L01 — Merge

Xét commit `B` có `timeout_ms=1000`. Nhánh `main` đi từ `B` tới `D`, đặt `timeout_ms=3000` để giảm timeout sai ở cổng vào. Nhánh `feature/retry` đi từ `B` tới `C`, đặt cùng dòng thành `timeout_ms=2000` để thử retry. Đồ thị trước tích hợp là `A → B → D` trên `main` và `B → C` trên `feature/retry`; `B` là tổ tiên chung gần nhất. Tên branch chỉ là ref có thể dịch chuyển; `D` và `C` là hai object commit vẫn còn nguyên khi ref đổi.

Nếu `main` vẫn ở `B`, tích hợp `C` có thể fast-forward: chỉ dịch ref `main` tới `C`, không tạo commit hai cha. Ở đây `main` đã tới `D`, nên fast-forward không đủ. Merge ba phía so snapshot ở `B`, `D` và `C`. Cả hai phía thay cùng dòng theo hai cách nên Git không thể chọn đúng ý định nghiệp vụ; nó dừng với conflict. Trước khi người vận hành giải quyết, `HEAD` vẫn là `D`, `MERGE_HEAD` ghi phía `C`, index có thể giữ ba stage cho tệp xung đột: stage 1 là bản ở `B`, stage 2 là phía `HEAD`/`D`, stage 3 là phía `MERGE_HEAD`/`C`. Dấu `<<<<<<<`, `=======`, `>>>>>>>` trong working tree chỉ là trình bày vùng chưa giải quyết, không phải cấu hình được phép đưa vào production.

Quyết định đúng không thể suy từ câu "lấy ours" hay "lấy theirs". Nhóm phải kiểm tra hợp đồng timeout và retry. Giả sử kiểm thử cho thấy upstream cần ngưỡng `2500` và retry chỉ dành cho thao tác idempotent. Người tích hợp viết `timeout_ms=2500`, cập nhật kiểm thử liên quan, rồi stage tệp đã giải quyết. Khi đó index có bản thường thay cho ba stage xung đột; `git diff --cached` cho thấy snapshot sắp ghi, và `git ls-files -u` không còn mục chưa giải quyết. Chạy kiểm thử cấu hình và hành vi retry trên kết quả cuối, xem diff để bảo đảm không còn marker; chỉ sau đó tạo merge commit `M` với hai cha theo thứ tự `D` và `C`. Ref `main` dịch tới `M`, còn ref `feature/retry` vẫn ở `C` nếu chưa xóa. Nếu chưa hiểu cách giải, `git merge --abort` có thể đưa merge về trước khi bắt đầu, nhưng với thay đổi chưa commit trước merge không nên mặc định rằng mọi thứ luôn khôi phục hoàn hảo; hãy bắt đầu từ working tree sạch.

Bài tập P04.C02.L01.Q01

Trên đồ thị `A → B → D` của `main` và `B → C` của feature, vì sao không thể fast-forward `main` tới `C` mà giữ `D`? Nếu giải conflict rồi merge thành `M`, nêu hai cha và trạng thái các branch ref. Index có gì khi conflict đang dở?

Bài P04.C02.L02 — Rebase

Thay vì merge, trên nhánh feature chưa chia sẻ có thể rebase `C` lên `D`: Git lấy thay đổi mà `C` đưa vào so với `B`, thử áp dụng lên `D`, rồi tạo commit mới `C′` có cha là `D`. Vì `D` đã sửa cùng dòng, rebase cũng có thể dừng để con người giải conflict. Sau khi giải và tiếp tục, ref feature trỏ tới `C′`; `C` cũ không bị sửa tại chỗ. Ngay cả khi snapshot cuối cùng giống kết quả merge, đồ thị khác: `A → B → D → C′` là một đường, trong khi merge tạo `M` có hai cha `D` và `C`. Giá trị băm của `C′` khác `C` vì nội dung commit, đặc biệt parent, khác.

Rebase thuận tiện để sắp xếp các commit của chính mình trước khi công bố. Nếu người khác đã dựa vào `C` trên một branch dùng chung, đẩy lịch sử rebase lên sẽ buộc ref từ xa rời đường cũ và làm bản sao của họ lệch nhau; cần phối hợp rõ trước mọi lần viết lại lịch sử đã chia sẻ. Với nhánh tình huống này đã có người cùng dùng, chọn merge để giữ cả `C` và `D` trong lịch sử công khai. Cả merge lẫn rebase chỉ xử lý cách ghép lịch sử và nội dung; không tự chứng minh bản `timeout_ms=2500` đúng. Chạy lại phép kiểm chứng trên snapshot tích hợp là điều bắt buộc.

Bài tập P04.C02.L02.Q01

Nếu rebase `C` lên `D` và giải cùng conflict, đồ thị mới và định danh commit feature thay đổi ra sao? Vì sao phương án này cần cân nhắc khác khi `C` đã được chia sẻ? Hai cách tích hợp có tự xác nhận timeout mới là đúng không?

Bài P04.C02.L03 — Phục hồi

Nếu một commit thường đã công bố gây lỗi, `git revert` tạo commit mới đảo tác động của commit được chọn trên trạng thái hiện tại. Commit lỗi vẫn có trong đồ thị, người khác có thể lấy commit đảo chiều bằng cách đồng bộ bình thường; revert cũng có thể xung đột nếu mã đã thay đổi thêm. Muốn revert chính merge commit `M` của ví dụ, phải chỉ định parent làm dòng chính: `git revert -m 1 M` xem `D` là mainline rồi đảo thay đổi mà merge đưa vào so với `D`. Cần xem diff và kiểm thử vì thao tác này không có nghĩa "quay toàn bộ repo về B", và về sau merge lại cùng nhánh có thể không tự mang lại các thay đổi đã bị đảo.

`git reset <commit>` ở dạng dịch `HEAD` đặt ref branch hiện tại về commit được chọn; chế độ `--soft`, mặc định `--mixed`, và `--hard` lần lượt ảnh hưởng khác nhau tới index và working tree. `--hard` còn có thể bỏ thay đổi local chưa commit ở các tệp được theo dõi. Dịch ref `main` từ `M` về `D` sẽ làm lịch sử branch công bố không còn đi qua `M`, dù object có thể còn tạm được tham chiếu ở nơi khác. Vì thế reset là công cụ sửa lịch sử cục bộ khi đã hiểu trạng thái; nó không thay revert trên nhánh đã chia sẻ. Khôi phục một tệp trong working tree hoặc bỏ stage là tác vụ khác với dịch branch ref; đừng dùng một lệnh dịch lịch sử để chữa nhầm một file đang sửa.

Khi nhận một conflict thật, trước hết xác định branch đang checkout, working tree có sạch trước khi tích hợp không, và hai commit đầu nhánh cùng merge base. Sau đó đọc ba phiên bản của tệp, xác định mỗi thay đổi phục vụ yêu cầu nào; conflict marker chỉ cho vị trí chồng lấn, không cho quyết định đúng. Kiểm tra diff đã stage và đồ thị cha sau commit, chạy test trên snapshot vừa tích hợp, rồi mới chia sẻ. Nếu test thất bại, phân biệt lỗi từ code đã commit với sửa đổi chỉ còn trong working tree; một `git status` sạch không thay thế kiểm thử. Nếu cần hủy ảnh hưởng trên branch chung, chuẩn bị một revert có review; nếu cần thử lại một merge đang dở, kiểm tra trạng thái trước khi abort. Sự an toàn đến từ việc biết ref, index và working tree nào đang thay đổi ở mỗi bước.

Bài tập P04.C02.L03.Q01

Merge commit `M` đã lên branch chung rồi gây lỗi. So sánh mục tiêu và hậu quả của revert `M` với reset `main` về `D`; vì sao revert merge cần chọn mainline? Cần kiểm chứng gì trước khi push?

PHẦN 05 — Tự động hóa hệ thống

Một chuỗi tác vụ báo đúng thất bại và có thể chạy lại thế nào?

Parser, argv, status, deadline và tác dụng phụ qua nhiều tiến trình là các hợp đồng khác nhau.

Chương P05.C01 — Bash và ngữ nghĩa lỗi của shell

Dự đoán argv, output và exit status trước khi chạy script.

Bài P05.C01.L01 — Parsing

Một script backup có thể in `BACKUP_OK` dù không tạo được bản sao dùng được. Lỗi không nhất thiết nằm ở lệnh sao lưu: shell có thể biến dữ liệu thành đối số khác, pipeline có thể che mã lỗi, và thao tác dọn dẹp có thể chạy sau một kết luận thành công sai. Chương này xem Bash như một chương trình điều phối tiến trình và file descriptor. Mọi đường dẫn và kết quả của tình huống bên dưới thuộc một thử nghiệm cô lập; chúng không mô tả trạng thái của lab hoặc máy production.

Bash đọc cú pháp trước: dấu nháy, toán tử `|`, `>`, `&&` và dấu kết thúc lệnh xác định cấu trúc. Sau đó shell mở rộng các word: biến, phép tính, command substitution, tách word và mở rộng tên tệp theo ngữ cảnh, rồi bỏ dấu nháy. Ký tự đặc biệt sinh ra từ giá trị biến không tự được phân tích lại thành toán tử shell; gọi `eval` để ép phân tích lần hai là mở một ranh giới thực thi mới. Dấu nháy kép giữ kết quả mở rộng biến thành một đối số, trừ những quy tắc riêng như `"$@"` giữ từng đối số ban đầu. Dấu nháy đơn giữ nguyên hầu hết ký tự bên trong, không mở rộng biến.

Giả sử `src='/tmp/du lieu'`. Với `tar -C $src`, Bash có thể đưa cho `tar` hai đối số `/tmp/du` và `lieu`; `tar -C "$src"` đưa đúng một đường dẫn. Nếu biến không được nháy chứa `*`, pathname expansion còn có thể thay một word thành nhiều tên file. Điều này xảy ra trước khi `tar` chạy; sửa tùy chọn của `tar` không chữa được lỗi truyền đối số. Phép gán đơn giản `src=$1` không tách word ở vế phải, nhưng dùng `$src` không nháy ở lời gọi sau đó vẫn nguy hiểm. Với tập tên file tùy ý, dùng mảng Bash và `"${files[@]}"`; không ghép danh sách thành một chuỗi rồi hy vọng shell tách lại đúng. `--` là quy ước được nhiều utility hỗ trợ để kết thúc phần tùy chọn, không phải phép bảo vệ do Bash tự áp cho mọi chương trình.

Một biến chưa đặt và một biến rỗng là hai trạng thái khác nhau. `set -u` báo lỗi khi mở rộng biến chưa đặt trong nhiều ngữ cảnh, nhưng không chứng minh đường dẫn tồn tại hoặc nội dung hợp lệ. Cú pháp `${1:?can tham so}` buộc có đối số thứ nhất và khác rỗng trước khi đi tiếp; nó vẫn không bảo đảm đối số ấy là một thư mục an toàn. Dữ liệu từ môi trường, tham số và file cấu hình phải đi qua kiểm tra theo hợp đồng của thao tác sắp làm.

Bài tập P05.C01.L01.Q01

Cho `src='/tmp/du lieu'`, giải thích số đối số đường dẫn mà `tar -C $src` và `tar -C "$src"` nhận sau mở rộng. Nếu tên file chứa `*`, vì sao chỉ thêm `set -u` không sửa lỗi? Nêu cách giữ từng phần tử của một danh sách tên file.

Bài P05.C01.L02 — I/O và status

Mỗi tiến trình nhận các file descriptor chuẩn: 0 để đọc, 1 để ghi thông thường, 2 để ghi lỗi. `cmd >file 2>&1` cho descriptor 1 trỏ tới file rồi nối descriptor 2 vào đích hiện tại của 1. Đảo thứ tự thành `cmd 2>&1 >file` khiến descriptor 2 giữ đích cũ của 1 trong khi 1 mới trỏ tới file. Bash thực hiện redirection trước khi thực thi lệnh; lỗi mở file có thể làm lệnh chính không chạy. Với pipeline, output của lệnh trái trở thành input của lệnh phải, và các phần thường chạy trong những môi trường tiến trình riêng. Đừng kỳ vọng biến gán bên trong một nhánh pipeline luôn trở lại shell cha; cấu hình `lastpipe` là một ngoại lệ có điều kiện.

Mặc định, exit status của `tar ... | tee log` là status của `tee`, không phải của `tar`. Nếu `tar` thoát 2 còn `tee` ghi log thành công và thoát 0, cả pipeline có status 0. `set -o pipefail` đổi status pipeline thành status khác 0 của lệnh lỗi nằm ngoài cùng bên phải; nó không nói lệnh nào hỏng nếu nhiều lệnh hỏng. Mảng `PIPESTATUS` trong Bash giữ status từng phần, nhưng phải sao chép ngay: chỉ cần chạy thêm một lệnh là trạng thái quan sát đã đổi. Status 0 nói rằng chương trình đã hoàn thành theo hợp đồng riêng của nó; không chứng minh bản backup khôi phục được. GNU `tar` phân biệt 0 thành công, 1 khi file thay đổi trong lúc tạo kho và 2 cho lỗi nghiêm trọng; `tee` thành công chỉ nói việc ghi dữ liệu nó nhận đã thành công theo hợp đồng của `tee`.

Bài tập P05.C01.L02.Q01

Trong script lỗi, giả sử `tar` trả 2 và `tee` trả 0. Dự đoán status pipeline, việc in `BACKUP_OK` và status cuối của script khi chưa bật `pipefail`. Bật `pipefail` đổi điều gì? Status 0 của `tar` sẽ và sẽ không chứng minh điều gì?

Bài P05.C01.L03 — Control flow

`set -e` yêu cầu shell thoát khi một lệnh đơn giản thất bại trong những ngữ cảnh nhất định. Nó bỏ qua lỗi ở phép thử `if`/`while`, ở phần không cuối của chuỗi `&&`/`||`, ở lệnh không cuối của pipeline khi không bật `pipefail`, và ở một số ngữ cảnh phủ định. Nếu một hàm được gọi làm điều kiện `if`, các lệnh bên trong cũng chịu ngữ cảnh bỏ qua này: `false` có thể không chặn lệnh sau. Đây là quy tắc điều khiển của Bash, không phải một ngoại lệ của utility. `set -euo pipefail` có ích để bắt một số sơ suất, nhưng không định nghĩa thế nào là dữ liệu hợp lệ hoặc việc khôi phục thành công.

Command substitution `result=$(cmd)` chạy `cmd` trong môi trường subshell và bỏ các newline ở cuối output; status của phép gán đơn giản phản ánh substitution cuối. Bash không ở POSIX mode thường tắt `errexit` trong môi trường substitution, trừ khi bật `inherit_errexit`; vì vậy các lệnh bên trong cần xử lý status rõ. Một bẫy khác là `local result=$(cmd)`: status người gọi thấy có thể là của builtin `local`, che status của `cmd`. Tách `local result` và `result=$(cmd) || ...` khi kết quả quan trọng. Không nhét một thao tác có side effect vào `echo "$(...)"` rồi chỉ kiểm tra status của `echo`.

Bash khởi tạo tiến trình con cho utility bên ngoài, chờ status và quyết định nhánh tiếp theo. `trap ... EXIT` chạy khi shell thoát bình thường hoặc do lỗi khiến shell thoát; `trap ... INT` và `TERM` xử lý tín hiệu tương ứng nếu shell có cơ hội nhận và xử lý chúng. `SIGKILL` không bắt được. Khi

Bash đang chờ một lệnh đồng bộ ở foreground, trap có thể bị hoãn tới lúc lệnh đó kết thúc. Một trap trên shell cha không tự hủy toàn bộ cây tiến trình con. Vì vậy trap phù hợp để dọn file tạm đã biết, nhưng không thay cơ chế quản lý process group, timeout và shutdown của trình quản lý dịch vụ.

Bài tập P05.C01.L03.Q01

Xét `f() { false; printf 'done\n'; }` rồi `if f; then printf 'ok\n'; fi` với `set -e`. Dự đoán các dòng in ra theo Bash và giải thích vì sao `false` không tự làm script thoát. Viết lại ý định "dừng nếu bước đầu thất bại" mà không dựa vào `set -e`.

Chương P05.C02 — Python và Go điều phối tiến trình

Đọc/viết CLI DevOps nhỏ bằng Python hoặc Go, xử lý argv, lỗi, timeout và test nhánh.

Bài P05.C02.L01 — Chọn ngôn ngữ

Chương 7 cho thấy một pipeline Bash có thể che mã lỗi của bước sao lưu. Chuyển cùng việc ấy sang Python hoặc Go không tự sửa sai: chương trình vẫn có thể nhận lỗi rồi bỏ qua và in thông báo thành công. Khác biệt hữu ích nằm ở cách biểu diễn đối số, kết quả, thời hạn và trạng thái của từng bước. Ta sẽ điều phối ba bước nhỏ: kiểm tra điều kiện, áp dụng thay đổi giả, xác minh kết quả. Không bước nào chạm hệ thống thật; helper chỉ ghi vào thư mục tạm được đánh dấu cho demo.

Bash phù hợp cho vài lệnh tuyến tính trên một máy, với đầu vào nhỏ và hợp đồng lỗi rõ. Khi phải giữ trạng thái qua nhiều bước, đọc cấu hình có cấu trúc, kiểm tra dữ liệu, phân loại lỗi và kiểm thử nhánh phục hồi, Python thường cho mã dễ đọc hơn. Thư viện chuẩn của Python đã có subprocess, xử lý tệp, JSON và kiểm thử; đổi ngôn ngữ chỉ để gọi một lệnh đơn lẻ không mang lại an toàn. Go phù hợp khi cần một chương trình biên dịch triển khai trên nhiều máy, kiểu dữ liệu và luồng lỗi tường minh, hoặc nhiều tác vụ I/O đồng thời với cancellation chung. Một binary Go vẫn phụ thuộc hệ điều hành, kiến trúc, tùy chọn build và có thể phụ thuộc thư viện động; "một file" không có nghĩa chạy được ở mọi nơi. Python cần interpreter và môi trường package tương thích. Quyết định dựa trên độ phức tạp trạng thái, môi trường triển khai, năng lực nhóm và chi phí vận hành, không dựa trên nhãn "ngôn ngữ DevOps".

Trong cả ba ngôn ngữ, chương trình điều phối không đồng nhất với tác vụ nó khởi chạy. Một lệnh con có thể chạy xong và trả status khác 0; không khởi chạy được; bị timeout; hoặc trả 0 nhưng kết quả nghiệp vụ sai. Từng trạng thái cần chính sách riêng. Exit status là quan sát từ tiến trình con, còn "triển khai thành công" là kết luận sau khi kiểm tra hậu điều kiện. Không ghép hai ý ấy thành một biến boolean duy nhất.

Hai ví dụ chỉ dùng thư viện chuẩn. Với Python, ghi rõ phiên bản interpreter, tạo virtual environment cho package của dự án và khóa dependency theo công cụ đóng gói đã chọn; virtual environment là thứ tái tạo được, không nên coi thư mục của nó là artifact có thể di chuyển nguyên vẹn giữa máy. Với Go, module ghi yêu cầu ở `go.mod`, checksum dependency được quản lý ở `go.sum`; cần ghi cả phiên bản toolchain, target OS/architecture và cờ build khi muốn tái tạo binary. Không đưa secret vào package hoặc Git. Test nên chạy trong thư mục tạm có dấu hiệu nhận diện, xác minh cả status lẫn trạng thái cuối, rồi xóa đúng thư mục ấy sau khi đối chiếu. Với thay đổi thật, thêm kiểm thử hậu điều kiện từ góc nhìn người dùng, quan sát rollback và quyền thao tác; kết quả demo chỉ chứng minh các nhánh được thử trong môi trường thử.

Bài P05.C02.L02 — Subprocess

Python `subprocess.run` nhận danh sách đối số; Go `exec.CommandContext` nhận tên chương trình cùng từng đối số. Hai API này mặc định không gọi shell. Giá trị như `"a; rm -rf ..."` đi thành một đối số, không thành lệnh thứ hai. Điều đó chặn một kiểu shell injection tại ranh giới này, nhưng utility con vẫn có thể hiểu dấu gạch đầu như tùy chọn, đọc file cấu hình độc hại hoặc tự gọi shell. Nếu cố dùng `shell=True` hay `sh -c` với chuỗi ghép từ đầu vào, ranh giới parsing của Chương 7 trở lại. Dùng argv tách rời, xác nhận giá trị theo schema và hợp đồng của chương trình con, chọn đường dẫn executable đáng tin, rồi dùng dấu kết thúc tùy chọn khi utility hỗ trợ.

Đường dẫn làm việc, `PATH`, biến môi trường, locale, quyền và tài nguyên của tiến trình con ảnh hưởng kết quả. Python `env=None` và Go `Cmd.Env=nil` cho con kế thừa môi trường hiện tại; điều này tiện trong demo nhưng có thể chuyển cả credential hoặc cấu hình ngoài dự kiến. Bí mật trong argv có thể lộ qua danh sách tiến trình hoặc log; bí mật trong environment cũng không phải két an toàn tuyệt đối. Đừng in nguyên command, exception, environment hoặc output khi chúng có thể chứa secret. Chỉ ghi mã bước, loại lỗi và request ID đã lọc; cấp secret qua cơ chế quản lý riêng với quyền tối thiểu. Trong ví dụ, output con bị bỏ đi vì ta đã biết helper nhỏ và chỉ cần status; công cụ thật cần log giới hạn kích thước, che dữ liệu nhạy cảm và đủ chứng cứ chẩn đoán.

Helper sau là tác vụ giả. Nó chỉ hoạt động khi thư mục đích có file `.demo-only`. Sau khi khởi tạo, nó phát `READY` rồi chờ lệnh `go`: nhờ vậy hạn tác vụ chỉ bắt đầu khi cả hai phía đã sẵn sàng. Biến `DEMO_STARTUP_DELAY` chỉ dùng trong kiểm thử để mô phỏng máy khởi tạo chậm. `apply_fail` ghi `partial` rồi trả 7. `apply_slow` ghi `partial`, ngủ hai giây, rồi mới định ghi `ready`. `verify` chỉ trả 0 nếu trạng thái là `ready`. Như vậy một lỗi hoặc timeout có thể để lại dấu vết của tác dụng phụ trước khi tiến trình con dừng. Không suy từ timeout rằng không có gì đã xảy ra.

```
"""Demo child: only write inside a marked temporary directory."""
import os
from pathlib import Path
import sys
import time

if len(sys.argv) != 3:
    sys.exit(2)
action, work_arg = sys.argv[1:]
work = Path(work_arg)
if not (work / ".demo-only").is_file():
    sys.exit(2)
state = work / "state.txt"
time.sleep(float(os.environ.get("DEMO_STARTUP_DELAY", "0")))
print("READY", flush=True)
if sys.stdin.readline() != "go\n":
    sys.exit(2)
if action == "precheck":
    sys.exit(0)
if action == "apply_ok":
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action in ("apply_fail", "apply_slow"):
    state.write_text("partial\n", encoding="utf-8")
    if action == "apply_fail":
        sys.exit(7)
    time.sleep(2)
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action == "verify":
    sys.exit(0 if state.is_file() and
             state.read_text(encoding="utf-8") == "ready\n" else 8)
sys.exit(2)
```

Chương trình Python gọi cùng helper theo thứ tự `precheck → apply → verify`. Với lệnh đơn giản, `subprocess.run(check=False)` trả `CompletedProcess` khi con kết thúc, kể cả status 7 hoặc 8; người gọi phải đọc `returncode`. Ở đây ta dùng `Popen` để đợi `READY` riêng, rồi mới gửi `go` và gọi `communicate(timeout=0.5)`. Quá hạn tác vụ, chương trình kill và reap con trực tiếp; lỗi khởi chạy hoặc mất tín hiệu sẵn sàng được gắn nhãn khác với lỗi bước. Nhãn `TIMEOUT` không phải exit status của helper. Nhánh `bad` ghi nhận lỗi nhưng vẫn chạy bước sau và in `DEPLOY_OK`; nhánh `checked` dừng khác 0 ngay ở lỗi đầu. Cả hai nhánh cố ý để lại `partial` để người học thấy kết luận sai và giới hạn của việc dừng sớm.

```
"""Demo only: 'bad' deliberately ignores a failed step."""
import queue
import subprocess
import sys
import threading

STARTUP_LIMIT = 10  # Chỉ chờ tín hiệu READY, không tính vào hạn tác vụ.
TASK_LIMIT = 0.5


def run_step(probe, work, action):
    try:
        child = subprocess.Popen(
            [sys.executable, probe, action, work],
            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
        )
    except OSError:
        return "START_ERROR"
    ready = queue.Queue(maxsize=1)
    threading.Thread(target=lambda: ready.put(child.stdout.readline()),
                     daemon=True).start()
    try:
        line = ready.get(timeout=STARTUP_LIMIT)
    except queue.Empty:
        child.kill()
        child.wait()
        return "START_TIMEOUT"
    if line.strip() != b"READY":
        child.kill()
        child.wait()
        return "START_ERROR"
    try:
        child.communicate(input=b"go\n", timeout=TASK_LIMIT)
    except subprocess.TimeoutExpired:
        child.kill()
        child.communicate()
        return "TIMEOUT"
    except OSError:
        child.kill()
        child.wait()
        return "IO_ERROR"

    return "OK" if child.returncode == 0 else f"EXIT {child.returncode}"

if len(sys.argv) != 5:
    sys.exit("usage: script.py bad|checked ok|fail|slow probe.py workdir")
policy, mode, probe, work = sys.argv[1:]
if policy not in ("bad", "checked") or mode not in ("ok", "fail", "slow"):
    sys.exit(2)

for action in ("precheck", "apply_" + mode, "verify"):
    status = run_step(probe, work, action)
    if status != "OK":
        print(f"{action}: {status}", file=sys.stderr)
        if policy == "checked":
            sys.exit(1)
print("DEPLOY_OK")
```

```
package main

import (
 "bufio"
 "context"
 "errors"
 "fmt"
 "io"
 "os/exec"
 "strings"
 "time"
)

func runStep(python, probe, work, action string) string {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()
 cmd := exec.CommandContext(ctx, python, probe, action, work)
 input, err := cmd.StdinPipe()
 if err != nil {
  return "START_ERROR"
 }
 output, err := cmd.StdoutPipe()
 if err != nil {
  return "START_ERROR"
 }
 cmd.Stderr = io.Discard
 if err = cmd.Start(); err != nil {
  return "START_ERROR"
 }
 ready := make(chan string, 1)
 go func() {
  line, _ := bufio.NewReader(output).ReadString('\n')
  ready <- line
 }()
 select {
 case line := <-ready:
  if strings.TrimSpace(line) != "READY" {
   cancel()
   cmd.Wait()
   return "START_ERROR"
  }
 case <-time.After(10 * time.Second):
```

```
 cancel()
 cmd.Wait()
 return "START_TIMEOUT"
}
if _, err = io.WriteString(input, "go\n"); err != nil {
 cancel()
 cmd.Wait()
 return "IO_ERROR"
}
input.Close()
done := make(chan error, 1)
go func() { done <- cmd.Wait() }()
select {
case err = <-done:
 if err == nil {
  return "OK"
 }
 var exit *exec.ExitError
 if errors.As(err, &exit) {
  return fmt.Sprintf("EXIT %d", exit.ExitCode())
 }
 return "IO_ERROR"
case <-time.After(500 * time.Millisecond):
 cancel()
 <-done
 return "TIMEOUT"
}
}
```

Với API ngắn hơn, `Run` trả `nil` khi lệnh chạy và kết thúc thành công; ở đây `Wait` có cùng ranh giới lỗi sau `Start`.

Bài tập P05.C02.L02.Q01

Với `orchestrate_python.py bad fail`, dự đoán thứ tự các bước, status của helper ở từng bước, output cuối và nội dung `state.txt`. Sau `Popen` và `communicate`, chương trình lấy kết quả thực thi từ đâu? Vì sao nhánh `bad` vẫn báo thành công dù đã nhận kết quả lỗi, còn nhánh `checked` dừng?

Bài tập P05.C02.L02.Q02

Một tên môi trường do người dùng nhập là `prod; echo LEAK`. So sánh truyền nó như một phần tử argv với ghép vào chuỗi `sh -c`. Vì sao argv tách rời vẫn phải kiểm tra schema và quy tắc tùy chọn của utility? Nêu một nơi có thể làm lộ secret dù không dùng shell.

Bài P05.C02.L03 — Timeout

Go dùng `exec.CommandContext` nhưng chưa hủy context trong giai đoạn đợi `READY`. Sau tín hiệu đó, hàm gửi `go`, bắt đầu bộ đếm 0,5 giây và đợi `Wait`; nếu hết hạn, nó hủy context để kill con trực tiếp rồi reap. Với API ngắn hơn, `Run` trả `nil` khi lệnh chạy và kết thúc thành công; ở đây `Wait` có cùng ranh giới lỗi sau `Start`. Status khác 0 thường cho `*exec.ExitError`, còn lỗi tìm executable hoặc I/O là loại lỗi khác. Mã phải kiểm tra kết quả, không chỉ ghi log rồi bỏ qua. Nhánh `bad` dưới đây mắc đúng lỗi quyết định đó; nhánh `checked` dừng. Output con ngoài tín hiệu đồng bộ bị bỏ khỏi log demo, không phải mẫu quan sát phù hợp cho mọi dịch vụ.

```
// Demo only: policy "bad" deliberately ignores a failed step.
package main

import (
 "fmt"
 "os"
)

func main() {
 if len(os.Args) != 6 {
  fmt.Fprintln(os.Stderr, "usage: app bad|checked ok|fail|slow python probe workdir")
  os.Exit(2)
 }
 policy, mode := os.Args[1], os.Args[2]
 if (policy != "bad" && policy != "checked") ||
  (mode != "ok" && mode != "fail" && mode != "slow") {
  os.Exit(2)
 }
 python, probe, work := os.Args[3], os.Args[4], os.Args[5]
 for _, action := range []string{"precheck", "apply_" + mode, "verify"} {
  status := runStep(python, probe, work, action)
  if status != "OK" {
   fmt.Fprintf(os.Stderr, "%s: %s\n", action, status)
   if policy == "checked" {
    os.Exit(1)
   }
  }
 }
 fmt.Println("DEPLOY_OK")
}
```

Python `run(timeout=...)` bao cả thời gian giao tiếp với con sau khi nó được tạo, nhưng khởi tạo tiến trình trên một số nền tảng có thể chưa bị ngắt ngay đúng deadline. Go `CommandContext` gắn context cho command và mặc định kill process khi context kết thúc; nó không tự rollback file đã ghi. Cả hai ví dụ chỉ quản lý tiến trình con trực tiếp. Nếu con khởi chạy cháu, giữ pipe mở hoặc gọi một API từ xa đã nhận yêu cầu, hết hạn ở phía điều phối không chứng minh mọi tác vụ bên ngoài đã dừng. Với Go, việc chờ pipe đóng và cách hủy có thể cần `WaitDelay` hoặc quy ước shutdown riêng; với Python, một process group có thể cần quản lý riêng. Không mặc định kill toàn bộ cây tiến trình bằng một lời gọi.

Sau lỗi, phải xác định trạng thái bền vững hiện tại rồi mới quyết định thử lại, tiếp tục hay quay lui. Retry một thao tác ghi có thể nhân đôi tác dụng nếu lần đầu đã hoàn tất ở hệ thống đích nhưng phản hồi bị mất, như tình huống HTTP ở Chương 5. Chỉ retry khi thao tác idempotent theo hợp đồng, hoặc có khóa chống lặp và khả năng truy kết quả cũ; dùng số lần và tổng thời gian hữu hạn. `finally` của Python và `defer` của Go phù hợp giải phóng tài nguyên cục bộ, nhưng không thay giao dịch hay rollback phân tán; `os.Exit` trong Go còn bỏ qua các `defer` đang chờ ở cùng tiến trình. Việc xóa file `partial` mù quáng có thể che trạng thái cần điều tra hoặc xóa công việc hợp lệ của lượt khác. Demo dừng và giữ dấu vết, không tự tuyên bố đã khôi phục.

Ở đây ta dùng `Popen` để đợi `READY` riêng, rồi mới gửi `go` và gọi `communicate(timeout=0.5)`.

Bài tập P05.C02.L03.Q01

Với bản Go `checked slow`, hãy lần theo `READY`, `go`, lúc bắt đầu bộ đếm và kết quả `Wait`. Vì sao `TIMEOUT` khác exit status của helper, còn file vẫn chứa `partial` dù điều phối thoát 1? Nêu điều gì không thể kết luận về tiến trình cháu hoặc dịch vụ từ xa.

Chương P05.C03 — Workflow an toàn và chạy lại

Thiết kế tác vụ nhiều bước có nhận diện partial failure.

Bài P05.C03.L01 — Hợp đồng bước

Ví dụ lỗi dưới đây nhận thư mục nguồn và thư mục đích đã tồn tại. Nó được thiết kế để quan sát lỗi status, không có lệnh xóa nguồn. Không dùng nó cho backup thật.

```
#!/usr/bin/env bash
# Ví dụ lỗi: chỉ chạy trên dữ liệu thử nghiệm.
set -e
src=$1
out=$2
tar -cf "$out/backup.tar" -C "$src" . 2>&1 | tee "$out/backup.log"
printf 'BACKUP_OK\n'
```

Trong thử nghiệm trên Bash 5.2.21, GNU tar 1.35 và GNU coreutils 9.4, nguồn đầu vào là một thư mục không tồn tại, còn thư mục đích là thư mục tạm rỗng. `tar` không thể đổi sang nguồn và thoát 2; `tee` ghi thông báo lỗi vào log rồi thoát 0. Vì không bật `pipefail`, status pipeline bằng 0; `set -e` không chặn `printf`, script in `BACKUP_OK` và thoát 0. Dòng `BACKUP_OK` vì thế là kết luận của shell dựa trên status bị che, không phải chứng cứ backup tốt. Trên hệ thống khác, nội dung và ngôn ngữ thông báo lỗi, cũng như file tar dở dang, có thể khác. Trong một script có bước xóa nguồn phụ thuộc status này, hậu quả có thể là mất dữ liệu; ví dụ thực thi ở đây cố ý không có bước ấy.

Hàm chạy từng bước tách rời chính sách `bad/checked`: nó chỉ trả loại kết quả quan sát được. Cờ `TIMEOUT` do bộ đếm của cha xác định; lỗi trả về từ `Wait` sau khi kill không phải exit status nghiệp vụ của helper.

Ma trận kiểm thử có kiểm soát phải đối chiếu cả status lẫn file trạng thái: `ok` in `DEPLOY_OK`, thoát 0 và có `ready`; `bad fail` in `DEPLOY_OK` dù `apply` trả 7, `verify` trả 8 và file còn `partial`; `checked fail` thoát 1 trước `verify`; `checked slow` thoát 1 vì timeout sau khi đã ghi `partial`. `bad slow` vẫn in thành công giả. Ca `checked ok` còn ép khởi tạo helper chậm hơn hạn cũ 0,3 giây để chứng minh đồng hồ tác vụ chưa chạy trong lúc khởi tạo. Đây là phép thử của demo, không phải chứng nhận cho quy trình triển khai thật.

Bài P05.C03.L02 — Retry

Khi script chạy lại, phải định nghĩa rõ trạng thái nào có thể tái sử dụng. Việc `backup.tar` đã tồn tại không chứng minh nó đầy đủ; việc bản mới được tạo cũng không có nghĩa được phép xóa bản cũ. Một cách an toàn hơn là ghi vào file tạm trong chính thư mục đích, kiểm tra thành công của lệnh tạo, rồi công bố bằng thao tác không ghi đè tên cuối. Điều này xử lý xung đột tên, nhưng không giải quyết nguồn thay đổi trong lúc đọc, mất điện trước khi dữ liệu được đồng bộ, hay đối thủ đổi đường dẫn giữa kiểm tra và sử dụng. Quyền truy cập thư mục, snapshot nhất quán, kiểm thử khôi phục và chính sách giữ bản cũ là những hợp đồng riêng.

Bài tập P05.C03.L02.Q01

Sau timeout, người vận hành định retry vô hạn bước ghi và xóa `partial` trước mỗi lần. Phân tích rủi ro nhân đôi, mất chứng cứ và chạy đồng thời; đề xuất điều kiện tối thiểu để thử lại, cùng phép kiểm chứng hậu điều kiện. Khi nào Bash vẫn là lựa chọn hợp lý cho một tác vụ tương tự?

Bài P05.C03.L03 — Phục hồi

Phiên bản sau giữ từng đường lỗi ở vị trí có thể kiểm tra. Nó yêu cầu GNU `mktemp`, `tar`, `ln`, `rm`; tên cuối cố định và không cho ghi đè. `mktemp` tạo file riêng trong thư mục đích để tránh va chạm tên file tạm. Mọi nhánh trước `ln` đều thoát khác 0 và `EXIT` trap chỉ dọn đúng đường dẫn tạm đã tạo. `tar -tf` kiểm tra archive đọc được về mặt cấu trúc, không chứng minh dữ liệu có thể phục hồi đúng nghiệp vụ. `ln -T` tạo hard link tới file tạm trên cùng filesystem và buộc đối số đích được hiểu là đúng một tên file, kể cả khi tên ấy là symlink tới thư mục; nếu `backup.tar` đã tồn tại, GNU `ln` mặc định thất bại, nên lần chạy lại không âm thầm thay bản trước. Thành công của `ln` là điểm công bố tên, sau đó trap gỡ tên tạm và để hard link cuối giữ inode. Đích phải là thư mục tin cậy; một cây đường dẫn bị thay đổi đồng thời hoặc do đối thủ kiểm soát cần thiết kế an toàn hơn.

```
#!/usr/bin/env bash
# GNU tar/coreutils; đích phải là thư mục tin cậy cùng filesystem.
set -u
src=${1:?can thu muc nguon}
out=${2:?can thu muc dich}
[[ -d $src && -d $out ]] || exit 2

tmp=$(mktemp -- "$out/.backup.XXXXXX") || exit 3
cleanup() { rm -f -- "$tmp"; }
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM

if tar -cf "$tmp" -C "$src" .; then
    :
else
    exit $?
fi
if tar -tf "$tmp" >/dev/null; then
    :
else
    exit 4
fi
ln -T -- "$tmp" "$out/backup.tar" || exit 5
printf 'BACKUP_OK\n'
```

Ví dụ kiểm chứng cô lập tạo một file `payload.txt` trong thư mục nguồn tạm, chạy script sửa lần đầu, rồi liệt kê archive và trích xuất sang một thư mục tạm khác để so nội dung. Lần thứ hai trả status 5 vì tên cuối đã tồn tại; bản cuối của lần đầu vẫn đọc được. Khi thay nguồn bằng đường dẫn không tồn tại, script dừng status 2 trước khi tạo file tạm. Những kết quả này kiểm tra đường thành công, lỗi đầu vào và xung đột tên cho môi trường thử; chúng không chứng minh tính bền vững sau mất điện, tính nhất quán của dữ liệu đang bị ghi, hay an toàn trước thay đổi đường dẫn có chủ ý. Một

backup production cần snapshot hoặc điểm dừng ghi phù hợp ứng dụng, checksum/manifest, thử khôi phục định kỳ, chính sách retention và đồng bộ tệp/thư mục theo hệ thống tệp thực tế. Bài học nối với Chương 2 về I/O bền vững, Chương 3 về đường dẫn/quyền và Chương 6 về khác biệt giữa một snapshot đã ghi với working tree đang đổi.

Bài tập P05.C03.L03.Q01

Trong script sửa, sau một lần thành công, lần chạy lại với cùng đích đi tới bước nào và status nào? Vì sao `tar -tf` cùng việc không ghi đè chưa đủ cho backup production? Phân biệt lỗi được `EXIT` trap dọn với rủi ro do `SIGKILL` hoặc nguồn thay đổi giữa chừng.

PHẦN 06 — Build, kiểm thử và phát hành

Một commit trở thành artifact đang phục vụ người dùng bằng chứng cứ nào?

Source, build inputs, artifact digest, test gate, deployment state và dữ liệu là các vòng đời tách biệt.

Chương P06.C01 — Build và danh tính artifact

Chứng minh production chạy đúng byte đã qua kiểm thử.

Bài P06.C01.L01 — Đầu vào build

Chương 6 cho ta một commit xác định mã nguồn; Chương 8 cho thấy chương trình điều phối có thể báo thành công dù hệ thống còn ở trạng thái từng phần. Phát hành một dịch vụ nối hai điều ấy bằng một chuỗi quyết định có chứng cứ: chọn commit, tạo artifact, kiểm thử chính artifact ấy, cho phép triển khai, chuyển lưu lượng, kiểm tra hành vi thật và chuẩn bị đường phục hồi. Mỗi mũi tên có thể thất bại độc lập. Một pipeline màu xanh không thay thế được câu hỏi người dùng đang nhận phiên bản nào và dữ liệu đã đổi ra sao.

Commit định danh một trạng thái của repository, không định danh duy nhất binary hay image. Hai lần build từ cùng commit có thể khác byte vì dependency được giải lại, toolchain khác, timestamp, thứ tự đóng gói hoặc cờ build khác. Muốn giảm biến thiên phải khóa đầu vào, phiên bản compiler, base image, cấu hình build và môi trường; nếu đòi tái lập từng byte, phải đo bằng cách build độc lập rồi so digest, thay vì suy từ một file lock. Tính tái lập là thuộc tính của toàn quá trình và những đầu vào được kiểm soát, không phải lời hứa của tên phiên bản.

Với Go, module ghi yêu cầu ở `go.mod`, checksum dependency được quản lý ở `go.sum`; cần ghi cả phiên bản toolchain, target OS/architecture và cờ build khi muốn tái tạo binary.

Bài P06.C01.L02 — Artifact

Artifact là tập byte đã được đóng gói để chạy. Digest tính từ byte cho phép kiểm tra ta đang dùng đúng nội dung đã kiểm thử; nhãn như `orders:1.8.4` chỉ là tên có thể bị trỏ lại nếu kho cho phép. Với image, descriptor chứa digest và kích thước để người nhận xác thực nội dung. Bản ghi provenance nối subject artifact với nguồn và cách build; nó giúp điều tra và kiểm tra chính sách, nhưng chỉ đáng tin trong phạm vi builder và quy trình tạo chứng cứ được tin cậy. Giữ riêng ba danh tính: commit nguồn, digest artifact và bản ghi triển khai ở môi trường. Khi quảng bá từ thử nghiệm sang production, dùng lại cùng digest; build lại từ cùng commit là một artifact mới, cần xét lại bằng chứng kiểm thử tương ứng.

Bài tập P06.C01.L02.Q01

Một pipeline build lại từ cùng commit khi đưa từ staging sang production, rồi chỉ so tên tag. Hãy chỉ ra mắt xích chứng cứ bị đứt và thiết kế bản ghi tối thiểu để chứng minh production chạy đúng artifact đã kiểm thử. Trường hợp nào hai lần build cùng commit có thể khác byte?

Chương P06.C02 — Test gates và CI/CD

Thiết kế test kiểm đúng ranh giới của thay đổi và quyền release.

Bài P06.C02.L01 — CI vs CD

CI tích hợp thay đổi bằng build, kiểm tra và phản hồi sớm cho mỗi commit hoặc đề nghị hợp nhất. CD đưa một artifact đã qua cổng tới một hay nhiều môi trường; continuous delivery có thể dừng ở phê duyệt trước production, còn continuous deployment tự đi tiếp khi chính sách cho phép. Build tạo đầu ra; release là quyết định gắn đầu ra ấy với một phiên bản được phép dùng; deployment thay đổi cái đang chạy. Ba sự kiện có thể cách nhau nhiều ngày. Phân biệt chúng giúp biết chính xác cổng nào đã kiểm tra byte nào.

Môi trường triển khai có schema dữ liệu, cấu hình, quyền, secret, quota và lưu lượng riêng.

Bài P06.C02.L02 — Test model

Một bộ test chỉ bảo đảm những đường mà nó thực sự chạy với dữ kiện cụ thể. Trong tình huống `orders-api`, unit test và readiness đều xanh, nhưng một thay đổi làm `amount_cents` sai. Unit test không có trường hợp giá trị gây lỗi; readiness chỉ gọi đường probe. Hai phép kiểm này vì thế không kiểm tra hậu điều kiện quan trọng: số tiền của đơn đã ghi có bằng giá trị đúng theo input và quy tắc nghiệp vụ hay không.

Thiết kế cổng kiểm thử bắt đầu từ hợp đồng dữ liệu. Với một fixture có input, kết quả mong đợi và định danh request, kiểm tra riêng phép tính, API và bản ghi sau khi giao dịch hoàn tất. Nếu dùng môi trường tích hợp, phải biết schema, cấu hình và dependency nào tham gia; một test chạy trên mock không chứng minh database thật có cùng hành vi. Khi canary nhận một phần lưu lượng, gửi phép thử qua đúng đường client, gắn nó với release nhận request và đối chiếu dữ liệu đã lưu. Tách kết quả của phép thử chủ động khỏi tỷ lệ lỗi trong traffic tự nhiên.

Một mẫu canary nhỏ có thể không gặp input gây lỗi, dù tất cả phép kiểm vừa chạy đều xanh. Nếu một request sai được phát hiện, dừng mở rộng, giữ request ID và trạng thái dữ liệu để điều tra. Kết luận phải nêu phạm vi: điều gì đã được test, điều gì chưa được test, và bằng chứng nào cần thêm trước khi tăng lưu lượng.

Bài tập P06.C02.L02.Q01

Trong tình huống `orders-api`, unit test và readiness đều xanh. Vì sao hai tín hiệu ấy không bác bỏ lỗi `amount_cents`? Nêu phép kiểm tra hậu triển khai phát hiện sai lệch và giới hạn của mẫu canary nhỏ.

Bài P06.C02.L03 — Gate

Các job là đồ thị phụ thuộc: kiểm tra tĩnh và unit test có thể chạy song song, nhưng kiểm thử tích hợp cần binary, database và cấu hình đã chuẩn bị; triển khai chỉ được dùng artifact mà các cổng đã kiểm tra. Unit test kiểm tra logic hẹp; integration test kiểm tra hợp đồng với thành phần thật hoặc mô phỏng có giới hạn; smoke test sau triển khai chỉ thử vài đường trọng yếu. Một test pass chứng minh những trường hợp đã chạy, không chứng minh không có lỗi trên dữ liệu production. Ghi input, phiên bản, kết quả và lý do bỏ qua cho từng cổng. Nếu thay artifact hoặc cấu hình có ảnh hưởng, không gắn nhãn "đã kiểm thử" từ lượt khác.

Môi trường triển khai có schema dữ liệu, cấu hình, quyền, secret, quota và lưu lượng riêng. Secret được cấp ở môi trường cần dùng và không nằm trong source, artifact hay log. Cổng phê duyệt production cần cho người duyệt thấy digest, khác biệt cấu hình, kết quả test, kế hoạch dừng và phương án phục hồi; nút bấm phê duyệt không tự kiểm tra những điều đó. Hai bản phát hành chạy đồng thời vào cùng môi trường có thể ghi đè nhau: dùng một khóa điều phối theo đích, xác nhận phiên bản hiện tại trước khi thay và định nghĩa rõ yêu cầu mới sẽ chờ hay hủy lượt cũ. Khóa pipeline không thay thế khóa ở hệ thống đích nếu còn công cụ triển khai khác.

Chương P06.C03 — Rollout và quyết định dừng

Chọn rolling, blue-green hay canary theo năng lực và khả năng quan sát.

Bài P06.C03.L01 — Chiến lược

Rolling update thay dần instance cũ bằng mới, điều chỉnh số instance có thể thiếu và số được tăng thêm. Nó tiết kiệm tài nguyên hơn việc giữ hai cụm đầy đủ, nhưng cũ và mới cùng phục vụ trong một khoảng; schema, thông điệp và cache phải tương thích hai chiều trong thời gian đó. Blue-green chuẩn bị một tập mới rồi đổi tuyến lưu lượng; chuyển lại tuyến có thể nhanh nếu tập cũ còn khỏe, đổi lại phải trả thêm tài nguyên và giữ đồng bộ dữ liệu dùng chung. Canary gửi một phần lưu lượng tới bản mới, so cùng thời điểm với nhóm cũ rồi tăng dần. Nó giới hạn phạm vi ban đầu, nhưng mẫu nhỏ hoặc lưu lượng không đại diện có thể bỏ sót lỗi; sự kiện hiếm vẫn có thể gây hại ngay ở 1%. Chiến lược nào cũng cần xác định lưu lượng nền, phiên dài, queue, job nền và thao tác ghi không đi qua bộ cân bằng tải.

Rollback ứng dụng là đưa lưu lượng hoặc cấu hình chạy về artifact trước đó.

Bài tập P06.C03.L01.Q01

So sánh rolling, blue-green và canary cho dịch vụ vừa nêu khi chỉ có đủ tài nguyên cho thêm một Pod. Chiến lược nào thực tế hơn, điều kiện tương thích dữ liệu nào phải giữ, và khi nào cần dừng dù tỷ lệ lỗi tổng thể chưa tăng rõ?

Bài P06.C03.L02 — Readiness

Một Pod hoặc process đã chạy chỉ chứng minh scheduler và runtime khởi động được nó. Readiness phải phản ánh khả năng phục vụ đường yêu cầu cần thiết; một probe chỉ trả 200 từ bộ nhớ có thể xanh dù kết nối database hỏng. Trong Kubernetes, Pod chưa ready không nhận lưu lượng qua Service tương ứng, nhưng client đi đường khác hoặc yêu cầu đã bay tới vẫn cần xử lý riêng. Sau chuyển lưu lượng, kiểm tra từ phía client: phiên bản trả về, lỗi, latency và kết quả nghiệp vụ; gắn quan sát với digest hoặc release ID, không chỉ đếm process còn sống. Giới hạn thời gian quan sát và tiêu chí dừng phải được chọn trước khi thay đổi.

Bài P06.C03.L03 — Điều phối

Phê duyệt một release là quyết định cho phép thử thay đổi trong điều kiện đã xét; nó không tự điều phối các lượt ghi lên cùng một môi trường. Giả sử `R1` và `R2` cùng được duyệt, `R2` hoàn tất trước rồi `R1` ghi đè sau. Cả hai pipeline có thể đều báo thành công, trong khi production cuối cùng chạy phiên bản cũ hơn chủ định. Vì vậy, trạng thái cần bảo vệ là phiên bản đang phục vụ trên môi trường đích, không chỉ là màu xanh của từng job CI.

Một trình điều phối nên serialize các release cùng đích, hoặc dùng điều kiện so sánh phiên bản ngay trước khi thay: chỉ thực hiện nếu phiên bản hiện tại còn bằng phiên bản mà kế hoạch dựa vào. Nếu điều kiện sai, dừng và lập lại kế hoạch, không ép ghi cho xong. Khóa trong CI chỉ ràng buộc những tác nhân đi qua cùng CI; một lượt triển khai bằng công cụ khác hoặc thao tác tay vẫn có thể chen

vào. Vì thế mỗi lần chuyển lưu lượng hoặc cập nhật artifact phải ghi nhận actor, thời điểm, bản đang chạy và bản mục tiêu.

Sau triển khai, kiểm tra lại artifact thực nhận request, readiness và một đường nghiệp vụ đại diện. Nếu metric hoặc dữ liệu chỉ ra hỏng, dừng tăng traffic theo ngưỡng đã thống nhất. Rollback ứng dụng chỉ thay mã chạy; phải xét riêng thay đổi schema và dữ liệu phát sinh trong thời gian phiên bản lỗi phục vụ.

Bài tập P06.C03.L03.Q01

Hai lượt release `R1` và `R2` cùng được phê duyệt vào production; `R2` hoàn tất trước nhưng `R1` ghi đè sau. Cổng phê duyệt thiếu điều kiện nào? Nêu cách điều phối theo môi trường và phép kiểm tra phiên bản ngay trước khi thay, đồng thời giải thích vì sao khóa ở CI chưa đủ nếu có người triển khai bằng công cụ khác.

Chương P06.C04 — Rollback và tương thích dữ liệu

Chứng minh hậu điều kiện sau khi lùi ứng dụng.

Bài P06.C04.L01 — Dữ liệu qua bản phát hành

Rollback ứng dụng là đưa lưu lượng hoặc cấu hình chạy về artifact trước đó. Nó không tự đảo một migration database đã commit, hàng đợi đã phát, email đã gửi hay giá trị nghiệp vụ đã ghi sai. Một migration kiểu mở rộng, chẳng hạn thêm cột nullable trong khi phiên bản cũ vẫn đọc được bảng, thường cho đường quay về ứng dụng dễ hơn việc đổi tên hoặc xóa cột đang được bản cũ dùng. “Rollback database” là một hành động khác: lệnh `ROLLBACK` chỉ hủy thay đổi của giao dịch còn mở; sau `COMMIT` cần migration bù, khôi phục dữ liệu có kiểm soát hoặc một quy trình sửa riêng. Không chạy phép đảo cơ học trên dữ liệu sống khi chưa biết các ghi hợp lệ đã xen vào.

Tình huống sau là dữ liệu giả định để suy luận, không phải log production hay benchmark. Dịch vụ `orders-api` đang chạy 1.8.3, build từ commit `c83`, artifact `A83` có digest `H83`. Lượt mới lấy commit `c84`, tạo `A84` với digest `H84` khác `H83`. Pipeline khóa dependency và toolchain, unit test pass; test tích hợp chỉ thử tạo đơn có tổng bằng 0 nên không bắt lỗi đổi đơn vị tiền. Artifact `A84` được đưa qua staging, nhưng production dùng dữ liệu và lưu lượng khác. Bản ghi release lưu `c84 → H84 → test-run T84 → production`, cùng người duyệt và thời điểm; dấu mũi tên chỉ quan hệ, không hàm ý các chuỗi ký hiệu này là digest SHA-256 thật.

Trước khi chuyển lưu lượng, migration thêm `receipt_token` nullable vào bảng `orders` rồi commit. 1.8.3 bỏ qua cột mới nên vẫn chạy với schema đã mở rộng. 1.8.4 có lỗi: đầu vào `amount_cents=1234` bị nhân thêm 100 trước khi ghi, thành `123400`. Bản canary nhận 5% lưu lượng. Trong một cửa sổ giả định, kiểm tra một đơn canary có giá trị ghi `123400` trong khi yêu cầu và sổ đối soát độc lập đều là `1234`; ba đơn khác cũng có chênh lệch. Readiness của Pod vẫn xanh vì endpoint probe không đọc đường ghi đơn. Không suy rằng mọi request trong 5% đều sai, cũng không suy tổng thiệt hại từ bốn mẫu; chúng đủ để dừng tăng canary và điều tra.

Bài P06.C04.L02 — Partial deployment

Readiness của Pod vẫn xanh vì endpoint probe không đọc đường ghi đơn.

Trong tình huống này, trạng thái triển khai và khả năng phục vụ của một đường nghiệp vụ là hai phép quan sát khác nhau. Trước tiên phải gắn mỗi request với artifact đang nhận nó; tiếp đó so kết quả của đường ghi với hậu điều kiện dữ liệu mà thay đổi hứa bảo toàn. Nếu chỉ thấy probe trả thành công, ta mới biết đúng đường probe còn phản hồi. Không thể từ đó suy rằng thao tác ghi đơn đã hoàn tất hoặc dữ liệu vẫn ở trạng thái trước triển khai.

Khi phát hiện sai khác, cần dừng mở rộng lưu lượng, ghi lại phiên bản, khoảng thời gian và tập request chịu ảnh hưởng. Quyết định quay về artifact cũ phải đi kèm đánh giá schema và dữ liệu đã ghi trong thời gian phiên bản mới chạy. Một lần đổi image có thể phục hồi mã xử lý nhưng không tự đảo các bản ghi đã thay đổi. Sau thao tác giảm thiểu, kiểm tra lại đường ghi và dữ liệu đại diện; chỉ báo đã phục hồi trong phạm vi đã được đo.

Bài P06.C04.L03 — Phục hồi

Trước hết đóng đường ghi của `H84` hoặc chuyển toàn bộ lưu lượng ghi về `H83`, tạm dừng lượt triển khai đang chạy và xác nhận không có worker/queue consumer `H84` tiếp tục ghi. Xác minh `H83` đáp ứng request mới với số tiền đúng và 1.8.3 tương thích schema có `receipt_token`. Giữ cột mới vì việc rút schema không cần cho phục hồi phục vụ; nó đã commit và có thể chứa dữ liệu hợp lệ. Sau đó khoanh tập bản ghi do `H84` tạo trong khoảng canary bằng release ID, thời gian, request ID và đối soát độc lập; sửa qua quy trình có phê duyệt, transaction và audit trail. Không chia mọi `amount_cents` cho 100: có thể có đơn hợp lệ do `H83` ghi xen vào. Hậu điều kiện gồm digest phục vụ hiện tại là `H83`, không còn đường ghi `H84`, đơn mới khớp nguồn đối soát và toàn bộ đơn bị ảnh hưởng đã được tìm và xử lý hoặc ghi rõ còn chờ. Một thông báo “rollback succeeded” chỉ chứng minh controller đã đổi cấu hình, không chứng minh bốn điều ấy.

Nếu migration đã xóa cột 1.8.3 cần, đưa `H83` trở lại sẽ không phục vụ được; phải khôi phục tính tương thích schema hoặc phát hành bản sửa tiến lên trước khi chuyển. Nếu yêu cầu từ `H84` đã gọi hệ thống thanh toán bên ngoài, việc đổi image không hủy giao dịch ở đó; cần truy trạng thái bằng khóa idempotency và quy trình bù theo hợp đồng của hệ thống đích. Vì vậy kế hoạch release phải nêu cả “quay lại code” lẫn “xử lý hậu quả đã commit”.

Bài tập P06.C04.L03.Q01

Sau khi chuyển lưu lượng về `H83`, người trực định chạy `ROLLBACK` SQL và chia tất cả số tiền cho 100. Phân tích vì sao hai thao tác đó không chứng minh phục hồi. Đề xuất hậu điều kiện đo được cho phần ứng dụng, database và tác dụng phụ bên ngoài.

PHẦN 07 — Container và OCI

Image trở thành tiến trình cô lập ra sao, và dữ liệu sống ở đâu?

Namespace đổi góc nhìn, cgroup điều tiết tài nguyên; image là artifact, container là lần chạy, mount tách vòng đời dữ liệu.

Chương P07.C01 — Namespace, cgroup v2 và runtime boundary

Chỉ ra cơ chế nào đổi góc nhìn, cơ chế nào giới hạn CPU/RAM.

Bài P07.C01.L01 — Tạo/tham gia

Namespace mới phải được tạo hoặc tiến trình phải tham gia namespace đã có. `clone`/`clone3` có thể tạo tiến trình con ngay trong namespace mới theo cờ `CLONE_NEW*`; `unshare` tách một phần ngữ cảnh của tiến trình gọi. Trường hợp PID có điểm khác dễ bỏ sót: `unshare(CLONE_NEWPID)` chỉ đặt namespace PID cho những đứa con tạo sau đó, không chuyển PID của tiến trình gọi; đứa con đầu tiên thành PID 1. `setns` dùng file descriptor của namespace đã tồn tại để tham gia; với network namespace nó đổi góc nhìn mạng của tiến trình gọi, còn với PID namespace nó chỉ đổi nơi những đứa con sau này được tạo. `fork` thông thường kế thừa namespace, `execve` thay chương trình đang chạy mà không tự tạo namespace mới. Vì thế chuỗi tạo hoặc tham gia namespace, tạo con, rồi `execve` chương trình ứng dụng có thể dẫn tới PID 1 bên trong; chỉ gọi `execve` trên tiến trình hiện tại không tạo ra kết quả ấy.

Tạo hoặc tham gia PID/network namespace cần capability phù hợp, thường là `CAP_SYS_ADMIN` trong user namespace liên quan; cấu hình user namespace và chính sách host cũng ảnh hưởng việc cho phép. Lỗi `EPERM` là kết quả cần xử lý, không phải lý do tự nâng quyền trên máy đang phục vụ. Một phép quan sát chỉ đọc trên Linux là chạy `readlink /proc/$$/ns/pid` và `readlink /proc/$$/ns/net` trong shell, rồi chạy hai lệnh ấy trong một shell con `sh -c`; mặc định danh tính namespace của cha và con trùng nhau. Phép thử chỉ chứng minh lượt `fork`/`exec` ấy kế thừa góc nhìn, không chứng minh có thể tạo hay tham gia namespace, và `/proc` phải phản ánh đúng tiến trình đang xét.

Góc nhìn phải gắn với tiến trình đang xét. Từ host Linux có quyền phù hợp, `/proc/3142/status` có thể ghi `NSpid: 3142 1`, cho biết hai PID theo thứ tự các cấp namespace; `/proc/3142/ns/pid` và `/proc/3142/ns/net` cho biết danh tính namespace. Một số container có PID namespace riêng nhưng chia sẻ network namespace với host; cũng có thể chia sẻ PID namespace. Cấu hình runtime quyết định từng loại: loại namespace không khai báo có thể được kế thừa. Không thể suy từ nhãn "container" rằng mọi namespace đều tách. PID 1 trong namespace còn là điểm cần xem khi xử lý tín hiệu và tiến trình con; một ứng dụng bỏ qua việc thu hồi con có thể tích lũy zombie dù host vẫn hoạt động bình thường.

Bài P07.C01.L02 — View

Chương 1 đặt tiến trình làm đơn vị chạy; Chương 2 đo tài nguyên mà các tiến trình tranh dùng. Container ghép hai cơ chế của nhân Linux quanh một nhóm tiến trình: namespace quyết định chúng nhìn thấy phần nào của hệ thống, còn cgroup tổ chức và kiểm soát mức dùng tài nguyên. Image ở Chương 9 là đầu vào bất biến để dựng môi trường chạy, không phải một tiến trình. Hiểu ba thực thể ấy giúp ta điều tra một dịch vụ chậm hoặc chết mà không gán mọi lỗi cho "container".

Namespace gắn với tiến trình và bọc một loại tài nguyên bằng góc nhìn riêng. Trong PID namespace, cùng một tiến trình có thể là PID 1 khi nhìn từ bên trong và PID 3142 từ namespace tổ

tiên; danh sách tiến trình nhìn thấy vì thế khác nhau. Trong network namespace, các interface, địa chỉ, route và socket thuộc một ngăn mạng riêng. `127.0.0.1` của tiến trình bên trong trỏ về loopback của ngăn ấy, không tự trỏ về host hay một container khác. Mount namespace thay bảng mount nhìn thấy; user namespace ánh xạ UID/GID; còn UTS và IPC tách các không gian tên tương ứng. Đây là phạm vi nhìn và định danh, không phải hạn mức CPU hoặc RAM.

Namespace không tạo tài nguyên từ hư không và không tự làm một sandbox hoàn chỉnh. Tiến trình vẫn dùng cùng nhân Linux với host; quyền, capability, seccomp, chính sách bảo mật nhân, mount và cấu hình runtime quyết định thêm bề mặt truy cập. Máy ảo thông thường chạy một nhân khách trên phần cứng ảo do hypervisor cung cấp. Ranh giới và chi phí vận hành khác nhau; trên máy để bàn, một hệ container Linux thậm chí có thể chạy bên trong máy ảo Linux. Vì vậy phải nói rõ đang xét ranh giới nào trước khi so sánh tính cô lập.

Bài tập P07.C01.L02.Q01

Từ `NSpid: 3142 1`, namespace mạng khác host và listener `127.0.0.1:8080`, hãy nêu ba điều có thể kết luận và hai điều chưa thể kết luận về khả năng client kết nối. Phép quan sát nào phân biệt không có đường chuyển tiếp với ứng dụng đã nhận kết nối nhưng trả HTTP 500?

Bài P07.C01.L03 — Limit

Trong môi trường container, cgroup là một cơ chế Linux dùng để tổ chức và kiểm soát tài nguyên của nhóm tác vụ. Một giới hạn CPU có thể khiến nhóm bị trì hoãn khi dùng hết quota trong chu kỳ; một giới hạn bộ nhớ có thể làm tăng thu hồi và cuối cùng dẫn tới lỗi cấp phát hoặc tiến trình bị kết thúc theo chính sách. Hai tình huống này có thể cùng biểu hiện thành tăng độ trễ, nhưng dấu vết và cách xử lý khác nhau. Không nên gọi mọi trường hợp “container chậm” là thiếu CPU.

Cgroup v2 đặt tiến trình trong một cây phân cấp. Controller ở cha phân phối hoặc giới hạn tài nguyên cho các nhóm con; giới hạn ở nhóm con không vượt được ràng buộc của tổ tiên. Đây là cơ chế kiểm soát mức dùng, không thay đổi PID, route hay danh sách file mà tiến trình nhìn thấy. `cpu.weight` là trọng số cạnh tranh khi có nhóm cùng cần CPU, còn `cpu.max` là quota trên mỗi chu kỳ. Chẳng hạn `cpu.max` bằng `50000 100000` cho phép tối đa 50.000 microsecond CPU trong chu kỳ 100.000 microsecond: xấp xỉ nửa lõi trung bình khi luôn có việc, không hứa mỗi request sẽ hoàn tất trong 50 ms. Khi hết quota trước khi hết chu kỳ, tiến trình có thể phải chờ dù host còn CPU nhàn rỗi ngoài cgroup.

`cpu.stat` có `usage_usec`; khi CPU controller được bật, nó còn có `nr_periods`, `nr_throttled` và `throttled_usec`. Muốn quy chậm cho quota, đọc hai mẫu trên cùng cgroup và cùng khoảng thời gian, tính hiệu số, đồng thời so với thời điểm request chậm. Một mẫu sau sự cố không cho biết counter đã tăng khi nào. `memory.current` là mức dùng hiện tại, còn `memory.max` là giới hạn cứng. Khi không thể thu hồi đủ bộ nhớ dưới giới hạn, nhân có thể kích hoạt OOM trong cgroup. `memory.events.local` cho biết sự kiện tại cgroup ấy; `oom_kill` tăng nghĩa là đã có tiến trình thuộc nhóm bị OOM killer giết. `max` tăng chỉ nói đã chạm giới hạn, không đồng nghĩa đã có tiến trình chết. Bộ nhớ hiện tại có thể giảm mạnh sau khi nạn nhân bị giết, nên một ảnh chụp thấp

không bác bỏ OOM vừa xảy ra. `memory.high` là ngưỡng thúc đẩy thu hồi và gây chậm; nó khác `memory.max` và không phải điểm kích hoạt OOM trực tiếp.

Tình huống giả định để tính toán, không phải output đo trên máy người học. Dịch vụ `orders` chạy trong một PID và network namespace riêng, dùng cgroup v2 `/service/orders`. Client nhận timeout xen kẽ; sau đó runtime báo một lần khởi động thất bại. Người vận hành có quyền đọc trạng thái từ host Linux. Trước hết họ gắn đúng PID host của lượt chạy và thời gian quan sát, tránh trộn counter của container cũ với lượt chạy mới. Với PID host 3142, `NSpid: 3142 1` và namespace `net:[4026533001]` khác host `net:[4026531992]`: đó là chứng cứ về góc nhìn PID và mạng khác nhau, chưa phải chứng cứ lỗi mạng. Từ góc nhìn network namespace của tiến trình, ứng dụng nghe `127.0.0.1:8080`; client ở namespace khác không thể dùng trực tiếp loopback ấy. Muốn chốt nhánh này phải kiểm tra đường publish/forward, địa chỉ bind, route và phép kết nối từ chính vị trí client. Nếu đường publish được cấu hình tới địa chỉ khác mà không có listener tương ứng, timeout có thể đến từ mạng; nếu TCP đã kết nối rồi nhận HTTP 500, lỗi nằm sau bước kết nối.

Trong một cửa sổ 10 giây của cùng lượt chạy, cấu hình là `cpu.max = 50000 100000`, `memory.max = 268435456` byte. Hai mẫu đọc từ cùng cgroup cho bảng sau; số là dữ liệu giả định, công thức hiệu số là phần cần kiểm chứng.

Counter	Đầu cửa sổ	Cuối cửa sổ
`cpu.stat usage_usec`	20.000.000	24.700.000
`cpu.stat nr_periods`	1.000	1.100
`cpu.stat nr_throttled`	40	120
`cpu.stat throttled_usec`	4.000.000	6.000.000
`memory.events.local max`	3	5
`memory.events.local oom`	0	1
`memory.events.local oom_kill`	0	1

Hiệu `usage_usec` là 4,7 triệu microsecond, tức trung bình 0,47 lõi trong 10 giây; quota cho tối đa xấp xỉ 0,5 lõi trung bình. Có 100 chu kỳ mới, 80 chu kỳ bị throttle, và `throttled_usec` tăng 2 triệu microsecond. Đây là bằng chứng quota đã tác động trong cửa sổ, phù hợp giả thuyết latency tăng; nó không cho biết request cụ thể nào chờ bao lâu, cũng không loại trừ I/O, khóa hoặc upstream chậm. Cần đặt timestamp request cạnh counter và, khi cần, đo thêm CPU pressure, hàng đợi và thời gian từng chặng. Không lấy tỷ số 80/100 làm "80% CPU đã dùng": nó đếm chu kỳ có throttle, không đếm CPU time.

`memory.events.local max` tăng 2, `oom` tăng 1 và `oom_kill` tăng 1. Ít nhất một tiến trình trong cgroup này đã bị OOM killer giết trong cửa sổ; không thể từ counter ấy tự biết PID nào, request nào hoặc lỗi cấu hình nào làm bộ nhớ tăng. Phải đối chiếu thời điểm, danh tính tiến trình, log kernel/runtime và mức dùng trước khi bị giết. Nếu runtime báo exit 137, mã ấy riêng lẻ chỉ tương thích với việc tiến trình bị `SIGKILL` theo quy ước shell; OOM counter và log cùng thời điểm mới

làm giả thuyết OOM mạnh hơn. Một lần khởi động sau đó thoát 2 với log `config missing`, trong khi `oom_kill` của cgroup mới không đổi: đó là bằng chứng của nhánh lỗi ứng dụng, không được gọi nhầm là OOM chỉ vì lượt trước từng OOM.

Để tái lập phép phân tích mà không gây OOM trên môi trường đang phục vụ, người học lấy hai bản chụp chỉ đọc của `cpu.max`, `cpu.stat`, `memory.max`, `memory.events.local` trên cùng đường cgroup và lưu thời điểm cùng PID/lượt chạy; rồi tính hiệu từng hàng như bảng. Trên Linux có quyền phù hợp, đọc `NSpid` và liên kết `/proc/<pid>/ns/{pid,net}`; quan sát route, listener từ đúng network namespace và thử kết nối từ vị trí client. Nếu đang ở cgroup namespace khác hoặc controller không được bật, đường dẫn/counter nhìn thấy có thể khác; ghi rõ ngữ cảnh hoặc dừng kết luận. Bảng này tái lập được phép tính, không tái lập một sự cố thật và không chứng minh cấu hình nào là an toàn cho production.

Bài tập P07.C01.L03.Q01

Tính hiệu `usage_usec`, `nr_periods`, `nr_throttled`, `throttled_usec` trong bảng. Vì sao quota 0,5 lõi và 80 chu kỳ throttled ủng hộ nhưng chưa chứng minh CPU là nguyên nhân duy nhất của timeout?

Bài tập P07.C01.L03.Q02

Phân biệt ý nghĩa `memory.events.local max`, `oom` và `oom_kill` tăng; đề xuất chứng cứ để gắn nạn nhân OOM với lượt chạy. Vì sao lần thoát 2 có log `config missing` không thể gộp với sự kiện OOM trước?

Chương P07.C02 — OCI image, build và registry

Lần từ Dockerfile tới manifest/digest và byte đã pull.

Bài P07.C02.L01 — Build

Build cache tái dùng kết quả bước trước khi các đầu vào mà builder theo dõi được coi là không đổi. Khi một bước thay đổi, các bước phụ thuộc sau nó có thể phải chạy lại. Cache giúp tiết kiệm thời gian, nhưng cache hit không chứng minh chương trình đúng, dependency ngoài đã được cập nhật hay hai lượt build tạo byte giống nhau. Chương 9 đã yêu cầu ghi commit, toolchain, đầu vào và digest thực tế; ở đây cần thêm base image theo digest và các đầu vào build ngoài repository nếu muốn kiểm soát biến thiên. Build tạo image ở nơi build; push phân phối nội dung lên registry; pull lấy manifest và blob về nơi chạy. Ba hành động ấy khác nhau, và "đã pull" không có nghĩa container đang chạy đã được thay.

Bài P07.C02.L02 — OCI

Chương 10 cho thấy container đang chạy là một nhóm tiến trình với góc nhìn và giới hạn tài nguyên được cấu hình. Chương này đi ngược dòng tới thứ được phân phối để tạo tiến trình ấy, rồi theo dữ liệu đi qua nhiều lần chạy. Một câu hỏi vận hành thường bị trả lời sai là "đã đổi image, sao dữ liệu cũ vẫn ở đó?" Câu trả lời cần tách ba định danh: byte của image, lần chạy của container và nơi thực sự lưu dữ liệu.

Image OCI gồm những thay đổi hệ thống tệp có thứ tự và cấu hình chạy như entrypoint, đối số, biến môi trường. Digest giúp nhận diện nội dung image đã lấy về, nhưng byte của image không tự tạo một namespace hay một cgroup. Runtime nhận bundle và cấu hình, chuẩn bị root filesystem, namespace, quyền và giới hạn, rồi khởi chạy chương trình. Container đang `running` là một tiến trình đã thực thi chương trình trong môi trường ấy; khi tiến trình chính dừng, runtime ghi trạng thái `stopped`. Image vẫn còn để dùng cho lượt chạy khác. Lớp ghi, volume và dữ liệu bên ngoài có vòng đời riêng, nên xóa hoặc chạy lại container không có nghĩa phục hồi dữ liệu như cũ.

Builder đọc Dockerfile cùng build context, thực hiện các bước để tạo filesystem và cấu hình image. Theo mô hình OCI, manifest chỉ tới cấu hình và danh sách layer có thứ tự; descriptor có digest để người nhận kiểm tra đúng byte đã tham chiếu. Một layer ghi thay đổi so với lớp trước, không phải một máy ảo thu nhỏ. Lệnh `COPY` đưa dữ liệu của context vào kết quả build; file bị bỏ qua bởi `.dockerignore` không có trong context. Nếu đưa secret vào một bước build rồi xóa ở bước sau, không được mặc định rằng secret đã biến mất khỏi mọi layer hoặc cache. Cần cơ chế secret của builder và xác minh artifact cuối theo quy trình tương ứng.

Khi registry trả về một index đa nền tảng, lựa chọn kiến trúc và manifest con còn phải được ghi nhận: cùng tag có thể dẫn tới nội dung khác trên hai nền tảng.

Bài P07.C02.L03 — Distribution

Registry lưu manifest và blob. Tag như `orders:2.1` là tên thuận tiện trỏ tới manifest và có thể được cập nhật; digest định danh nội dung manifest đã chọn. Khi registry trả về một index đa nền tảng, lựa chọn kiến trúc và manifest con còn phải được ghi nhận: cùng tag có thể dẫn tới nội dung khác trên hai nền tảng. Dùng digest của artifact đã kiểm thử để triển khai và xác nhận digest ở đích; vẫn cần đánh giá nguồn gốc, lỗ hổng, cấu hình và quyền vì đúng byte không đồng nghĩa hành vi an toàn. Một registry không sẵn sàng có thể làm lần pull mới thất bại dù container hiện hữu còn chạy trên node đã có image; trạng thái cache cục bộ và chính sách pull quyết định chi tiết.

Bài tập P07.C02.L03.Q01

Một pipeline build và push `orders:2.2`; máy chủ vẫn chạy container tạo từ digest cũ. Phân biệt trạng thái registry, image cục bộ và tiến trình đang phục vụ. Cần đối chiếu những định danh nào để khẳng định bản mới đã nhận request?

Chương P07.C03 — Container lifecycle và dữ liệu

Dự đoán dữ liệu sau stop, recreate, remove, mount và đổi image.

Bài P07.C03.L01 — Lần chạy

Trạng thái `running` của runtime cũng không phải readiness của ứng dụng. Tiến trình có thể còn sống nhưng chưa bind socket, đang đợi DB, trả lỗi nghiệp vụ hoặc nghe sai địa chỉ. Việc publish port qua host hay qua bộ điều phối thêm một đường chuyển tiếp ngoài network namespace; thấy socket lắng nghe ở `127.0.0.1` bên trong chưa chứng minh client ngoài namespace đến được. Cần nối các phép quan sát từ tiến trình, route và socket tới phép thử request thực tế, như đã học ở Chương 4–5.

Runtime tạo môi trường theo cấu hình, khởi chạy tiến trình và theo dõi trạng thái như `created`, `running`, `stopped`. Dừng tiến trình rồi khởi động lại cùng container có thể giữ writable layer của lần tạo ấy; xóa hoặc recreate container tạo lớp ghi khác. Thay image không tự sửa writable layer của một container đang tồn tại; phải xác định container nào thực sự đang phục vụ. Khi container bị xóa, dữ liệu chỉ nằm trong writable layer của nó không còn được giữ như một kho bền vững. Đặt database hoặc upload quan trọng ở đó là thiết kế dễ mất dữ liệu khi thay container.

Container đang `running` là một tiến trình đã thực thi chương trình trong môi trường ấy; khi tiến trình chính dừng, runtime ghi trạng thái `stopped`.

Bài tập P07.C03.L01.Q01

Một image digest đã được kiểm thử nhưng runtime báo `running`, readiness vẫn đỏ. Giải thích vì sao cả digest lẫn trạng thái runtime không đủ chứng minh request được phục vụ. Nêu vai trò khác nhau của image, namespace và cgroup trong tình huống này.

Bài P07.C03.L03 — Mount

Volume là vùng dữ liệu do Docker quản lý, được mount vào đường dẫn trong container. Named volume có định danh để một container khác có thể gắn lại; nó tồn tại độc lập với container cho tới khi bị xóa riêng. Bind mount thay bằng một đường dẫn cụ thể trên host của Docker daemon. Nó hữu ích cho mã nguồn hoặc cấu hình cần đọc từ host, nhưng bị ràng buộc vào đường dẫn, quyền và hệ điều hành của host ấy; khi dùng Docker Desktop, host của daemon có thể là Linux VM phía sau giao diện máy người dùng. Một bind mount có quyền ghi cho tiến trình trong container có thể sửa file host tương ứng, nên cấu hình chỉ cần đọc nên mount read-only. Cả hai loại mount che nội dung có sẵn tại điểm mount trong container; không nên thấy một thư mục “trống” rồi vội kết luận byte cũ đã bị xóa. Với empty named volume, Docker có thể sao chép nội dung sẵn trong image tại điểm mount vào volume theo mặc định; hành vi này khác việc chuyển dữ liệu từng được ghi ở writable layer của container cũ.

Đường dẫn ứng dụng không nói nơi lưu. `/var/lib/orders/orders.db` có thể thuộc writable layer, named volume, hoặc một thư mục host. Cần đọc cấu hình mount và danh tính container/volume rồi mới quyết định backup, migration hay xóa. Volume vẫn nằm trên một host hoặc backend cụ thể; “persist” không tự có nghĩa có bản sao, backup, giao dịch nhất quán hay khả

năng chạy sang node khác. Database đang ghi cần phương pháp snapshot hoặc backup phù hợp với chính database, cùng phép restore thử. Xóa container, image và volume là ba hành động khác nhau; đặc biệt các tùy chọn xóa volume phải được xem như thao tác xóa dữ liệu.

Bài tập P07.C03.L03.Q01

Giải thích vì sao 41 bản ghi ở lượt A không tự xuất hiện ở lượt B. Nếu container A chỉ dừng, câu trả lời khác gì trường hợp A đã bị xóa? Hành động nào phải tránh khi chưa có backup?

Chương P07.C04 — Compose và ứng dụng nhiều service

Phân biệt cấu hình project với trạng thái service/volume thực tế.

Bài P07.C04.L01 — Mô hình project

Compose file khai báo service, image hoặc cách build, port, network và volume cho một ứng dụng. `docker compose up` đối chiếu cấu hình để tạo hoặc thay container, mạng và volume cần thiết trên Docker Engine được chọn. Compose gán project để phân biệt tập tài nguyên; mặc định có mạng cho các service cùng project, chúng tìm nhau bằng tên service. Cổng host và cổng trong container là hai địa chỉ khác nhau: trong ví dụ dưới, client trên host dùng `127.0.0.1:18080`, còn tiến trình trong container phải nghe cổng 8080 trên địa chỉ có thể nhận kết nối từ mạng container. Đổi IP container sau recreate không nên buộc service khác sửa cấu hình; dùng tên service trong mạng chung. `localhost` trong service vẫn là chính network namespace của nó.

Đoạn Compose sau chỉ minh họa cú pháp. Image `orders:2.1` và thư mục `./config` là đầu vào giả định của dự án học, không có ứng dụng có thể chạy ngay trong repository này.

Bài P07.C04.L02 — Khởi động

`orders_data` được mount tại đường ứng dụng ghi; `./config` là bind mount từ host, chỉ đọc trong container. `name: orders-case` cố định tên project trong ví dụ; chạy với project khác có thể tạo volume khác dù phần tên `orders_data` trong file giống nhau. Trước khi triển khai, phải xem cấu hình Compose đã được nội suy và kiểm tra image, mount, project, quyền, port cùng nơi daemon đang chạy. Đoạn này chưa có healthcheck; Compose thấy container chạy không chứng minh API hoặc database ready. `depends_on` mặc định giải quyết thứ tự khởi động, không tự kiểm tra ứng dụng đã sẵn sàng; điều kiện healthcheck có thể chờ một phép kiểm tra được khai báo, nhưng phép kiểm tra ấy vẫn chỉ chứng minh điều nó đo.

Compose hữu ích để mô tả và chạy ứng dụng trên một Docker Engine, kể cả Engine ở máy khác. File Compose tự nó không là bộ lập lịch nhiều node, không tự phân phối bản sao khi host hỏng và không tự nhân bản volume cục bộ. Nếu triển khai trên Swarm hoặc một hệ điều phối khác, phải đánh giá cơ chế scheduler, mạng nhiều host và storage của hệ đó thay vì suy từ cú pháp Compose. Chương 15–17 mới đi sâu vòng điều khiển và placement của Kubernetes.

Bài tập P07.C04.L02.Q01

Một Compose file có `depends_on` và hai service cùng mạng mặc định. Nêu điều nó đảm bảo, điều nó không đảm bảo về readiness, dữ liệu và phục hồi sau khi host hỏng. Đề xuất một phép kiểm chứng từ góc nhìn client và một phép kiểm chứng backup.

Bài P07.C04.L03 — Dữ liệu

```
name: orders-case
services:
  orders:
    image: orders:2.1
    ports:
      - "127.0.0.1:18080:8080"
    volumes:
      - orders_data:/var/lib/orders
      - ./config:/etc/orders:ro
volumes:
  orders_data:
```

Tình huống sau là dữ liệu giả định để suy luận, không phải output Docker đã chạy. Ở lượt A, image `orders:2.0` không khai báo mount cho `/var/lib/orders`; ứng dụng ghi 41 bản ghi vào `orders.db` trong writable layer của container `orders-a`. Sau khi thay container bằng lượt B dùng image `orders:2.1`, đường dẫn ấy trống. Không có bằng chứng 41 bản ghi bị xóa khỏi một volume: chúng chưa từng ở volume; lớp ghi gắn với container A không phải dữ liệu được tự chuyển sang B. Nếu A còn tồn tại dưới dạng container đã dừng, có thể còn khả năng đọc lại lớp ghi; nếu A đã bị xóa, không được hứa khôi phục mà không có backup hoặc cơ chế lưu khác.

Ở lượt C, dự án dùng cấu hình Compose ở mục trước và ghi 41 bản ghi mới vào named volume `orders-case_orders_data`. Lượt D thay image `orders:2.1` bằng digest của bản `2.2`, recreate container nhưng vẫn gắn đúng volume ấy; ứng dụng nhìn thấy 41 bản ghi. Điều đó chưa chứng minh image `2.2` đã đọc dữ liệu đúng schema hoặc volume được backup. Nếu người vận hành đổi project name thành `orders-test`, Compose có thể gắn một volume mới `orders-test_orders_data`; đường dẫn trong container vẫn `/var/lib/orders` nhưng file cũ không xuất hiện. Volume của project trước có thể vẫn còn. Việc thêm bind mount `./empty-data:/var/lib/orders` vào cùng điểm mount cũng thay nơi nhìn thấy; không được đồng thời gắn hai nguồn vào cùng đích và đoán nguồn nào thắng. Chọn một cấu hình rõ ràng, kiểm tra `Mounts` thực tế và ngừng thay đổi trước khi điều tra.

Phép kiểm chứng tối thiểu bắt đầu bằng container ID, image digest và thời điểm recreate; tiếp đến là loại mount, `Source`, `Destination`, tên project và volume. So số bản ghi và checksum hoặc ID nghiệp vụ từ ứng dụng với bản backup độc lập, rồi thử đọc volume cũ bằng phương thức không ghi nếu cần. `docker compose config` giúp xem cấu hình sẽ áp dụng nhưng không chứng minh trạng thái runtime hoặc byte dữ liệu. Không chạy `down -v`, `volume prune` hay xóa thư mục host để "làm sạch" trước khi đã xác định chính xác nguồn dữ liệu. Trong môi trường thử riêng, có thể tái hiện ba lượt với dữ liệu vô hại và so trạng thái trước/sau; ở bản thảo này chưa có thực nghiệm Docker nên không đưa output giả làm log thật.

Bài tập P07.C04.L03.Q01

Lượt D đổi image nhưng vẫn thấy 41 bản ghi; lượt kế tiếp đổi project name thì không thấy. Dùng volume identity và mount destination để giải thích hai hiện tượng. Vì sao một bind mount rỗng vào cùng đích không chứng minh volume cũ đã mất dữ liệu?

PHẦN 08 — Cloud và AWS

Một request qua ranh giới danh tính, mạng, tính toán và dữ liệu của cloud ra sao?

Identity policy và đường mạng là hai bộ lọc độc lập; compute và storage có vòng đời, miền lỗi, chi phí khác nhau.

Chương P08.C01 — AWS identity và IAM evaluation

Lần theo principal từ EC2 tới S3/STS và quyết định Allow/Deny.

Bài P08.C01.L01 — Danh tính

Một ứng dụng chạy được trên EC2 vẫn có thể không đọc được object S3. Từ góc nhìn người dùng, cả kết nối mạng bị chặn lẫn yêu cầu bị từ chối đều là "không lấy được báo cáo", nhưng chúng xảy ra ở những ranh giới khác nhau. Chương này theo một request từ client tới Application Load Balancer (ALB), tới tiến trình trên EC2 trong private subnet, rồi tới S3. Mỗi chặng phải trả lời ba câu hỏi: địa chỉ đích là gì, gói tin có đi tới đó không, và principal nào được phép thực hiện hành động gì trên tài nguyên nào. Ví dụ đặt trong một VPC IPv4 giả định ở Region `us-east-1`; tên subnet, bucket, policy và mọi quan sát bên dưới đều là dữ liệu học tập, không phải cấu hình hay log AWS đã chạy.

EC2 cung cấp máy ảo, network interface và vòng đời instance; ứng dụng là tiến trình chạy trên hệ điều hành của instance. EC2 ở private subnet không tự có quyền gọi API AWS. Khi gắn IAM role ứng dụng qua instance profile, phần mềm có thể lấy temporary credentials từ Instance Metadata Service (IMDS) và ký request tới dịch vụ AWS. Role có trust policy xác định ai được nhận role; permissions policy xác định hành động mà phiên role được phép yêu cầu. STS phát hành phiên tạm thời khi một principal gọi `AssumeRole` theo quan hệ tin cậy và quyền thích hợp. Với role gắn cho EC2, nền tảng cấp và làm mới credentials cho ứng dụng; không cần nhúng access key dài hạn vào image hay tự gọi `AssumeRole` để lấy lại chính role đó. Vẫn phải bảo vệ IMDS và tránh ghi credential vào log. Quyền của người triển khai hạ tầng, quyền của role instance và quyền của client HTTP là ba danh tính khác nhau.

Bài P08.C01.L02 — Quyết định

Một lần `s3:GetObject` không được đánh giá bằng tên role đơn lẻ. AWS xác thực chữ ký, xác định principal của phiên, action, ARN object, điều kiện của request, rồi xét các policy có hiệu lực. Mặc định là từ chối; một `Allow` hợp lệ phải áp dụng đúng action, resource và điều kiện. Một explicit `Deny` áp dụng ở bất kỳ policy liên quan nào thắng `Allow`. Identity policy, bucket policy, permissions boundary, session policy, SCP/RCP của tổ chức và endpoint policy có thể cùng tác động, nhưng cách kết hợp phụ thuộc loại principal, cùng hay khác account và dịch vụ cụ thể. Chẳng hạn resource policy cấp quyền trực tiếp cho role session có quy tắc khác với cấp cho role ARN; không thể dạy một phép giao hoặc hợp duy nhất cho mọi trường hợp. Trust policy kiểm soát bước nhận role, không thay cho quyền đọc object; bucket policy và endpoint policy cũng không tự tạo tuyến mạng. `AccessDenied` chỉ nói một yêu cầu đã tới lớp có thể từ chối nó; cần lấy principal thực, action, ARN, Region, request ID và loại policy được chỉ ra nếu thông báo có đủ chi tiết. Không suy chỉ từ chữ `AccessDenied` rằng IAM identity policy là nguyên nhân.

Bài tập P08.C01.L02.Q01

Sau khi TCP/TLS đã hoạt động, `GetObject` nhận `AccessDenied` dù role có `Allow`. Hãy lần theo các policy cần xét, xác định kết luận từ explicit `Deny` trong dữ kiện bài và nêu một điều chưa thể kết luận chỉ từ HTTP 403.

Bài P08.C01.L03 — Điều tra

Ở trạng thái sau khi có đường endpoint, giả sử quan sát mới là TCP/TLS hoàn tất và dịch vụ trả HTTP 403 `AccessDenied` cho cùng `GetObject`. Mạng đã đủ để nhận phản hồi; 403 không chứng minh đường đang đi qua endpoint mong muốn, cũng không tự chỉ ra policy nào thắng. Cần đối chiếu principal phiên role, bucket/Region/key, endpoint policy và các policy áp dụng; kiểm tra dấu vết request ở đúng tài khoản nếu có. Với dữ kiện identity policy ở trên, explicit `Deny` khớp object sẽ thắng `Allow`; bỏ hoặc thu hẹp deny phải qua review quyền tối thiểu, rồi kiểm chứng lại bằng `GetObject` cho đúng key và kiểm tra dữ liệu trả về. Nếu request tới STS regional endpoint vẫn timeout trong cùng private subnet, đó là đường mạng khác: S3 gateway endpoint không cấp kết nối cho STS. Từng phép sửa chỉ xử lý ranh giới mà bằng chứng xác nhận.

Tình huống không chạy trên tài khoản AWS. Khi kiểm chứng trên môi trường có thật, ghi thời điểm, Region, AZ ID, subnet/route table/endpoint ID, địa chỉ DNS, kết quả TCP/TLS, HTTP status và request ID mà không ghi secret hoặc payload nhạy cảm. Cấu hình có thể đổi trong khi điều tra; so cùng một cửa sổ thời gian và chạy lại đúng request sau mỗi thay đổi. Bất kỳ `AccessDenied` nào cũng có thể do bucket policy, SCP, session policy, endpoint policy hoặc KMS nếu object mã hóa bằng customer managed key; không được chuyển nguyên nhân giả định của ví dụ thành quy tắc chẩn đoán mọi sự cố.

Chương P08.C02 — VPC, subnet và đường đi AWS

Chứng minh private workload có hoặc không có đường tới S3/STS.

Bài P08.C02.L01 — Topology

VPC là không gian mạng của tài khoản trong một Region; subnet là một dải địa chỉ thuộc một Availability Zone (AZ). Route table gắn với subnet quy định đích IP/prefix đi tới target nào, theo tuyến khớp dài nhất; subnet không gắn tường minh dùng main route table. Public subnet trong mô hình IPv4 có tuyến mặc định tới Internet Gateway (IGW), nhưng một EC2 muốn giao tiếp trực tiếp ra internet còn cần địa chỉ public phù hợp và các rule cho phép. Private subnet không có đường trực tiếp ấy. Public ALB có thể nhận client ở các subnet public rồi kết nối tới private IP của target EC2; target không cần public IP. Health check xanh chỉ chứng minh phép thử được cấu hình, không chứng minh `s3:GetObject` hoặc tác vụ nghiệp vụ đã thành công.

IGW là cổng nối VPC với internet. Public NAT Gateway theo kiến trúc theo AZ nhận lưu lượng IPv4 outbound từ private subnet qua tuyến `0.0.0.0/0` của subnet ấy; subnet đặt NAT cần đường ra IGW. NAT cho phép kết nối do phía private khởi tạo và phản hồi đi về, không biến instance private thành server nhận kết nối inbound tùy ý. NAT Gateway là một phụ thuộc sẵn sàng và có chi phí tài nguyên/lưu lượng; nếu nhiều AZ cùng dựa vào một NAT theo AZ, AZ chứa NAT trở thành điểm lỗi chung. AWS còn có kiểu NAT và lựa chọn kiến trúc khác; kết luận này dành cho public NAT Gateway theo AZ đang xét, không áp dụng máy móc cho mọi kiểu. Với S3 cùng Region, gateway VPC endpoint có thể thêm tuyến prefix list S3 vào route table đã liên kết, để lưu lượng đi tới S3 qua endpoint mà không cần NAT/IGW. Endpoint nằm trong VPC và Region cụ thể; tạo endpoint nhưng quên liên kết route table của subnet vẫn không tạo đường đi cho subnet đó. Endpoint policy là một lớp lọc quyền bổ sung cho lưu lượng đi qua endpoint, không thay IAM/bucket policy. Nếu ứng dụng còn gọi STS regional endpoint, S3 gateway endpoint không tự mở đường tới STS; cần xét NAT hoặc interface endpoint tương ứng cùng DNS và quyền.

Bài P08.C02.L02 — Chính sách gói

Security Group (SG) gắn với network interface và gồm các luật cho phép; nó theo dõi kết nối, nên gói phản hồi của kết nối được phép không cần một luật chiều ngược riêng.

VPC Flow Logs, nếu đã bật ở đúng network interface và khoảng thời gian, có thể giúp xét ACCEPT/REJECT ở tầng mạng, nhưng không giải thích được nội dung policy IAM hay mọi nguyên nhân timeout.

Với một request từ instance, hãy tách hai câu hỏi: gói tin có đi qua ranh giới mạng hay không, và danh tính gọi API có được phép hay không. Security Group (SG) gắn với network interface và gồm các luật cho phép; nó theo dõi kết nối, nên gói phản hồi của kết nối được phép không cần một luật chiều ngược riêng. NACL xét lưu lượng ở ranh giới subnet theo các luật và hướng riêng; vì vậy cần kiểm tra cả cổng đích lẫn cổng tạm thời của chiều phản hồi trong cùng đường đi. Route table quyết định next hop, không tự tạo một luật cho phép ở SG hoặc NACL.

Nếu phép quan sát ở network interface cho thấy luồng bị từ chối, đó là bằng chứng để tiếp tục đối chiếu luật và thời điểm, chưa phải bằng chứng về IAM. Ngược lại, khi request tới được endpoint và nhận `AccessDenied`, ta phải xét principal, action, resource cùng các lớp policy áp dụng. Giữ nguyên Region, subnet, địa chỉ đích và thời gian khi so các phép đo; đổi đồng thời nhiều yếu tố sẽ làm mất quan hệ nhân quả.

Bài P08.C02.L03 — Private access

DNS đổi tên thành địa chỉ; nó không chứng minh TCP tới địa chỉ ấy kết nối được. Route 53 Resolver/AmazonProvidedDNS có thể trả lời trong VPC, còn DNS tùy chỉnh, private DNS của endpoint, Region trong SDK và cache có thể làm ứng dụng nhận một đích khác với dự tính. Security Group (SG) gắn với network interface và gồm các luật cho phép; nó theo dõi kết nối, nên gói phản hồi của kết nối được phép không cần một luật chiều ngược riêng. Network ACL (NACL) áp cho subnet, có thứ tự luật allow/deny và không giữ trạng thái: cả chiều đi lẫn chiều về, gồm cổng ephemeral của client, phải được cho phép. “Route đúng” chưa chứng minh SG/NACL đúng; “DNS trả IP” chưa chứng minh route hay quyền đúng. VPC Flow Logs, nếu đã bật ở đúng network interface và khoảng thời gian, có thể giúp xét ACCEPT/REJECT ở tầng mạng, nhưng không giải thích được nội dung policy IAM hay mọi nguyên nhân timeout.

Ở trạng thái đầu, giả sử DNS trên instance trả địa chỉ cho S3 regional endpoint, ứng dụng bắt đầu TCP 443 nhưng hết hạn kết nối; log ứng dụng chỉ ghi timeout và không có HTTP response từ S3. Đó là bằng chứng về đích DNS và việc chưa thiết lập được kết nối, không phải bằng chứng IAM đã cho phép hoặc từ chối. Đọc subnet association và route table cho thấy chỉ có tuyến local; endpoint được tạo nhưng không gắn với table ấy. Kiểm tra SG outbound, NACL outbound 443/inbound ephemeral, trạng thái endpoint và đường đi vẫn cần làm trước khi quy nguyên nhân duy nhất. Trong bài toán đã cho, thiếu tuyến tới S3 là lời giải cho timeout; sửa bằng cách liên kết đúng route table theo thay đổi được phê duyệt, không mở rộng `0.0.0.0/0` hoặc thêm NAT chỉ để lấp một khoảng trống bằng suy đoán.

Với S3 cùng Region, gateway VPC endpoint có thể thêm tuyến prefix list S3 vào route table đã liên kết, để lưu lượng đi tới S3 qua endpoint mà không cần NAT/IGW.

Nếu ứng dụng còn gọi STS regional endpoint, S3 gateway endpoint không tự mở đường tới STS; cần xét NAT hoặc interface endpoint tương ứng cùng DNS và quyền.

Nếu request tới STS regional endpoint vẫn timeout trong cùng private subnet, đó là đường mạng khác: S3 gateway endpoint không cấp kết nối cho STS.

Bài tập P08.C02.L03.Q01

Với cấu hình ở mục 12.4, DNS trả địa chỉ S3 nhưng TCP 443 timeout. Hãy nêu ba phép kiểm tra phân biệt route, SG/NACL và DNS. Vì sao tăng quyền role chưa phải phép sửa có căn cứ?

Chương P08.C03 — EC2, load balancer và miền lỗi

Phân biệt instance running, target healthy và request nghiệp vụ đúng.

Bài P08.C03.L02 — Traffic

Giả sử ở `us-east-1`, client gọi HTTPS tới public ALB; target EC2 `orders-api` nằm trong subnet `10.24.2.0/24` của một AZ, không có public IPv4. Tiến trình đọc `reports/day-01.json` từ bucket giả định `orders-archive-example`, rồi trả báo cáo. ALB và target có đường tới nhau; health check chỉ gọi `/health`, không đọc S3. Role ứng dụng cho phép `s3:GetObject` trên prefix báo cáo, nhưng một statement khác trong chính identity policy giả định `Deny` đúng action và object ấy. VPC có S3 gateway endpoint, song route table của subnet `10.24.2.0/24` chưa liên kết với endpoint; subnet cũng không có tuyến NAT. Đây là dữ kiện thiết kế giả định, không phải kết quả AWS đã đo.

Bài tập P08.C03.L02.Q01

ALB `/health` trả 200; API báo cáo lỗi, còn STS timeout. Phân biệt đường client–ALB–EC2, EC2–S3, EC2–STS và nêu phép kiểm chứng sau sửa.

Chương P08.C04 — Block, file và object storage

Chọn EBS/EFS/S3 theo cách truy cập và vòng đời dữ liệu.

Bài P08.C04.L01 — Block

EC2 instance có CPU, RAM, hệ điều hành, ENI và trạng thái chạy; dừng hoặc thay instance không tự quyết định số phận mọi dữ liệu. Instance store, nếu loại instance có cung cấp, gắn với vòng đời phần cứng/instance và không phải nơi đặt dữ liệu cần tồn tại sau mất instance. EBS là block volume: hệ điều hành thấy thiết bị, tạo filesystem rồi ứng dụng đọc/ghi như đĩa. Volume EBS thuộc một AZ và chỉ attach cho instance tương thích trong cùng AZ; không thể coi nó như filesystem dùng chung giữa các AZ. Dữ liệu EBS có thể còn sau khi instance dừng, nhưng khi terminate phải xem thuộc tính `DeleteOnTermination` của từng volume. Redundancy của volume trong AZ giúp chịu lỗi thiết bị, không tự tạo bản khôi phục khỏi xóa nhầm, lỗi ứng dụng hay mất AZ. Snapshot là bản sao theo thời điểm cần được lên lịch, bảo vệ và thử restore; snapshot của volume đang ghi không tự chứng minh nhất quán giao dịch của database.

Bài P08.C04.L02 — File

EFS cung cấp filesystem chia sẻ qua NFS, không phải block device. Client cần DNS, tuyến tới mount target, SG cho NFS và quyền ở lớp filesystem/ứng dụng. EFS Regional giữ dữ liệu qua nhiều AZ trong Region; tạo mount target ở các AZ cần truy cập giúp tránh phụ thuộc vào đường liên AZ. EFS One Zone đặt dữ liệu và mount target trong một AZ, nên không có cùng mô hình chịu mất AZ. Hai loại này có phạm vi sẵn sàng khác nhau, và quyền NFS không thể thay bằng IAM đơn giản. Một mount thành công vẫn chưa xác nhận dữ liệu được backup hay ứng dụng ghi đúng file.

Bài P08.C04.L03 — Object

S3 lưu object trong bucket ở Region đã chọn; ứng dụng gọi API theo key, không mount mặc định như block device hay NFS. S3 Standard được thiết kế phân tán qua nhiều AZ, còn một số storage class có mô hình một AZ hoặc thời gian truy xuất khác; phải đọc lớp lưu trữ thực tế thay vì suy từ chữ “S3”. Durability là xác suất giữ byte theo thiết kế, availability là khả năng truy cập dịch vụ khi cần, còn backup là khả năng trở về một trạng thái lịch sử đã chọn. Replication có thể chép cả thay đổi sai hoặc xóa tùy cấu hình; không thay cho versioning, lưu bản sao độc lập theo chính sách, kiểm tra quyền khôi phục và diễn tập restore. EBS tính theo dung lượng/hiệu năng được cấp và snapshot; EFS theo lớp lưu trữ, thông lượng/truy cập; S3 theo lớp, dung lượng, request và truy xuất; NAT và truyền liên AZ có thể thêm chi phí. Không chọn dịch vụ chỉ bằng giá mỗi GB, cũng không bịa giá cố định cho một Region chưa kiểm tra bảng giá hiện hành.

Bài tập P08.C04.L03.Q01

Một nhóm đặt database trên EBS cùng AZ với EC2, file dùng chung trên EFS Regional và báo cáo trên S3 Standard. AZ của EC2 hỏng, sau đó một job xóa nhầm object báo cáo. Mỗi lớp có thể tiếp tục phục vụ hoặc phục hồi theo điều kiện nào? Giải thích vì sao redundancy không đủ thay backup.

PHẦN 09 — Terraform và trạng thái hạ tầng

Cấu hình, state và hạ tầng thực lệch nhau thì quyết định thay đổi thế nào?

Provider dựng DAG rồi đọc state/API và thực hiện các tác vụ có thể thất bại từng phần; state là ánh xạ nhạy cảm.

Chương P09.C01 — HCL, provider và dependency DAG

Đọc tài nguyên, data source và cạnh implicit/explicit trong plan.

Bài P09.C01.L01 — Ngôn ngữ

Một `resource` khai báo object Terraform quản lý và vòng đời tạo, đọc, sửa, xóa của nó thông qua provider.

Một cấu hình HCL diễn đạt quan hệ mong muốn, không phải bản ghi đầy đủ về thứ đang tồn tại. `variable` nhận đầu vào của lần chạy; expression kết hợp các giá trị; `output` công bố giá trị sau khi Terraform tính được. `data` đọc một đối tượng do nơi khác quản lý để dùng trong đồ thị, còn `resource` yêu cầu provider quản lý một đối tượng trong state của cấu hình này. Các tên trong HCL là địa chỉ logic để các khối tham chiếu nhau; chúng không tự chứng minh rằng API phía xa đã tạo đúng đối tượng.

Khi một expression trong resource tham chiếu thuộc tính của resource khác, Terraform có cơ sở suy ra cạnh phụ thuộc. Nếu thứ tự cần thiết không thể hiện qua dữ liệu tham chiếu, người viết phải diễn đạt quan hệ ấy rõ ràng và giải thích vì sao. Đọc plan bằng cách đối chiếu địa chỉ logic, ID từ state và hành động dự kiến. Chỉ sau apply và phép kiểm tra hậu điều kiện tại hạ tầng thật mới được kết luận thay đổi đã đạt mục tiêu.

Bài P09.C01.L03 — DAG

Một tệp HCL mô tả điều ta muốn quản lý; state ghi Terraform đang gắn địa chỉ nào với object nào; API của nhà cung cấp trả về trạng thái hạ tầng tại thời điểm được hỏi. Ba thực thể ấy có thể khác nhau. `plan` đối chiếu chúng để đề xuất hành động, không phải bản ghi đã thay đổi thành công. Muốn dùng Terraform an toàn, người vận hành phải đọc cả nguyên nhân và tác dụng phụ của mỗi hành động, nhất là `replace` trên tài nguyên giữ dữ liệu.

Một `resource` khai báo object Terraform quản lý và vòng đời tạo, đọc, sửa, xóa của nó thông qua provider. `data source` đọc object có sẵn để làm đầu vào, nhưng không nhận quyền sở hữu vòng đời object ấy. Provider là chương trình riêng kết nối API, diễn giải schema và quyết định thuộc tính nào cập nhật tại chỗ, thuộc tính nào buộc thay object. Cấu hình không tự gọi API; Terraform khởi tạo provider rồi lập đồ thị tác vụ. Tham chiếu thuộc tính như `terraform_data.schema.output` tạo cạnh phụ thuộc suy diễn. `depends_on` tạo cạnh tường minh khi phụ thuộc có thật mà không biểu đạt được qua giá trị. Thêm cạnh tùy tiện có thể trì hoãn phép đọc data source và làm plan có nhiều giá trị chưa biết; thứ tự đồ thị không chứng minh dịch vụ đã sẵn sàng cho request.

Ví dụ HCL dưới đây chỉ dùng `terraform_data` của Terraform, không kết nối cloud và không tạo máy, mạng hay lưu trữ. Nó cho phép quan sát riêng quy tắc về đồ thị. `schema.output` là giá trị từ resource thứ nhất; `triggers_replace` khiến resource `service` được thay khi giá trị đó đổi. Cạnh `schema` tới `service` được suy ra từ tham chiếu. Cạnh `service` tới `audit` là chủ ý mô phỏng một bước kiểm tra phải chờ bước dịch vụ, nên khai báo tường minh. `audit` chỉ lưu chuỗi `ready`, không chạy health check thật; đừng diễn giải tên ấy thành bằng chứng readiness.

```
terraform {
  required_version = ">= 1.5, < 1.17"
}

variable "schema_revision" {
  type    = string
  default = "v1"
}

resource "terraform_data" "schema" {
  input = var.schema_revision
}

resource "terraform_data" "service" {
  triggers_replace = terraform_data.schema.output
}

resource "terraform_data" "audit" {
  input      = "ready"
  depends_on = [terraform_data.service]
}

output "service_id" {
  value = terraform_data.service.id
}
```

Với Terraform CLI tương thích, sao chép ví dụ vào một thư mục tạm riêng rồi chạy `terraform fmt -check`, `terraform init -backend=false`, `terraform validate` và `terraform plan`. Chỉ `apply` nếu muốn thử state cục bộ của ba `terraform_data` này. Đổi `schema_revision` từ `v1` sang `v2` rồi đọc plan: `schema` đổi `input`, `service` có thể phải replace do `triggers_replace`; `audit` chỉ có cạnh thứ tự. Giá trị ID có thể chưa biết trước apply. Đây là dự đoán từ cấu hình và hợp đồng resource, không phải transcript chạy thực tế; kết quả kiểm tra trên máy dựng được ghi riêng ngoài sách.

Bài tập P09.C01.L03.Q01

Trong ví dụ HCL, cạnh nào suy ra từ tham chiếu, cạnh nào khai báo tường minh? Đổi `schema_revision` sang `v2` tác động thế nào tới `service` và vì sao chuỗi `ready` không chứng minh ứng dụng sẵn sàng?

Chương P09.C02 — Plan, state và backend

Giải thích tại sao một plan có thể thất bại giữa chừng và state nhạy cảm.

Bài P09.C02.L01 — Ba trạng thái

State ánh xạ địa chỉ `resource` trong cấu hình tới ID object thực và giữ metadata cần thiết để cập nhật hoặc xóa. Nó không phải bản sao đầy đủ của hạ tầng, cũng không phải cấu hình mong muốn. Trong plan thông thường, Terraform đọc state và hỏi API để làm mới góc nhìn về object đang quản lý, sau đó so với cấu hình. Bởi thế, sửa object ngoài Terraform tạo drift có thể xuất hiện khi refresh; thuộc tính không được provider đọc hoặc API che giấu có thể không hiện. `-refresh=false` tránh phép đọc trong lượt đó nhưng có thể làm plan dựa trên state cũ; `refresh-only` cập nhật state theo thực tế mà không kéo thực tế về cấu hình, nên vẫn phải review trước apply. Data source có thể được đọc lúc plan hoặc bị hoãn tới apply nếu đầu vào chưa biết; không nên hiểu mọi giá trị hiện trong plan là một snapshot nguyên tử của toàn hệ thống.

Bài P09.C02.L02 — Plan/apply

Plan là dự báo dựa trên cấu hình, state, dữ liệu provider trả về và quyền tại thời điểm lập. Từ plan đến apply, người khác hoặc dịch vụ khác có thể đổi object; API có thể từ chối, hết quota, timeout hoặc trả kết quả không chắc chắn. Terraform thực hiện các tác vụ theo dependency graph, thường có song song giữa các nhánh độc lập, chứ không mở một giao dịch chung trên các API. Nếu ba resource đã tạo mà resource thứ tư thất bại, ba resource đầu có thể còn nguyên; phải đọc kết quả apply, state và API thực rồi lập kế hoạch tiếp. Exit code lỗi không đồng nghĩa mọi hành động đã rollback. Saved plan giúp ràng buộc lựa chọn đã review, nhưng vẫn không đóng băng hạ tầng ngoài Terraform hoặc bảo đảm API sẽ thành công.

`lifecycle` điều chỉnh vòng đời, không thay phép phân tích tác động. `create_before_destroy` đổi thứ tự replacement khi API và tên/định mức cho phép; nó không tự chuyển dữ liệu. `prevent_destroy` chặn plan phá object khi rule còn trong cấu hình, nhưng không bảo vệ object trước thao tác ngoài Terraform hay khi resource block bị loại bỏ. `ignore_changes` hợp lý khi một thuộc tính được giao cho bộ điều khiển khác, song nó có thể giấu drift có ý nghĩa; phải ghi rõ chủ sở hữu thuộc tính và đường giám sát. `destroy` là yêu cầu xóa các resource còn được quản lý trong state, không phải thao tác dọn thử nghiệm vô hại.

Data source có thể được đọc lúc plan hoặc bị hoãn tới apply nếu đầu vào chưa biết; không nên hiểu mọi giá trị hiện trong plan là một snapshot nguyên tử của toàn hệ thống.

`-refresh=false` tránh phép đọc trong lượt đó nhưng có thể làm plan dựa trên state cũ; `refresh-only` cập nhật state theo thực tế mà không kéo thực tế về cấu hình, nên vẫn phải review trước apply.

Bài P09.C02.L03 — Backend

State cục bộ thường là tệp JSON; state từ backend dùng chung cho phép nhiều người và CI cùng làm việc trên một bộ ánh xạ. Remote backend phải được chọn theo khả năng kiểm soát truy cập, mã hóa, lưu phiên bản, khôi phục, tính sẵn sàng và cơ chế khóa. Backend nào có locking thì Terraform có thể khóa state trước thao tác ghi để ngăn hai lượt Terraform cùng sửa một state; khóa không ngăn người sửa trực tiếp trên console, không khóa tất cả API và không phải backup. Không tắt khóa hoặc force-unlock khi chưa biết chủ khóa và tình trạng lượt chạy. Nếu state remote ghi thất bại sau khi API đã nhận thay đổi, phải bảo toàn bản state Terraform giữ lại để phục hồi và đối chiếu, thay vì chạy apply lần hai một cách mù quáng.

State và tệp plan có thể chứa secret, kể cả giá trị được đánh dấu `sensitive`; cờ này chủ yếu che hiển thị trong output chứ không làm state biến thành kho bí mật. Tệp cấu hình backend cũng có thể để lộ credential nếu nhúng trực tiếp. Giới hạn quyền đọc/ghi state, dùng cơ chế cấp credential ngắn hạn và lưu phiên bản/backup có thử restore. Một bản sao state hỏng hoặc cũ có thể khiến Terraform tưởng object chưa tồn tại, đề xuất tạo trùng hoặc phá nhầm; khi phục hồi cần xác định lineage, serial, thời điểm và object ID, kiểm tra khóa, lưu bản hiện tại an toàn rồi mới khôi phục bản đúng. Không chỉnh JSON state bằng tay khi còn công cụ và quy trình chính thức.

`import` nối một object đã tồn tại với một địa chỉ resource, không tự tạo thiết kế HCL đúng và không tự bảo vệ dữ liệu. Với import block, khai báo địa chỉ và ID, bảo đảm cấu hình resource khớp object, review plan và kiểm tra chỉ có một địa chỉ quản lý mỗi object trước apply. Nếu object vẫn thuộc một state khác, import vào state mới sẽ tạo hai chủ quản lý cạnh tranh. `state rm` chỉ bỏ ánh xạ quản lý, không xóa object thực; dùng nó như cách "sửa" một plan nguy hiểm mà không tìm nguyên nhân sẽ để lại hạ tầng mồ côi. Mỗi thao tác phục hồi hay chuyển state cần ghi quyền sở hữu và hậu điều kiện ở cả state lẫn API.

Bài tập P09.C02.L03.Q01

Một apply tạo thành công network và volume nhưng API tạo instance timeout; đồng thời remote backend báo lỗi ghi state. Vì sao không chạy lại ngay? Hãy nêu thứ tự đối chiếu và hậu điều kiện trước lượt tiếp theo.

Chương P09.C03 — Drift, import và refactor

Phân biệt thay đổi ngoài Terraform với đổi địa chỉ cấu hình.

Bài P09.C03.L01 — Drift

Giả sử Region `us-east-1`, Terraform CLI 1.16.0 và AWS provider 6.19.0 quản lý `aws_instance.orders` có `user_data` cấu hình là phiên A, `user_data_replace_on_change = true`, và root EBS `delete_on_termination = true`. Ứng dụng còn giữ 41 đơn chưa được sao lưu trên root volume; đó là thiết kế nguy hiểm có chủ ý để phân tích, không phải mẫu nên triển khai. State lưu instance ID `i-demo-01` và user data A. Người vận hành dừng instance, sửa user data thành B bằng console rồi khởi động lại; cấu hình HCL vẫn A. Trong mô hình provider đã nêu, nếu refresh đọc được B và plan hiển thị thay đổi `user_data` yêu cầu replacement, apply có thể terminate instance cũ; root EBS với cờ xóa theo termination có nguy cơ mất 41 đơn. Các ID và số liệu là giả định, không phải output Terraform/AWS đã chạy. Nếu provider không phát hiện thuộc tính ấy, hoặc object đã đổi cách khác, kết luận về plan phải theo kết quả thực, không áp dụng máy móc.

Nếu B đúng và cần quản lý lâu dài, sửa HCL cho khớp B qua review, refresh/plan lại và chỉ apply khi không còn replacement ngoài chủ đích. Nếu B sai nhưng dữ liệu cần giữ, thiết kế đường chuyển dữ liệu có snapshot nhất quán và restore thử, hoặc tách dữ liệu khỏi root volume trước khi tính tới thay instance. Nếu state trỏ sai ID vì một lượt apply trước bị gián đoạn, đối chiếu API và lịch sử state trước khi cân nhắc import/khôi phục; không nhập một object đang được quản lý ở địa chỉ khác. Nếu vẫn phải replace, ghi kế hoạch dừng ghi, chuyển lưu lượng, backup, restore, kiểm tra đơn 41/41 và phương án rollback ứng dụng lẫn dữ liệu; rollback mã không tự trả lại EBS đã bị xóa. Sau apply, so instance ID, root volume ID/chính sách xóa, trạng thái ứng dụng, dữ liệu đơn, request đại diện và plan mới; plan trống riêng nó chưa chứng minh nghiệp vụ đúng.

Chương P09.C04 — Thay đổi hạ tầng có dữ liệu

Quyết định dừng khi plan đề xuất replace instance chứa dữ liệu.

Bài P09.C04.L01 — Replacement

Đừng áp ngay chỉ vì plan là "đưa về cấu hình". Trước tiên đóng băng lượt triển khai, xác định workspace/backend/state serial, phiên provider, đúng account/Region/instance ID và trạng thái apply khác. Đọc plan có refresh để biết thuộc tính nào đổi, replacement do schema provider hay lifecycle nào, object nào sẽ bị destroy. Đối chiếu thuộc tính thực qua API và lịch sử thay đổi; hỏi người sửa B vì có thể B là bản sửa lỗi cần giữ. Kiểm tra root block mapping thực tế, vị trí 41 đơn, cơ chế ghi đang hoạt động, snapshot/backup và phép restore độc lập trước khi quyết định. `prevent_destroy` có thể là rào chắn bổ sung khi block còn hiện diện; không lấy nó làm bằng chứng dữ liệu đã an toàn. Không dùng `-refresh=false`, `-target` hoặc `ignore_changes` để che nguyên nhân rồi áp plan.

Tình huống chưa được chạy trên AWS và không có chi phí cloud phát sinh để chứng minh. Thuộc tính `user_data` theo provider này mặc định dẫn tới stop/start khi đổi; chính lựa chọn `user_data_replace_on_change = true` mới làm thay instance. Bỏ qua chi tiết ấy sẽ dạy sai nguyên nhân. Ngay cả khi replacement tạo máy mới trước, root volume cũ vẫn phải được bảo vệ và dữ liệu phải được chuyển/kiểm tra riêng.

Trong mô hình provider đã nêu, nếu refresh đọc được B và plan hiển thị thay đổi `user_data` yêu cầu replacement, apply có thể terminate instance cũ; root EBS với cờ xóa theo termination có nguy cơ mất 41 đơn.

Bài tập P09.C04.L01.Q01

Plan với refresh báo `aws_instance.orders` cần replace sau khi user data bị sửa ngoài Terraform. Hãy phân biệt cấu hình, state và thực tế; xác định điều kiện có thể mất 41 đơn, ba bằng chứng phải thu trước apply và hai hướng xử lý tùy B đúng hay sai.

Bài P09.C04.L02 — Điều phối

Một backend có khóa state giúp các lượt Terraform cùng sử dụng backend đó tránh ghi state đồng thời. Nó không khóa API của cloud và không ngăn người có quyền sửa security group trên console. Nếu một người sửa ngoài Terraform sau lúc plan được tạo, hạ tầng thực đã khác điều kiện đầu vào của plan. Dùng saved plan mà không kiểm lại phạm vi, state và thay đổi bên ngoài có thể áp một hành động khác điều người duyệt đã hình dung.

Trong tình huống hai pipeline cùng hướng đến một workspace, cần biết chúng có thật sự dùng cùng backend, cùng khóa và cùng state hay không. Nếu có, một lượt có thể phải đợi lượt kia; đó chỉ là bảo vệ tuần tự cho state. Sau khi khóa được nhả, lượt tiếp theo vẫn cần đọc trạng thái mới và đánh giá lại plan. Sửa đổi trực tiếp ở console là một actor thứ ba ngoài khóa ấy. Hãy ghi rõ object bị sửa, account/Region, thời điểm và lý do, rồi đối chiếu configuration, state và API thực trước khi quyết định nhập thay đổi vào cấu hình hay hoàn nguyên drift.

State có thể chứa giá trị nhạy cảm. `sensitive = true` chủ yếu hạn chế hiển thị ở một số đầu ra, không thay kiểm soát truy cập vào state. Versioning giúp tìm lại phiên bản cũ nhưng không tự mã hóa, giới hạn người đọc hoặc bảo đảm bản khôi phục đúng. Cần thiết kế quyền, bảo vệ lưu trữ, khóa và phương án phục hồi như các cơ chế riêng.

Bài tập P09.C04.L02.Q01

Hai pipeline cùng dùng một backend có locking; một người sửa security group trực tiếp trên console. Khóa bảo vệ được gì, drift nào nó không ngăn, và vì sao `sensitive = true` hoặc state versioning riêng lẻ chưa đủ bảo vệ dữ liệu?

PHẦN 10 — Ansible và hội tụ cấu hình

Làm sao áp trạng thái mong muốn lên các máy đang tồn tại mà không gây restart thừa?

Inventory chọn host, module quan sát rồi đổi trạng thái, handler hoãn tác động; kết quả theo từng host không phải giao dịch.

Chương P10.C01 — Inventory và đường điều khiển Ansible

Chỉ ra ai kết nối tới ai, bằng quyền nào và biến nào được dùng.

Bài P10.C01.L01 — Topology

Terraform ở chương trước quản lý ánh xạ giữa resource và object hạ tầng bằng state. Khi máy đã tồn tại, Ansible thường làm việc ở lớp khác: nối tới máy, kiểm tra file, package hoặc service hiện có rồi yêu cầu chúng đạt cấu hình mong muốn. Không có một state tập trung tương đương Terraform ghi toàn bộ máy đã như thế nào; mỗi module tự quan sát trạng thái liên quan ở lúc chạy và báo `ok`, `changed` hoặc `failed`. Vì vậy chạy lại playbook là một phép thử quan trọng, nhưng chỉ thấy `changed=0` chưa chứng minh ứng dụng phục vụ đúng.

Inventory liệt kê managed node và nhóm, đồng thời có thể gắn biến như địa chỉ, người dùng hoặc cách kết nối. Một play chọn nhóm trong inventory, đặt biến và tập task; playbook chứa một hoặc nhiều play. Control node đọc playbook, nạp module và plugin, rồi kết nối tới managed node. Với máy Linux, kết nối thường qua SSH; `ansible_connection=local` là lựa chọn khác khi chạy trên chính máy điều khiển. Kết nối thành công chỉ chứng minh đường điều khiển đã mở, không chứng minh ứng dụng đã sẵn sàng. Module là đơn vị thực hiện hành động cụ thể như `file`, `template` hoặc `systemd_service`; task gọi module cùng tham số và điều kiện. Module chạy với danh tính của connection user, trừ khi task hoặc play dùng `become` để nâng quyền. `become` thay danh tính thực thi trên đích, không tự cấp quyền sudo và không thay chính sách phân quyền của hệ điều hành.

Ansible và Terraform có thể cùng có mặt: Terraform tạo instance, network và volume; Ansible cấu hình ứng dụng trên instance được chọn. Cần xác định một chủ quản lý cho từng thuộc tính. Nếu Terraform viết cùng file qua user data còn Ansible cũng ghi file ấy, hai bộ điều khiển có thể thay nhau tạo drift. Inventory có thể lấy từ hạ tầng nhưng tên host trong inventory không phải ID object trong Terraform state; cần kiểm tra đích thật trước khi chạy. Không dùng một kết quả `ping` của Ansible làm bằng chứng dịch vụ HTTP hay cơ sở dữ liệu hoạt động.

Bài P10.C01.L02 — Đơn vị chạy

Facts là dữ liệu Ansible thu được từ managed node, chẳng hạn hệ điều hành hoặc network interface, nếu bật `gather_facts` hoặc gọi module thu thập; fact không phải một nguồn luôn mới khi dùng cache.

Một play chọn tập host từ inventory rồi chạy các task theo thứ tự đối với các host đó. Mỗi task gọi module với tham số đã tính từ biến và facts; module so trạng thái quan sát được với yêu cầu của task, sau đó trả về `ok`, `changed` hoặc `failed`. Vì vậy bản playbook là ý định hội tụ, còn recap là báo cáo của lần chạy cụ thể, không phải bản sao state lưu lâu dài như Terraform.

Khi dùng fact trong điều kiện, cần biết fact được lấy tại thời điểm nào. Một fact đã cache có thể phản ánh trạng thái trước drift; một host không truy cập được có thể không có fact mới để đánh giá. Nếu một task được báo `changed`, tìm đúng thuộc tính đã đổi và xem handler nào được thông báo. Nếu

lượt chạy thứ hai vẫn `changed` dù đầu vào không đổi, phải kiểm tra module, template, nguồn biến và điều kiện task trước khi gọi playbook là idempotent.

Bài P10.C01.L03 — Quyền/biến

`become` thay danh tính thực thi trên đích, không tự cấp quyền sudo và không thay chính sách phân quyền của hệ điều hành.

Một task có thể đọc biến từ nhiều mức như inventory, play, role và lời gọi thực thi. Giá trị cuối cùng cần được xác định theo thứ tự ưu tiên thực sự của phiên bản Ansible đang dùng, thay vì đoán từ vị trí xuất hiện trong tệp. Trước khi sửa quyền một đường dẫn, ghi rõ danh tính kết nối, danh tính sau privilege escalation, chủ sở hữu mong muốn và mode hậu điều kiện. Nếu thiếu quyền, task phải báo lỗi; chạy lại với quyền rộng hơn mà không xét ranh giới truy cập có thể che giấu sai cấu hình.

`become` chỉ yêu cầu thực thi dưới danh tính khác nếu cơ chế trên máy đích cho phép. Nó không biến một lệnh tùy ý thành thao tác an toàn để chạy lặp lại. Secret dùng cho kết nối hoặc escalation cần được chuyển qua kênh quản lý phù hợp và tránh in vào log hoặc diff. Khi kiểm tra một lần hội tụ, đối chiếu mode và nội dung file thực tế sau cả lượt đầu lẫn lượt lặp lại; nhãn `ok` một mình không chứng minh hậu điều kiện quan trọng đã đúng.

Chương P10.C02 — Hội tụ, handler và thất bại từng host

Chứng minh lượt đầu changed, lượt hai ok và drift sửa đúng phạm vi.

Bài P10.C02.L01 — Idempotency

Một task idempotent có thể chạy lại với cùng đầu vào và trạng thái quan sát mà không gây thêm thay đổi ngoài ý muốn. `file` với `mode: '0750'` chỉ sửa quyền khi khác; `template` so nội dung tạo ra với file đích rồi báo `changed` khi cần ghi. `ok` nghĩa là module không báo thay đổi ở lượt đó, không phải lời bảo đảm toàn hệ thống đúng. `failed` ngăn các task tiếp theo trên host theo quy tắc mặc định, nhưng không tự đảo những task trước đã thành công. `changed` là kết quả do module hoặc `changed_when` báo, không phải phép đo độc lập rằng người dùng được phục vụ tốt hơn. Một `command` hoặc `shell` gọi utility thường không có hiểu biết trạng thái đích: nó có thể báo changed mỗi lần và lặp tác dụng phụ, trừ khi người viết tạo điều kiện, kiểm tra và ranh giới lỗi đúng.

Trong thư mục chứa playbook, `ansible-playbook -i localhost, playbook.yml --syntax-check` kiểm tra cú pháp; dấu phẩy khiến `localhost,` là danh sách host một phần tử. Chạy với `-i localhost,` mà không dùng inventory production. Để kiểm thử quyền trên Linux, có thể thêm `-e sandbox=<thư_mục_tạm_riêng>` trỏ tới filesystem hỗ trợ mode POSIX, không cần chuyển repository hoặc dùng root. Sau lượt đầu, đọc recap từng task, nội dung `atlas.conf`, mode thực của `data` và marker. Chạy lại cùng biến: nếu đã hội tụ, task template và quyền là `ok`, handler không được thông báo. Để thử drift một cách cô lập, đổi `atlas.conf` sang `listen_port=9090`, chạy lại và so marker; chỉ đổi mode thư mục `data` rồi chạy lại phải sửa quyền mà không chạm marker. Trên filesystem không biểu diễn quyền POSIX đúng, thí nghiệm mode phải ghi SKIP thay vì diễn giải sai kết quả. Đây là thí nghiệm về module và handler, không chứng minh một daemon đã reload thành công.

Task `file` sửa quyền thư mục dữ liệu nhưng không notify: thay quyền riêng nó không phải bằng chứng cần restart.

Bài tập P10.C02.L01.Q01

Trong ví dụ cục bộ, lượt đầu tạo file và thư mục có thể báo gì? Lượt thứ hai cùng đầu vào nên có trạng thái nào cho template và handler? Vì sao `changed=0` vẫn không chứng minh Atlas phục vụ HTTP đúng?

Bài P10.C02.L02 — Handler

Task chạy theo thứ tự đã khai báo trên từng host; các host có thể tiến hành song song theo chiến lược và số fork. `when` bỏ qua task khi điều kiện không đúng, nhưng điều kiện dựa trên biến đăng ký từ task trước có thể khác trong check mode. `notify` chỉ xếp handler khi task báo changed; cùng một handler được thông báo nhiều lần thường chỉ chạy một lần tại điểm flush. Handler mặc định chạy sau nhóm task thích hợp, hoặc sớm hơn qua `meta: flush_handlers`. Nếu host lỗi trước lúc handler được chạy, handler thường không chạy trên host đó; `force_handlers` thay hành vi này nhưng có thể khiến restart trên cấu hình chưa được xác minh. Bởi vậy phải thiết kế thứ tự kiểm tra, điểm restart và hậu điều kiện, không chỉ bật `force_handlers` để "chắc ăn".

Playbook dưới đây chỉ nhắm `localhost`, dùng connection local, `become: false` và ghi trong `.qa-ch14/fixture` thuộc giáo trình. `atlas.conf` là cấu hình mẫu; handler chỉ chạm marker `reload-requested`, không restart dịch vụ. Template gán cổng và thư mục dữ liệu từ biến. Task `file` sửa quyền thư mục dữ liệu nhưng không notify: thay quyền riêng nó không phải bằng chứng cần restart. Task template mới notify handler khi nội dung hoặc quyền của file cấu hình đổi. Phép `stat`/`assert` kiểm tra hậu điều kiện quyền sau lượt thật; nó được bỏ qua trong check mode vì lúc đó thay đổi chưa xảy ra.

```
---
- name: Converge an isolated Atlas configuration
  hosts: localhost
  connection: local
  gather_facts: false
  become: false
  vars:
    sandbox: "{{ playbook_dir }}/../../.qa-ch14/fixture"
    listen_port: 8080
  tasks:
    - name: Keep the sandbox directory
      ansible.builtin.file:
        path: "{{ sandbox }}"
        state: directory
        mode: '0750'
    - name: Restore data directory access
      ansible.builtin.file:
        path: "{{ sandbox }}/data"
        state: directory
        mode: '0750'
    - name: Converge application configuration
      ansible.builtin.template:
        src: templates/atlas.conf.j2
        dest: "{{ sandbox }}/atlas.conf"
        mode: '0640'
      notify: Record reload request
    - name: Observe final data directory mode
      ansible.builtin.stat:
        path: "{{ sandbox }}/data"
      register: data_directory

    - name: Reject a failed permission repair
      ansible.builtin.assert:
        that: data_directory.stat.mode == '0750'
      when: not ansible_check_mode

  handlers:
    - name: Record reload request
      ansible.builtin.file:
        path: "{{ sandbox }}/reload-requested"
        state: touch
      when: not ansible_check_mode

listen_port={{ listen_port }}
data_dir={{ sandbox }}/data
```

Bài tập P10.C02.L02.Q01

Chỉ mode thư mục `data` thành `0700`; `atlas.conf` không đổi. Playbook sửa gì, handler có chạy không? Vì sao `command: chmod 0750 ...` có `notify` lại có thể restart thừa?

Bài P10.C02.L03 — Failure

`--syntax-check` kiểm tra cấu trúc playbook, chưa kết nối và chưa xác nhận đường file, quyền hoặc hành vi module trên host thật. `--check` dự đoán thay đổi mà không ghi trên managed node với module hỗ trợ; module không hỗ trợ có thể bị bỏ qua, task dùng kết quả task trước có thể không được đánh giá như lúc apply. `--diff` hiển thị chênh lệch khi module hỗ trợ và có thể làm lộ dữ liệu nhạy cảm trong template, nên không bật tùy tiện cho secret. Check mode không dự báo được API sẽ từ chối, ổ đĩa hết chỗ, handler restart thất bại hay request sau restart sai. Chạy trên một host đại diện và kiểm tra sau thay đổi vẫn cần thiết.

Nếu host lỗi trước lúc handler được chạy, handler thường không chạy trên host đó; `force_handlers` thay hành vi này nhưng có thể khiến restart trên cấu hình chưa được xác minh.

Bài tập P10.C02.L03.Q01

Một lượt `--check --diff` báo sẽ sửa config, nhưng apply thật thất bại sau khi một host đã restart. Hãy nêu ba giới hạn của check/diff, trạng thái từng phần cần ghi lại và hậu điều kiện trước khi chạy tiếp.

Chương P10.C03 — Vai trò, secrets và triển khai theo đợt

Tổ chức cấu hình nhiều host với blast radius kiểm soát được.

Bài P10.C03.L01 — Tái sử dụng

Variables có thể đến từ play, inventory, group/host vars, role, CLI và nguồn khác; ưu tiên của chúng phải được kiểm tra khi cùng tên xuất hiện nhiều nơi. Facts là dữ liệu Ansible thu được từ managed node, chẳng hạn hệ điều hành hoặc network interface, nếu bật `gather_facts` hoặc gọi module thu thập; fact không phải một nguồn luôn mới khi dùng cache. Role gom task, handler, template và biến theo cấu trúc tái sử dụng; nó không tạo ranh giới quyền mới, và handler của role tham gia phạm vi handler của play. Một role dùng lại rộng rãi vẫn cần review quyền, biến mặc định và tác dụng phụ trên nhóm host thực tế.

Bài P10.C03.L02 — Phân đợt

Giả sử hai máy `atlas-a` và `atlas-b` phục vụ sau load balancer. Config đúng là `listen_port=8080`; service chạy dưới user `atlas`, cần quyền đọc file config và đi qua thư mục dữ liệu. Một playbook lỗi chạy `command: chmod 0700 /srv/atlas/data` ở mọi lượt rồi `notify` restart, còn task template đặt `owner: root`, `group: root`, `mode: '0600'` cho config. Trên máy chỉ cần đổi quyền thư mục, `command` vẫn có thể báo changed và kích hoạt handler không cần thiết; restart sau đó có thể khiến service user không đọc được config hoặc data. Nếu cả hai host restart cùng lúc, khả năng phục vụ có thể mất. Đây là chuỗi giả định, không phải log đã đo; `chmod` có thể thành công về exit status dù tạo quyền sai cho ứng dụng.

Phân tích theo thứ tự: xác nhận inventory thực sự chọn hai máy, connection user và `become` dùng quyền nào; xem task nào báo changed và vì sao; xác nhận mode/owner/group cùng UID/GID của service, không chỉ nhìn ký hiệu `0700` hoặc `0600`. So file trước và sau, số lần notify, điểm handler chạy, trạng thái systemd và request từ client. Nếu template bị ghi sai quyền, sửa ownership/mode trước khi restart. Dùng module `file` và `template` để hội tụ về chủ sở hữu/quyền đã kiểm tra; chỉ task đổi nội dung config mới notify handler `ansible.builtin.systemd_service` với `state: restarted` khi ứng dụng thực sự cần đọc lại config. Task bảo đảm service đang chạy dùng `state: started`, không dùng `restarted` ở mọi lượt. Với nhiều máy, dùng nhóm thử trước rồi `serial: 1` hoặc chiến lược phù hợp, rút từng máy khỏi lưu lượng nếu cần và kiểm tra request đại diện trước máy tiếp theo. `serial: 1` giới hạn số host trong một batch, không tự rút lưu lượng hoặc bảo đảm health check đúng.

Một sửa đổi an toàn bắt đầu từ backup cấu hình và đường khôi phục, kiểm tra syntax của cấu hình bằng chính ứng dụng nếu nó có chế độ validate đáng tin, kiểm tra quyền file và dữ liệu, rồi mới restart. Nếu validate task thất bại, không nên ghi đè file tốt hoặc gọi handler. Sau restart, kiểm tra tiến trình, log, cổng lắng nghe và request qua đúng load balancer; trạng thái `active` của systemd chỉ là một phần bằng chứng. Khi một host lỗi giữa play, các task đã hoàn tất trên host khác không tự rollback. Ghi lại host nào đã đổi nội dung, host nào mới sửa quyền, handler nào đã chạy và phiên bản config hiện tại; khoanh tác động rồi phục hồi từng host theo trạng thái thực. Secrets trong biến,

template và diff cần được quản lý bằng cơ chế bảo mật riêng, không đặt mật khẩu vào inventory Git hay in diff/log.

Bài tập P10.C03.L02.Q01

Playbook lỗi ở mục 14.4 báo task `chmod` changed và restart hai máy. Hãy dựng chuỗi nguyên nhân có thể dẫn đến gián đoạn, nêu bằng chứng cần thu để phân biệt lỗi quyền với lỗi nội dung config, và đề xuất thứ tự sửa trên từng máy.

PHẦN 11 — Kubernetes và cloud native

Khai báo biến thành Pod sẵn sàng phục vụ nhờ những vòng điều khiển nào?

API lưu spec, controller/scheduler/kubelet/dataplane hội tụ bất đồng bộ; status, endpoint và request là các chứng cứ khác nhau.

Chương P11.C01 — API, etcd và reconciliation

Theo một spec từ apply tới các controller và node.

Bài P11.C01.L01 — Control plane

Ở Chương 14, một lần chạy Ansible quan sát rồi sửa cấu hình trên các máy đã chọn. Kubernetes đặt khai báo trong API để nhiều bộ điều khiển liên tục quan sát và sửa phần việc của mình. Người vận hành gửi `spec` mô tả điều muốn có; `status` là quan sát mà hệ thống cập nhật sau đó. Khoảng cách giữa hai phần là chuyện bình thường trong một thời gian, không phải bằng chứng `kubectl apply` đã thất bại. Ngược lại, API chấp nhận object cũng không chứng minh ứng dụng đã phục vụ. Muốn chẩn đoán phải lần theo đối tượng nào đã xuất hiện, ai chịu trách nhiệm bước kế tiếp, và điều kiện nào còn chưa đạt.

Giả sử ta khai báo Deployment `atlas` có hai bản sao. `kubectl` gửi yêu cầu đến API server. API server xác thực người gọi, kiểm tra quyền và qua các bước admission/validation áp dụng, rồi lưu object bằng kho trạng thái của cluster, thường là etcd. etcd giữ dữ liệu API; nó không chạy container và không phải kho dữ liệu của ứng dụng. Phản hồi apply thành công xác nhận yêu cầu khai báo đã được API chấp nhận, không phải xác nhận hai Pod đã chạy. Bản sao etcd và backup của nó liên quan đến khả năng khôi phục control plane; chúng không tự backup database bên trong workload.

Controller manager chạy các vòng điều khiển. Deployment controller thấy Deployment mới thì tạo hoặc điều chỉnh ReplicaSet; ReplicaSet controller tạo Pod để số bản sao tiến gần `spec.replicas`. Controller thường đọc/ghi qua API server thay vì trực tiếp mở tiến trình ứng dụng. Scheduler theo dõi Pod chưa có node, chọn node theo tài nguyên được yêu cầu và các ràng buộc, rồi ghi quyết định gán node vào API. Chọn node không có nghĩa image đã tải hoặc ứng dụng đã mở cổng. Kubelet trên node nhận Pod được gán, làm việc với container runtime qua Container Runtime Interface để chuẩn bị sandbox, tải image và chạy container; nó báo trạng thái quan sát về API. Runtime thực thi container, còn kubelet điều phối vòng đời Pod trên node. Network plugin và triển khai Service trên node tham gia đường truyền gói tin; không thể lấy kết quả scheduler làm bằng chứng đường dữ liệu đã thông.

Bài P11.C01.L02 — Reconciliation

`spec` và `status` có chủ thể ghi khác nhau. Khi sửa `spec`, `metadata.generation` của object phù hợp tăng; trường `status.observedGeneration` của một số controller, khi được công bố, giúp nhận ra controller đã quan sát thế hệ khai báo nào. Không phải mọi kind/status đều cung cấp cùng một trường hoặc cập nhật cùng lúc, nên phải đọc theo hợp đồng của kind.
`metadata.resourceVersion` là chuỗi phiên bản nội bộ để watch và cập nhật có điều kiện; chỉ so bằng nhau hoặc gửi lại nguyên dạng theo API, không biến nó thành đồng hồ hoặc lấy số lớn hơn để kết luận Pod đã mới hơn Deployment. Một lần `get` là ảnh chụp tại thời điểm đọc; quan sát thay đổi cần watch hoặc đọc lại, không suy từ một snapshot.

Bài tập P11.C01.L02.Q01

`kubectl apply` trả thành công, Deployment mong muốn hai Pod nhưng `availableReplicas=0`. Hãy nêu chuỗi object và thành phần cần quan sát để phân biệt controller chưa tạo Pod, scheduler chưa chọn node và kubelet chưa chạy container. Mỗi bước có thể kết luận đến đâu?

Bài P11.C01.L03 — Node boundary

Có hai đường cần vẽ riêng trong đầu. Đường điều khiển là client → API server → object trong kho trạng thái → controller/scheduler → kubelet, kèm các lần ghi status ngược về API. Đường dữ liệu của một request là client → địa chỉ/port Service → cơ chế chuyển tiếp trên node → IP/port Pod → ứng dụng, tùy kiểu Service và cách cluster triển khai mạng. API server không chuyển mọi HTTP request của ứng dụng qua nó. Một control plane khỏe có thể đi kèm Service không có backend; một Pod chạy được cũng không chứng minh client đi tới được nó.

Scheduler theo dõi Pod chưa có node, chọn node theo tài nguyên được yêu cầu và các ràng buộc, rồi ghi quyết định gán node vào API.

Kubelet trên node nhận Pod được gán, làm việc với container runtime qua Container Runtime Interface để chuẩn bị sandbox, tải image và chạy container; nó báo trạng thái quan sát về API.

Chương P11.C02 — Pod, Deployment và readiness

Phân biệt Running, Ready, Available và rollout complete.

Bài P11.C02.L01 — Workload objects

Pod là đơn vị được lập lịch; các container của Pod chia sẻ ngữ cảnh mạng và các volume được khai báo cho Pod. Pod có danh tính và vòng đời riêng. Deployment giữ ý định cập nhật một tập Pod qua các ReplicaSet; ReplicaSet giữ số Pod theo selector và Pod template. Quan hệ quản lý được ghi ở `metadata.ownerReferences`: Pod thường thuộc ReplicaSet, ReplicaSet thuộc Deployment. Khi Pod mất, controller có thể tạo Pod khác, với UID/IP khác. Selector là phép chọn theo label, không phải quan hệ sở hữu: Service chọn Pod có label khớp dù Pod đó thuộc controller nào. Nếu selector của Service sai, việc Deployment đủ bản sao không sửa được đường nối này.

Pod `Running` nói Pod đã gắn vào node và container đã được tạo, với ít nhất một container còn chạy hoặc đang khởi động lại.

Bài P11.C02.L02 — Probe

Pod `Running` nói Pod đã gắn vào node và container đã được tạo, với ít nhất một container còn chạy hoặc đang khởi động lại. Nó không nói request thành công. Readiness probe kiểm tra điều kiện để đưa Pod vào tập backend bình thường; probe thất bại có thể giữ Pod `Running` nhưng `Ready=False`. Trong EndpointSlice, một endpoint có các điều kiện như `ready`, `serving`, `terminating`; không đếm mọi địa chỉ tìm thấy như backend đang phục vụ. Một Service có selector được EndpointSlice controller cập nhật theo các Pod khớp; địa chỉ Service và `targetPort` vẫn phải đi tới đúng cổng ứng dụng. `containerPort` trong Pod là khai báo thông tin cổng, không tự làm ứng dụng lắng nghe.

Đây là dữ liệu giả định để luyện đọc trạng thái, không phải output từ một cluster đã chạy. Ở namespace `demo`, bản sửa đổi khai báo Deployment `atlas` hai bản sao; Pod template có label `app=atlas`, container dự kiến nghe TCP 8080, readiness HTTP gọi `/ready` trên 8080. Service `atlas` công bố port 80, `targetPort: 8080`, nhưng selector bị gõ `app=atals`. Ứng dụng vừa đổi đường readiness thành `/health/ready`, còn `/ready` trả 404. Hai lỗi độc lập cùng ngăn lưu lượng. Manifest minh họa không đặt `metadata.namespace`; nếu thử trong môi trường cô lập đã được cho phép, dùng `kubectl -n demo apply -f` với đúng đường dẫn manifest, hoặc khai báo namespace trong từng object. Không sửa cả hai lỗi theo trực giác trước khi biết từng ranh giới đã xảy ra gì.

Manifest tối giản dưới đây cho thấy trực tiếp hai sai khác ấy. Container minh họa tạo file ở `/health/ready` rồi phục vụ thư mục qua HTTP; probe lại gọi `/ready`, còn Service chọn nhãn khác nhãn Pod. Đây là cấu hình cố ý lỗi để đọc spec, không phải kết quả đã chạy trong cluster; image tag cũng chưa được xác minh bằng một lần pull.

```
# Minh họa giả định trong namespace demo; cố ý sai probe và Service selector.
# Image tag dùng để đọc spec, chưa được pull hoặc chạy trên cluster.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: atlas
spec:
  replicas: 2
  selector:
    matchLabels:
      app: atlas
  template:
    metadata:
      labels:
        app: atlas
    spec:
      containers:
        - name: atlas
          image: python:3.12-alpine
          command: ["/bin/sh", "-ec"]
          args:
            - |
              mkdir -p /tmp/site/health
              printf 'ready\n' > /tmp/site/health/ready
              cd /tmp/site
              exec python -m http.server 8080
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /ready # Sai: file phục vụ ở /health/ready.
              port: 8080
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
---
apiVersion: v1
kind: Service
metadata:
  name: atlas
spec:

  selector:
    app: atals # Sai: Pod mang app=atlas.
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

Ở bước đầu, apply trả thành công và `get deployment` cho thấy `spec.replicas=2` nhưng `status.availableReplicas=0`. Đây chỉ xác nhận khai báo và trạng thái chưa hội tụ. Xem ReplicaSet và Pod có `ownerReferences`/label đúng để biết Deployment controller và ReplicaSet controller đã tạo gì. Nếu không có Pod, kiểm tra Deployment/ReplicaSet events và quota hoặc admission; đừng nhảy ngay sang mạng. Nếu Pod có `spec.nodeName` rỗng và event

`FailedScheduling`, xét requests CPU/memory, node affinity/taint và năng lực node theo đúng event. Nếu đã có node, scheduler không còn là giả thuyết chính cho lần đó.

Trong dữ kiện tiếp theo, hai Pod có node, container state `Running`, không có event `FailedScheduling` hay `Failed` khi pull image; image ID đã được kubelet ghi. Những quan sát này loại bớt lỗi gán node và tải image cho hai Pod được xem, nhưng không chứng minh mọi lần pull trong tương lai hoặc mọi node đều tốt. Nếu gặp `ErrImagePull` hay `ImagePullBackOff` trong sự cố khác, kiểm tra image reference/digest, credential registry, quyền và đường mạng từ node; event và container waiting reason quan trọng hơn việc đoán từ tên Deployment. Ở ca này, điều kiện Pod `Ready=False` và event readiness báo HTTP 404 cho `/ready`. Ứng dụng trả 200 tại `/health/ready` trong kiểm tra từ đúng network context của Pod. Quan hệ nhân quả có thể kiểm được: kubelet đang thử sai path, nên Pod không trở thành endpoint `ready`. Sửa probe theo hợp đồng ứng dụng, áp dụng bản sửa và đợi Pod thế hệ mới đạt Ready; đừng xóa probe để có màu xanh giả.

Một thứ tự quan sát ngắn là: đọc object và generation → tìm ReplicaSet/Pod do controller tạo → xem `spec.nodeName`, events và container state → so Pod `Running` với điều kiện `Ready` và probe → so Service selector với label → xem tất cả EndpointSlice của Service và điều kiện endpoint → gửi request qua đúng Service. Lệnh thích hợp có thể là `kubectl -n demo describe pod <pod>`, `kubectl -n demo get pod <pod> -o yaml`, `kubectl -n demo get svc atlas -o yaml` và `kubectl -n demo get endpointslices -l kubernetes.io/service-name=atlas -o yaml`. Kết quả từng lệnh là bằng chứng ở thời điểm chạy, không phải suy luận thay cho cluster chưa có. Khi thu log và YAML cần tránh chia sẻ Secret hoặc token.

Bài P11.C02.L03 — Rollout

Deployment `Available=True` phản ánh mức sẵn sàng tối thiểu theo chiến lược và thời gian `minReadySeconds`, không đồng nghĩa tất cả Pod đều mới. `Progressing=True` có thể còn đang rollout; chỉ riêng giá trị True không chứng minh rollout hoàn tất. Với Deployment v1.34, điều kiện `Progressing=True` kèm reason `NewReplicaSetAvailable` biểu thị rollout đã hoàn tất vào thời điểm controller báo; `kubectl rollout status` cho một phép đợi/kiểm tra phù hợp hơn nhìn một cột. Ngay cả khi rollout hoàn tất, Service selector, cổng, DNS, policy mạng và request thực từ client vẫn cần kiểm chứng. `Available` có thể đổi sau đó khi Pod gặp lỗi; không lấy một ảnh chụp tốt làm cam kết vĩnh viễn.

Các vòng điều khiển chạy bất đồng bộ; có thể tạm thời thấy Deployment mới, ReplicaSet mới nhưng Pod cũ, hoặc status chưa theo kịp spec. Watch có thể đứt, API có thể quá tải, node có thể hết tài nguyên, registry không truy cập được hoặc quyền bị từ chối. Controller có thể thử lại và báo điều kiện lỗi, nhưng không thể tạo tài nguyên vượt quota hay làm ứng dụng lỗi trở nên đúng. `progressDeadlineSeconds` giúp nhận biết rollout không tiến triển; nó không tự rollback một Deployment đã lỗi. Cần định nghĩa ngưỡng dừng, cách sửa hoặc rollback theo bằng chứng và kiểm tra request sau can thiệp. Mỗi API write và bước điều khiển có thể thành công một phần; reconciliation không phải giao dịch trên cả cluster.

Bài tập P11.C02.L03.Q01

Deployment báo `Available=True`, `Progressing=True`. Có được kết luận rollout hoàn tất và người dùng nhận response đúng không? Giải thích vai trò reason, thế hệ đã quan sát, EndpointSlice và request từ client.

Chương P11.C03 — Scheduler và ngân sách tài nguyên

Giải thích Pending vì requests/taint dù CPU tức thời thấp.

Bài P11.C03.L01 — Placement

Chương 15 theo một khai báo từ API tới Pod và Service. Nhưng một Pod chỉ phục vụ ổn định khi đồng thời tìm được node phù hợp, có CPU và bộ nhớ để làm việc, có volume cần thiết và nhận được request qua đường mạng thực. Bốn điều kiện này do những thành phần khác nhau xử lý; `Running` hay một con số replica không gộp chúng thành một bảo đảm. Chương này lần theo từng ranh giới rồi mới dùng metric CPU để hiểu HPA. Các số liệu trong tình huống là dữ liệu giả định có thể tính lại, không phải output từ cluster của người học.

Node công bố `capacity` và `allocatable`. `allocatable` là phần dành được cho Pod sau khi trừ ngân sách dự trữ phù hợp cho hệ thống và kubelet, không phải CPU đang rảnh tức thời. Scheduler lọc các node khả thi bằng requests CPU/bộ nhớ của Pod, tài nguyên còn có thể cấp theo các Pod đã được đặt, ràng buộc volume, taint, affinity và các quy tắc khác; sau đó chấm điểm để chọn node. Một node đang dùng CPU thực tế thấp vẫn có thể bị loại nếu tổng requests đã chiếm hết allocatable. Ngược lại, Pod được đặt không có nghĩa nó luôn có đủ CPU khi nhiều workload cùng tranh chấp.

Ví dụ giả định: node A có `allocatable.cpu=2000m`, các Pod đã đặt yêu cầu tổng `1700m`, nên chỉ còn `300m` để nhận request mới. Node B có `2000m`, đã yêu cầu `1400m` nên còn `600m`, nhưng mang taint `dedicated=infra:NoSchedule`. Pod mới yêu cầu `500m` và không có toleration tương ứng: A thiếu `200m`, B bị taint chặn. Kể cả đo usage của A và B chỉ `200m`, Pod vẫn chưa thể đặt theo dữ kiện này. Cần đọc `FailedScheduling` event, requests thực của mọi container, `status.allocatable`, taint và affinity của chính Pod; một event tại một thời điểm không chứng minh nguyên nhân duy nhất của mọi lần thử sau. Toleration chỉ bỏ một rào cản taint, không ép Pod lên B; node affinity dạng `requiredDuringSchedulingIgnoredDuringExecution` là ràng buộc bắt buộc lúc đặt, còn `preferred` chỉ ảnh hưởng lựa chọn khi có node phù hợp. Không nới ràng buộc an toàn hoặc sửa taint của node production chỉ để biến cột Pending thành Running.

Xét một chuỗi quan sát giả định của dịch vụ `atlas` trong namespace `demo`; mỗi ảnh chụp dưới đây là dữ kiện học tập, không phải log thật. Đầu tiên, client thấy timeout. Deployment mong muốn ba replica, một Pod mới Pending với event `FailedScheduling`. Node A còn `300m` theo requests, Pod cần `500m`; node B còn `600m` nhưng có taint mà Pod không toleration. Việc `kubectl top node` cho usage thấp không phủ định hai rào cản này. Hướng xử lý là bố trí năng lực/node phù hợp hoặc xem lại ràng buộc theo yêu cầu vận hành, rồi kiểm tra Pod thực sự được gán node; không vội tăng HPA max.

Bài tập P11.C03.L01.Q01

Node A còn `300m` allocatable chưa được request; B còn `600m` nhưng có taint `NoSchedule`. Pod yêu cầu `500m` và không có toleration. Vì sao usage thực 10% không giúp scheduler đặt Pod? Nêu quan sát cần kiểm tra trước khi đổi ràng buộc.

Bài P11.C03.L02 — Runtime budget

`requests.cpu` dùng để đặt chỗ trên node và thường ảnh hưởng trọng số chia CPU khi cạnh tranh; `limits.cpu` là trần CPU time được kernel áp bằng cgroup trong các khoảng quota. Nếu chương trình cần thêm CPU nhưng bị trần, counter `cpu.stat.nr_throttled` và `throttled_usec` tăng trên cgroup tương ứng; đó là bằng chứng đã bị quota chặn trong cửa sổ đo, chưa chứng minh mọi request chậm đều do CPU. Kết hợp hiệu `usage_usec` để tính số lõi trung bình theo thời gian như Chương 2, cùng latency từng chặng và cửa sổ metric tương ứng. `limits.memory` khác: khi cấp phát không thể đáp ứng dưới giới hạn, kernel có thể giết tiến trình; đối chiếu `OOMKilled`, restart count, event và counter `memory.events` của đúng cgroup, không đọc một giá trị `memory.current` sau khi process đã chết rồi kết luận chưa từng thiếu bộ nhớ. Memory request chủ yếu tham gia scheduling và đánh giá áp lực node; vượt request không tự động bị OOM như vượt limit có thể dẫn tới OOM.

Với QoS truyền thống ở mức container, Pod `Guaranteed` có request và limit CPU lẫn memory bằng nhau cho từng container; `BestEffort` không khai báo chúng; trường hợp còn lại có ít nhất một khai báo phù hợp là `Burstable`. QoS tham gia ưu tiên khi node chịu áp lực và xử lý OOM/eviction, nhưng không thay thế requests/limits, dữ liệu sử dụng thực hoặc policy của kernel. Kubernetes v1.34 còn có tùy chọn tài nguyên ở mức Pod sau feature gate; khi dùng tính năng ấy phải đối chiếu hợp đồng phiên bản thay vì áp máy móc tiêu chí container trên mọi cluster. Pod có thể `Pending` vì không đủ requests, `Running` nhưng bị CPU throttling, hoặc restart vì OOM; cả ba có thể biểu hiện cùng một latency từ phía client nhưng cần bằng chứng khác nhau.

Bài P11.C03.L03 — Resource metric tối thiểu

CPU usage là mức CPU time tiêu thụ trong một khoảng đo: hiệu counter CPU time chia cho thời gian thực. Nếu một container tích lũy thêm 3 giây CPU trong 10 giây tường, trung bình nó dùng `0,3` lõi, tức `300m`. Một mẫu không cho biết request nào chờ lâu, mức dùng giữa hai mẫu hay khả năng sẽ tăng tải sau đó. Metrics Server thường thu resource metrics từ kubelet và trình bày qua `metrics.k8s.io`; HPA lấy metric qua API này, còn `kubectl top` là một cách xem nhanh. Metrics Server là add-on có thể thiếu hoặc trễ; timestamp/cửa sổ mẫu phải được đối chiếu với sự cố. Không lấy CPU `limits.cpu` hoặc tổng CPU node làm mẫu số của CPU utilization trong HPA: với target kiểu utilization, mẫu số là CPU `requests.cpu` của các container liên quan. Thiếu request thích hợp khiến utilization của Pod không xác định và HPA không thể quyết định từ metric đó theo cách dự kiến.

Chương P11.C04 — Service, EndpointSlice và NetworkPolicy

Lần theo tên Service tới Pod socket và policy chặn.

Bài P11.C04.L01 — Discovery

Sau khi hai Pod `Ready=True`, Service vẫn không phục vụ. So trực tiếp `spec.selector` của Service với `metadata.labels` của Pod trong cùng namespace: `app=atals` khác `app=atlas`. EndpointSlice gắn với Service không có endpoint khớp. Đây là một lỗi ranh giới chọn backend, không phải lỗi TLS hay CPU. Sửa selector thành `app=atlas`; chờ EndpointSlice có hai địa chỉ Pod với `ready=True`, rồi thử kết nối qua Service port 80 và kiểm tra request ứng dụng từ vị trí client thực. Nếu đã có endpoint `ready` mà request vẫn lỗi, mới tiếp tục xem `port`/`targetPort`, network policy, DNS, đường mạng node và ứng dụng ở cổng đích. Service port 80 chuyển tới 8080 theo cấu hình, không phải do tên `atlas` hay readiness tự suy ra.

Trong cluster có DNS phù hợp, tên Service được phân giải theo namespace thành địa chỉ Service, thường là ClusterIP cho Service thông thường. DNS trả lời chỉ chứng minh bước tìm tên; nó không chứng minh Service có backend. Service selector chọn Pod theo label, EndpointSlice chứa địa chỉ/port và điều kiện endpoint, rồi thành phần dataplane trên node chuyển tiếp kết nối tới Pod phù hợp. `ready=True` của endpoint là tín hiệu để chọn backend bình thường; vẫn phải kiểm tra ứng dụng nghe đúng `targetPort` và request đại diện thực sự thành công. Các triển khai có thể dùng kube-proxy hoặc dataplane khác; không suy ra mọi cluster dùng cùng luật iptables hay cùng kiểu eBPF từ API Service.

Bài tập P11.C04.L01.Q01

Hai Pod `Running`, điều kiện `Ready=False`, readiness HTTP trả 404. Service cũng có selector không khớp label Pod. Sau khi sửa probe, vì sao request qua Service vẫn lỗi? Nêu bằng chứng cần có trước khi kết luận đường Service đã hoạt động.

Bài P11.C04.L02 — Dataplane

Pod network do plugin tương thích CNI triển khai, gồm cấp địa chỉ và kết nối giữa Pod/node theo mô hình mạng của cluster. CNI là giao diện giữa runtime và plugin, không chỉ định một sản phẩm, một tuyến gói tin duy nhất hay tự mang chính sách firewall. `NetworkPolicy` là API chọn Pod và cho phép lưu lượng ingress/egress theo những điều kiện khai báo; nó chỉ có tác dụng khi plugin mạng của cluster hỗ trợ thực thi chính sách. Mặc định, Pod không bị cô lập ở một chiều nếu chưa có policy chọn nó cho chiều đó. Khi đã bị cô lập, các allow rule cộng lại, không phải danh sách lệnh deny theo thứ tự. DNS có thể trả đúng Service IP, EndpointSlice có backend Ready nhưng một policy vẫn chặn client → Pod hoặc Pod → dependency; ngược lại một timeout không tự chứng minh policy là nguyên nhân. So namespace/label, port, chiều ingress và egress, khả năng của plugin, rồi thử từ đúng nguồn tới đúng đích. Có thể có độ trễ giữa API nhận policy và dataplane áp dụng; trạng thái object policy không phải bằng chứng gói tin đã bị chặn tại thời điểm ấy.

Bài P11.C04.L03 — Policy

Ở ảnh chụp thứ ba, DNS trả ClusterIP, EndpointSlice của Service có ba địa chỉ `ready=True`, request trực tiếp từ một Pod khác tới backend đáp, nhưng client Pod ở namespace `web` kết nối qua Service bị timeout. Các phép đo này thu hẹp lỗi về đường từ client tới Service/backend; chúng chưa chỉ ra plugin mạng hay policy nào. Đối chiếu policy chọn client/đích theo đúng namespace và chiều, `targetPort`, route và dataplane thực của cluster. Chỉ khi phép thử trước/sau trên cùng nguồn/đích và quan sát policy implementation khớp mới quy nguyên nhân cho NetworkPolicy. Sau khi request đi được, latency vẫn cao là vấn đề mới, không thể tiếp tục kết luận "mạng lỗi" từ timeout cũ.

`NetworkPolicy` là API chọn Pod và cho phép lưu lượng ingress/egress theo những điều kiện khai báo; nó chỉ có tác dụng khi plugin mạng của cluster hỗ trợ thực thi chính sách.

Bài tập P11.C04.L03.Q01

DNS trả Service IP, ba endpoint `ready=True`, request từ một Pod nội bộ thành công, nhưng client namespace khác timeout. Hãy thiết kế phép kiểm tra phân biệt DNS, selector, cổng, NetworkPolicy và dataplane; nêu điều không thể suy từ một timeout.

Chương P11.C05 — PVC, CSI và dữ liệu workload

Phân biệt PVC Bound với volume mount và dữ liệu phục hồi được.

Bài P11.C05.L01 — Tài nguyên

PersistentVolume (PV) là object tài nguyên lưu trữ ở phạm vi cluster; PersistentVolumeClaim (PVC) là yêu cầu của workload trong namespace, nêu dung lượng, access mode và StorageClass. StorageClass mô tả cách provision volume và chính sách liên quan; provisioner/CSI driver có thể tạo PV động cho PVC. Binding nối một PVC với một PV đáp ứng điều kiện, nhưng sau đó node còn phải attach hoặc mount bằng driver và hạ tầng lưu trữ thật. `PVC Bound` không chứng minh file đã mount, quyền đọc/ghi đúng, dữ liệu có nội dung mong đợi hay có bản backup. Khi Pod kẹt `ContainerCreating`, cần xem events `FailedAttachVolume` hoặc `FailedMount`, CSI controller/node plugin và topology của PV trước khi đổ lỗi ứng dụng.

Access mode nói về cách volume được dùng hoặc ghép claim với volume: `ReadWriteOnce` cho phép read-write từ một node, không nhất thiết chỉ một Pod nếu nhiều Pod ở cùng node. `ReadWriteOncePod` giới hạn một Pod, cần CSI driver và sidecar hỗ trợ; `ReadOnlyMany`/`ReadWriteMany` còn tùy năng lực provider. Access mode không tự mã hóa dữ liệu, không tạo quyền file ở trong container, và không phải bản sao dữ liệu. Với `WaitForFirstConsumer`, binding/provisioning được trì hoãn để xét đồng thời yêu cầu của Pod và topology volume; một PVC Pending trước khi có consumer có thể là trạng thái chủ ý. Đặt thẳng `spec.nodeName` bỏ qua scheduler có thể khiến PVC kiểu này Pending mãi; cần chọn cơ chế ràng buộc node qua scheduler. Nếu topology không có node phù hợp, thêm replica chỉ tạo thêm Pod Pending.

Bài P11.C05.L02 — Thực thi

Sau khi có node phù hợp, PVC dùng StorageClass `WaitForFirstConsumer` mới được chọn theo topology. PVC Bound nhưng Pod tiếp tục kẹt `ContainerCreating`, event giả định báo `FailedMount` từ CSI. Điều đã biết: binding hoàn tất; điều chưa biết: volume đã attach/mount và chứa dữ liệu đúng. Kiểm tra volume handle, node/topology, driver và sự kiện theo UID Pod/PVC. Không xóa claim hoặc PV để "thử lại" khi chưa có bản sao và chính sách reclaim rõ ràng. Sau khi mount được xác nhận, Pod có thể Running và Ready, nhưng timeout của client vẫn cần chẩn đoán đường dữ liệu.

`PVC Bound` không chứng minh file đã mount, quyền đọc/ghi đúng, dữ liệu có nội dung mong đợi hay có bản backup.

Bài tập P11.C05.L02.Q01

Một Pod có PVC Bound nhưng kẹt `ContainerCreating` với event `FailedMount`. Có thể kết luận dữ liệu đã an toàn và application đã lỗi không? Phân biệt binding, attach/mount, access mode và kiểm tra dữ liệu.

Bài P11.C05.L03 — Vòng đời

Thay Pod chỉ phục hồi số bản sao của tiến trình theo template. File trong writable layer của container mới không tự mang dữ liệu container cũ; `emptyDir` sống theo Pod và mất khi Pod bị loại khỏi node. PersistentVolume/PVC có vòng đời và chính sách thu hồi riêng, nhưng gắn volume không tự tạo backup nhất quán hoặc đảo được một lần ghi sai. Trước khi xóa/recreate Pod hoặc rollback image trong hệ thống có dữ liệu, xác nhận dữ liệu nằm ở đâu, bản sao/backup có thể restore, và phiên bản ứng dụng cũ có đọc được dữ liệu mới hay không. Kiểm chứng cuối cùng là request và dữ liệu nghiệp vụ được phục vụ từ đúng đường đi, không chỉ một hàng `Running` hoặc `Available`.

StatefulSet cho Pod danh tính ổn định và có thể tạo PVC riêng theo ordinal qua `volumeClaimTemplates`, hữu ích khi mỗi replica cần dữ liệu riêng. Nó không tự biến một volume một AZ thành lưu trữ HA, không tự backup, không bảo đảm database replica đã đồng bộ. Reclaim policy như `Retain` hoặc `Delete`, chính sách giữ PVC của StatefulSet, snapshot và backup phải được kiểm tra riêng trước khi scale down hoặc xóa. Phục hồi phải thử ở đường đọc dữ liệu nghiệp vụ, không dừng ở việc PVC chuyển sang Bound.

Bài tập P11.C05.L03.Q01

Một Pod bị xóa rồi ReplicaSet tạo Pod khác; dữ liệu chỉ nằm trong `emptyDir` của Pod cũ. Hệ thống đã tự phục hồi điều gì, chưa phục hồi điều gì? Đề xuất hậu điều kiện cho một ứng dụng giữ dữ liệu trước khi cho phép thay Pod trên production.

Chương P11.C06 — HPA và điều khiển mở rộng

Tính replica từ CPU utilization khi biết request và chất lượng metric.

Bài P11.C06.L01 — Phép đo

Không lấy CPU `limits.cpu` hoặc tổng CPU node làm mẫu số của CPU utilization trong HPA: với target kiểu utilization, mẫu số là CPU `requests.cpu` của các container liên quan.

Trước khi giải thích một quyết định tăng replica, phải đặt cùng một cửa sổ đo cho ba đại lượng: CPU đã dùng của Pod, CPU request dùng làm mẫu số và số replica đủ điều kiện trong phép tính. Một mức sử dụng bằng 80% request không có nghĩa dùng 80% toàn node. Nếu request đặt quá thấp, tỷ lệ có thể cao dù lượng CPU tuyệt đối nhỏ; nếu metric vắng hoặc đã cũ, kết quả không thể được xem như số 0.

HPA điều chỉnh số replica theo metric và cấu hình mục tiêu; hành động đó vẫn phải đi qua Deployment, scheduler và khả năng khởi động Pod mới. Một đề xuất replica cao hơn chưa chứng minh request của người dùng đã nhanh hơn. Khi truy lỗi, kiểm tra dữ liệu metric và thời điểm lấy mẫu trước, sau đó xem desired/current/ready replica cùng sự kiện của Pod. Nếu Pod mới không Ready, nguyên nhân có thể nằm ở đường khởi động, probe hoặc phụ thuộc ứng dụng, không tự nằm trong công thức HPA.

Bài P11.C06.L02 — Vòng tính

Ví dụ tính toán giả định, tất cả ba Pod đã Ready và có metric hợp lệ trên cùng cửa sổ 10 giây. Mỗi Pod khai báo `requests.cpu=500m`, target utilization là 50%, giới hạn HPA từ 2 đến 5 replica. Hiệu CPU time của ba Pod lần lượt là 3,0; 4,5 và 1,5 giây. Chia cho 10 giây cho usage `300m`, `450m`, `150m`; chia cho request `500m` cho utilization 60%, 90%, 30%. Trung bình là `(60+90+30)/3=60%`. Công thức cơ sở cho ba replica là `ceil(3 × 60/50)=ceil(3,6)=4`. Tỷ số 1,2 nằm ngoài tolerance mặc định 10% của controller v1.34; 4 cũng nằm trong khoảng 2–5. Nếu `limits.cpu=1000m` cho mỗi Pod, con số limit ấy không đổi phép tính utilization. Đây là một phép tính lý tưởng trên dữ kiện đầy đủ, không phải lời hứa controller đã tạo đúng một Pod mới ở lượt quan sát ấy.

HPA là vòng điều khiển chạy theo chu kỳ và ghi số replica mong muốn qua `scale` subresource; Deployment/StatefulSet và scheduler mới tạo/đặt Pod. Trước quyết định cuối, controller loại Pod đang xóa hoặc Failed, xét Pod chưa Ready và metric thiếu, tính lại theo giả định bảo thủ để giảm mức scale, rồi áp tolerance, min/max, chính sách tốc độ scale và cửa sổ ổn định khi scale down. Status HPA có thể hiển thị trung bình metric ban đầu, khác tỷ số đã điều chỉnh để quyết định. Nếu metric thiếu, sample cũ, Pod vừa khởi động hay nhiều metric cùng cấu hình, không lấy mỗi công thức cơ sở để đoán số replica cuối. HPA không tăng node allocatable, không sửa PVC, DNS hoặc NetworkPolicy; nó có thể tạo thêm Pod Pending khi ràng buộc đặt chỗ vẫn tồn tại. Khi HPA quản lý số replica, tránh liên tục apply một `spec.replicas` cố định trong manifest vì hai bộ điều khiển sẽ tranh cùng giá trị.

Bài tập P11.C06.L02.Q01

Ba Pod cùng request `500m` CPU; trong 10 giây mỗi Pod dùng 3,0; 4,5; 1,5 giây CPU. Target HPA 50%, min=2, max=5. Tính usage, utilization và desired replicas cơ sở; giải thích vì sao controller có thể chưa tạo đủ Pod hoặc request vẫn chậm.

Bài P11.C06.L03 — Hội tụ

Cuối cùng, ba Pod Ready có các hiệu CPU time 3,0; 4,5; 1,5 giây trên 10 giây như mục 16.5. Phép tính cho khuyến nghị cơ sở bốn replica, nhưng phải kiểm tra timestamp metric, điều kiện HPA, hiệu counter throttling và latency từng chặng trước khi nói CPU là nút thắt. Nếu HPA ghi desired=4 mà Pod thứ tư Pending vì requests hoặc PVC, HPA đã đổi ý định nhưng chưa tăng năng lực phục vụ. Hậu điều kiện là Pod mới Scheduled, volume mount, Ready, EndpointSlice cập nhật và request từ client có latency/response đạt yêu cầu qua nhiều cửa sổ, cùng với dữ liệu vẫn nguyên vẹn. Một giá trị `desiredReplicas=4` riêng nó chưa chứng minh điều đó.

Bài tập P11.C06.L03.Q01

Pod mới do HPA yêu cầu Pending do thiếu node phù hợp, trong khi một Pod cũ tăng `nr_throttled` và `oom_kill` không đổi. Hãy tách hai ranh giới tài nguyên, những phép đo tiếp theo và một quyết định vận hành có hậu điều kiện rõ.

Chương P11.C07 — Gateway API và ranh giới request ngoài cluster

Phân biệt controller conditions với đường request của client.

Bài P11.C07.L01 — API objects

Chương 15–16 cho thấy object được API nhận, Pod được đặt và Service có endpoint vẫn là những sự kiện khác nhau. Khi người dùng ở ngoài cluster gửi request, còn có ranh giới định tuyến vào cluster. Helm giúp tạo và cập nhật một nhóm object theo bản phát hành; Gateway API khai báo nơi lưu lượng đi vào và cách chọn backend. Helm không phải bộ định tuyến, còn Gateway API không tự chạy proxy. Để một request thành công, bản phát hành, controller Gateway, dataplane, Service, Pod và ứng dụng phải cùng thỏa điều kiện tại thời điểm thử.

Gateway API là nhóm API mở rộng: cluster cần CRD tương ứng và một triển khai controller hỗ trợ phiên bản, loại Route và tính năng đang dùng. `GatewayClass` là object phạm vi cluster, chỉ ra `controllerName` của triển khai. Quản trị viên có thể chuẩn bị class; tên class trong bản khai báo không làm controller xuất hiện. `Gateway` là object có namespace, chọn class và khai báo các `listener`: cổng, protocol, hostname, chính sách Route được phép gắn, và khi cần thì cấu hình TLS. Controller được chọn có thể tạo load balancer mới, sửa proxy hiện có hoặc lập trình hệ thống mạng khác; API không quy định một proxy cố định cho mọi triển khai.

`HTTPRoute` mô tả một hay nhiều luật HTTP. `parentRefs` chọn Gateway, có thể chỉ rõ listener qua `sectionName`; Gateway listener cũng phải cho phép Route gắn vào. `hostnames` và `matches.path` thu hẹp request phù hợp, còn `backendRefs` chỉ nơi gửi tiếp, thường là Service cùng namespace và cổng Service. Địa chỉ DNS của client phải tới địa chỉ thực của Gateway; HTTP `Host` hoặc TLS SNI phải phù hợp với listener và Route. Có Route đúng mà DNS trỏ tới một proxy cũ thì request vẫn không đi qua nó. Có Gateway nhận request mà không có Route phù hợp thì phản hồi có thể là 404 do dataplane; cần kiểm tra status, request và cấu hình triển khai trước khi quy nguyên nhân từ mã HTTP.

Với HTTPS termination, listener của Gateway dùng TLS Secret để bắt tay với client; sau điểm giải mã, HTTPRoute có thể so hostname/path và chuyển request tới backend. Secret thường ở cùng namespace với Gateway; tham chiếu Secret ở namespace khác cần grant phù hợp và hỗ trợ từ triển khai. SNI, chứng chỉ, tên host và nơi chấm dứt TLS phải được xét cùng nhau. TLS ở Gateway không mặc nhiên mã hóa chặng Gateway → Service/Pod. Ví dụ chart dưới đây cố ý chỉ dùng HTTP để không giả vờ có chứng chỉ hoặc bảo đảm bảo mật của chặng bên trong; muốn dùng trong hệ thống thật phải thiết kế TLS và ranh giới tin cậy riêng.

Gateway tạo listener HTTP cổng 80, giới hạn Route cùng namespace. Namespace lấy từ release để tên Gateway và Route không bị tách nhầm.

Bài tập P11.C07.L01.Q01

Chart đã render Gateway và HTTPRoute hợp lệ về YAML, nhưng GatewayClass chưa tồn tại. Dự đoán thứ tự bằng chứng cần đọc trước khi chẩn đoán 404 từ client. Điều gì chưa thể kết luận từ `helm template`?

Bài P11.C07.L02 — Attachment

Quan hệ gắn Route có hai phía: Route gọi tên parent, còn listener giới hạn loại Route và namespace qua `allowedRoutes`. Mặc định thông dụng là chỉ Route cùng namespace; trong ví dụ dưới đây `from: Same` được viết tường minh. Nếu Route ở namespace khác, Gateway có thể cho phép namespace bằng `allowedRoutes`, nhưng điều đó không tự cho Route quyền trỏ tới Service ở namespace thứ ba. Tham chiếu chéo namespace tới backend hoặc TLS Secret cần `ReferenceGrant` tại namespace chứa object đích, với nguồn và đích được khai báo cụ thể. Trường hợp Route ở namespace khác gắn Gateway là ngoại lệ của cơ chế ReferenceGrant và do `allowedRoutes` điều khiển. Không mở `allowedRoutes: All` hoặc tạo grant rộng chỉ để làm mất lỗi cấu hình; quyền gắn Route là quyền chiếm một phần bề mặt lưu lượng.

`GatewayClass.status.conditions` cho biết class có được triển khai chấp nhận hay không. Trên Gateway, đọc `Accepted`, `Programmed`, status của từng listener và địa chỉ được cấp nếu có. `Accepted=True` nghĩa cấu hình được controller nhận ở mức tương ứng; `Programmed=True` nói controller đã lập trình cấu hình xuống dataplane theo hợp đồng của nó, không chứng minh DNS, firewall, TLS hay Pod phía sau hoạt động. Trên HTTPRoute, đọc `status.parents` đúng parent/controller, điều kiện `Accepted` và `ResolvedRefs`; `ResolvedRefs=False` có thể chỉ backend hoặc tham chiếu khác không tồn tại hay không được phép. Một Route có nhiều parent có thể được parent này nhận nhưng parent kia từ chối. Luôn so `observedGeneration` trong condition với `metadata.generation` trước khi dùng status để đánh giá bản spec mới.

HTTPRoute gắn đúng listener qua `parentRefs`, so hostname và chuyển path bắt đầu bằng `/` tới cổng Service. `backendRefs.port` là cổng Service, không phải `targetPort` của Pod:

Bài tập P11.C07.L02.Q01

HTTPRoute ở namespace `app` trỏ tới Gateway ở namespace `edge`, listener chỉ cho Route cùng namespace. Một kỹ sư thêm ReferenceGrant ở `edge` cho Route này. Vì sao Route có thể vẫn không attach? Nếu Route còn trỏ Service ở namespace `data`, cần cơ chế quyền nào khác?

Bài tập P11.C07.L02.Q02

Sau upgrade, `HTTPRoute.status.parents` cho parent đúng có `Accepted=True`, `ResolvedRefs=False`, còn Service dự kiến không có trong namespace của Route. Suy luận nhánh lỗi hợp lý và phép kiểm tra tiếp; vì sao không nên chỉ tăng timeout?

Bài P11.C07.L03 — Request path

Ingress vẫn là API Kubernetes đang tồn tại và vẫn được controller tương ứng triển khai; Gateway API mở rộng mô hình vai trò Gateway/Route và cho phép chia quyền cấu hình rõ hơn. Không đổi một Ingress thành Gateway chỉ bằng cách đổi `kind`. Cần kiểm tra loại Route, tính năng được controller cụ thể hỗ trợ, đối tượng TLS và cách dataplane nhận lưu lượng. Cả Ingress lẫn Gateway đều cần một controller thực thi; sự tồn tại của object trong API server chỉ xác nhận bản khai báo được lưu.

Nếu host và attachment đều đúng nhưng nhận 503, lần tiếp theo là `backendRefs` → Service/port → EndpointSlice có endpoint Ready → Pod readiness → ứng dụng. Trong nhiều triển khai, không có backend khỏe có thể hiện thành 503, nhưng mã ấy không tự định danh lỗi: proxy khác, policy, timeout kết nối backend hoặc ứng dụng cũng có thể góp phần. `ResolvedRefs=False` gợi ý Service/Secret sai tên, sai namespace hoặc thiếu grant; `Accepted=False` trên Route gợi ý Route chưa attach, `Programmed=False` trên Gateway gợi ý controller/dataplane chưa hoàn tất. Nếu tất cả condition đều hiện True nhưng request timeout, kiểm tra địa chỉ và DNS, đường client → Gateway, firewall, listener thực tế, rồi thử từ vị trí gần backend để tách chặng lỗi. Một status True có `observedGeneration` cũ không được dùng để minh oan revision 5.

Bài tập P11.C07.L03.Q01

Người dùng báo timeout nhưng Gateway `Programmed=True` ở generation mới, HTTPRoute Accepted/ResolvedRefs True và Service có endpoint Ready. Hãy thiết kế phép thử phân biệt đường client tới Gateway, Gateway tới Pod và lỗi ứng dụng; chỉ rõ giới hạn của mỗi condition.

Chương P11.C08 — Helm chart và vòng đời release

Theo chart/values đến manifest/revision rồi phân biệt với trạng thái thật.

Bài P11.C08.L01 — Chart

Một chart là gói đầu vào. `Chart.yaml` chứa tên và version của chart, `values.yaml` đặt giá trị mặc định, còn `templates/` là mẫu được Helm render bằng Go template và values cuối cùng. Giá trị do người dùng truyền qua `-f` hoặc `--set` có thể ghi đè mặc định; `--reuse-values` khi upgrade có thể kéo theo một giá trị cũ mà người sửa chart tưởng đã bỏ. `helm lint` kiểm tra chart ở mức tĩnh; `helm template` cho thấy YAML render từ release name, namespace và values đã chọn mà không cần cluster. Bản YAML ấy chưa được API server chấp nhận, chưa có CRD/controller và chưa thể nói request sẽ đi đâu.

Chart `atlas-gateway` chỉ tạo Gateway và HTTPRoute trong namespace của release; nó trỏ tới Service `atlas:80` đã có từ Chương 15. Đây là ví dụ cấu hình, chưa có kết quả chạy trên cluster. Trước khi thử, quản trị viên phải có Gateway API CRD, `GatewayClass` tên `example-gateway` do một controller thật quản lý, namespace `demo`, Service `atlas` và backend Ready. Chữ `example-gateway` không phải tên class mặc định của Kubernetes. Manifest Chương 15 không đặt namespace; nếu thử nó trong cluster cô lập đã được phép, chạy `kubectl -n demo apply -f` với đường dẫn tệp đó, sau khi đã hiểu hai lỗi cố ý và sửa trên bản sao thử nghiệm. Đừng áp manifest lỗi lên workload đang phục vụ.

`Chart.yaml` và `values.yaml` định nghĩa đầu vào, không chứa credential:

```
apiVersion: v2
name: atlas-gateway
description: Teaching example for an existing Atlas Service
type: application
version: 0.1.0
appVersion: "1.0"

# A matching GatewayClass and the atlas Service must already exist.
gatewayClassName: example-gateway
hostname: atlas.example.test
backend:
  service: atlas
  port: 80
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: {{ .Release.Name }}-web
  namespace: {{ .Release.Namespace }}
spec:
  gatewayClassName: {{ .Values.gatewayClassName | quote }}
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      hostname: {{ .Values.hostname | quote }}
      allowedRoutes:
        namespaces:
          from: Same

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: {{ .Release.Name }}-route
  namespace: {{ .Release.Namespace }}
spec:
  parentRefs:
    - name: {{ .Release.Name }}-web
      sectionName: http
  hostnames:
    - {{ .Values.hostname | quote }}
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: {{ .Values.backend.service | quote }}
          port: {{ .Values.backend.port }}
```

Trên máy có Helm 3, `helm lint` và `helm template atlas-edge ./examples/gateway-helm --namespace demo` kiểm tra chart và cho manifest dự kiến. Thử ghi đè hostname bằng một values file thử nghiệm rồi so bản render; kiểm tra `metadata.namespace=demo`, `atlas-edge-web`, tên listener `http`, host và backend. Không dùng render cục bộ như bằng chứng API/CRD/controller chấp nhận. Nếu triển khai trong cluster được cho phép, cần kiểm tra GatewayClass đã Accepted, Gateway/listener đã Programmed, HTTPRoute parent Accepted/ResolvedRefs, địa chỉ Gateway, DNS trỏ đúng địa chỉ và request có `Host: atlas.example.test` từ một client xác định. Không tự cài CRD, controller hay tạo Gateway ngoài môi trường được giao chỉ để chứng minh ví dụ sách.

Bài P11.C08.L02 — Release

`--wait` có thể chờ một số điều kiện Ready của tài nguyên Kubernetes trong thời hạn cấu hình, nhưng không thay thế request đại diện qua Gateway, kiểm tra dữ liệu hay xác nhận backend nghiệp vụ. Một chart render đúng vẫn có thể fail khi API server không có CRD hoặc cấm quyền; một upgrade được ghi nhận vẫn có thể để controller Gateway từ chối Route hay Pod chưa Ready. Vì vậy cần giữ riêng bốn ảnh chụp: chart/values đã chọn, manifest của revision, trạng thái object hiện tại và phản hồi từ đúng client.

`helm lint` kiểm tra chart ở mức tĩnh; `helm template` cho thấy YAML render từ release name, namespace và values đã chọn mà không cần cluster.

Bài P11.C08.L03 — Rollback

Khi `helm install` tạo release, Helm gửi manifest đã render đến API server và ghi lại lịch sử release. `helm upgrade` render lại với chart/values mới và tạo revision tiếp theo; `helm history` và `helm get manifest` giúp phân biệt revision đã ghi với cấu hình đang có trong cluster. Chart version, `appVersion`, revision và image digest là bốn danh tính khác nhau; một revision mới có thể chỉ đổi hostname, không thay image. `helm rollback` tạo một revision mới dựa trên revision cũ, rồi cập nhật các object Helm quản lý; nó không quay ngược cơ sở dữ liệu, request đã xử lý, dữ liệu trên volume hay thay đổi bên ngoài chart. `helm uninstall` bỏ các object trong phạm vi release theo quy tắc Helm, không phải lệnh dọn mọi dữ liệu liên quan. Giá trị, template và manifest có thể được giữ trong release metadata, thường là Kubernetes Secret của Helm 3; Secret không phải nơi an toàn để ghi credential vào chart/values rồi đưa output `helm template` hoặc `helm get` lên log công khai.

Giả sử revision 4 của release `atlas-edge` dùng host `atlas.example.test` và Service `atlas:80`; một request kiểm thử qua địa chỉ Gateway đã thành công. Revision 5 được tạo bằng `helm upgrade` với file values mới đặt `hostname: api.example.test`. Lệnh hoàn tất và hai object mới tồn tại, nhưng DNS của khách hàng vẫn đưa `atlas.example.test` tới đúng Gateway. Đây là dữ liệu giả định, không phải log Helm, kubectl hoặc proxy đã chạy. So chart và values được dùng cho revision 5 với `helm get manifest` của revision đó: cả listener lẫn Route nay chọn `api.example.test`. Request mang Host cũ không khớp cấu hình mới, nên một phản hồi 404 tại Gateway là giả thuyết mạnh; vẫn phải xác định chính gateway nào đã trả mã và điều kiện listener/Route ở generation mới. Sửa values cho đúng host và upgrade có kiểm tra, hoặc rollback sang revision đã biết rồi xác nhận request cũ; chỉ nhìn chữ `deployed` của Helm không đủ.

Khi rollback, ghi rõ revision đích, manifest cũ, các object live trước/sau và thời điểm lưu lượng chuyển. Nếu revision 5 kèm migration schema hay đã ghi dữ liệu vào backend, đổi Route về host cũ không khôi phục dữ liệu. Hậu điều kiện cần có request từ client ở đúng hostname/SNI và path, response mong đợi, EndpointSlice còn backend Ready, và kiểm tra dữ liệu nếu ứng dụng có ghi. Đây là phép xác nhận một lần tại thời điểm đo, không phải bảo đảm lâu dài.

Bài tập P11.C08.L03.Q01

Revision 7 đổi hostname và image, đồng thời ứng dụng ghi dữ liệu theo schema mới. `helm rollback` về revision 6 hoàn tất. Nêu ba hậu điều kiện độc lập phải chứng minh trước khi nói dịch vụ đã phục hồi.

PHẦN 12 — Observability và telemetry

Dữ liệu quan sát được tạo, vận chuyển và diễn giải có giới hạn nào?

Metric tổng hợp, log sự kiện và trace đường request là các phép chiếu có sai lệch lấy mẫu, mất dữ liệu và khác thời gian.

Chương P12.C01 — Phép đo, nhãn và chuỗi thời gian

Định nghĩa phép đo và rủi ro sai lệch trước khi xem dashboard.

Bài P12.C01.L01 — Đo

Classic histogram gồm các bucket tích lũy theo ngưỡng `le`, `_sum` và `_count`.

Summary có thể xuất quantile do client tính; không thể cộng trực tiếp các quantile của nhiều target để có quantile toàn hệ thống.

Một metric chỉ có ý nghĩa khi biết loại, đơn vị, nhãn và điểm tăng của nó. Counter diễn tả số tích lũy có thể reset khi tiến trình khởi động lại; để hỏi tốc độ trong một cửa sổ phải xét chênh lệch theo thời gian theo cách nhận biết reset. Gauge mô tả giá trị có thể tăng hoặc giảm, ví dụ số công việc đang chờ ở thời điểm lấy mẫu. Không gọi tổng của các gauge ở các thời điểm khác nhau là một tốc độ.

Classic histogram gồm các bucket tích lũy theo ngưỡng `le`, `_sum` và `_count`. Vì bucket tích lũy, số ở ngưỡng lớn đã bao gồm quan sát của các ngưỡng nhỏ hơn; cộng tất cả bucket sẽ đếm lặp. Có thể tổng hợp bucket cùng ranh giới từ nhiều target trước khi ước lượng quantile, nhưng kết quả phụ thuộc các ngưỡng đã chọn. Summary có thể xuất quantile do client tính; không thể cộng trực tiếp các quantile của nhiều target để có quantile toàn hệ thống. Trước khi vẽ biểu đồ, kiểm tra đơn vị giây/millisecond và population của nhãn để tránh một con số đẹp nhưng sai nghĩa.

Bài P12.C01.L02 — Tập hợp

Mỗi tổ hợp tên metric và giá trị nhãn tạo một time series. Gauge có thể tăng hoặc giảm theo trạng thái hiện tại, như số request đang chạy. Counter cộng dồn và có thể về giá trị nhỏ khi tiến trình khởi động lại; không lấy một giá trị counter đơn lẻ làm tốc độ. Classic histogram gồm các bucket tích lũy theo ngưỡng `le`, `_sum` và `_count`. Một request nằm trong mọi bucket có ngưỡng lớn hơn hoặc bằng thời gian của nó; bucket `+Inf` phải bằng `_count`. Summary có thể xuất quantile do client tính; không thể cộng trực tiếp các quantile của nhiều target để có quantile toàn hệ thống. Chương này chỉ tính p95 từ classic histogram, không trộn với native histogram.

Chọn nhãn để phân nhóm hữu ích như `service`, `route`, `status_class`; tránh `request_id` hoặc user ID thay đổi theo từng request. Nếu 50.000 request tạo 50.000 giá trị `request_id` cho một metric, riêng chiều nhãn đó có thể tạo khoảng 50.000 chuỗi thay vì vài chuỗi theo status; nhân thêm instance, method và bucket còn tăng số chuỗi, bộ nhớ và chi phí truy vấn. Đây là phép đếm tổ hợp giả định, không phải benchmark Prometheus. Không đưa secret hay định danh người dùng vào nhãn chỉ để lọc dashboard.

Bài tập P12.C01.L02.Q01

Một thay đổi thêm `request_id` vào metric HTTP và Grafana báo chậm. Tính bậc số chuỗi nếu có 50.000 ID khác nhau, hai instance, hai status class và bốn bucket; đề xuất nhãn thay thế và cách kiểm chứng sau sửa. Giả sử mọi tổ hợp đều xuất hiện; giải thích vì sao tích này chỉ là ước lượng trần theo giả định.

Bài P12.C01.L03 — Thu thập

Ở Chương 16, HPA dùng CPU utilization tính từ CPU usage và request. Đó chỉ là một tín hiệu điều khiển. Người vận hành cần biết phép đo được tạo ở đâu, có tới nơi lưu trữ không, câu truy vấn biến mẫu rời rạc thành đại lượng nào và lúc nào dữ liệu không còn đủ để kết luận. Một biểu đồ đẹp không sửa được một lần scrape thất bại hay một nhãn sai.

Ứng dụng hoặc exporter đo một sự kiện rồi giữ giá trị theo tên metric và tập nhãn. Target công bố snapshot qua HTTP; Prometheus chọn target bằng cấu hình hoặc service discovery, định kỳ scrape endpoint và gắn timestamp cho mẫu nhận được. Metric `up` do Prometheus tạo cho kết quả scrape: 1 khi scrape thành công, 0 khi thất bại với target vẫn còn trong danh sách. `up=1` chỉ xác nhận việc lấy metric, không chứng minh request nghiệp vụ qua Gateway thành công. Target bị loại khỏi discovery là trường hợp khác với target còn được theo dõi nhưng `up=0`.

Prometheus lưu mẫu thành các chuỗi thời gian trên đĩa cục bộ với write-ahead log và block; thời hạn lưu do cấu hình retention chi phối. Retention không biến dữ liệu thành backup: lỗi đĩa hoặc xóa TSDB vẫn cần kế hoạch sao lưu/độ sẵn sàng riêng. Recording rule định kỳ lưu kết quả biểu thức thành chuỗi mới để giảm chi phí truy vấn lặp lại; alerting rule đánh giá điều kiện và chuyển cảnh báo sang trạng thái chờ/phát theo `for`. Rule cũng chịu ảnh hưởng của dữ liệu thiếu và thứ tự đánh giá, không xác nhận nguyên nhân gốc. Grafana truy vấn Prometheus qua data source rồi vẽ panel; nó không phải nguồn gốc của phép đo.

Chương P12.C02 — Prometheus, PromQL và Grafana

Tính rate, tỷ lệ lỗi và p95 có giới hạn từ mẫu thật/giả định.

Bài P12.C02.L01 — PromQL

Dữ liệu CSV dưới đây là ví dụ tính toán giả định, không phải output scrape trên cluster. Hai instance `a` và `b` được lấy mẫu ở giây 0, 60, 120, 180; trong ví dụ, tên instance được gắn khi scrape thay cho địa chỉ target mặc định. `ok` và `err` là hai counter HTTP; bốn cột `le_*` là bucket tích lũy của thời gian request tính bằng giây; `cpu_s` là CPU time tích lũy. Để số dễ tự kiểm, từng phút mỗi instance xử lý 54 request 2xx và 6 request 5xx; số đếm bucket tăng 20, 45, 58, 60. Mọi request của bảng thuộc một trong hai status class và đi vào histogram. Trong vận hành, hãy xác nhận phạm vi instrument trước khi lấy histogram count làm mẫu số HTTP.

```
t_s,instance,ok,err,le_0_1,le_0_5,le_1,le_inf,cpu_s
0,a,95,5,30,70,90,100,100
60,a,149,11,50,115,148,160,112
120,a,203,17,70,160,206,220,124
180,a,257,23,90,205,264,280,136
0,b,190,10,60,140,180,200,200
60,b,244,16,80,185,238,260,212
120,b,298,22,100,230,296,320,224
180,b,352,28,120,275,354,380,236
```

Tại giây 180, biểu thức `rate(http_requests_total[3m])` dùng mẫu trong khoảng `(0,180]`, tức giây 60, 120, 180 theo quy tắc range selector. Trên chuỗi đều và không reset này, `rate` của 5xx ở mỗi instance là `(23−11)/120 = 0,1` và `(28−16)/120 = 0,1` request/giây. Cộng theo instance cho 0,2 lỗi/giây. Tương tự tổng request mỗi instance là `(280−160)/120 = 1` request/giây, cả hai là 2 request/giây; tỷ lệ lỗi bằng 0,2/2 = 10%. `increase(counter[3m])` là `rate` nhân 180 giây, có thể cho số lẻ vì Prometheus ngoại suy tới biên cửa sổ. Không đọc nó như hiệu thô 120 giây giữa hai mẫu đầu/cuối. Nếu cần kiểm phép đếm thô của cả 0→180, tổng lỗi tăng 36 trên tổng request tăng 360, cũng là 10% trong bộ dữ liệu tuyến tính này.

Trong hệ có nhãn `job="atlas"` được gắn lúc scrape, các biểu thức sau gom theo status và instance sau khi tính tốc độ riêng từng chuỗi:

```
sum(rate(http_requests_total{job="atlas",status_class="5xx"}[3m]))
/
sum(rate(http_requests_total{job="atlas"}[3m]))

histogram_quantile(0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket{job="atlas"}[3m]))
)
```

Phép cộng trước `rate` là sai khi hai tiến trình reset counter khác lúc: reset của một instance có thể bị tổng của instance khác che khuất. `rate` và `increase` nhận biết bước giảm counter như reset trong phạm vi mẫu, nhưng không thể cứu cửa sổ quá ít mẫu hoặc instrument sai. Khi chọn status class, tử số và mẫu số phải cùng service, route, loại request và cửa sổ; một truy vấn thiếu nhãn lọc có thể cho tỷ lệ số học đúng mà sai đối tượng.

Recording rule định kỳ lưu kết quả biểu thức thành chuỗi mới để giảm chi phí truy vấn lặp lại; alerting rule đánh giá điều kiện và chuyển cảnh báo sang trạng thái chờ/phát theo `for`.

Bài tập P12.C02.L01.Q01

Một kỹ sư tính `rate(sum(http_requests_total)[3m:])` rồi lấy trung bình `p95` của hai Pod. Nêu hai lỗi cơ chế và truy vấn/quan sát phù hợp hơn. Nếu một Pod mất scrape, giới hạn của kết quả mới là gì?

Bài P12.C02.L02 — Histogram

Ở `(0,180]`, tổng tốc độ bucket của hai instance lần lượt là 0,667; 1,5; 1,933; 2 request/giây cho ngưỡng 0,1; 0,5; 1; `+Inf`. Vì p95 của 2 request/giây nằm tại 1,9 request/giây, nó thuộc bucket `(0,5;1]`. Với giả định nội suy đều trong bucket của `histogram_quantile`, kết quả là `0,5 + (1,9−1,5)/(1,933…−1,5) × 0,5 ≈ 0,962 giây`. Có thể kiểm bằng 120 giây quan sát: tổng 240 request, 180 ở ≤0,5 giây và 232 ở ≤1 giây; vị trí 228 cho `0,5+(228−180)/(232−180)×0,5 = 0,9615` giây. Đây là ước lượng từ ranh giới bucket, không phải p95 chính xác của từng request; bucket rộng làm sai số nội suy lớn. Giữ `le` khi `sum` bucket; bỏ nó làm mất biên để tính quantile.

CPU time tăng 24 giây trên hai instance trong 120 giây mẫu, tức 0,4 lõi trung bình; con số ấy không loại trừ chờ I/O, lock, backend, mạng hoặc một nhóm request chậm. p95 tăng cùng lúc CPU bình thường là dấu hiệu cần điều tra latency theo route và upstream; không được gán nguyên nhân chỉ bằng hai đường biểu đồ.

Snapshot exposition giả định của target `a` ở giây 180 dưới đây có cú pháp metric, bucket và count nhất quán với dòng `a` cuối CSV. Endpoint ứng dụng không tự xuất nhãn `instance` hoặc `job`: Prometheus gắn chúng theo target khi scrape; `up` cũng do Prometheus tạo. Snapshot không có các mẫu quá khứ nên riêng nó không thể trả lời truy vấn `rate`.

```
# HELP http_requests_total Completed HTTP requests.
# TYPE http_requests_total counter
http_requests_total{status_class="2xx"} 257
http_requests_total{status_class="5xx"} 23
# HELP http_request_duration_seconds Request duration in seconds.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.1"} 90
http_request_duration_seconds_bucket{le="0.5"} 205
http_request_duration_seconds_bucket{le="1"} 264
http_request_duration_seconds_bucket{le="+Inf"} 280
http_request_duration_seconds_sum 130
http_request_duration_seconds_count 280
# HELP process_cpu_seconds_total CPU time consumed by the process.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 136
```

Bài tập P12.C02.L02.Q01

Từ CSV, tại giây 180 hãy tính rate 5xx, tổng rate, tỷ lệ lỗi, CPU trung bình và p95 classic histogram cho hai instance. Nêu vì sao `increase(...[3m])` không nhất thiết bằng hiệu thô giữa mẫu đầu và cuối trong cửa sổ.

Bài P12.C02.L03 — Dashboard

Tình huống giả định: sau một thay đổi nhãn, dashboard báo p95 tăng, CPU giữ quanh 0,4 lõi, target `b` có `up=0` ở giây 240 và số series tăng nhanh. Trước hết khóa cùng data source, time range, step, filter `service/route` và múi giờ trong Grafana; panel vẽ `rate(...[3m])` ở các mốc đánh giá khác nhau sẽ nhìn những cửa sổ khác nhau. Xem Explore hoặc truy vấn thô để so số mẫu, nhãn và series tham gia, rồi kiểm tra target discovery, endpoint `/metrics`, kết quả scrape và log exporter. Nếu `b` vẫn được discovery và scrape lỗi, `up{instance="b"}=0`; sau failed scrape, series do `b` xuất có thể stale và biến mất khỏi instant vector. Range vector có thể còn mẫu cũ một thời gian. Tỷ lệ lỗi hoặc p95 chỉ từ `a` có thể trông tốt lên dù người dùng của `b` vẫn lỗi. Không biến "không có mẫu" thành 0 request hay 0 latency.

Song song, xem metric/label nào tạo nhiều tổ hợp mới; nếu có `request_id`, bỏ nhãn ấy ở điểm instrument, giới hạn nhãn được phép và đánh giá ngân sách chuỗi trước khi triển khai lại. Sau đó xác minh cả hai target scrape được, số series ổn định và request nghiệp vụ qua đúng Route; xem latency/error theo route và so với log/trace của upstream để tìm nguyên nhân p95. Tăng retention hay đổi panel không sửa được explosion ở nguồn. Grafana hiển thị câu trả lời của truy vấn cho cửa sổ đã chọn, nên một panel "No data" là tín hiệu kiểm tra scrape, selector và range, không phải bằng chứng dịch vụ khỏe hoặc chết.

Grafana truy vấn Prometheus qua data source rồi vẽ panel; nó không phải nguồn gốc của phép đo.

Bài tập P12.C02.L03.Q01

Ở giây 240, `up{instance="b"}=0` nhưng panel lỗi 5xx giảm. Liệt kê các phép kiểm để phân biệt dịch vụ `b` hồi phục, `b` không được scrape và truy vấn bị lọc nhãn; chỉ rõ điều chưa thể kết luận từ `up`.

Chương P12.C03 — Logs, traces và OpenTelemetry

Nối log và span của một request, kể cả khi một nhánh timeout nhưng cuối trả 200.

Bài P12.C03.L01 — Log

Chương 18 cho thấy p95 tăng có thể cùng tồn tại với CPU bình thường và tỷ lệ 5xx không đổi. Metric trả lời "bao nhiêu request chậm trong tập được đo", nhưng không chứa đường đi của từng request. Log ghi một sự kiện tại một điểm: khi nào một lần gọi timeout, module nào báo lỗi, biến trạng thái nào đã quan sát. Trace nối các công đoạn thực thi thành các span có quan hệ cha con. Ba tín hiệu hỗ trợ nhau; một log `WARN`, một span dài hoặc một p95 cao tự nó chưa chứng minh nguyên nhân gốc.

Một log record cần thời gian, mức nghiêm trọng, nội dung sự kiện và các thuộc tính đủ để định vị nguồn phát. OpenTelemetry phân biệt thời điểm sự kiện được ghi với thời điểm hệ thu nhận quan sát nó; hai thời điểm có thể khác vì hàng đợi hoặc truyền mạng. `INFO`, `WARN` hay `ERROR` thể hiện quyết định của nơi phát log, không thay cho HTTP status hoặc trạng thái span. Log có cấu trúc giữ các trường riêng như `service`, `event`, `request_id` và `trace_id`, để lọc chính xác hơn việc tìm chuỗi trong một câu thông báo. Không đặt payload, token hoặc người dùng cụ thể vào trường chỉ để việc tìm kiếm dễ hơn.

`request_id` do Gateway hoặc ứng dụng tạo giúp tìm log cùng một request; nó có thể xuất hiện ở header ứng dụng nhưng không phải định dạng propagation chuẩn của trace. `trace_id` định danh một trace, còn `span_id` gắn log với đúng công đoạn nếu logger biết context hiện hành. Một trace có thể phục vụ nhiều log và nhiều span; khi context bị mất, log vẫn được tạo nhưng thiếu ID hoặc mang ID mới. Không đưa `request_id` vào nhãn metric Prometheus: một ID khác cho mỗi request gây cardinality lớn như đã phân tích ở Chương 18.

```
[
  {
    "time": "2026-10-19T10:00:00.030Z",
    "severity": "INFO",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "c1c1c1c1c1c1c1c1",
    "request_id": "req-17",
    "event": "db_call_started"
  },
  {
    "time": "2026-10-19T10:00:00.335Z",
    "severity": "WARN",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "d2d2d2d2d2d2d2d2",
    "request_id": "req-17",
    "event": "upstream_timeout",
    "attempt": 1
  },
  {
    "time": "2026-10-19T10:00:00.405Z",
    "severity": "INFO",
    "service": "api",
    "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
    "span_id": "b7ad6b7169203331",
    "request_id": "req-17",
    "event": "response_sent",
    "http_status": 200
  }
]
```


Bài P12.C03.L02 — Trace

Span biểu diễn một thao tác có thời điểm bắt đầu/kết thúc, tên, thuộc tính, event và status; `trace_id` chung nối nó vào trace. `span_id` định danh span trong trace, `parent span ID` cho biết công đoạn gọi trực tiếp. Span event là dấu thời gian gắn trong một span, chẳng hạn lúc lên lịch retry; nó không phải một log record độc lập, và có thể không tồn tại trong trace được lưu nếu span bị bỏ. Status tóm tắt kết quả của thao tác theo instrumentation, không phải severity của một log event. Các span lồng nhau nên không được cộng mọi duration để ra latency của request. Nếu Gateway span chạy 420 ms và API span nằm bên trong 393 ms, tổng hai số đã đếm trùng phần lớn thời gian. Với các nhánh chạy song song, đường quyết định thời gian hoàn tất còn phụ thuộc việc chờ và thứ tự, không phải tổng độ dài mọi span. Span client gọi DB chỉ đo thời gian nhìn từ client, không tự phân rã được thời gian DB xử lý, chờ connection pool và mạng nếu không có instrument tương ứng.

Khi Gateway gọi API, nó inject trace context vào carrier HTTP; API extract rồi tạo server span nhận parent từ request. Tiếp đó API phải truyền context hiện hành tới client DB/upstream và qua hàng đợi hoặc tác vụ bất đồng bộ khi có. Nếu chỉ tạo span mới mà không truyền context, trace có thể tách thành hai cây dù request vẫn chạy thành công. Context propagation truyền định danh và cờ xử lý, không vận chuyển toàn bộ span hay bảo đảm span đã được lưu.

W3C Trace Context quy định header `traceparent` gồm version, trace ID 16 byte, parent ID 8 byte và trace flags. Trong ví dụ này, Gateway gửi tới API `traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`. API tạo span mới có cùng trace ID và parent là span Gateway. Bit sampled trong `01` là tín hiệu propagation/sampling, không phải biên nhận của backend. Header `tracestate` tùy chọn mang trạng thái của hệ tracing theo cú pháp chuẩn; không dùng nó để chuyển secret hoặc payload. Nếu không có header hợp lệ, dịch vụ có thể tạo root trace mới theo cấu hình, làm đứt mạch tương quan cần điều tra.

Sampling giảm lượng trace được ghi, gửi và lưu. Quyết định đầu trace có thể bỏ qua cả một request chậm nếu tỷ lệ lấy mẫu thấp; quyết định ở Collector sau khi thấy thêm span có thể chọn giữ trace chậm/lỗi nhưng phải giữ dữ liệu tạm và chờ đủ lâu. Một `traceparent` có cờ sampled không bảo đảm mọi SDK, exporter, Collector và backend thực sự nhận được span. Khi mẫu trace không đại diện ngẫu nhiên cho toàn bộ lưu lượng, tỷ lệ lỗi hoặc p95 tính từ trace đã lưu có thể lệch so với metric HTTP được instrument đầy đủ. Muốn suy ra phân phối người dùng, phải hiểu chính sách sampling của từng tuyến.

Ví dụ sau là dữ liệu giả định có thể tính độc lập, không phải trace lấy từ cluster hay giao diện backend. Các mốc là mili giây tương đối từ khi Gateway nhận request; giả định đồng hồ các nguồn đã được chuẩn hóa cho bài toán. `G` là Gateway server span; `A` là API server span; `D` là DB client span do API tạo; `U1` và `U2` là hai lần gọi upstream do API tạo. Cột `status` là span status, không phải HTTP status. Không có DB server span hoặc upstream server span trong dữ liệu này.

```
alias,parent,span_id,stage,start_ms,end_ms,status
G,-,00f067aa0ba902b7,gateway,0,420,UNSET
A,G,b7ad6b7169203331,api,12,405,UNSET
D,A,c1c1c1c1c1c1c1c1,db_call,30,280,UNSET
U1,A,d2d2d2d2d2d2d2d2,upstream_1,285,335,ERROR
U2,A,e3e3e3e3e3e3e3e3,upstream_2,345,385,UNSET
```

Gateway mất `420−0=420 ms`, API nằm trong khoảng `12..405` và mất 393 ms. DB client span `30..280` mất 250 ms. Lần gọi upstream đầu `285..335` mất 50 ms và bị timeout, nghỉ 10 ms rồi retry `345..385` mất 40 ms; đoạn retry quan sát được từ 285 tới 385 chiếm 100 ms. Tổng duration của năm span là `420+393+250+50+40=1153 ms`, không phải latency client. Chuỗi công việc đã thấy nằm trên đường hoàn tất của request 420 ms; phần `0..12`, `12..30`, `280..285`, `335..345`, `385..405` và `405..420` chưa được phân rã đủ để gán nguyên nhân. DB call dài nhất trong các child span được đo, nhưng có thể chứa chờ pool hoặc mạng; chưa có chứng cứ DB server là nơi xử lý chậm.

Ba log JSON giả định dưới đây dùng cùng trace ID; `span_id` nối lần lượt tới `D`, `U1` và `A`. Tại 335 ms, API ghi timeout lần thứ nhất; tại 405 ms nó trả HTTP 200 sau retry. `U1` có span status `ERROR`, còn `A` và `G` là `UNSET`: một thao tác con thất bại rồi được phục hồi không buộc request cuối cùng thành 5xx. Ngược lại HTTP 500 là trường HTTP; trạng thái span là phân loại riêng của instrumentation/semantic conventions và phải kiểm tra theo kind, ngữ cảnh và phiên bản. Một timeout do client chủ động hủy cũng cần phân biệt với timeout lỗi upstream.

Mẫu này có thể phù hợp với một phần request chậm của hiện tượng Chương 18, nhưng một trace 420 ms không chứng minh p95 toàn hệ, không chứng minh CPU bình thường là nguyên nhân hoặc loại bỏ các request không được sampling. Chỉ khi cùng route, phiên bản, cửa sổ thời gian và nhóm người dùng với metric p95, nhiều trace đại diện cùng log tương ứng mới hỗ trợ giả thuyết DB hoặc upstream đóng góp. Trace không có span DB server vẫn để ngỏ phần rã 250 ms; cần query latency/pool wait, log DB hoặc instrument thêm với ngân sách dữ liệu phù hợp.

Bài tập P12.C03.L02.Q01

Từ CSV, tính duration của `G`, `A`, `D`, `U1`, `U2` và khoảng của hai lần gọi upstream. Vì sao cộng năm duration không cho latency của client? Chỉ ra hai đoạn thời gian chưa có span riêng.

Bài tập P12.C03.L02.Q02

Log ở 335 ms báo timeout, nhưng log ở 405 ms ghi HTTP 200. Hãy lần theo parent/child và phân biệt HTTP status, span status, severity. Có thể kết luận DB server chậm 250 ms không?

Bài P12.C03.L03 — Pipeline

Instrumentation gọi OpenTelemetry API để tạo span, thêm event hoặc phát log; SDK thực thi chính sách sampling, processor và exporter. Instrumentation tự động của framework có thể tạo span HTTP/DB mà không cần viết tay mọi thao tác, nhưng độ phủ, tên thuộc tính và hành vi khi lỗi phụ thuộc ngôn ngữ, thư viện, phiên bản và cấu hình. Resource attributes như `service.name` và `service.version` mô tả nguồn phát chung; thuộc tính span như `http.route` mô tả thao tác riêng. Semantic conventions giúp nhiều dịch vụ gọi cùng một khái niệm bằng tên nhất quán, nhưng phiên bản convention và chế độ ổn định phải được kiểm tra trước khi ghép truy vấn qua nhiều SDK.

SDK có thể xuất OTLP qua HTTP hoặc gRPC tới Collector. Collector dùng `receiver` nhận tín hiệu, `processor` biến đổi/lọc/batch rồi `exporter` gửi tới backend; thành phần đã khai báo nhưng chưa gắn vào `service.pipelines` không xử lý dữ liệu. Có thể có pipeline riêng cho traces, logs và metrics. Exporter helper của Collector hỗ trợ queue, timeout và retry theo cấu hình; batch giảm số lần gửi nhưng tạo độ trễ và phần dữ liệu chưa chuyển đi. Khi backend chậm, queue hữu hạn có thể đầy, retry hết hạn hoặc dữ liệu bị từ chối. Queue trong bộ nhớ mất theo tiến trình; hàng đợi bền cần storage được cấu hình và cũng không thay cho chính sách lưu dữ liệu của backend.

Cần phân biệt bốn ranh giới: ứng dụng đã tạo span; SDK đã đưa span cho exporter; Collector đã nhận yêu cầu OTLP; backend đã lưu và truy vấn thấy span. Một kết quả gửi thành công đến Collector chỉ xác nhận mức chấp nhận của chặng đó. OTLP còn có phản hồi partial success cho phần bị từ chối và lỗi có/không thể retry; ngay cả Collector đã nhận đủ cũng có thể lỗi khi xuất đi. Chẩn đoán mất trace cần đo queue, dropped items, retry/export failures và trạng thái backend ở đúng cửa sổ. Đừng biến lỗi Collector thành lỗi request nghiệp vụ: chọn xử lý bất đồng bộ và giới hạn queue/timeout để đường chính không mặc nhiên chờ telemetry. Tuy nhiên exporter đồng bộ hoặc cấu hình chặn khi queue đầy có thể kéo dài request; phải kiểm tra SDK và Collector cụ thể, không khẳng định khả năng chịu lỗi nếu chưa thử.

Khi p95 tăng mà CPU và error rate bình thường, trước tiên cố định time range, route, status và số request của metric. Chọn một request chậm đại diện, tìm log theo request ID và trace ID rồi kiểm tra timeline, parent, HTTP response và các lần retry. So thời điểm timeout với metric độ trễ DB/upstream và log tại đúng dịch vụ; kiểm tra queue và pool wait trước khi quy chậm cho CPU. Nếu thiếu span, kiểm tra header propagation qua Gateway, sampling, SDK/exporter, Collector receiver/processor/queue, OTLP rejection và backend retention. Không có span chỉ chứng minh không thấy span ở vị trí tìm kiếm, không chứng minh thao tác chưa chạy. Timestamp từ nhiều máy có thể lệch; đối chiếu duration trong span, đồng hồ hệ thống và thứ tự sự kiện từ cùng nguồn trước khi suy một lời gọi kết thúc trước khi bắt đầu.

Bài tập P12.C03.L03.Q01

API trả request thành công nhưng tìm trace chỉ thấy span Gateway. Nêu ít nhất bốn ranh giới cần kiểm tra theo thứ tự từ propagation đến backend. Cờ sampled `01` trong `traceparent` cho phép kết luận gì và không cho phép kết luận gì?

Chương P12.C04 — Alerting và sức khỏe pipeline quan sát

Tạo tín hiệu hành động thay vì báo động từ một dashboard.

Bài P12.C04.L01 — Rule

Với alert, tách "không lấy được metric" (`up`, target discovery/absence) khỏi "ứng dụng có lỗi" (tỷ lệ 5xx và lưu lượng tối thiểu). Recording rule có thể lưu tốc độ đã tổng hợp theo service/route, nhưng tên và nhãn của rule phải giữ đúng phạm vi; nếu mất target, kết quả tổng hợp cũng thiếu. Một alert p95 nên có điều kiện lưu lượng và chất lượng dữ liệu, ngưỡng theo SLO đã xác định, thời gian `for` phù hợp; ví dụ này không ấn định SLO cho hệ thật.

Recording rule định kỳ lưu kết quả biểu thức thành chuỗi mới để giảm chi phí truy vấn lặp lại; alerting rule đánh giá điều kiện và chuyển cảnh báo sang trạng thái chờ/phát theo `for`.

Bài P12.C04.L03 — Pipeline

Một pipeline telemetry gồm nhiều chặng: ứng dụng tạo tín hiệu, SDK hoặc agent chọn và gửi, collector nhận và xử lý, backend lưu, rồi công cụ truy vấn hiển thị. Mỗi chặng có miền lỗi riêng. Request thành công không chứng minh span đã được lưu; một trace xuất hiện cũng không chứng minh toàn bộ request đã được lấy mẫu. Vì vậy, khi p95 metric tăng nhưng các trace đã lưu chủ yếu nhanh, không được kết luận metric sai chỉ từ tập trace đó.

Trước hết xét sampling: head sampling có thể bỏ qua nhiều request trước khi biết độ trễ cuối cùng; tail sampling có thể có quy tắc khác cho trace lỗi hoặc chậm. Đối chiếu cấu hình và số lượng trace theo cùng cửa sổ với metric toàn population. Tiếp theo xét đường vận chuyển: queue đầy, retry hoặc drop ở exporter/collector có thể làm thiếu span; đọc counter của chính pipeline và log lỗi gửi. Cuối cùng xét truy vấn và retention: time range, nhãn dịch vụ, sampling của backend và thời gian lưu có thể khiến tập nhìn thấy không đại diện. Kiểm từng giả thuyết bằng số vào/ra của chặng tương ứng.

Liên kết metric, log và trace qua service, route, thời gian và trace ID trong log/trace. Không đặt request ID vào nhãn metric có cardinality cao; cũng không đưa dữ liệu cá nhân vào thuộc tính chỉ để tiện tìm kiếm. Nếu tín hiệu thiếu, ghi trạng thái đo là chưa đủ bằng chứng thay vì biến khoảng trống thành số 0.

Bài tập P12.C04.L03.Q01

p95 metric tăng, còn tập trace đã lưu có phần lớn request nhanh. Nêu ba cách dữ liệu quan sát bị sai lệch và một phép kiểm cho từng cách. Đề xuất cách liên kết log/trace/metric mà không đưa request ID vào nhãn Prometheus hoặc lộ PII.

PHẦN 13 — SRE và phục hồi

Độ tin cậy được định nghĩa, bảo vệ và khôi phục ra sao?

SLI/SLO định nghĩa tác động người dùng; response dùng chứng cứ, DR dùng bản sao có thử khôi phục, capacity dùng tải và giới hạn.

Chương P13.C01 — SLI, SLO và error budget

Chọn phép đo gần trải nghiệm người dùng và tính burn rate.

Bài P13.C01.L01 — SLI

Bản thảo nội dung A1; dữ liệu dưới đây là giả định để học, không phải phép đo production.

Một API trả HTTP 200 có thể vẫn gây mất tiền nếu giá trị đơn hàng sai; ngược lại HTTP 503 ở một tác vụ nền không thuộc hành trình mua hàng có thể không làm hỏng thao tác khách hàng đang quan tâm. Vì thế trước khi chọn biểu đồ, ta viết hợp đồng sự kiện: ai là người dùng, hành động nào quan trọng, điểm đo ở client, edge hay server, và cách xử lý timeout, retry, cache, request trùng, bot, dữ liệu lỗi. **SLI** là phép đo có định nghĩa đó, không phải tên của một metric có sẵn. Availability thường là tỷ lệ thao tác hợp lệ thành công; latency là tỷ lệ thao tác hợp lệ hoàn tất dưới ngưỡng thời gian. Error rate là phần bù của một SLI nhị phân nếu định nghĩa "bad" bao trùm mọi trường hợp không good. Throughput đo lượng công việc trên đơn vị thời gian; nó mô tả tải, không tự là chất lượng phục vụ.

Với request-based SLI, tập `valid` gồm các lần thử thuộc phạm vi đã công bố; `good` là tập con thỏa cả kết quả nghiệp vụ và ngưỡng thời gian nếu SLI gộp hai điều kiện. `bad = valid - good`. Tỷ lệ good là `good/valid`, có đơn vị 1; không cộng các tỷ lệ của từng Pod rồi chia đều khi lưu lượng khác nhau. Chỉ loại request có lý do sản phẩm rõ (ví dụ endpoint nội bộ hoặc đầu vào bị từ chối đúng theo hợp đồng), ghi riêng số bị loại. Timeout phía client sau khi request rời máy thường phải được tính là bad nếu SLI hứa đáp ứng cho lần thử ấy, ngay cả khi server không có log; đo chỉ ở server có thể làm mẫu số co lại trong lúc mạng hỏng. Retry chỉ là good của một *lần thử* nếu SLI đo lần thử; nếu SLI đo một hành trình người dùng, phải gom các lần thử thành một thao tác và công bố quy tắc gom. Không đổi mẫu số sau sự cố để làm SLI đẹp hơn.

Time-based SLI chia lịch thành các khoảng đo (chẳng hạn phút) và xét phút nào thỏa điều kiện phục vụ. Nó hữu ích khi mục tiêu là thời gian sẵn sàng, nhưng một phút có một request được cân ngang phút có mười nghìn request. Request-based SLI cân theo số thao tác, nên phản ánh tác động tại đỉnh tải tốt hơn nhưng im lặng khi không có request. Hai cách không đổi chéo bằng một phép nhân tùy tiện; nếu dùng probe tổng hợp, giữ nó thành luồng đo riêng và nói rõ phần hành vi người dùng mà probe không kiểm được.

Bài P13.C01.L02 — SLO

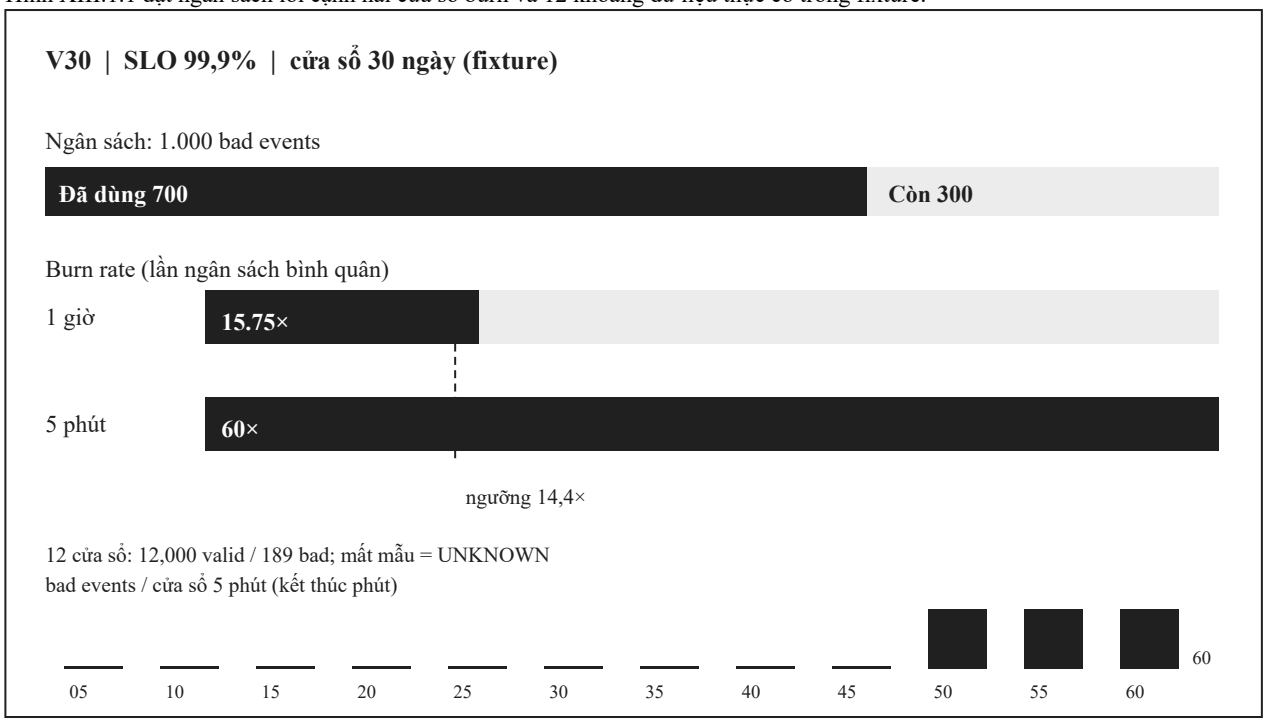
SLO là mục tiêu định lượng áp lên SLI, population và cửa sổ: ví dụ "ít nhất 99,9% request checkout hợp lệ trong 30 ngày gần nhất trả kết quả đúng trong 500 ms, đo tại edge". **SLA** là thỏa thuận với bên nhận dịch vụ có hậu quả khi vi phạm; chỉ có SLO nội bộ chưa tự thành SLA. SLO cũng không chứng minh mỗi request đúng: một mục tiêu 99,9% vẫn cho phép một phần thao tác lỗi. Rolling 30 ngày tại thời điểm `t` dùng `(t−30 ngày, t]` và dịch mỗi khi đánh giá; tháng lịch cố định đóng ở ranh giới tháng. Cùng số lượng bad nhưng budget còn lại có thể khác vì population, cửa sổ và lượng request khác. Cần chốt múi giờ, độ trễ nạp dữ liệu và quy tắc event sát biên trước khi so số liệu.

Với SLO dạng tỷ lệ `S` và `V` valid events trong cửa sổ, ngân sách lý thuyết là `B=(1−S)×V` bad events; đã dùng `E=V−G`, còn `B−E`. Nếu `B` không nguyên, chính sách làm tròn phải được định nghĩa trước, không tự nâng trần để nới mục tiêu. Với một cửa sổ có ít lưu lượng, tỷ lệ có phương sai lớn: một bad trong 10 request là 10% lỗi, không đủ để suy cả hệ thống hỏng liên tục. Với SLO theo thời gian, ngân sách là `(1−S)×độ dài cửa sổ` theo đơn vị thời gian, không lấy số request làm mẫu số. Với latency SLI, bad là request quá ngưỡng (hoặc timeout theo hợp đồng), không lấy trung bình các p95 rời rạc làm phần trăm good toàn hệ thống.

`monthly.csv` là **toàn bộ dữ liệu giả định** của rolling 30 ngày tại một thời điểm: 988.000 valid/511 bad trước giờ cuối, 12.000 valid/189 bad ở giờ cuối. Tổng `V=1.000.000`, `E=700`, `G=999.300`, SLI `99,93%`. Với `S=99,9%`, `B=1.000` bad và còn `300` bad. Phần giờ cuối khớp 12 dòng năm phút trong `five_minute.csv`; con số này chứng minh phép tính trên fixture, không cho biết lưu lượng tháng sau. Nếu có request hoàn toàn mất khỏi instrumentation, các con số vẫn có thể đẹp một cách sai lệch.

Bài P13.C01.L03 — Burn

Hình XIII.1.1 đặt ngân sách lỗi cạnh hai cửa sổ burn và 12 khoảng dữ liệu thực có trong fixture.



Hình XIII.1.1 — V30: 700 bad đã dùng và 300 còn lại; burn 1h/5m từ dữ liệu giả định.

Trong một cửa sổ ngắn, `error_ratio = bad/valid`. **Burn rate** của SLO request-based là `error_ratio/(1−S)`, không có đơn vị. Burn 1 nghĩa tỷ lệ lỗi hiện tại bằng phần lỗi được phép bình quân; nó không nói budget hiện đã cạn. Giờ cuối trong fixture có `189/12.000 = 1,575%` lỗi, burn `15,75`; năm phút cuối có `60/1.000 = 6%`, burn `60`. Cảnh báo 1 giờ/5 phút với ngưỡng burn `14,4` sẽ thỏa cả hai phía (`15,75 > 14,4` và `60 > 14,4`). Đây là một tín hiệu can thiệp nhanh, **không phải** bằng chứng duy nhất của nguyên nhân hay xác suất tháng sẽ vi phạm SLO. Công thức "burn × độ dài cửa sổ / 30 ngày" ước lượng tỷ lệ ngân sách thời gian bị tiêu nếu traffic và error ratio đại diện, ổn định; với request budget và traffic biến thiên, phải dùng bad events thật trên toàn population thay cho ước lượng thời gian ấy.

Ví dụ PromQL sau giả sử ứng dụng xuất hai counter riêng `sli_valid_total` và `sli_bad_total`, cùng nhãn `service` và `route`, counter bad tăng đúng một lần cho mỗi valid bad. Đây là **mẫu biểu thức chưa chạy trên Prometheus**, không phải output thật:

```
(
  sum(rate(sli_bad_total{service="checkout",route="write"}[1h]))
  /
  sum(rate(sli_valid_total{service="checkout",route="write"}[1h]))
  > (14.4 * 0.001)
)
and
(
  sum(rate(sli_bad_total{service="checkout",route="write"}[5m]))
  /
  sum(rate(sli_valid_total{service="checkout",route="write"}[5m]))
  > (14.4 * 0.001)
)
```

`rate()` phải áp trên từng counter trước `sum` để nhận ra reset từng instance. Hai cửa sổ dài/ngắn cùng vượt ngưỡng giúp xác nhận lỗi còn đang tiếp diễn và giảm thời gian alert treo sau khi hồi phục. Trên Prometheus thật, cửa sổ phải có đủ sample, vector matching/labels phải đúng, mẫu số dương và target phải còn được scrape; thiếu chuỗi hoặc mất scrape là trạng thái đo **unknown**, không là lỗi bằng 0. Cần cảnh báo độc lập cho tính đầy đủ telemetry và kiểm từ phía client. Không tính trung bình tỷ lệ của các replica; lấy tổng tốc độ bad chia tổng tốc độ valid cùng population. SLO 6h/30m là một nhánh alert khác, không được kết luận từ fixture chỉ có 1h.

Một page burn nhanh và chỉ còn 300 bad làm việc mở rộng canary không hợp lý cho tới khi phân biệt lỗi thật và lỗi đo. Người trực có thể dừng tăng traffic, so request/dữ liệu theo release, giảm tác động bằng rollback hoặc flag nếu hậu điều kiện đã biết. Quyết định tạm dừng release là chính sách sản phẩm/SRE đã thống nhất, không phải tính năng tự động của phương trình. Khi ít lưu lượng, một bad có thể tạo burn rất lớn; đánh giá giá trị giao dịch, probe riêng, số sự kiện và độ tin cậy dữ liệu trước khi đánh thức cả đội. Khi không có valid event, tỷ lệ không xác định chứ không bằng 100% good.

Bài tập C13.1

Trong fixture 30 ngày, giả sử chỉ giữ 11 dòng đầu của `five_minute.csv` vì dòng cuối chưa được scrape, nhưng monthly summary vẫn được dashboard giữ nguyên. Có được báo “không còn burn” hay không? Hãy nêu phép đối chiếu, trạng thái đo và quyết định trước khi tiếp tục canary.

Bài tập C13.2

Một API có hai replica: A xử lý 900 valid/9 bad, B xử lý 100 valid/10 bad trong cùng cửa sổ. Tính tỷ lệ lỗi toàn dịch vụ và giải thích vì sao trung bình cộng 1% và 10% là sai. Nếu SLO 99%, còn ngân sách trong cửa sổ 1.000 valid là bao nhiêu bad?

Bài tập C13.3

Với năm phút cuối trong fixture, tính burn và so ngưỡng 14,4. Nếu 1 giờ trước đó chỉ có 12.000 valid/100 bad (thay cho 189), alert hai cửa sổ có page không? Giải thích vì sao không thể tự suy nhánh 6h/30m.

Chương P13.C02 — Incident response và postmortem

Tổ chức phát hiện, phân vai, giảm tác động và học từ sự cố.

Bài P13.C02.L01 — Phản ứng

Tình huống, log tóm tắt và số liệu trong chương là fixture giả định, không phải kết quả chạy production.

Một alert chỉ là tín hiệu được một quy tắc phát ra. Một incident bắt đầu khi có tác động hoặc rủi ro đủ để cần phối hợp ứng cứu; incident có thể được khai báo từ cuộc gọi khách hàng dù alert im lặng. **Impact** mô tả ai bị ảnh hưởng, hành động nào hỏng, bao nhiêu sự kiện và trong bao lâu. **Severity** là mức tác động đã xác định theo chính sách tổ chức; **priority** là thứ tự xử lý có thể thay đổi vì thời gian, nghĩa vụ pháp lý hoặc rủi ro tiếp diễn. Không gán P1 chỉ vì một biểu đồ màu đỏ. Ngược lại, sai số tiền đã lưu là lý do hợp lý để đề xuất P1 dù HTTP 5xx bằng 0; người trực vẫn phải đối chiếu tiêu chí P1 thực tế của tổ chức.

Fixture `observations.csv` theo cùng một cửa sổ năm phút: v1 phục vụ 900 request, 1 số tiền sai; v2 phục vụ 100 request, 30 số tiền sai. Cả hai có `http_5xx=0`. Nếu SLI chỉ đếm status thành công thì sẽ báo 100% good, trong khi phép đối soát nghiệp vụ cho thấy 31/1.000 request sai, tức 3,1%. Trước canary, v1 ở cửa sổ trước có 1/900 sai; đây là baseline nhỏ, không chứng minh v1 hoàn hảo. Tỷ lệ v2 sai 30%, v1 khoảng 0,111% cùng thời điểm. Ta mới có một tương quan mạnh với release, chưa có root cause. Việc tập v2 là 10% lưu lượng cần kiểm chứng phân tuyến thực tế, không chỉ cấu hình cân bằng tải.

Tại 10:07, trong bài toán giả định, người trực tuyên bố incident và mở một tài liệu trạng thái sống: tác động đã biết, phần chưa biết, giả thuyết, người phụ trách, thời điểm cập nhật tiếp theo. Incident commander (IC) giữ mục tiêu và quyền điều phối; Ops lead thực hiện thay đổi kỹ thuật trong phạm vi được giao; Communications lead báo đều cho khách hàng và các nhóm liên quan, nêu điều đã xác minh, điều đang điều tra và mốc cập nhật. Nếu chỉ có một người, họ có thể tạm kiêm vai nhưng phải ưu tiên gọi hỗ trợ khi đồng thời vừa đổi hệ thống vừa báo cáo trở nên không an toàn. Escalation không phải chuyển trách nhiệm cho "ai đó": nêu rõ cần quyền gì, quyết định nào, thời hạn nào. Nhật ký chung ghi cả thử nghiệm thất bại và lý do dừng để người sau không lặp thao tác nguy hiểm.

Bài P13.C02.L02 — Giảm tác động

Từ `request_evidence.csv`, hai request v2 mang 4.100 và 5.900 cents được lưu thành 41 và 59 cents, nhưng đều trả HTTP 200; request v1 cùng đầu vào 4.100 giữ 4.100. DB time 42–49 ms và p95 toàn cửa sổ 180–190 ms không cho thấy tăng chậm lớn, nhưng vài mẫu không đủ loại bỏ mọi vấn đề database. Giữ bốn giả thuyết: DNS/edge đưa sai đích; DB làm chậm hoặc ghi sai; adapter v2 đổi đơn vị; feature flag chỉ áp cho nhóm canary. DNS sai toàn service khó giải thích khác biệt v1/v2 trong cùng cửa sổ, nhưng fixture chưa có edge destination hay revision cho từng request; cần đo chúng trước khi loại giả thuyết này. DB chậm không giải thích nhân 1/100 nhất quán; vẫn có thể có lỗi ghi logic ở tầng DB. Feature flag và adapter đều còn khả dĩ cho tới khi so config, diff và tái hiện cặp input–output trong fixture độc lập. Một diff **giả định** `store(amount_cents)` → `store(amount_cents // 100)` khớp hai mẫu v2; đây là bằng chứng cơ chế của tình huống, không là log/code production đã quan sát. Aggregate chỉ phân loại **30 request v2 sai**; số order hoặc bản ghi phân biệt bị ảnh hưởng là **UNKNOWN** vì retry và cập nhật lặp có thể cùng trỏ một order. Cần nối từng request ID với order ID rồi đối soát trạng thái lưu trước khi khẳng định phạm vi dữ liệu.

Đường giảm tác động chọn theo ranh giới dữ liệu. Tại 10:09 runbook **đề xuất** giữ digest, manifest, config flag, log/trace, phân tuyến và snapshot dữ liệu; hạn chế truy cập bản sao có thông tin khách hàng. Tại 10:12 **đề xuất** dừng đường ghi mới của v2 và đưa traffic về v1; chưa có bằng chứng thao tác đã hoàn tất. Hậu kiểm phải chứng minh không còn worker/queue consumer v2; chỉ rollback routing mà bỏ qua job nền là chưa đủ. Nếu lỗi nằm sau một flag độc lập, tắt flag có thể nhanh hơn, nhưng phải chứng minh phiên v2 không còn ghi sai. Failover sang vùng khác sẽ vô ích nếu cùng binary/config sai hoặc dữ liệu đã được sao chép; nó chỉ phù hợp nếu failure domain được chứng minh độc lập. Traffic shifting, rollback, flag và failover là các lựa chọn có tiền điều kiện/hậu điều kiện, không phải bốn nút “chữa lỗi” tương đương.

Mốc 10:16 là **phép hậu kiểm cần làm**, chưa có request ID, revision, input/stored hay kết quả đọc lại sau mitigation. Cần thử request mới qua v1 từ nhiều client/route, kiểm digest/revision đang phục vụ và xác nhận không còn v2 writer; chỉ sau đó mới có thể nói đường request mới phục hồi. Tại 10:22, dữ liệu hiện có vẫn chỉ cho biết 30 request v2 sai trong cửa sổ; chưa biết số order/bản ghi phân biệt, chưa có đối soát hay sửa dữ liệu. Vì vậy incident đang ở trạng thái *mitigation được đề xuất; phục vụ mới và phục hồi dữ liệu chưa xác minh*. Dashboard 5xx và CPU xanh không thay thế hai phép kiểm ấy. Một SLI nghiệp vụ đúng phải tiếp tục báo phần bad đã xảy ra; chính sách SLO xác định khi nào mở lại release, không được xóa dữ liệu lịch sử để “reset” ngân sách.

Bài P13.C02.L03 — Học lại

Hình XIII.2.1 giữ riêng quan sát, giả định, đề xuất và hậu kiểm còn thiếu.

Incident timeline | fixture, không phải production

09:55	**QUAN SÁT**	Baseline v1 ghi nhận 900 valid và 1 sai số tiền
10:00	**GIẢ ĐỊNH**	change rel-84 tạo revision 5 đưa v2 nhận 10 phần trăm request
10:03	**QUAN SÁT**	Hỗ trợ nhận báo cáo số tiền sai
10:05	**QUAN SÁT**	Chốt cửa sổ 5 phút v1/v2 và lấy mẫu K-26 đến K-28
10:07	**ĐỀ XUẤT**	Tuyên bố incident P1 và phân IC/Ops/Comms
10:09	**ĐỀ XUẤT**	Giữ manifest digest config log trace và bản sao dữ liệu
10:12	**ĐỀ XUẤT**	Đề xuất chặn mọi v2 writer và chuyển request mới về v1; chưa có hậu kiểm
10:16	**CẦN XÁC MINH**	Cần thử request mới qua v1 và xác minh không còn v2 writer; chưa có kết quả
10:22	**CẦN ĐỐI SOÁT**	Đã phân loại 30 request v2 sai; số order hoặc bản ghi phân biệt chưa rõ

31/1.000 request sai; số order phân biệt UNKNOWN; hậu kiểm 10:16 chưa chạy.

Hình XIII.2.1 — Trạng thái bằng chứng trong incident; 10:16 còn cần xác minh.

Timeline trong `timeline.csv` tách observation, giả định, đề xuất thao tác và verification còn cần làm. Cùng một câu "rollback xong" có thể có ba timestamp: lệnh hoàn tất, request mới đúng, dữ liệu cũ được sửa. Postmortem không ép ba mốc đó thành một. Khi điều tra sau giảm tác động, hỏi vì sao một thay đổi đổi đơn vị lọt qua: phép thử nào thiếu, canary gate chỉ nhìn status nào, telemetry có dấu hiệu gì nhưng không có cảnh báo, ai có quyền nâng traffic, và dữ liệu nào đáng lẽ được đối soát trước khi ghi. Giả thuyết root cause là adapter v2 đổi cents thành đơn vị nhỏ hơn; cần tái hiện trên input cụ thể và so cả diff/config trước khi ghi là xác nhận. Các điều kiện đóng góp có thể gồm test chỉ xác nhận 200, thiếu SLI số tiền, không có stop rule cho canary 10%; đó là lỗ hổng hệ thống cần khắc phục, không là lời buộc tội cá nhân.

Action item phải có chủ sở hữu, thời hạn và phép kiểm: (1) test âm cho 4.100 cents và kiểm dữ liệu lưu; (2) canary gate tính tỷ lệ sai nghiệp vụ theo release với mẫu số và missing-data policy; (3) runbook chặn mọi v2 writer và đối soát toàn bộ order ID; (4) diễn tập rollback app **và** kiểm sửa dữ liệu. "Cẩn thận hơn" không kiểm chứng được. Incident chỉ được đóng khi người phụ trách dữ liệu xác nhận phạm vi bản ghi, kế hoạch/sửa chữa, hậu điều kiện và thông báo cho người bị ảnh hưởng theo quy trình. Một postmortem không phải tài liệu để tìm người chịu lỗi; nó là hồ sơ bằng chứng và các thay đổi giảm xác suất hoặc tác động của lần sau.

Bài tập C13.4

Tính tỷ lệ sai của v1, v2 và toàn canary window. Một người kết luận "DB chậm là root cause" vì có vài request chậm; người khác kết luận "DNS sai" vì khách hàng báo lỗi. Dữ liệu nào chống lại hai kết luận ấy, dữ liệu nào còn thiếu? Hãy đề xuất thứ tự ba phép kiểm ít gây tác động nhất.

Bài tập C13.5

Dashboard ghi HTTP 5xx bằng 0 trong cửa sổ quan sát. Runbook đề xuất chuyển về v1 lúc 10:12; phép thử request mới lúc 10:16 chưa có kết quả. Aggregate có 30 request v2 sai nhưng chưa biết số order phân biệt. IC có được đánh dấu incident `resolved` và mở lại v2 không? Tách bốn việc: mitigation, xác minh request mới, đối soát/khôi phục dữ liệu và điều kiện đóng incident.

Bài tập C13.6

So sánh rollback ứng dụng, tắt feature flag và failover trong fixture. Nêu tiền điều kiện và một hậu điều kiện có thể đo cho mỗi lựa chọn; nếu dữ kiện thiếu thì ghi rõ không thể chọn an toàn.

Chương P13.C03 — Backup, DR và phục hồi dữ liệu

Phân biệt availability, durability, RPO, RTO và restore kiểm chứng.

Bài P13.C03.L01 — Mục tiêu

Toàn bộ thời điểm, đơn hàng và kết quả trong chương là fixture giả định. Chưa chạy restore PostgreSQL hay diễn tập trên hệ thống thật. Mô hình trung tâm: chỉ một bản sao có thể khôi phục, đủ nhất quán và nằm ngoài miền lỗi cần chống mới có giá trị đối với mục tiêu phục hồi.

Khi ứng dụng mất kết nối database, khởi động lại Pod có thể khôi phục process nhưng không tái tạo một đơn hàng đã bị xóa. **High availability** giữ dịch vụ tiếp tục phục vụ khi một thành phần hỏng, thường nhờ bản sao hoặc chuyển vai; **disaster recovery** khôi phục sau biến cố vượt khả năng HA. Replication đưa thay đổi tới bản sao để giảm gián đoạn, đồng thời có thể sao cả thao tác xóa sai hoặc dữ liệu hỏng. Backup là một điểm dữ liệu có thể chọn để phục hồi; nhãn "backup thành công" chỉ nói đã tạo được bản sao theo một quy trình, chưa chứng minh đọc lại đúng nội dung. Snapshot là cách ghi trạng thái ở một thời điểm, nhưng tính nhất quán phụ thuộc tầng tạo snapshot và ứng dụng. Những cơ chế này bổ sung cho nhau; không suy ra backup từ số replica hay từ việc storage tuyên bố durable.

Mục tiêu giả định là **RPO không quá 15 phút**, **RTO không quá 45 phút**. RPO đo khoảng từ thời điểm mất khả năng ghi lúc 10:00 lùi đến commit cuối cùng thực sự khôi phục được. RTO đo từ 10:00 đến lúc đường nghiệp vụ đọc **và ghi** được kiểm chứng, không dừng ở thông báo "database started". Nếu chỉ đọc được một trang health check, chưa đạt hậu điều kiện. Kịch bản dùng base backup nhất quán lúc 09:00 và chuỗi WAL đã lưu liên tục đến commit 09:52. Sự cố lúc 10:00; phát hiện 10:04, bắt đầu restore 10:08, replay xong 10:27, xác minh dữ liệu và ghi/đọc ứng dụng xong 10:37. Theo fixture, RPO thực tế là 8 phút, RTO thực tế là 37 phút. Đây là phép tính từ timeline, không phải bảo đảm RPO/RTO cho mọi lần sự cố. Nếu WAL thực sự thiếu một đoạn trước 09:52, recovery point phải lùi hoặc restore thất bại; không được ghi 09:52 chỉ vì dashboard archive xanh.

Miền lỗi quyết định bản sao nào còn sống. Hai file trên cùng volume không bảo vệ khi volume mất; replica đồng bộ cùng account và quyền xóa không bảo vệ trước một lệnh xóa lan truyền. Bản sao phục hồi cần được kiểm tra ở miền lỗi tách biệt theo rủi ro đang xét, với chính sách quyền, phiên bản, thời hạn lưu và khóa giải mã còn dùng được. "Offsite" không tự động nghĩa là độc lập trước credential bị chiếm; cần thử kịch bản mất quyền, mất key và xóa nhầm bên cạnh lỗi máy.

Bài P13.C03.L02 — Bản sao

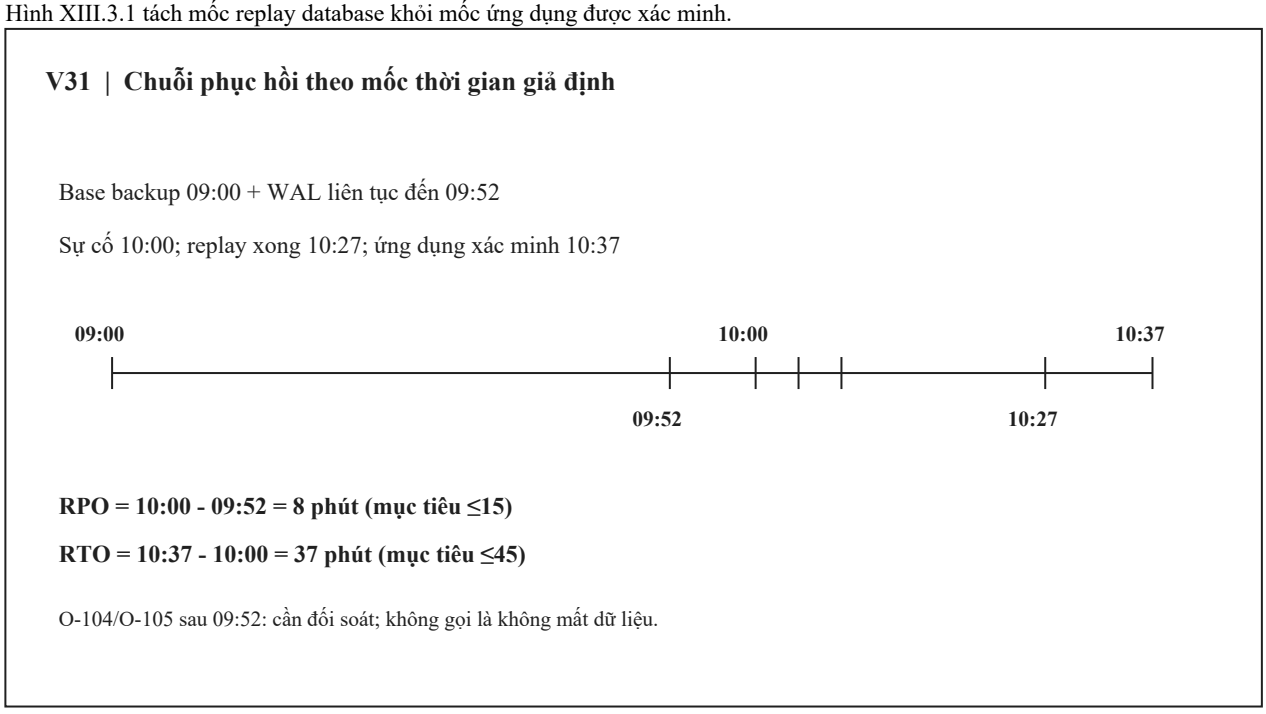
Một snapshot filesystem có thể **crash-consistent**: các byte giống lúc máy tắt đột ngột, database phải replay log và vẫn có thể phát hiện giao dịch chưa commit. Nó không mặc nhiên **application-consistent**: ví dụ hai hệ thống cùng ghi một đơn, một snapshot chụp sau DB commit nhưng trước khi đối soát message có thể giữ hai phía ở hai thời điểm khác nhau. Một base backup PostgreSQL hợp lệ cộng WAL liên tục cho phép khôi phục đến điểm mong muốn trong phần WAL còn có; việc sao chép riêng data directory của database đang chạy mà không theo quy trình backup không thay thế cơ chế ấy. Nếu cần nhất quán xuyên database, object store và queue, phải xác định invariant nghiệp vụ và cách đối soát hoặc đóng băng ghi; một snapshot từng hệ thống không tự tạo giao dịch liên hệ thống.

Trong fixture `orders.csv`, O-101 đến O-103 commit chậm nhất 09:52 và còn ở bản khôi phục. O-104 lúc 09:53 và O-105 lúc 09:59 không có trong recovery point; chúng nằm trong khoảng mất dữ liệu 8 phút đã chấp nhận theo RPO, nhưng vẫn cần đối chiếu nguồn giao dịch bên ngoài để xử lý. Một bản sao khác đến 09:59 có thể ít mất hơn, nhưng chỉ chọn được nếu kiểm tra tính toàn vẹn, thứ tự log và miền lỗi. Không đổi điểm khôi phục chỉ dựa vào timestamp của file hoặc “latest” trên UI.

Retention xác định ta có thể quay về bao xa trước một lỗi xóa âm thầm được phát hiện muộn. Versioning hoặc bản sao khó sửa giúp giữ một phiên bản cũ, nhưng không loại rủi ro hết hạn retention, phá khóa mã hóa hoặc cấu hình cho cùng principal được xóa mọi phiên bản. Cần ghi danh mục backup, checksum hoặc kiểm tính toàn vẹn tương đương, chain WAL, phiên bản schema, key và quyền đọc. Hash khớp cho file chỉ xác nhận byte được so khớp; nó không chứng minh giao dịch nghiệp vụ đúng hoặc ứng dụng dùng được.

Bài P13.C03.L03 — Diễn tập

Hình XIII.3.1 tách mốc replay database khỏi mốc ứng dụng được xác minh.



Hình XIII.3.1 — V31: WAL, điểm khôi phục và hậu kiểm ứng dụng; RPO/RTO giả định.

Runbook bắt đầu bằng **dừng thao tác phá dữ liệu**: cô lập writer và job ghi, khóa quyền xóa trên bản sao, giữ ảnh hiện trạng và log trước khi thử restore. Nếu chưa xác định được backup chain, khóa giải mã, recovery target hoặc người phê duyệt ghi đè, dừng trước bước thay production. Diễn tập trên môi trường tạm độc lập: liệt kê base backup và WAL, kiểm tính liên tục, dựng một instance sạch, replay đến commit 09:52, kiểm tra trạng thái database, các order ID và tổng tiền. Sau đó thử bằng tài khoản ứng dụng một phép đọc và ghi có thể hoàn tác, đối soát message/queue, xác nhận route tới instance đúng và chỉ mới cân nhắc mở traffic. Nếu bài kiểm gây thay đổi dữ liệu, dùng fixture độc lập rồi dọn theo chính sách của môi trường thử; không chạy thử trực tiếp trên bản sao duy nhất.

Ở 10:27 chỉ có kết quả replay; 10:37 mới có hậu điều kiện ứng dụng. Báo người chịu trách nhiệm dữ liệu rằng O-104/O-105 chưa có trong điểm khôi phục và cần quy trình tái tạo/đền bù, tránh tuyên bố "không mất dữ liệu". Nếu client đã nhận xác nhận ghi sau 09:52, phải đối soát với nguồn ngoài DB để không tự phát hành trùng đơn. Khi chỉ có binary/app phục hồi còn nội dung sai, incident vẫn mở. Sau diễn tập, lưu thời điểm, hash, phiên bản, query kiểm tra, sai lệch, quyền đã dùng, thời gian từng bước và kết luận theo từng mục tiêu. Một lần PASS trong môi trường nhỏ không bảo đảm thời gian restore khi dữ liệu lớn hơn hoặc miền lỗi khác; lặp diễn tập với kích thước và điều kiện gần thực tế, không dùng nhãn PASS vĩnh viễn.

Bài tập C13.7

Từ `timeline.csv`, tính RPO/RTO thực tế và đánh giá hai mục tiêu. Nếu dashboard chỉ ghi "restore xong" lúc 10:27, vì sao không thể dùng 27 phút làm RTO? Nếu thiếu WAL 09:45–09:46, có còn chứng minh được recovery point 09:52 không?

Bài tập C13.8

Đối chiếu `orders.csv`: đơn nào chắc chắn hiện diện ở recovery point, đơn nào cần đối soát bên ngoài? Một kỹ sư đề xuất chuyển sang replica cùng account rồi xóa bản chính để tiết kiệm thời gian. Nêu ít nhất ba điều kiện cần xác minh trước bất kỳ thao tác hủy dữ liệu nào.

Bài tập C13.9

Thiết kế hậu kiểm cho một restore: phân biệt kiểm byte, kiểm database, kiểm ứng dụng và kiểm nghiệp vụ. Nêu một tình huống mà checksum PASS nhưng vẫn chưa được mở ghi production.

Chương P13.C04 — Hiệu năng, capacity và thử giới hạn

Phân biệt nghẽn CPU, I/O, hàng đợi và giới hạn dữ liệu khi tải tăng.

Bài P13.C04.L01 — Workload

Mọi số liệu trong `load.csv` là fixture tính toán giả định, không phải benchmark đã chạy. Mô hình trung tâm: request đến, chờ, dùng tài nguyên rồi hoàn tất hoặc bị từ chối; capacity là mức tải giữ được SLO và khả năng phục hồi với biên an toàn đã kiểm, không phải một giá trị CPU.

Một request có **latency** từ điểm người dùng bắt đầu đo tới kết quả họ nhận; một service có **throughput** là số request hoàn tất mỗi giây, còn **arrival/offered rate** là số request đi vào mỗi giây. Khi hàng đợi tăng, hai rate này khác nhau. **Concurrency** là số request đang ở trong hệ thống, gồm đang chờ và đang phục vụ; utilization là phần tài nguyên bận theo một định nghĩa quan sát cụ thể. Một CPU 70% không cho biết thread đang đợi DB, khóa, đĩa, network hay một shard nóng. Cần đo cùng cửa sổ và cùng population; trung bình latency có thể ổn dù p95/p99 vi phạm mục tiêu của một nhóm người dùng.

Trong fixture, mỗi pha dài 60 giây và không có retry, reject hay timeout ẩn. Pha A nhận và hoàn tất 100 request/s, latency trung bình 0,120 s, p95 0,180 s. Pha B nhận và hoàn tất 160 request/s, trung bình 0,250 s, p95 0,520 s. Nếu hai pha đã ổn định và số vào bằng số ra, Little's Law cho số request trung bình trong hệ thống **L = λW**: A là 100 request/s × 0,120 s = 12 request; B là 160 × 0,250 = 40 request. `L` không phải số thread hay CPU core; nó gồm cả chờ. Cùng định nghĩa "vào hệ thống" và "rời hệ thống" phải áp cho λ, W và L. Từ hai con số này không suy được phân bố độ trễ hay bottleneck; dữ liệu p95 và counter tài nguyên vẫn cần thiết.

Pha C nhận 220/s nhưng chỉ hoàn tất 180/s trong 60 giây. Nếu không từ chối, timeout, hủy hoặc retry ẩn và cùng ranh giới đo, **số request chưa hoàn tất** tăng `(220−180)×60 = 2.400 request`. `outstanding_delta` là chênh lệch tổng request đang chờ **và** đang xử lý, không phải phép đo riêng độ dài queue. Hệ thống chưa ổn định: không lấy 220×0,550 để gọi là concurrency trung bình theo Little's Law; 0,550 s chỉ đo trên nhóm đã hoàn tất và bỏ lại các request chưa hoàn tất. Ngay cả 180×0,550 cũng không tự cho tổng request đang có khi outstanding thay đổi. Phải đo riêng độ dài queue đầu/cuối, số in-flight, drop, timeout và retry; nếu một counter thiếu, số 2.400 chỉ là hệ quả **có điều kiện** của fixture.

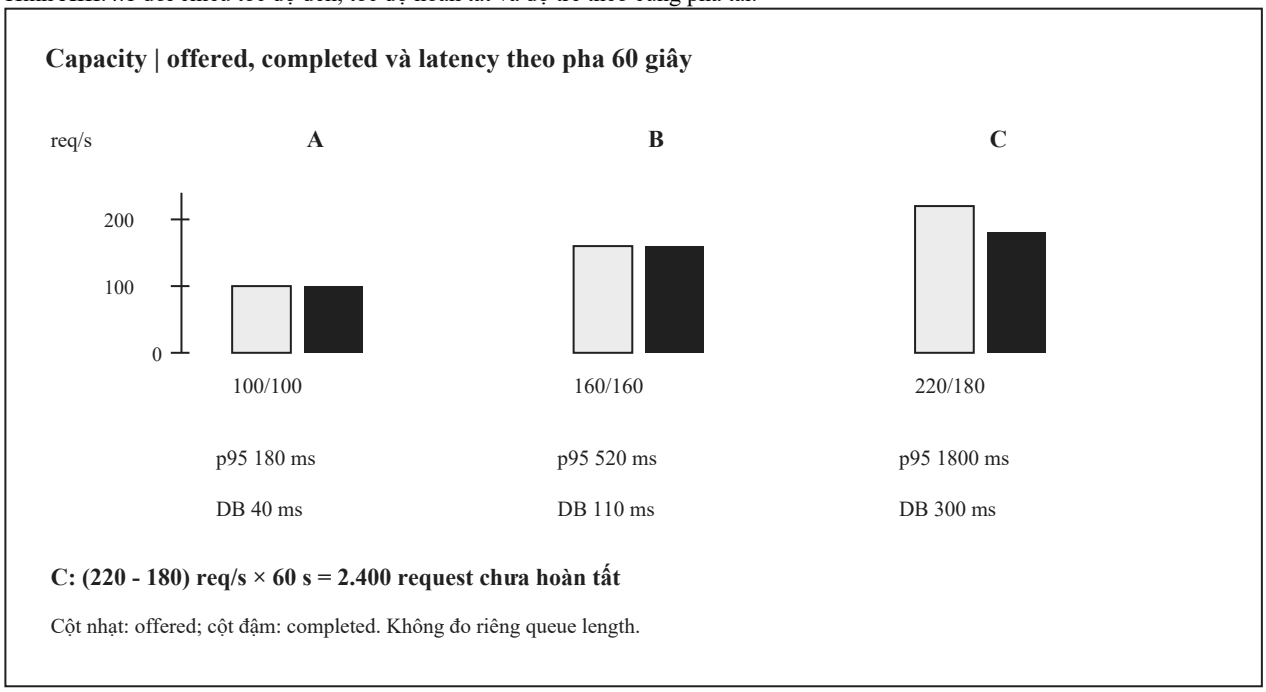
Bài P13.C04.L02 — Capacity

Ở A→B, CPU giả định tăng 45%→65% nhưng p95 từ 180→520 ms; DB wait p95 từ 40→110 ms. Ở C, CPU chỉ 70% trong khi DB wait p95 300 ms, p95 request 1.800 ms và số chưa hoàn tất tăng. Tương quan cho thấy cần kiểm tra DB/connection pool và queue trước khi thêm replica; không chứng minh DB là root cause vì client mix, lock, hot key hoặc đo sai cũng có thể tạo dấu hiệu giống vậy. CPU phải được đọc kèm quota/throttling, per-core skew, run queue và thời gian chờ I/O. Memory phải phân biệt RSS/page cache, reclaim, OOM và GC; I/O cần độ trễ, queue depth, throughput; network cần loss/retransmit, RTT và connection errors. Một graph CPU thấp là một observation, không phải chứng thư còn capacity.

Capacity forecast bắt đầu từ nhu cầu và hậu điều kiện. Nếu peak hiện tại là 160 request/s và dự báo tăng 25%, peak mới là **200 request/s**. Fixture chỉ có pha B ổn định 160/s; pha C 220/s có số request chưa hoàn tất tăng. Vì vậy không thể nội suy rằng 200/s đạt SLO hoặc suy ra cần đúng 1,25 lần số replica. Nếu SLO giả định là p95 ≤600 ms và error rate <0,1% tại peak, B qua được điều kiện latency trong fixture, nhưng chưa có số liệu error rate và độ dài chạy nên cả SLO chưa được chứng minh. C cần stop vì số chưa hoàn tất tăng và p95. Biên an toàn phải xét mất một instance, rollout, autoscaler trễ, burst, skew tenant và shared dependency. Tăng replica có chi phí và có thể đẩy thêm tải vào DB; đặt queue hữu hạn và shed tải có kiểm soát có thể giữ phần request được phục vụ đúng hạn, nhưng sẽ ảnh hưởng availability SLI. Quyết định cần xem cả ngân sách lỗi P13.C01 và chi phí, không tối ưu p95 bằng cách lặng lẽ bỏ request khỏi mẫu số.

Bài P13.C04.L03 — Experiment

Hình XIII.4.1 đối chiếu tốc độ đến, tốc độ hoàn tất và độ trễ theo cùng pha tải.



Hình XIII.4.1 — Tải A/B/C: offered và completed; chênh lệch C là tổng outstanding.

Một **load test** chạy mức tải gần dự báo để kiểm SLO; **stress test** tăng vượt vùng dự kiến để tìm ngưỡng bão hòa và hành vi từ chối; **soak test** giữ tải đủ lâu để lộ leak, tích queue, hết connection/disk hay drift chậm. Tên test không tự chứng minh kết quả: dữ liệu phải có traffic mix, kích thước payload, think time hoặc arrival model, trạng thái cache, phiên bản artifact, topology, warm-up, thời lượng và điểm đo phía client. Một máy phát tải bão hòa có thể tạo số liệu giả; đo cả generator. Histogram p95 phải giữ population và bucket phù hợp; không lấy trung bình p95 giữa các pod như p95 toàn fleet.

Đề xuất thí nghiệm **chưa chạy** cho peak 200/s: dùng môi trường cô lập với cấu hình và dữ liệu đại diện nhưng không chứa dữ liệu khách hàng; kiểm baseline 160/s, tăng từng bậc 180→200→220/s và giữ mỗi bậc đủ lâu để thấy queue đầu/cuối. Ghi offered, accepted, completed, rejected, timed-out, retries, latency client p50/p95/p99, in-flight, queue, CPU quota/throttle, memory/GC, disk/network

và DB wait theo cùng cửa sổ. Stop ngay nếu p95 vượt 600 ms hai cửa sổ liên tiếp, backlog tăng quá giới hạn định trước, dữ liệu sai, hoặc generator bão hòa; giữ khả năng phục hồi trước khi tăng tiếp. Sau giảm tải, xác nhận queue rút, request mới đúng và không có job nền tiếp tục gây tải. Chỉ khi dữ liệu thật đạt hậu điều kiện và thử thêm mất một replica mới chốt mức capacity. Không failure-inject production để "cho chắc" nếu chưa định blast radius và đường quay lui.

Bài tập C13.10

Từ A và B tính concurrency trung bình theo Little's Law. Ở C tính mức tăng số request chưa hoàn tất trong 60 giây, rồi giải thích vì sao 220×0,550 không là concurrency đã chứng minh. Có thể kết luận riêng độ dài queue tăng bao nhiêu không? Nêu hai counter có thể bác bỏ giả định của phép tính.

Bài tập C13.11

Một người nói "CPU 70% nên chỉ cần thêm 25% replica để chịu peak 200/s". Dựa vào dữ liệu A–C, chỉ ra ba lỗ hổng trong suy luận, một giả thuyết bottleneck và phép đo có thể bác bỏ nó. Kết luận có được triển khai 200/s chưa?

Bài tập C13.12

Thiết kế một phép thử capacity cho peak 200/s với tiêu chí dừng, hậu kiểm và thông tin cần ghi để tái lập. Phân biệt load, stress và soak; nêu một ví dụ khi kết quả PASS ở môi trường thử vẫn chưa đủ để công bố production capacity.

Lời giải

Đáp án P01.C01 — Công việc, thời gian và đường đi dữ liệu

P01.C01.L01.Q01. Wall time 100 ms, CPU time 32 ms. Giữ nguyên 68 ms chờ và giảm CPU xuống 16 ms cho 84 ms. Đây là tính toán theo giả định; không dự báo phép đo thật.

P01.C01.L02.Q01. Theo mô hình cộng đã cho, độ trễ còn 20 + 60 = 80 ms. Trên hệ thống thật, thay đổi thời gian CPU có thể đổi lịch chạy, hành vi cache, mức cạnh tranh hoặc thời gian chờ của các yêu cầu khác; 80 ms chỉ là kết quả của giả định giữ mọi yếu tố kia cố định.

P01.C01.L03.Q01. Cần kiểm việc đồng bộ nội dung tệp và mục nhập thư mục sau đổi tên, đồng thời kiểm lỗi từng lời gọi trên filesystem mục tiêu. HTTP thành công không chứng minh dữ liệu đã bền vững sau mất điện.

Đáp án P01.C02 — Chương trình, tiến trình và ranh giới nhân

P01.C02.L01.Q01. Artifact trên đĩa và lần thực thi đang chạy là hai thực thể khác nhau. Việc ghi đè một đường dẫn không chứng minh process image đã đổi. Cần kiểm tra PID và thời điểm khởi động, lệnh và artifact mà dịch vụ đang dùng, rồi so với định danh bản triển khai. Nếu ứng dụng có endpoint phiên bản, chỉ dùng nó khi biết endpoint phản ánh đúng build đang chạy. Khởi động lại phải theo quy trình triển khai, có kiểm tra sức khỏe và đường quay lui.

P01.C02.L02.Q01. Sau `fork` thành công có hai tiến trình: cha và con. Nếu con gọi `execve` và lời gọi thất bại, chính nhánh con nhận giá trị lỗi và có thể báo lại; cha chỉ biết nếu có cơ chế thông báo hoặc chờ kết thúc. `execve` thay chương trình trong tiến trình gọi, nên không tăng số tiến trình. Trên đường thành công, lệnh sau `execve` trong chương trình cũ không chạy tiếp.

Đáp án P02.C01 — Syscall, đường dẫn và hệ thống tệp

P02.C01.L02.Q01. Descriptor của tiến trình tham chiếu open file description đang mở, không tra cứu lại tên đường dẫn cho mỗi lần ghi. Đổi tên hoặc tạo tệp mới cùng tên không chuyển descriptor cũ. Cách xử lý là yêu cầu dịch vụ mở lại log bằng cơ chế được nó hỗ trợ, hoặc khởi động lại có kiểm soát sau khi bảo đảm log đã được thu thập và khả năng phục vụ không bị gián đoạn ngoài mức chấp nhận. Không xóa cưỡng bức dữ liệu trước khi xác định descriptor và nhu cầu lưu giữ.

P02.C01.L03.Q01. Chưa đủ. Lời gọi ghi thành công không tự bảo đảm dữ liệu và metadata đã tới thiết bị bền vững. Khi tên tệp được tạo hoặc đổi, cần kiểm tra hợp đồng đồng bộ tệp và thư mục chứa mục tên đó, cùng hành vi hệ thống tệp thực tế. Không nên biến `fsync` trên tệp thành lời hứa tuyệt đối cho mọi bước của thao tác thay tên.

Đáp án P02.C02 — Danh tính và quyền Linux

P02.C02.L02.Q01. Nhóm `atlas` có thể đọc tệp theo bit nhóm của `0640`, nhưng không có quyền tìm kiếm trên `/srv/atlas` khi thư mục là `0700 root:root`; phân giải đường dẫn có thể dừng ở thư mục đó với `EACCES`. Kiểm tra quyền từng thành phần và danh tính thực của dịch vụ; nếu hợp đồng cho phép, cấp quyền tìm kiếm trên thư mục cho đúng nhóm hoặc chuyển cấu hình sang cây thích hợp. Không đổi tệp thành `0666` hay dùng `chmod -R 777`: hai cách đó vừa không sửa quyền đi qua thư mục, vừa mở rộng truy cập.

P02.C02.L03.Q01. Điểm chặn đầu là quyền tìm kiếm ở /srv/atlas. Kiểm dưới đúng danh tính và ngữ cảnh dịch vụ sau một thay đổi quyền hẹp được duyệt. chmod 777 cho tệp cuối không cấp quyền đi qua thư mục

cha.

Đáp án P02.C03 — Tài nguyên và chẩn đoán hiệu năng Linux

P02.C03.L02.Q01. Không gian địa chỉ ảo đo vùng địa chỉ mà tiến trình có thể dùng, không phải số trang vật lý đang cư trú. Cần phân biệt kích thước ánh xạ ảo và bộ nhớ cư trú; nếu phân tích áp lực chung, còn phải xét trang dùng chung, cache và cách cgroup tính bộ nhớ.

P02.C03.L03.Q01. Writeback chậm là một giả thuyết. Cần ghép thời điểm request và thời gian fsync/I/O cùng ranh giới process, cgroup, thiết bị. Chưa thể quy toàn bộ 80 MiB dirty hay độ trễ cho request này vì các counter có thể gồm tác vụ khác.

P02.C03.L04.Q01. `cpu.stat usage_usec` chia cho 60.000.000 µs cho 0,30 và 0,32 lõi trung bình; `nr_throttled` không tăng nên chưa có dấu vết quota CPU tại cgroup đã đo. `memory.current` cuối kỳ tăng 5 MiB, còn `memory.events high` và `oom` không tăng; chưa thấy chạm hai ngưỡng này, nhưng chưa loại trừ thu hồi ở máy hoặc cgroup tổ tiên. `io.pressure some total` chia cho 60.000.000 µs cho 2% và 30% thời gian có ít nhất một tác vụ nghẽn I/O. Cùng với p95 tăng, đó là lý do ưu tiên điều tra I/O. PSI không gắn thời gian chờ với từng yêu cầu hay thiết bị; bước tiếp theo là đo thời gian thao tác đọc theo yêu cầu, đối chiếu `io.stat` và tình trạng thiết bị trong cùng cửa sổ. Chưa thể quy toàn bộ độ trễ cho I/O.

Đáp án P02.C04 — systemd và vòng đời dịch vụ

P02.C04.L02.Q01. `Type=exec` xác nhận systemd đã thiết lập và thực thi được binary chính, theo hợp đồng của loại unit này. Nó không chứng minh ứng dụng đọc cấu hình thành công, mở cổng hay trả HTTP đúng. Xem nhật ký và trạng thái tiến trình chính, rồi gọi health endpoint hoặc phép thử chức năng từ đúng đường đi của người dùng. Một PID `active` không đủ thay cho phép thử khả năng phục vụ.

P02.C04.L03.Q01. Với binary thiếu, bước chuẩn bị/thực thi của systemd thất bại trước khi mã ứng dụng chạy. Với `openat` trả `EACCES`, binary đã chạy; nhân từ chối một thao tác tệp của tiến trình, ứng dụng phải ghi lỗi và chọn cách thoát. Nếu tiến trình thoát lỗi, `Restart=on-failure` có thể thử lại, rồi bị giới hạn tần suất khởi động. Tăng `RestartSec` chỉ đổi nhịp thử; đường dẫn hoặc quyền sai vẫn sai. Cần sửa đúng artifact, danh tính và quyền sau khi kiểm tra nguyên nhân.

Đáp án P02.C05 — Vận hành host và phục hồi có kiểm chứng

P02.C05.L01.Q01. Trước thay đổi cần ít nhất revision đang chạy, hash/diff cấu hình và trạng thái dữ liệu hoặc bản sao có thể restore. Sau thay đổi cần request qua đường thật thành công và dữ liệu đọc đúng; active riêng lẻ chưa đủ.

P02.C05.L02.Q01. Hash chỉ kiểm byte archive không đổi sau tạo; nó không chứng minh snapshot nhất quán hay đủ WAL. Phải theo cơ chế backup hỗ trợ của PostgreSQL và restore thử ở đường dẫn/môi trường cô lập, không đè server đang chạy.

P02.C05.L03.Q01. Chưa. Cần đối chiếu nội dung bản ghi cũ, kiểm ghi/đọc mới trong phạm vi an toàn, xác nhận revision phục vụ và quyền còn hẹp. Rollback config không tự phục hồi dữ liệu.

Đáp án P03.C01 — Địa chỉ, subnet và đường đi gói IP

P03.C01.L02.Q01. Route `/24` cụ thể hơn route mặc định `/0`, nên được ưu tiên trong mô hình đã nêu. Trên Linux còn phải xét policy routing, bảng route, địa chỉ nguồn và namespace. `ip route get` trong đúng ngữ cảnh cho biết quyết định hiện tại, không bảo đảm gói thực sự qua được gateway.

P03.C01.L03.Q01. Client chọn gateway `10.20.1.1` làm next hop vì đích nằm ngoài prefix trực tiếp. Nó cần MAC của gateway trên liên kết cục bộ để đóng frame đầu tiên. MAC của server ở mạng xa không được ARP trực tiếp qua các router; mỗi chặng có frame và phép phân giải láng giềng riêng.

Đáp án P03.C02 — Transport, TCP, UDP và NAT

P03.C02.L01.Q01. SYN chiếm một vị trí nên sau `seq=1000`, byte hoặc số thứ tự tiếp theo của client là 1001; server báo `ack=1001`. Bốn byte ở 1001–1004 làm số tiếp theo là 1005, nên ACK cộng dồn là 1005. Nếu ACK mất rồi cùng đoạn dữ liệu được gửi lại với `seq=1001`, server vẫn báo `ack=1005` vì không xuất hiện byte mới liên tục; TCP nhận diện đoạn lặp và không giao bốn byte lần hai cho ứng dụng. Không thể suy từ một capture đơn lẻ rằng mất ACK là nguyên nhân duy nhất của retransmission.

P03.C02.L02.Q01. Kiểm tra route trả từ server tới địa chỉ nguồn mà server thực sự thấy, rồi kiểm tra ánh xạ NAT và lọc trên gateway hoặc các chặng về. Có thể đối chiếu capture hai phía của NAT theo thời gian và tuple trước/sau dịch. SYN-ACK đã rời server nhưng chưa tới client nghĩa là bắt tay TCP chưa hoàn tất ở client; HTTP chưa bắt đầu nên chưa thể quy nguyên nhân cho ứng dụng HTTP.

P03.C02.L02.Q02. Giả thuyết mạnh nhất là server chỉ bind trên loopback, nên không có listener cho `198.51.100.20:443`. Vẫn kiểm tra quy tắc lọc có thể chủ động gửi RST. Sau khi sửa cấu hình bind trong môi trường được phép, xác nhận `ss -ltn` có đúng địa chỉ/cổng, thấy SYN, SYN-ACK, ACK và thử yêu cầu ứng dụng. `systemctl active` chỉ phản ánh trạng thái unit/process, không chứng minh socket nhận từ xa hay ứng dụng phục vụ đúng.

P03.C02.L03.Q01. Server trả tới `203.0.113.5:40001`, không gửi thẳng tới `10.20.1.10:51514`. Gateway cần ánh xạ còn hiệu lực và đường đưa gói về mạng trong để dịch đích trở lại. Ánh xạ NAT là trạng thái ở thiết bị dịch; `SYN-SENT` hay `ESTABLISHED` là trạng thái TCP của hai endpoint. Có mục NAT không chứng minh server đã chấp nhận kết nối.

Đáp án P03.C03 — Tên, TLS và ngữ nghĩa HTTP

P03.C03.L01.Q01. Ghi tên và loại bản ghi mà ứng dụng hỏi, resolver nó dùng, thời điểm cùng TTL còn lại, rồi đối chiếu địa chỉ mà chính tiến trình đã kết nối. Có thể kiểm tra dữ liệu authoritative và cache của resolver để định vị chỗ sai. Một máy khác có cache, cấu hình resolver, đường đi hoặc thời điểm khác; kết quả của nó không đại diện cho client đang lỗi.

P03.C03.L02.Q01. Đối chiếu tên URL, SNI mà client gửi, chứng chỉ proxy chọn, tên trong chứng chỉ, chuỗi tin cậy, thời hạn và đồng hồ của client. `Host` là thông tin HTTP sau TLS; đổi nó không làm chứng chỉ đã chọn hợp lệ cho tên yêu cầu. Tắt xác minh biến lỗi rõ ràng thành một kết nối không được xác thực, không sửa binding hoặc chứng chỉ sai.

Đáp án P03.C04 — Reverse proxy và điều tra request hai chặng

P03.C04.L03.Q01. Timeout trước TLS sau khi DNS có địa chỉ mới chỉ cho biết chưa có kênh TCP hoàn tất; xem SYN/SYN-ACK ở client và proxy. Certificate error chứng minh đã tới giai đoạn xác thực TLS, chưa có HTTP request hợp lệ; xem SNI và chứng chỉ. `502` chứng minh một HTTP response mang mã ấy đã tới client, cần log proxy xác định ai tạo và phản hồi upstream nào bị coi không hợp lệ. `504` cũng là HTTP response; đối chiếu thời gian proxy, upstream và nguồn tạo status. Không suy upstream chết từ `502`: ứng dụng có thể còn chạy nhưng trả header sai, đóng kết nối, hoặc chính upstream trả mã đó.

P03.C04.L03.Q02. Lần đầu có thể đã tạo đơn hàng dù proxy hết chờ, nên retry sang upstream khác có nguy cơ tạo bản ghi thứ hai. Chỉ retry tự động khi thao tác idempotent theo hợp đồng hoặc có khóa chống lặp và khả năng truy lại kết quả lần đầu; cần giới hạn số lần, thời gian và quan sát cả hai lần. Tăng timeout đổi ngưỡng proxy chờ, có thể giữ kết nối lâu hơn và che triệu chứng, nhưng không làm truy vấn dữ liệu hoặc phụ thuộc upstream nhanh hơn.

Đáp án P04.C01 — Git object, index và ref

P04.C01.L02.Q01. `HEAD` trỏ tới `B` với giá trị `1000`; index có snapshot `2000` tại lúc `git add`; working tree đã đổi thành `3000`. Commit ngay lấy index nên ghi `2000`, còn `3000` vẫn là thay đổi chưa stage. `git diff --cached` cho thấy `1000 → 2000`, `git diff` cho thấy `2000 → 3000`; stage lại rồi xem diff nếu định commit `3000`.

Đáp án P04.C02 — Tích hợp và sửa lịch sử

P04.C02.L01.Q01. `D` và `C` đều là con của `B`, không commit nào là tổ tiên của commit kia; dịch `main` thẳng tới `C` sẽ bỏ `D` khỏi lịch sử đi theo `main`. Merge sau giải conflict tạo `M` có cha thứ nhất `D`, cha thứ hai `C`; `main` trỏ tới `M`, feature vẫn trỏ tới `C`. Khi dừng vì conflict, `HEAD` còn ở `D`; index giữ stage 1 từ `B`, stage 2 từ `D` và stage 3 từ `C` cho đường dẫn xung đột, thay cho một bản đã giải ở stage thường.

P04.C02.L02.Q01. Rebase tạo `C'` có cha `D`, nên nhánh feature đi theo `A → B → D → C'`; `C'` có định danh mới vì parent của commit đã đổi. Nếu `C` đã được người khác dùng, dịch ref tới `C'` tạo lịch sử khác với lịch sử họ đang có và buộc phối hợp khi cập nhật nhánh chung. Merge giữ cả `C` lẫn `D` và ghi quyết định tích hợp ở `M`. Cả hai cách đều có thể cho snapshot sai; cần test timeout và retry trên kết quả cuối.

P04.C02.L03.Q01. `git revert -m 1 M` tạo một commit mới đảo tác động merge so với cha thứ nhất `D`; phải chọn mainline vì `M` có hai cha và "đảo merge" phụ thuộc cha nào được giữ làm nền. Xem diff vì nó không đơn giản đưa toàn kho về `B`, và merge lại nhánh cũ có thể không tự phục hồi thay đổi đã đảo. `git reset D` dịch ref `main` về `D`; nếu đã push `M`, lịch sử branch chia sẻ bị viết lại và bản sao của người khác có thể lệch. Cần review diff, xác nhận branch/ref và cha, chạy kiểm thử liên quan trên snapshot sau sửa, rồi kiểm tra trạng thái trước push.

Đáp án P05.C01 — Bash và ngữ nghĩa lỗi của shell

P05.C01.L01.Q01. `tar -C $src` nhận hai word `/tmp/du` và `lieu` sau word splitting trong trường hợp giá trị đã cho; `tar -C "$src"` nhận một word `/tmp/du lieu`. Nếu biến chứa ký tự glob như *, lần mở rộng tên tệp có thể biến một word thành nhiều đường dẫn. `set -u` chỉ phát hiện một số lần dùng biến chưa đặt, không chặn tách word hay globbing. Lưu danh sách bằng mảng và truyền `"${files[@]}"` để bảo toàn ranh giới từng phần tử.

P05.C01.L02.Q01. Không có `pipefail`, status pipeline là 0 từ `tee`, nên `set -e` không thoát; `printf` in `BACKUP_OK` và script kết thúc 0. Có `pipefail`, pipeline trả 2 trong giả định này, `set -e` ở vị trí top-level khiến script thoát trước `printf`. Không nên chỉ dựa vào tương tác `set -e` vì nó có ngoại lệ ngữ cảnh; kiểm tra status tường minh rõ hơn. `tar` trả 0 chứng minh thao tác của `tar` đạt hợp đồng trả status của nó trong lần chạy ấy, không chứng minh restore hoàn chỉnh, nguồn nhất quán hoặc bản ghi đã bền vững qua mất điện.

P05.C01.L03.Q01. Hàm được gọi làm điều kiện `if`, nên `errexit` bị bỏ qua trong thân hàm ở ngữ cảnh này: `false` trả 1 nhưng `printf` vẫn chạy và trả 0. Hàm kết thúc 0, do đó in `done` rồi `ok`. Có thể viết `f() { if bước_đầu; then :; else return $?; fi; printf 'done\n'; }`, trong đó `bước_đầu` là lệnh thật cần kiểm tra. Cách khác là `bước_đầu || return $?` trong hàm, với chủ ý lan truyền lỗi rõ ràng.

Đáp án P05.C02 — Python và Go điều phối tiến trình

P05.C02.L02.Q01. `precheck` trả 0; `apply_fail` ghi `partial` rồi trả 7; `verify` đọc trạng thái ấy và trả 8. `Popen` khởi chạy helper, `communicate` chờ và thu dữ liệu; sau khi con kết thúc, `returncode` mang status cần xét. Nhánh `bad` ghi hai lỗi ra stderr nhưng bỏ qua status trong quyết định cuối, vẫn in `DEPLOY_OK` và thoát 0. Nhánh `checked` dùng status 7 để dừng với mã 1, nên không chạy `verify`; file vẫn chứa `partial`, vì dừng điều phối không đảo tác dụng phụ đã có.

P05.C02.L02.Q02. Khi truyền `prod; echo LEAK` là một phần tử argv với API không gọi shell, dấu chấm phẩy là dữ liệu trong đối số; với `sh -c` và chuỗi ghép không được bảo vệ, nó có thể trở thành dấu tách lệnh. Dù dùng argv, utility vẫn có thể diễn giải giá trị bắt đầu bằng `-` như tùy chọn hoặc chấp nhận tên ngoài schema; cần xác thực và dùng ranh giới tùy chọn theo hợp đồng utility. Secret có thể lộ qua argv trong danh sách tiến trình, environment được kế thừa, hoặc log in nguyên command/exception.

P05.C02.L03.Q01. Helper phát `READY` sau khởi tạo và đợi `go`; cha chỉ bắt đầu bộ đếm 0,5 giây sau khi gửi `go`. `apply_slow` ghi `partial` trước khi ngủ hai giây; khi bộ đếm hết, cha hủy context, đợi `Wait` và trả nhãn `TIMEOUT`. Lỗi `Wait` sau khi kill không phải exit status 7 của nhánh `apply_fail`; `ctx.Err()` khi chủ động hủy là `context.Canceled`, nên không dùng nó một mình để suy ra nguyên nhân. Kill không hoàn nguyên file. `CommandContext` mặc định tác động tới tiến trình con trực tiếp; không thể kết luận tiến trình cháu hay yêu cầu từ xa đã dừng nếu không có cơ chế và quan sát riêng.

Đáp án P05.C03 — Workflow an toàn và chạy lại

P05.C03.L02.Q01. Timeout có thể đến sau khi hệ thống đích đã ghi nhưng trước phản hồi, nên retry vô hạn có thể tạo nhiều tác dụng, kéo dài quá thời gian cho phép và chồng lên lượt khác. Xóa `partial` mù quáng làm mất dấu vết và có thể xóa phần việc hợp lệ. Chỉ thử lại với điều kiện lỗi được phân loại là tạm thời, thao tác idempotent hoặc có khóa chống lặp, số lần/tổng thời gian hữu hạn và trạng thái hiện tại đã được đối chiếu; sau đó kiểm tra hậu điều kiện từ góc nhìn người dùng và log tương ứng. Bash vẫn hợp lý cho vài lệnh tuyến tính với ít trạng thái, đầu vào nhỏ, status được kiểm tra tường minh và môi trường chạy rõ ràng; không cần đổi ngôn ngữ chỉ vì script có một lệnh con.

P05.C03.L03.Q01. Lần thứ hai vẫn tạo và kiểm tra file tạm; tới `ln`, tên `backup.tar` đã có nên GNU `ln` mặc định trả khác 0 và script thoát 5. `EXIT` trap xóa file tạm của lần ấy; hard link của bản đầu còn nguyên. `tar -tf` chỉ kiểm tra khả năng đọc cấu trúc archive; cần thử khôi phục và so dữ liệu, kiểm tra snapshot nhất quán và bền vững theo hệ thống tệp. `SIGKILL` không chạy trap, có thể để lại file tạm; nguồn bị sửa đồng thời có thể cho bản sao không nhất quán dù cleanup đúng. Việc phát hiện và xử lý hai rủi ro đó đòi hỏi cơ chế khác.

Đáp án P06.C01 — Build và danh tính artifact

P06.C01.L02.Q01. Build lại tạo một tập byte mới; kết quả test staging chỉ gắn với digest cũ, còn tag có thể bị trỏ lại. Ghi commit, đầu vào và phiên bản build, digest artifact được test, test-run, approval, môi trường và digest thực sự được triển khai; promotion phải dùng lại digest đã test hoặc chạy lại cổng phù hợp cho artifact mới. Dependency không khóa, timestamp, toolchain hoặc cờ build khác có thể đổi byte dù commit giống nhau. Muốn kết luận tái lập từng byte phải so digest từ các lượt build độc lập có đầu vào kiểm soát.

Đáp án P06.C02 — Test gates và CI/CD

P06.C02.L02.Q01. Unit test không phủ đơn giá trị 1234; readiness chỉ chứng minh điều kiện của probe, không kiểm tra phép ghi tiền. Gửi một đơn canary có đầu vào biết trước, đối chiếu bản ghi và sổ độc lập, gắn kết quả với `H84`; 123400 thay vì 1234 là bằng chứng lỗi cụ thể. Bốn mẫu cho phép dừng rollout nhưng không ước lượng chính xác tỷ lệ lỗi toàn hệ thống, nhất là khi luồng canary hoặc phân phối đơn hàng không đại diện.

Đáp án P06.C03 — Rollout và quyết định dừng

P06.C03.L01.Q01. Blue-green đầy đủ cần giữ hai tập có năng lực phục vụ, nên thiếu tài nguyên nếu chỉ thêm được một Pod. Rolling có thể tăng một Pod rồi giảm dần bản cũ; canary có thể dùng chính Pod đó ở tỷ lệ lưu lượng nhỏ nếu bộ định tuyến hỗ trợ, sau đó mở rộng từng bước. Trong cả hai, 1.8.3 và 1.8.4 cùng truy cập schema nên việc thêm cột nullable phải giữ bản cũ đọc/ghi được; thông điệp, job và cache cũng cần xét. Sai một đơn tiền đã đối chiếu là lý do dừng ngay, dù tỷ lệ lỗi HTTP tổng thể vẫn đẹp.

P06.C03.L03.Q01. Approval cho phép từng lượt nhưng không đảm bảo thứ tự và không xác nhận phiên bản đích tại lúc ghi. Cần serialize release theo môi trường và dùng kiểm tra điều kiện: chỉ thay nếu digest hiện tại đúng phiên bản mà lượt này dự kiến. Nếu `R2` đã thay trước, `R1` phải dừng hoặc được đánh giá lại, không ghi đè im lặng. Khóa CI chỉ quản lý các lượt đi qua CI; công cụ hoặc người triển khai trực tiếp vẫn có thể thay đích, nên hệ thống đích cũng phải kiểm tra danh tính bản hiện tại và quyền thao tác.

Đáp án P06.C04 — Rollback và tương thích dữ liệu

P06.C04.L03.Q01. `ROLLBACK` SQL chỉ hủy giao dịch còn mở, trong khi migration và các đơn đã commit. Chia toàn bảng cho 100 phá các đơn đúng do `H83` ghi xen kẽ và không xử lý thanh toán bên ngoài. Hậu điều kiện ứng dụng: chỉ `H83` nhận ghi, xác nhận digest ở đường phục vụ và đơn mới đúng. Hậu điều kiện dữ liệu: tập đơn do `H84` ảnh hưởng được khoanh bằng chứng cứ, từng giá trị đối soát và sửa có audit; cột mới còn tương thích. Hậu điều kiện bên ngoài: mỗi thanh toán liên quan có trạng thái xác minh bằng ID, xử lý bù nếu hợp đồng cho phép, các trường hợp chưa biết được ghi riêng.

Đáp án P07.C01 — Namespace, cgroup v2 và runtime boundary

P07.C01.L02.Q01. `NSpid` cho thấy cùng tiến trình có PID 3142 ở góc nhìn tổ tiên và PID 1 ở góc nhìn bên trong; danh tính network namespace khác host; listener chỉ bind loopback của ngăn mạng đang quan sát. Chưa thể kết luận client có hay không có đường publish/forward, cũng chưa biết request đã tới ứng dụng. Kiểm tra cấu hình chuyển tiếp và socket/route ở đúng hai đầu, rồi thử từ vị trí client: nếu TCP không thiết lập, điều tra đường mạng và listener; nếu có HTTP 500, kết nối TCP và tầng HTTP đã hoạt động đủ để có phản hồi, tiếp tục xem proxy/upstream và log ứng dụng. HTTP 500 riêng nó chưa chỉ rõ tiến trình nào sinh lỗi.

P07.C01.L03.Q01. Hiệu lần lượt là 4.700.000 microsecond CPU, 100 chu kỳ, 80 chu kỳ throttled và 2.000.000 microsecond throttling; mức dùng trung bình là 4,7/10 = 0,47 lõi. `50000/100000 = 0,5` lõi là trần trung bình theo chu kỳ. Throttling trong 80 chu kỳ cho thấy quota thực sự chặn công việc ở cửa sổ đo, nhưng không đo thời gian chờ của từng request và không loại trừ I/O, khóa hay upstream. Cần đối chiếu cửa sổ metric với trace/latency từng chặng trước khi quy nguyên nhân.

P07.C01.L03.Q02. `max` tăng 2 nghĩa là hai lần mức dùng chạm giới hạn; `oom` tăng 1 ghi nhận tình huống cấp phát sắp thất bại có xét OOM; `oom_kill` tăng 1 xác nhận một tiến trình trong cgroup bị OOM killer giết. Muốn xác định nạn nhân cần ghép timestamp, PID/lượt chạy, log kernel và runtime; một số đo `memory.current` sau sự kiện có thể đã giảm. Lần khởi động sau thoát 2 với log `config missing` và counter OOM của cgroup mới không đổi thuộc nhánh lỗi cấu hình ứng dụng; sự kiện OOM cũ không tự giải thích lần thoát này.

Đáp án P07.C02 — OCI image, build và registry

P07.C02.L03.Q01. Build tạo artifact, push đưa manifest/blob lên registry, pull đưa nội dung về máy chạy; các bước ấy không tự thay tiến trình. Ghi commit nguồn, digest manifest đã kiểm thử, digest image được dùng tạo container mới, container ID, thời điểm chuyển lưu lượng và kết quả request từ client. Tag `orders:2.2` có thể trỏ lại nên tên tag riêng lẻ không đủ. Nếu máy chủ vẫn phục vụ container từ digest cũ, bản mới chưa được chứng minh là đang nhận request dù registry đã có nó.

Đáp án P07.C03 — Container lifecycle và dữ liệu

P07.C03.L01.Q01. Digest nhận diện byte image đã kiểm thử, không chứng minh cấu hình runtime, dependency ngoài, mạng hay dữ liệu hiện thời đúng. `running` chỉ xác nhận chương trình đã được thực thi và chưa dừng; readiness đỏ cần phép thử ứng dụng cụ thể, ví dụ bind, phụ thuộc và request đại diện. Image cung cấp root filesystem và tham số mặc định; namespace đặt góc nhìn tài nguyên của tiến trình; cgroup giới hạn/phân phối tài nguyên. Bất kỳ tầng nào cũng có thể góp phần làm request thất bại, nên cần quan sát đúng tầng rồi mới kết luận.

P07.C03.L03.Q01. Lượt A ghi trong writable layer riêng của container A. B được tạo mới từ image nên có lớp ghi mới, không tự nhận dữ liệu từ A. Nếu A chỉ dừng và còn container, lớp ghi có thể còn đọc được để điều tra; nếu A bị xóa, không được hứa khôi phục từ lớp ghi ấy. Tránh xóa container cũ, chạy `down -v`, prune hoặc ghi đè đường dẫn nguồn trước khi xác định dữ liệu và có bản sao đã thử restore.

Đáp án P07.C04 — Compose và ứng dụng nhiều service

P07.C04.L02.Q01. Mạng mặc định cho các service cùng project đường kết nối và tên để tìm nhau; `depends_on` mặc định sắp thứ tự khởi động, không đảm bảo API hay database đã ready. Ngay cả healthcheck chỉ bao phủ phép đo được khai báo. Compose file cũng không tự backup volume hoặc tự lập lịch bản sao sang host khác khi host chết. Gửi request đại diện từ đúng vị trí client và kiểm tra dữ liệu trả về; tạo backup nhất quán của dữ liệu thật rồi thử restore trong môi trường riêng, so ID và checksum hoặc hậu điều kiện nghiệp vụ.

P07.C04.L03.Q01. D vẫn gắn `orders-case_orders_data` tại `/var/lib/orders`, nên đổi byte image không thay byte của named volume. Project `orders-test` tạo tài nguyên tên khác theo project; ứng dụng nhìn cùng đường dẫn nhưng có thể đọc volume mới. Bind mount rỗng đặt tại `/var/lib/orders` che nội dung ở điểm mount; volume cũ có thể vẫn tồn tại, chỉ không còn là nguồn dữ liệu hiện được gắn. Xác minh loại mount, `Source`, `Destination`, tên volume và đọc dữ liệu cũ theo cách không ghi trước khi sửa cấu

hình.

Đáp án P08.C01 — AWS identity và IAM evaluation

P08.C01.L02.Q01. Ghi principal phiên role, action `s3:GetObject`, ARN bucket/key, Region, điều kiện request và request ID. So identity policy, bucket policy, endpoint policy cùng session policy, boundary, SCP/RCP nếu áp dụng; nếu mã hóa bằng KMS key của khách hàng thì xét cả quyền và key policy thích hợp. Statement `Deny` trong identity policy của ví dụ khớp cùng object, nên thắng `Allow`. HTTP 403 riêng nó chưa xác định policy nào gây từ chối trong một sự cố khác, cũng không chứng minh request dùng đúng endpoint. Sau thay đổi quyền được review, phải đọc đúng key và kiểm tra nội dung, không chỉ thấy hết lỗi.

Đáp án P08.C02 — VPC, subnet và đường đi AWS

P08.C02.L03.Q01. Xác nhận instance dùng đúng DNS resolver và đúng S3 regional endpoint; so địa chỉ trả về với Region cấu hình. Đọc route table thực sự gắn với subnet và liên kết gateway endpoint S3, không suy từ việc endpoint tồn tại trong VPC. Kiểm tra SG outbound TCP 443 và NACL ở cả hai chiều, đặc biệt inbound ephemeral khi instance là client; nếu có flow logs ở đúng ENI/cửa sổ, dùng chúng làm dấu vết phụ. DNS đã trả địa chỉ chỉ loại bớt lỗi phân giải ở lượt ấy; timeout xảy ra trước khi có phản hồi dịch vụ, nên chưa có bằng chứng IAM đã được xét. Trong dữ kiện đề, thiếu tuyến endpoint và không có NAT giải thích timeout.

Đáp án P08.C03 — EC2, load balancer và miền lỗi

P08.C03.L02.Q01. `/health` xanh chứng minh ALB có target đáp ứng phép thử ấy, không chứng minh bước S3 của báo cáo. EC2–S3 có thể qua gateway endpoint của route table đúng, còn EC2–STS cần đường riêng như NAT hoặc interface endpoint phù hợp; timeout STS chưa chứng minh role thiếu quyền. Đối chiếu DNS, route, SG/NACL và kết quả TCP/TLS từng đích; với phản hồi `AccessDenied` mới chuyển sang kiểm tra principal và policy. Hậu điều kiện là request báo cáo từ client thực nhận đúng object/phiên bản, không có lỗi ẩn trong log và quyền chỉ mở cho action/resource cần thiết; ghi Region, request ID, thời điểm và kiểm chứng dữ liệu mà không lộ secret.

Đáp án P08.C04 — Block, file và object storage

P08.C04.L03.Q01. Instance ở AZ hỏng không phục vụ; EBS volume của AZ ấy không thể attach trực tiếp cho instance AZ khác, cần snapshot đã có hoặc cơ chế sao chép/phục hồi khác để dựng volume ở AZ hoạt động, cộng với dữ liệu nhất quán và ứng dụng sẵn sàng. EFS Regional có dữ liệu phân tán nhiều AZ nhưng client ở AZ khác cần mount target, đường NFS, SG/quyền và ứng dụng thay thế hoạt động. S3 Standard được thiết kế qua nhiều AZ, nhưng object xóa nhầm cần versioning/bản sao lịch sử hoặc backup đã bảo vệ và thử restore; redundancy của trạng thái hiện tại không tự giữ một phiên bản đúng trong quá khứ. Khả năng phục hồi còn phụ thuộc RPO/RTO thực đo.

Đáp án P09.C01 — HCL, provider và dependency DAG

P09.C01.L03.Q01. `service` tham chiếu `terraform_data.schema.output` nên Terraform suy ra cạnh `schema` → `service`; `audit` dùng `depends_on` để khai báo cạnh `service` → `audit`. Khi `schema_revision` đổi, `schema.input` đổi; `service.triggers_replace` nhận giá trị mới nên `service` được thay sau khi giá trị xác định. `audit.input = "ready"` chỉ là dữ liệu state, không thực hiện request, nên không chứng minh ứng dụng hay phụ thuộc của nó sẵn sàng. Plan thực tế còn tùy giá trị chưa biết cho đến apply.

Đáp án P09.C02 — Plan, state và backend

P09.C02.L03.Q01. Terraform không có giao dịch chung trên API và backend. Network/volume có thể đã tồn tại; timeout của API instance không đủ kết luận instance không được tạo; lỗi ghi state còn khiến ánh xạ lưu trữ khác thực tế. Giữ log và state phục hồi, tạm ngừng lượt triển khai, xác định khóa và phiên state cuối cùng. Đọc API theo ID/tag và thời điểm, đối chiếu từng object với state, tránh tạo thêm object trùng. Sửa ánh xạ bằng quy trình phục hồi/import đã review nếu cần, rồi lập plan mới và kiểm tra hậu điều kiện: đúng một chủ quản lý cho mỗi object, đúng dependency, volume/instance nhận diện chính xác, dữ liệu còn nguyên và request đại diện hoạt động. Chỉ tiếp tục khi nguyên nhân lỗi backend/API đã được xử lý.

Đáp án P09.C04 — Thay đổi hạ tầng có dữ liệu

P09.C04.L01.Q01. HCL vẫn muốn user data A, state ghi liên kết `aws_instance.orders` với `i-demo-01` và giá trị đã lưu, còn instance thực có B nếu API xác nhận. Chỉ khi provider đọc được khác biệt và thuộc tính `user_data_replace_on_change = true` áp dụng thì thay user data có thể dẫn tới replacement; mặc định provider chỉ stop/start. Nếu instance bị terminate trong khi root EBS thật có `delete_on_termination = true` và 41 đơn chỉ ở đó, dữ liệu có thể mất. Trước apply phải xác nhận đúng account/Region/ID và plan có refresh, root block mapping/chính sách xóa và bản sao nhất quán đã thử restore; thêm lịch sử ai sửa B. Nếu B là cấu hình mong muốn, cập nhật HCL rồi plan lại. Nếu B sai, bảo vệ/chuyển dữ liệu và lập kế hoạch phục hồi được kiểm tra trước khi thay máy; không che drift bằng `ignore_changes` chỉ để plan xanh.

P09.C04.L02.Q01. Locking của backend, nếu hỗ trợ và cùng dùng đúng state, ngăn hai lượt Terraform cùng sửa state; nó không khóa console hoặc tất cả API AWS, nên thay security group ngoài Terraform vẫn là drift. Lượt plan kế tiếp có refresh và quyền đọc phù hợp mới có cơ hội thấy đổi khác. `sensitive = true` che hiển thị nhưng state/plan vẫn có thể chứa giá trị; cần giới hạn quyền, mã hóa và tránh nhúng credential. Versioning giữ bản cũ để khôi phục nhưng không ngăn đọc trái phép hay chứng minh restore chạy được; phải bảo vệ bản sao, kiểm tra quyền và thử phục hồi.

Đáp án P10.C02 — Hội tụ, handler và thất bại từng host

P10.C02.L01.Q01. Lượt đầu trên filesystem POSIX có thể báo changed cho việc tạo sandbox, thư mục dữ liệu, config và handler marker; `stat`/`assert` chỉ quan sát nên ok nếu hậu điều kiện đúng. Lượt thứ hai cùng biến, khi không có drift, task template là ok, handler không được notify và không chạy. `changed=0` chỉ nói các module không báo sửa ở phạm vi chúng quản lý; không thử kết nối HTTP, dữ liệu, phụ thuộc hay sức khỏe tiến trình Atlas. Trên ổ gắn không giữ mode POSIX, thí nghiệm quyền có thể thất bại dù module đặt mode và không được tính là kiểm chứng hội tụ.

P10.C02.L02.Q01. `file` sửa mode thư mục từ `0700` về `0750`; template không đổi nên handler không được notify, marker không bị chạm. Lệnh `command: chmod 0750 ...` không tự so trạng thái trước/sau theo hợp đồng cấu hình; nếu chạy và báo changed ở mọi lượt, handler bị gọi không cần thiết dù mode đã đúng. Cần dùng module có ý thức trạng thái hoặc điều kiện cùng kiểm chứng rõ ràng, và xác nhận service user thực sự cần quyền nào.

P10.C02.L03.Q01. Syntax/check không chứng minh kết nối, quyền hoặc API thật; module có thể không hỗ trợ check hay task phụ thuộc kết quả trước cho dự báo khác; diff chỉ hiển thị phần module hỗ trợ và còn có nguy cơ lộ secret. Khi apply lỗi sau một restart, ghi host nào đã đổi file/quyền, handler nào đã chạy, dịch vụ nào còn phục vụ và config đang có. Khoanh host lỗi, kiểm tra log và request, phục hồi từ bản cấu hình đúng nếu cần; trước lượt tiếp phải chứng minh file và quyền đúng, service healthy qua request đại diện, inventory/biến đúng, và kế hoạch rollout còn đủ năng lực phục vụ. Lỗi trên host này không tự rollback host trước.

Đáp án P10.C03 — Vai trò, secrets và triển khai theo đợt

P10.C03.L02.Q01. Task `chmod 0700` có thể bỏ quyền đi qua của group service; template `root:root 0600` có thể khiến user `atlas` không đọc được config. Task command báo changed rồi notify restart; khi process cũ còn chạy nó có thể chưa cần mở lại file, nhưng sau restart sẽ gặp lỗi quyền. Nếu hai máy cùng restart, không còn target phục vụ. Thu mode/owner/group và UID/GID tại từng host, diff config, recap task/handler, log systemd và thử request; không suy từ mỗi chữ changed. Sửa quyền và nội dung có kiểm tra trước, chỉ restart máy thử khi config thật sự đổi, xác nhận request rồi lần lượt tới máy khác; nếu host lỗi, dừng rollout và ghi trạng thái từng máy.

Đáp án P11.C01 — API, etcd và reconciliation

P11.C01.L02.Q01. Đọc Deployment ở đúng namespace, phân biệt `spec.replicas=2` với status và generation mà controller đã quan sát. Tìm ReplicaSet thuộc Deployment qua owner reference; nếu chưa có, xem điều kiện/events Deployment và khả năng controller/API bị chặn. Nếu ReplicaSet có nhưng không có Pod, xem selector, quota, lỗi tạo Pod và events của ReplicaSet. Với Pod đã có, `spec.nodeName` rỗng cùng `FailedScheduling` đưa chẩn đoán sang scheduler, requests và ràng buộc node; khi đã có node, đọc container waiting reason/events để tách image pull hoặc lỗi runtime. Không bước nào riêng lẻ chứng minh Service có backend hay ứng dụng đáp request.

Đáp án P11.C02 — Pod, Deployment và readiness

P11.C02.L03.Q01. `Available=True` chỉ nói mức sẵn sàng tối thiểu theo chiến lược; `Progressing=True` có thể còn đang cập nhật. Cần xem reason như `NewReplicaSetAvailable`, các bản sao mới/cũ và `kubectl rollout status`; so generation nếu status kind này có trường quan sát phù hợp để tránh đọc trạng thái cũ. Rollout xong cũng chưa kiểm tra selector, EndpointSlice ready, cổng hoặc response nghiệp vụ từ client. Phải kiểm chứng các ranh giới ấy trong cùng cửa sổ quan sát, không lấy một status làm lời bảo đảm vĩnh viễn.

Đáp án P11.C03 — Scheduler và ngân sách tài nguyên

P11.C03.L01.Q01. Scheduler xét requests và node allocatable đã trừ phần được request bởi Pod khác, không dùng mức CPU tức thời 10% để lấp chỗ. A thiếu `500-300=200m`; B đủ theo requests nhưng taint `NoSchedule` loại Pod không toleration. Đọc event `FailedScheduling`, requests từng container, allocatable, tổng requests đã đặt, taint/toleration, affinity và ràng buộc volume ở đúng thời điểm. Nếu đổi ràng buộc, phải xác nhận node mới thật sự thích hợp về quyền, topology và năng lực rồi thấy Pod được gán và phục vụ; toleration riêng nó không bảo đảm đặt lên B.

Đáp án P11.C04 — Service, EndpointSlice và NetworkPolicy

P11.C04.L01.Q01. Probe sửa đúng path làm Pod có thể đạt Ready, nhưng Service vẫn chọn `app=atals` trong khi Pod mang `app=atlas`. Controller không thể sinh backend khớp selector sai. So selector và labels trong cùng namespace, rồi quan sát EndpointSlice gắn với Service có địa chỉ Pod với `ready=True`; sau đó kiểm tra Service port 80 tới `targetPort` 8080 và gửi request từ đúng vị trí client. `Running` và thậm chí Pod Ready riêng lẻ chưa chứng minh đường Service.

P11.C04.L03.Q01. DNS trả Service IP loại bớt lỗi phân giải ở lượt đó. So Service selector với nhãn Pod, `port`/`targetPort` và tất cả EndpointSlice/điều kiện Ready; thử kết nối từ chính client và từ nguồn đã hoạt động tới cùng đích/port. Đọc policy ingress ở đích và egress ở nguồn theo namespace/label, xác nhận plugin thực thi NetworkPolicy, rồi quan sát route/dataplane của cluster nếu còn lỗi. Một timeout không định danh policy, CNI hay ứng dụng; backend đáp từ một nguồn cũng chưa chứng minh nguồn khác được phép tới.

Đáp án P11.C05 — PVC, CSI và dữ liệu workload

P11.C05.L02.Q01. `Bound` chỉ xác nhận PVC đã ghép với PV, không xác nhận CSI attach/mount, quyền file, nội dung hay backup. `FailedMount` yêu cầu xem event theo UID, CSI node/controller, volume handle, topology và trạng thái attach; không nên xóa PVC/PV khi chưa hiểu reclaim policy và dữ liệu. RWO giới hạn read-write theo node chứ không đồng nghĩa một Pod; driver/storage vẫn có thể từ chối mount vì nguyên nhân riêng. Sau khi mount thành công, đọc dữ liệu/hậu điều kiện nghiệp vụ và thử restore bản sao riêng trước khi tuyên bố an toàn dữ liệu.

P11.C05.L03.Q01. ReplicaSet có thể phục hồi số Pod theo template bằng một Pod mới, nhưng `emptyDir` gắn với Pod cũ đã mất khi Pod bị loại khỏi node; dữ liệu ứng dụng không được tái tạo bởi replica count. Trước khi cho phép thay Pod, xác định dữ liệu nào cần bền, chọn lưu trữ phù hợp và thử restore bản sao nhất quán; kiểm tra ràng buộc truy cập volume, tính tương thích phiên bản/schema và request nghiệp vụ sau khi Pod mới Ready. PVC/PV cũng không tự chứng minh backup hoặc rollback dữ liệu.

Đáp án P11.C06 — HPA và điều khiển mở rộng

P11.C06.L02.Q01. Hiệu CPU time chia cửa sổ 10 giây cho `300m`, `450m`, `150m`. Với request `500m` mỗi Pod, utilization lần lượt 60%, 90%, 30%, trung bình 60%; tỷ số với target 50% là 1,2, nên `ceil(3×1,2)=4` trong khoảng 2–5. CPU limit nếu có không nằm ở mẫu số. Đây là tính toán cơ sở: metric có thể cũ/thiếu, Pod chưa Ready, tolerance, chính sách tốc độ và ổn định thay quyết định cuối; Pod thứ tư còn cần scheduler, volume, readiness và đường mạng. Đối chiếu status HPA, timestamp metric, Pod/events, EndpointSlice và latency client trước khi kết luận mở rộng đã giúp.

P11.C06.L03.Q01. Pod mới Pending là thất bại ở bước đặt chỗ: xem requests, allocatable, taint, affinity và topology PVC. `nr_throttled` tăng ở Pod cũ cho thấy quota CPU có chặn trong cửa sổ đọc counter; cần so

`throttled_usec`, `usage_usec`, CPU limit và latency cùng cửa sổ để đánh giá tác động. `oom_kill` không đổi không chứng minh mọi bộ nhớ đều đủ nhưng không có bằng chứng OOM kill trong counter này. Có thể tăng năng lực node phù hợp hoặc điều chỉnh request/limit dựa trên nhu cầu đo và quyền thay đổi; hậu điều kiện gồm Pod mới Scheduled/Ready, request qua Service đạt latency mục tiêu, dữ liệu nguyên vẹn, và counter throttling/metric được đọc lại ở cùng thời gian.

Đáp án P11.C07 — Gateway API và ranh giới request ngoài cluster

P11.C07.L01.Q01. Đọc manifest render để biết Gateway dùng class nào, rồi xác nhận CRD đã có, `GatewayClass` ấy tồn tại và status Accepted ở generation hiện hành; sau đó mới xem Gateway/listener, HTTPRoute parent/conditions và địa chỉ dataplane. Từ vị trí client, xác nhận DNS, địa chỉ đích, Host/SNI và nơi sinh 404. `helm template` chỉ tạo YAML cục bộ với giả định capabilities, không hỏi API server và không chứng minh CRD, class, controller, listener hay request thực tồn tại.

P11.C07.L02.Q01. Tham chiếu Route → Gateway khác namespace là quan hệ attachment do listener `allowedRoutes` quyết định; ReferenceGrant ở `edge` không mở cửa khi listener còn `from: Same`. Người quản lý Gateway phải cho phép namespace `app` theo chính sách hẹp phù hợp, rồi đọc status parent của Route. Tham chiếu Route → Service ở namespace `data` là quan hệ backend riêng, cần ReferenceGrant trong namespace `data` cho đúng nguồn HTTPRoute và đích Service; sau đó xác minh Service/EndpointSlice. Grant không thay thế selector, endpoint hay khả năng kết nối.

P11.C07.L02.Q02. `Accepted=True` chỉ nói parent nhận đủ phần cấu hình để có tác dụng; `ResolvedRefs=False` báo ít nhất một tham chiếu sai hoặc không được phép. Với Service vắng mặt, kiểm tra `backendRefs.name`, namespace mặc định là namespace Route, port và message/reason của condition theo parent, cùng khả năng một ReferenceGrant cần thiết. Đối chiếu Service thực và endpoint sau khi sửa. Tăng timeout không tạo object bị thiếu và không đổi quyền tham chiếu; cũng chưa được suy từ condition này rằng dataplane đã phản hồi mã HTTP cụ thể.

P11.C07.L03.Q01. So DNS với địa chỉ Gateway và thử TCP/TLS/HTTP từ chính client, ghi IP, Host/SNI, thời gian và nơi sinh timeout. Từ điểm gần Gateway kiểm tra listener/dataplane và request tới Service, rồi từ trong cluster so Service/EndpointSlice với Pod, thử trực tiếp cổng ứng dụng và đọc log/latency; thay từng điểm quan sát để khoanh chặng bị ngắt. Gateway Programmed là báo cáo controller về lập trình dataplane, Route Accepted/ResolvedRefs về attachment/tham chiếu, endpoint Ready về chọn backend; không điều kiện nào kiểm tra đường client, policy/firewall, socket ứng dụng hay response nghiệp vụ tại thời điểm ấy.

Đáp án P11.C08 — Helm chart và vòng đời release

P11.C08.L03.Q01. Thứ nhất, xác nhận revision mới do rollback tạo và manifest/values/địa chỉ Gateway, hostname, Route/backend sau rollback thực sự trở về cấu hình mong muốn. Thứ hai, kiểm tra condition ở generation mới, Service endpoint Ready và request qua đúng DNS, SNI, Host, path từ client đại diện trả nội dung mong đợi. Thứ ba, kiểm tra schema và dữ liệu mà revision 7 đã ghi còn tương thích với image cũ, hoặc thực hiện phục hồi dữ liệu bằng kế hoạch riêng rồi xác minh nghiệp vụ. Helm không đảo migration hoặc tác dụng phụ chỉ vì rollback báo hoàn tất.

Đáp án P12.C01 — Phép đo, nhãn và chuỗi thời gian

P12.C01.L02.Q01. Nếu mọi tổ hợp đều có dữ liệu, 50.000 × 2 × 2 × 4 = 800.000 chuỗi bucket; còn `_sum`, `_count` và request counter làm tăng thêm. Thực tế mỗi ID thường chỉ gắn một status và một instance, nên số này là trần giả định của phép nhân, không phải phép đo. Thay ID bằng nhãn hữu hạn như route template, method, status class; giữ request ID ở log/trace. Sau sửa so số series, scrape duration, bộ nhớ và độ trễ truy vấn cùng điều kiện tải; xác nhận dashboard vẫn phân nhóm cần thiết.

Đáp án P12.C02 — Prometheus, PromQL và Grafana

P12.C02.L01.Q01. `rate` phải xử lý reset trên từng chuỗi trước khi `sum`; tổng counter rồi tính rate có thể che reset riêng lẻ. p95 không cộng hoặc lấy trung bình vì phân phối và số request của Pod khác nhau; dùng `histogram_quantile(0.95, sum by (le)(rate(..._bucket[3m])))` cho classic histogram với cùng phạm vi. Nếu một Pod mất scrape, phép gộp chỉ phản ánh các bucket còn quan sát được, có thể bias p95 và không kết luận toàn dịch vụ; kiểm tra `up`, số target/series tham gia và request thực.

P12.C02.L02.Q01. Trong `(0,180]`, mỗi instance tăng 12 lỗi và 120 request từ giây 60 đến 180: mỗi instance 0,1 lỗi/giây và 1 request/giây; gộp thành 0,2 và 2 request/giây, tỷ lệ 10%. CPU tăng 24 giây mỗi instance trên 120 giây, tổng 0,4 lõi. Bucket gộp trong cửa sổ 120 giây tăng 80, 180, 232, 240; p95 là vị trí 228 trong khoảng 180–232, nội suy 0,9615 giây. `increase` ngoại suy `rate` theo toàn cửa sổ 180 giây, nên cho khoảng 36 lỗi và 360 request trong ví dụ này, trong khi hiệu thô giữa giây 60 và 180 là 24 và 240. Khi scrape không đều hoặc reset, chênh lệch còn khác.

P12.C02.L03.Q01. Kiểm tra target có còn trong service discovery, `up` của đúng job/instance, thời điểm scrape cuối, lỗi HTTP/TLS/timeout của endpoint, series `http_requests_total` có mẫu mới không, và selector/job/route/time range của panel. Thử request nghiệp vụ tới `b` qua đường thật, quan sát log và health của ứng dụng. `up=0` chỉ xác nhận scrape thất bại cho target còn được theo dõi; không chứng minh dịch vụ đã ngừng hoặc đã hồi phục. Khi `b` vắng mẫu, tổng lỗi của `a` có thể giảm giả tạo so với trước dù tình trạng người dùng xấu hơn.

Đáp án P12.C03 — Logs, traces và OpenTelemetry

P12.C03.L02.Q01. `G=420–0=420 ms`, `A=405–12=393 ms`, `D=280–30=250 ms`, `U1=335–285=50 ms`, `U2=385–345=40 ms`; hai lần gọi cùng khoảng chờ giữa chúng kéo dài từ 285 đến 385, tức 100 ms. Tổng các duration là 1153 ms vì `A`, `D`, `U1`, `U2` nằm trong `G` và các child nằm trong `A`, nên thời gian bị đếm lặp. Có thể chọn 0..12 ms (Gateway trước API), 335..345 ms (chờ trước retry) hoặc 385..405 ms (API trước khi gửi response) là đoạn chưa có span riêng. Không được tự gán các đoạn ấy cho mạng, queue hoặc CPU.

P12.C03.L02.Q02. `G` là root, `A` là child của `G`; `D`, `U1`, `U2` là child của `A`. Log timeout gắn span `U1` ở 335 ms, severity `WARN`, span status `ERROR` do lần gọi này lỗi; retry `U2` kết thúc ở 385 ms và API ghi response HTTP 200 ở 405 ms. HTTP status của response cuối, span status và mức log là ba trường với hợp đồng khác nhau. `D` là DB client span 250 ms: nó đo thời gian từ API quan sát, có thể gồm chờ pool, mạng và xử lý DB; chưa có DB server span để khẳng định DB server xử lý 250 ms.

P12.C03.L03.Q01. Kiểm tra header `traceparent` đi khỏi Gateway và vào API; API extract context và tạo server span đúng parent; chính sách sampling và việc SDK kết thúc/flush span; exporter gửi OTLP; Collector receiver, processor, queue, retry và dropped items; cuối cùng backend ingest, index, retention và truy vấn đúng time range/trace ID. `01` biểu thị sampled flag trong context được truyền, không phải xác nhận span API đã được ghi, gửi, nhận hoặc lưu. Request thành công không thay thế kiểm tra pipeline telemetry.

Đáp án P12.C04 — Alerting và sức khỏe pipeline quan sát

P12.C04.L03.Q01. Thứ nhất, head sampling thấp hoặc tail sampling thiên về một loại request có thể làm trace đã lưu không đại diện; đối chiếu cấu hình/chỉ số sampling và metric toàn lưu lượng. Thứ hai, mất context hoặc span ở exporter/Collector/backend có thể làm trace ngắn giả; kiểm tra parent, queue, dropped/export failures và truy vấn cùng time range. Thứ ba, khác route, instance, phiên bản, cửa sổ hoặc đồng hồ lệch khiến hai tập không so được; khóa selector, thời gian và kiểm tra đồng hồ. Dùng nhãn metric hữu hạn như route template và status class để định vị nhóm; từ log có quyền truy cập lấy request ID/trace ID của request đại diện, rồi truy trace. Không đưa ID hay PII vào nhãn metric hoặc payload telemetry.

Đáp án P13.C01 — SLI, SLO và error budget

C13.1. Không. Tổng 11 dòng là 11.000 valid và 129 bad; summary nói giờ cuối 12.000/189, nên thiếu 1.000 valid/60 bad chứ không có bằng chứng lỗi đã hết. Trước tiên kiểm scrape/target, thời điểm sample, counter reset và đối chiếu log/edge từ đúng population. Đánh dấu SLI ngắn hạn `UNKNOWN`, giữ nguyên đánh giá rủi ro canary và không tăng traffic cho tới khi phép đo hoặc tín hiệu độc lập đủ tin cậy. Không dùng dòng vắng như 0 bad. Monthly summary cũng cần provenance: nếu được cập nhật từ một đường dữ liệu khác, nó không tự sửa được khoảng mù năm phút.

C13.2. Tổng 19 bad/1.000 valid = 1,9%; phép tính `(1%+10%)/2=5,5%` trao trọng số 50/50 dù A nhận 90% lưu lượng. SLO 99% cho phép `0,01×1.000=10` bad, nên đã vượt 9 bad. Không đổi SLO hay loại B khỏi population sau khi nhìn số.

C13.3. Năm phút có `60/1.000=0,06`; burn `0,06/0,001=60`, vượt 14,4. Nếu giờ có 100/12.000 = 0,008333…; burn `8,333…`, không vượt 14,4, nên điều kiện AND không page dù năm phút xấu. Cần dữ liệu 6h và 30m cùng population mới xét nhánh kia; một mẫu 1h không tái tạo được hai cửa sổ ấy. Ngay cả khi page, nguyên nhân và độ tin cậy đo vẫn phải kiểm tra.

Đáp án P13.C02 — Incident response và postmortem

C13.4. v1 có 1/900 ≈ 0,111%; v2 có 30/100 = 30%; chung là 31/1.000 = 3,1%. DB p95 45–50 ms và các mẫu K-26..28 42–49 ms không khớp giải thích “chậm” cho sai số tiền nhân 1/100; tuy nhiên chưa loại được trigger/logic ghi trong DB. DNS sai toàn đường khó tạo kết quả v1 đúng/v2 sai trong cùng cửa sổ; fixture chưa có edge destination hay revision từng request nên chưa loại được lỗi chọn đích theo nhóm. Ba kiểm tra ưu tiên: xác thực request ID→release/config/edge destination; đối chiếu input và dữ liệu lưu gắn với từng ID trong 30 request v2 sai, không suy số order phân biệt; đọc diff và chạy một input 4.100 cents trên hai bản trong fixture cô lập. Không tạo thêm đơn thật để thử.

C13.5. Chưa. (1) **Mitigation:** 10:12 mới là đề xuất chặn v2 writer và chuyển traffic, cần bằng chứng thao tác hoàn tất cùng hậu kiểm không còn writer v2. (2) **Request verification:** 10:16 còn pending; cần request ID mới, revision/digest, input và giá trị đọc lại đúng qua các route. (3) **Data reconciliation:** 30 là số request sai, không phải số bản ghi phân biệt; phải ánh xạ request→order, đối soát nguồn độc lập rồi sửa hoặc đền bù theo chính sách dữ liệu. (4) **Incident closure:** cần người chịu trách nhiệm xác nhận tác động, hậu điều kiện phục vụ mới, xử lý dữ liệu và thông báo; thiếu một trong bốn không ghi `resolved`. Mở v2 chỉ sau test hồi quy, canary gate nghiệp vụ và quyết định theo risk/budget; không dùng 5xx=0 làm bằng chứng.

C13.6. Rollback/reroute về v1 cần v1 tương thích schema và mọi writer v2 được dừng; hậu kiểm revision/digest, traffic share v2=0, đơn mới đúng. Flag chỉ dùng nếu có bằng chứng flag là nguyên nhân và mọi đường ghi đọc giá trị flag mới; hậu kiểm hai request ở nhánh flag và trạng thái worker. Failover chỉ nếu

vùng mới không mang binary/config sai và dữ liệu nhất quán; hậu kiểm từ client tới đúng vùng và đối soát số tiền. Fixture chưa cho bằng chứng cô lập vùng, nên failover không phải lựa chọn an toàn. Cả ba không tự sửa dữ liệu liên quan tới 30 request sai; số bản ghi phân biệt vẫn UNKNOWN.

Đáp án P13.C03 — Backup, DR và phục hồi dữ liệu

C13.7. Từ 10:00 lùi đến commit khôi phục cuối 09:52 là 8 phút, đạt RPO ≤15 phút trong fixture. Từ 10:00 đến 10:37 là 37 phút, đạt RTO ≤45 phút trong fixture. 10:27 mới replay xong; app chưa chứng minh đọc/ghi, route và invariant nghiệp vụ. Một lỗ WAL trong chuỗi cần thiết có thể chặn replay qua lỗ hoặc buộc chọn điểm sớm hơn; không có bằng chứng 09:52 còn khôi phục được, nên kết luận 8 phút phải chuyển thành UNKNOWN cho đến khi thử chain thật.

C13.8. O-101, O-102, O-103 có commit ≤09:52 và được fixture gắn `expected_restored=true`. O-104/O-105 ở 09:53/09:59 cần đối soát receipt, payment, queue hay event log độc lập, không tự tái phát hành. Trước khi xóa bản chính: xác minh backup/replica độc lập miền lỗi và có quyền/khóa đọc; thử restore và nội dung đơn; giữ ảnh hiện trạng và bằng chứng; kiểm tra replica chưa sao lỗi xóa; phê duyệt mục tiêu phục hồi và kế hoạch quay lui. Dữ kiện hiện có không cho phép xóa bản chính.

C13.9. Hash/size giúp kiểm byte hoặc manifest; `pg_verifybackup` dùng với bản `pg_basebackup` dạng plain có manifest, và kiểm WAL bằng bản công cụ cùng phiên bản, giúp phát hiện một số hư hỏng backup nhưng không là phép kiểm nghiệp vụ. DB phải mở được, replay đúng target, query/order count và constraint hợp lệ; ứng dụng phải đọc/ghi qua đúng endpoint với quyền đúng; nghiệp vụ phải đối soát amount, receipt và tác dụng phụ. Checksum PASS nhưng schema ứng dụng không tương thích hoặc O-104 đã thanh toán bên ngoài mà DB khôi phục không có đơn: chưa mở ghi production vì nguy cơ mất/trùng giao dịch. Cần điều tra và quy trình đền bù trước.

Đáp án P13.C04 — Hiệu năng, capacity và thử giới hạn

C13.10. A: 100 request/s × 0,120 s = 12 request; B: 160 × 0,250 s = 40 request, với điều kiện cửa sổ ổn định, cùng ranh giới vào/ra và latency trung bình của toàn population hoàn tất. C: `(220−180) request/s × 60 s = 2.400 request` tăng trong tổng chưa hoàn tất (đang chờ + đang xử lý) **nếu** không có reject, drop, timeout, hủy hay retry ẩn. Không biết bao nhiêu trong 2.400 đang chờ riêng ở queue; phải đo queue và in-flight đầu/cuối. `220×0,550=121` không là L đã chứng minh vì arrival≠departure, số chưa hoàn tất tăng và W chỉ đo những request đã hoàn tất. Counter reject/timeout hoặc đo tổng outstanding đầu/cuối khác +2.400 sẽ bác bỏ giả định; retry có thể làm offered bị đếm hai lần.

C13.11. CPU tổng 70% che per-core/quota skew và waiting; C đã có số request chưa hoàn tất tăng dù completion chỉ 180/s; thêm replica có thể tăng áp lực lên DB hoặc connection pool, còn 200/s chưa được đo. Giả thuyết DB wait/connection pool bão hòa phù hợp p95 DB wait 300 ms; bác bỏ bằng phân rã trace theo stage, pool wait, DB active/lock/IO và so cùng traffic mix. Không tuyên bố 200/s đạt SLO khi không có error rate, thời lượng ổn định, tail ở 200 hay test mất một replica.

C13.12. Chốt artifact/topology/payload, generator đủ dư địa, offered/accepted/completed/rejected và SLI client. Baseline 160 rồi 180/200, stress có kiểm soát đến 220 và stop nếu p95 >600 ms hai cửa sổ, backlog tăng, lỗi dữ liệu hoặc generator bão hòa. Soak giữ 200 sau khi đã chứng minh ổn định để xem leak/queue/GC; hậu kiểm giảm tải, queue rút và nghiệp vụ đúng. PASS trong môi trường thử không chứng minh production nếu DB nhỏ hơn, tenant mix khác, cache nóng hơn hoặc không có mất replica/rollout như thực tế. Không đem fixture A–C làm kết quả test thật.